Manual

MES Development Suite AIS: AIP2

MDS-AIS 8.1

Version 1.4.23049

Last changed on: 01.09.2020

Copyright

Contents

1 Overview – AIP2

1.1 Features

You can use the MES Development Suite to change and extend the data collection and the data display on the shop floor client AIP2.

The document **MDS-AIS_81_AIP2** describes the functions that the MES Development Suite Business Applications & Services provides to change and extend the data collection and the data display on the shop floor client AIP2.

- Using configuration files, you can change the layout displayed on the shop floor client AIP2. The configuration files are available as XML or INI files depending on intended use.
- Using the dialog configuration on the MOC, you can change and define the dialogs and workflows to enter and display data.
- For the data collection via dynamic dialogs, you can use the user exits provided to implement dynamic actions. The user exits are implemented in a script language. The script language is similar to the programming language Visual Basic and easy to learn.
- Also for the configured GUI (tile GUI), user exits are provided that you can use to implement dynamic behavior. The user exits of the tile GUI are implemented in the script language **PasScript**. The script language is similar to the programming language Pascal and easy to learn.
- Using deployment mechanisms, you can automatically deploy the customer-specific configurations and user exits to the shop floor clients.

The document **MDS-AIS_81_AIP2** is the reference manual of the functions provided. To learn all about the MES Development Suite Business Applications & Services, MPDV offers specific trainings. MPDV recommends to attend this training to be able to successfully use this product.

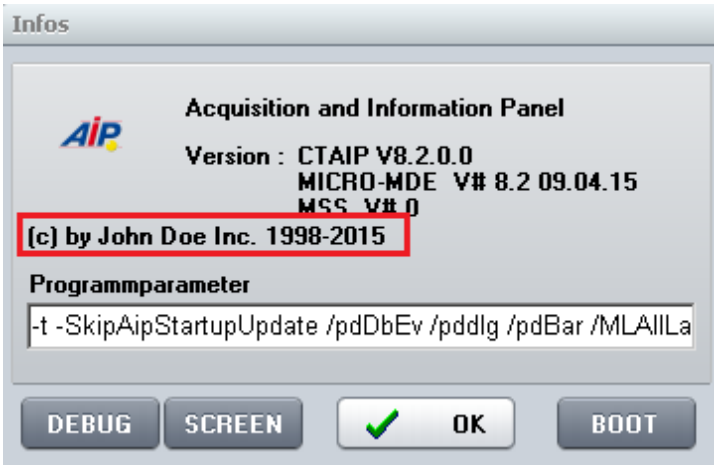
2 AIP2 -Local Configuration

2.1 Local Configuration ctaip.ini

The most important hardware and system settings are defined for each terminal in the CTAIP.INI file of the C:\MPDV\AIP2 directory.

Changes to the configuration file ctaip.ini are only enabled after rebooting the terminal software.

Layout configuration

Entry	Comment
Section [system]	
CompanyInfo=(c) by John Doe Inc. 1998-2015	Overwrites the copyright text in the “About" dialog 
parameters= … -t | +t	The virtual keyboard can be disabled/enabled, irrespective of the terminal type.
DEMO_ALL=ON	The information in the lower bar are normally hidden in demo mode. This option, however, causes all pieces of information to be displayed even in demo mode (version number and date, server, terminal number, online lights).
parameters= … -SkipDynDlgExtraction…	Internal option preventing the extraction of dialog fields/buttons when updating dialog data. (Only applies if WF directory ".\spool\$wf.$$$\“ is available)
parameters= … -SkipAipStartupUpdate…	Internal option preventing the configuration files (*.ini,*.cfg,*.cfi) of the module DLLs (packets*.dll) and application DLLs (functions*.dll) from being synchronized when starting the terminal.
VirtScreenSize=640	All windows are started with the indicated resolution

Entry	Comment
VirtScreenRatio=16:9	The display ratio remains if the configuration VirtScreenSize is used to reduce the window. The width-to-height (aspect) ratio can be changed using VirtScreenRatio. Consequently, the width-to-height ratio 16:9 can be tested with a 4:3 monitor and vice versa. The value can be configured as "16:9" or as floating point value (e.g. 1,77777). Note: Information screens are specifically scaled if monitor screens in portrait format (e.g. 9:16) are used. This special scaling can be disabled in hytnrcfg.ini: [Tnr Konfiguration 0]->PortraitAlignment=off
Section [SKIN]	Configurations for the terminal's skin (see ctaiplay.ini) The online configuration of the terminal can be reached by the info dialog (click MPDV icon). ALT+F1 opens the control elements for the skin. Please note: Only in the design/GUI of AIP 8.1 and/or the dynamic dialogs
Saturation=0	Skin configuration / Default = 0
Hue=0	Skin configuration / Default = 0
Name=mpdv	Skin to be used / Default = mpdv
Active=false	Skins can only disabled in ctaip.ini! Default = true

Entry	Comment
Section [system]	
`Usr=21`	Distinct terminal number
`Hostname=192.9.200.24`	Internet address of the server
`Offlinetimeout=600`	The interval after which online access should be attempted the next time. The interval is specified in seconds
`Showcursor=on`	Show or hide mouse pointer in terminal application: on: mouse pointer active off: mouse pointer inactive
`Loadfile=` `ctnet\win\aip2.txt`	Configuration file to download the application from the server. The paths is relative to the installation directory of the system.
`Watchdog=on`	ON: Watchdog is activated OFF: Watchdog is not activated

Entry	Comment
Demo=off	'on': Offline demo mode; always off in the production environment!
parameters=-t	The –t parameter switches off the virtual keyboard.
TMOUT_C=xxx	Timeout for CONNECT to the server If not specified, default = 10 seconds → Increase to 20 seconds for routing
TMOUT_S=xxx	Timeout for SEND to the server If not specified, default = 10 seconds → Increase to 20 seconds for routing
TMOUT_R=xxx	Timeout for RECEIVE of the server If not specified, default = 120 seconds
TMOUT_F=xxx	Timeout for FILESERVER operations to the server If not specified, default = 10 seconds → Increase to 20 seconds for routing
Section [barcode]	**Configuration of customized barcode prefixes.**
BarKenn90=MNR4 ... BarKenn99=ANR3	BarKenn90 > defines the prefix (here: 90); The ID from the dialog (= acronym) is assigned.
Section [comports]	
com1=0 com2=MSS com3=BAR Com3=LEGIC Com4=RFLESER	Assignment of serial interfaces to connected devices: MSS – machine interface BAR, LEGIC, RFLESER – various reading devices
Section [MSS-INIT]	
MSS_DIALOG=10	If the terminal is switched off longer than 15 minutes, a dialog is displayed on terminal restart. The user must then decide whether the counter pulses, which were recorded when the terminal was closed, are posted or discarded. The dialog closes automatically with "Yes" after an entered time has elapsed; in this case the counting impulses are posted. This value configures the time in seconds the dialog is open. If the terminal is switched off for less than 15 minutes, no dialog is opened; the counting pulses recorded in the switch-off phase are accepted and posted without confirmation. Please note: The value can also be configured in hytnrcfg.ini. Entries in the hytnrcfg.ini file take priority.

Entry	Comment
MSS_FILEAGE_MIN=5 MSS_FILEAGE_OVERTIME=delete	Additional configuration for **MSS_DIALOG** If the backup file for counting pulses of the terminal is older than 5 minutes, no dialog is opened and the back up file deleted. Quantities recorded at the time when the terminal was closed are not used/posted. Please note: The value can also be configured in hytnrcfg.ini. Entries in the hytnrcfg.ini file take priority.
Section [ext. software]	
Button=Editor WindowName=Editor SearchParts=On	Configuration of the button in the top line: A previously started program can be brought to the foreground at the push of a button. Button: button caption WindowName: Name of the program (e.g. from the taskbar). SearchParts=On: Parts of WindowName are sufficient SearchParts=Off: WindowName must be entered completely. The option "SearchParts=On" is recommended for programs such as MSWord that change the title bar subject to the document that is currently being loaded.
ProgFileName=c:\Programme\wincmd\Wincmd32.exe	The program that is started if the program mentioned above cannot be called to the foreground.
AutoStart=on	This option starts the program (ProgFileName) when starting the terminal program.
Section [DLL]	
BusDLL=PCC.EXE	PCC.EXE must be entered here if the terminal is configured with the option "operated as HYDRA-MDE terminal". If necessary, the AIP2 enters this value independently upon starting the program.
Section [PDV]	
PDVProtokoll=ON	Enables PDV logging (prot_pdv.txt) (by default=OFF)
PDVIOPDir=c:\IOPSim\	Path for DNC
Section [CAQ]	
;Supported barcodes	
FieldWNRBarcodeOnly=Y	If this entry is set the tool number may only be entered using a scanner.
FieldNestBarcodeOnly=Y	If this entry is set the cavity number may only be entered using a scanner.
FieldNummBarcodeOnly=Y	If this entry is set the number may only be entered using a scanner.
FieldKNRBarcodeOnly=Y	If this entry is set the badge number may only be entered using a scanner.

Entry	Comment
BarcodeWNR=	This field specifies which acronym is entered into the tool number field by the scanner.
BarcodeNest=	This field specifies which acronym is entered into the cavity number field by the scanner.
BarcodeNumm=	This field specifies which acronym is entered into the number field by the scanner.

2.2 PNG – Files / Bitmaps

The use of PNC files is recommended by MPDV. By default PNG files have a size of 24 x 24 px.

2.2.1 File pict.zip

The file "pict.zip“ is updated by the installation tool "inst32.exe“ while downloading and includes all default PNG files.

The default PNG files can be overwritten in the file pict_cust.zip. Several PNG files have the extension ".small.png" (e.g. aip.small.png). These PNG files are used with a screen resolution of 640x480.

2.2.2 File pict_cust.zip

The file "pict_cust.zip“ is loaded from the server directory (e.g. \<serverDir>\1\custom) when starting the program (as is the case for the hycust.mld).

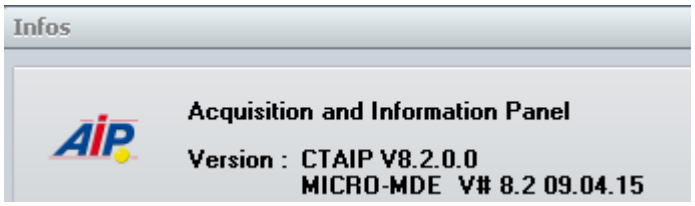
Customized PNG files may be stored in this file and loaded by the AIP2 terminal. Default PNG files may also be "overwritten".

Please note: file sizes are not adjusted.

Customize header

The AIP icon displayed in the header can be replaced by storing a separate AIP.png file in the pict_cust.zip file.

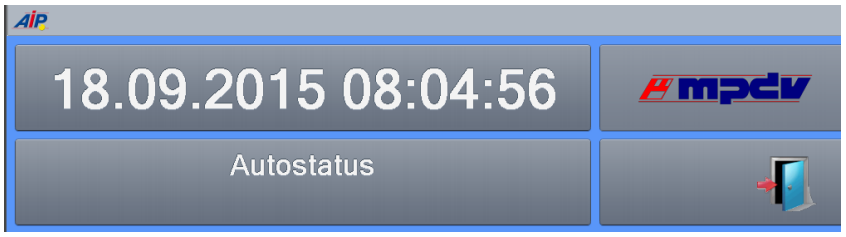
This AIP icon will also be replaced in the "About" dialog.



Customize footer

The MPDV icon displayed in the footer can be replaced by storing a separate company.png file in the pict_cust.zip file.

Customize PZE dialog



The MPDV icon displayed in the PZE dialog can be replaced by storing a separate pze_mpdv.png file in the pict_cust.zip file. In case the PZE terminal is operated with a screen resolution of 640x480, a customized pze_mpdv.small.png file has to be integrated in the pict_cust.zip file.

2.3 Multilingualism (*.mld files)

The below-mentioned files are required for the translation of the application. The table shows the priorities for the translation, ownership and the relevant storage locations.

Priority	File	Server directory	Owner	Description
1	ctaipkd.mld	./ctnet/win/aip2/custom	Customer	Customer-specific translations
2	hycust.mld	./1/custom/	MPDV	Customized translations by MPDV. The file is used by the AIP2, CTAIP and CTWIN terminal.
3	ctaip.mld	./ctnet/win/aip2	MPDV	Standard translation file

3 AIP2 - GUI Configuration

3.1 Overview

The layout for the new GUI of the AIP2 terminal (tile design) is stored in XML files. XML files can be edited using a standard text editor. Microsoft's *XML Notepad 2007* can also be used. This XML editor provides a clearer presentation, a user-friendly copy function for entire objects and the possibility to move complete objects. *XML Notepad 2007* was used for the generation of screenshots included in this document.

In the configuration, font sizes and positions are given in points in relation to a screen resolution of 600 points in height. Values must be scaled and then rounded to whole dots when using a screen with a higher resolution. Therefore, the proportions can slightly vary depending on the screen resolution.

Colors are specified in XML files in a reversed RGB notation (**B**lue/**G**reen/**R**ed instead of **R**ed/**G**reen/**B**lue). If the entry is in the hexadecimal format, the two places behind the symbol $ define the color blue, the next two places the color green and red is defined by the last two places.

3.2 Filing XML files in the server

Like the INI files ctaiplay.ini and ctaipbut.ini, the XML files that define the GUI are located on the server in the sub directory ctnet\win\aip2. XML files are filed in the sub directory *gui.*

3.2.1 Scope Concept

The INI files in the subdirectory <SystemNo>\custom\aip2 and the XML files in the subdirectory <SystemNo>\custom\aip2\gui can be overridden customer-specifically in deviation from the standard. Various *scopes* are provided in order that the different changes do not overwrite each other.

Scope	Directory	Examples
Standard	ctnet\win\aip2\<x>.<y>	ctnet\win\aip2\ctaiplay.ini ctnet\win\aip2\gui\l_anr.xml
Standard scope	<SystemNo>\custom\aip2\<x>.<y>	1\custom\aip2\ctaiplay.ini 1\custom\aip2\gui\l_anr.xml
Custom Scope	<SystemNo>\custom\aip2\<x>@**custom**.<y>	1\custom\aip2\ctaiplay@custom.ini 1\custom\aip2\gui\l_anr@custom.xml
VAR scope	<SystemNo>\custom\aip2\<x>@**var**.<y>	1\custom\aip2\ctaiplay@var.ini 1\custom\aip2\gui\l_anr@var.xml
Local scope	<SystemNo>\custom\aip2\<x>@**local**.<y>	1\custom\aip2\ctaiplay@local.ini 1\custom\aip2\gui\l_anr@local.xml

Standard scope

The Standard Scope is used to create a copy of the standard. This ensures that changes to the standard, sometimes supplied with a service pack, do not interfere with the collection terminal and therefore, the collection software remains unchanged. After generating the copy, the required changes to the standard must be either copied again or synchronized. The changes can then be integrated into the Standard Scope.

Custom Scope

The Custom Scope is reserved for MPDV to file customer specific configurations. A file is stored in the Custom Scope if the file name includes **@custom** before the extension.

VAR scope

The VAR Scope is reserved for partners (Value Added Reseller) to store changes for customers of partners. A file is stored in the VAR scope if **@var** is inserted before the extension.

Local scope

The Local Scope is reserved for customers to store their own customized files. A file is stored in the Local Scope if **@local** is inserted before the extension.

The priority of the different scopes is ascending from the standard scope to the local scope. A file in the local scope takes priority over a file included in the standard scope.

INI and CFG files are processed differently to XMLfiles:

INI and CFG files are merged per section, that means sections in the standard are totally replaced by potentially existent sections deriving from individual scopes. Settings in the Local Scope have highest priority as they are processed at last and therefore overwrite settings from the scope located above.

XML files are not merged but accepted. That means only files are processed located in the list of scopes at the bottom.

The only exception is the file *globaldefines.xml.* The content of that file is merged with the settings of the individual scopes. It is therefore possible to overwrite individual settings (i.e. font size or color) without copying the complete file. If you would like to overwrite a certain element of the file *globladefines.xml,* please copy the file, delete all elements to be accepted from the standard file and then store the file in the relevant Scope.

3.2.2 Specific layouts of terminals or terminal groups

Like the INI files ctaiplay.ini and ctaipbut.ini, the XML files that define the GUI are stored on the HYDRA server in the subdirectory <SystemNo>\custom\aip2. Standard XML files are filed in the sub directory *gui.*

If different GUI layouts are required for specific terminals or terminal groups, the non-standard XML files are stored in the following subdirectories:

Reference	Subdirectory	Example
Terminal group	tgrp_<*Terminal group*>\gui	tgrp_900\gui
Terminal	tnr_<*Terminal number*>\gui	tnr_100\gui

XML files stored in these sub directories replace standard files with the exception of the *globaldefines.xml* file whose content is merged with the relevant standard files.

3.2.3 Loading configuration files during restart

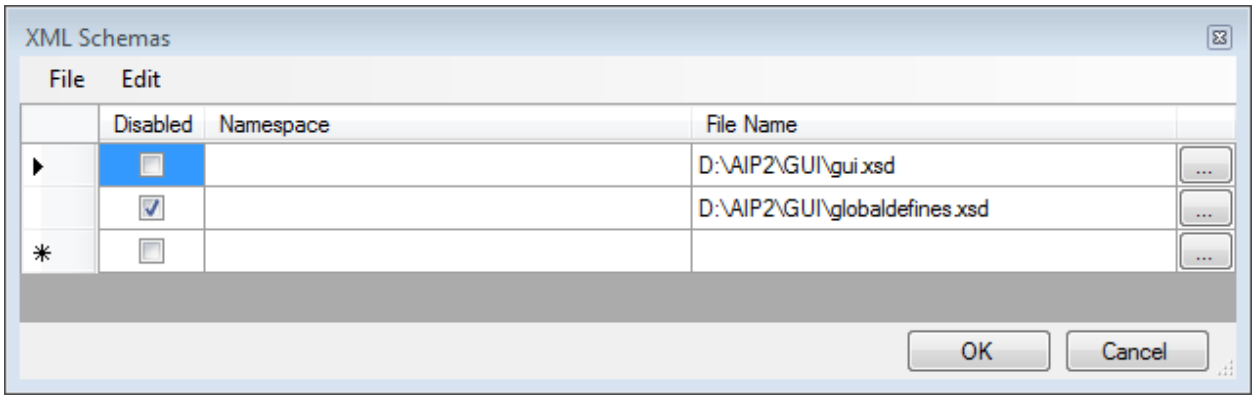
Every time the AIP2 is started, INI, CFG and XML files are updated from the server and automatically activated in the terminal.

When changing the layout, please note that the changed layouts are not overwritten when the AIP is started. Updating of configuration files can be deactivated in the file ctaip.ini by adding the parameter *SkipAipStartupUpdate* to the entry *parameters*= in the section *[system].*

3.2.4 Syntax check via XML Schema Definition (XSD)

An XML Schema Definition (XSD) defines the structure of an XML file. Depending on the editor used, a syntax check of the edited XML file is performed. In addition, you can select the value of specific fields via a selection list.

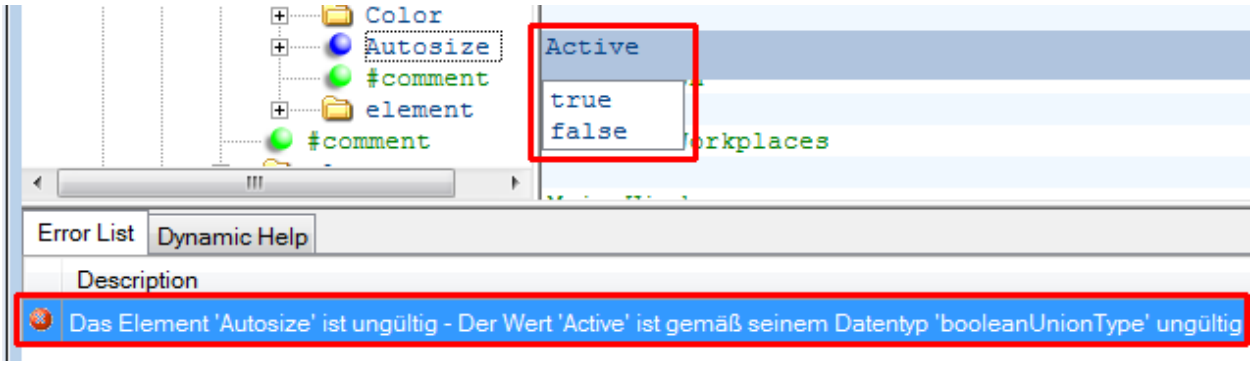
In the *XML Notepad 2007* of *Microsoft,* you can enter XML Schema Definitions via the menu item *View – Schemas…* and enable or disable them:



The file *globaldefines.xsd* includes the schema for the file *globaldefines.xml*. The file *gui.xsd* includes the schema for the other XML files used for the GUI configuration. The two XSD files are located in the HYDRA server in the same directory as the corresponding XML files.

The two XSD files *globaldefines.xsd* and *gui.xsd* are not compatible. For this reason, one of these files must always be *disabled*.

The following example shows a syntax check and a selection list:



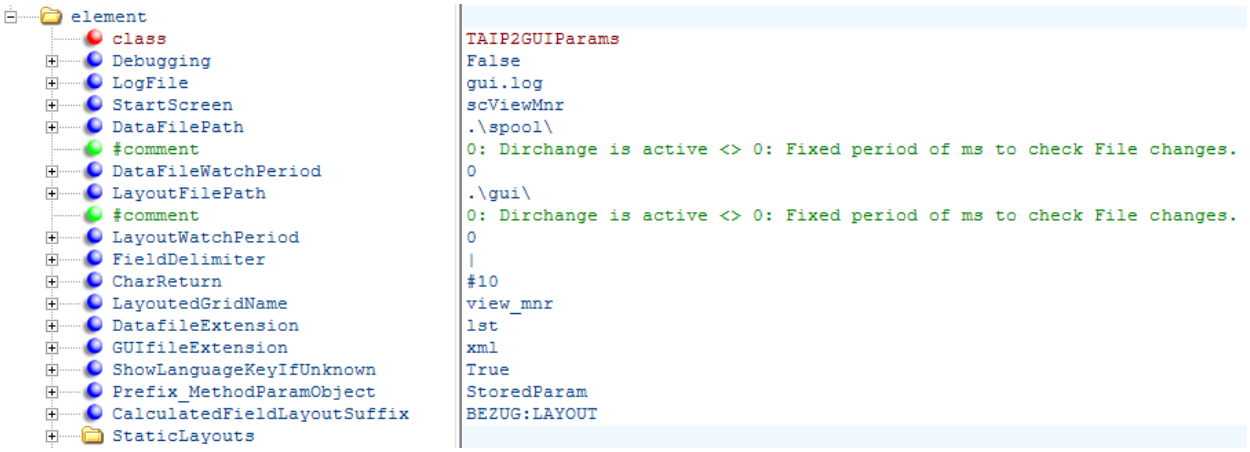
3.3 Settings

The file *globaldefines.xml* includes general settings, constants, data sources, calculated fields and functions.

Changes in the file *globaldefines.xml* are only active after restart of the AIP2.

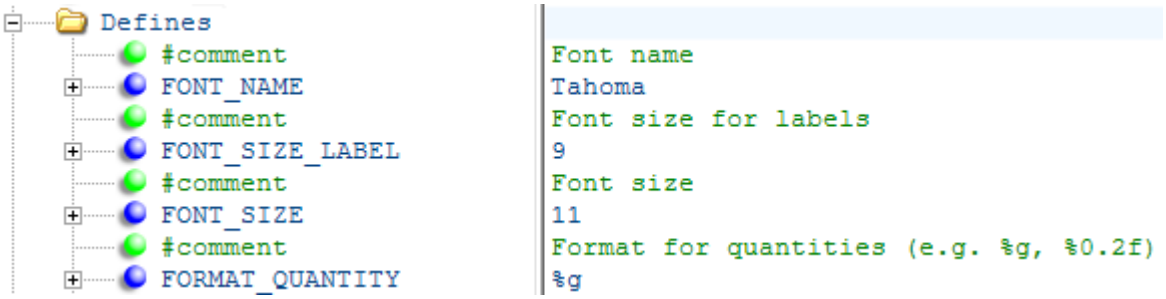
3.3.1 General settings

Standard settings control program processing and may not be changed.



3.3.2 Constants (Defines)

The section *Defines* specifies the Constants used for different layout configurations. For example, you can change color or font size at a central location using these constants.



The following constants can be set:

FONT_NAME

This constant sets the font of the GUI. The standard setting is „Tahoma“.

FONT_SIZE_LABEL

This constant sets the font size for the GUI labeling. The standard setting is 8.

FONT_SIZE

The constant FONT_SIZE sets the font size for the displayed data in the GUI. The standard setting is 10.

FONT_SIZE_HEADING

FONT_SIZE_HEADING defines the font size for the headings on the right hand side. The standard setting is 10.

FORMAT_QUANTITY

This constant sets the format for the display of quantities.

Standard setting is „%g“.

"%g": Automatically formats the shortest display: If the quantity is an integer, then without decimal places and without decimal separators. If the quantity is not an integer, the existing decimal places are output after a decimal separator (maximum 15 decimal places). No thousand separator is output.

"%0.0f": No decimal places, without thousands separator.

„%0.2f“: Always two decimal places, without thousands separator.

„%0.2n“: The format n is the same as the format f, but the resulting string contains thousands separators, if a thousands separator is configured in the regional settings.

The set format only affects configured layouts and not dynamic dialogs. A delimiter for thousands is not available for dynamic dialogs.

FORMAT_CYCLE

FORMAT_CYCLE defines the output format for cycle times (target cycle and actual cycle) if they are not output as durations (as in the standard system) but in seconds. Formatting is issued like in the previous field. Standard setting is "%0.3f". Please refer to section "1.2.4 CalculatedFields" for further information.

The set format only affects configured layouts and not dynamic dialogs.

COLOR_MENU_BUTTON

This constant defines the background color for specific buttons on the left hand side, like the button "<back" and "PZE". For example the button "< Back" and the button "PZE" on the start page belong here. Standard setting is "$C0C0C0" (light gray).

COLOR_MENU

This constant controls the background color of the buttons used for selecting workplaces in the "Home"-page and for calling functions if an object was selected. Standard setting is "$E0E0E0" (lighter gray).

COLOR_MENU_ACTIVE

This constant can set a background color for the selected workplace. Standard setting is "$909090" (gray).

COLOR_BACKGROUND

This constant can set a background color to display data on the right hand side. The color should correspond with the COLOR_MENU_ACTIVE. Standard setting is "$909090" (gray).

COLOR_MARGINS

This constant sets the color for the borders of the layout. Standard setting is "$FFFFFF" (white).

COLOR_HEADING

This constant defines the background color for the upper headings on the right hand side. Standard setting "$833014" (dark blue).

COLOR_HEADING_2

This constant defines the background color for the headings in the middle on the right hand side. Standard setting is "$974428" (blue).

COLOR_HEADING_3

This constant defines the background color for the lower headings on the right hand side. Standard setting is "$AA583B" (light blue).

COLOR_FONT_HEADING

This constant defines the font color of the headings on the right hand side. Standard setting is "$FFFFFF" (white).

COLOR_TILE

This constant defines the background color of the individual tiles on the right hand side. Standard setting is "$F8F8F8" (very light gray).

COLOR_FONT

COLOR_FONT sets the standard font color. Standard setting is "$202020" (dark gray).

COLOR_STATUS_PRODUCTION

This constant defines the color for the status *production.* Standard setting is "1077248" (dark green, $107000). With this constant, the color must be entered as a decimal value to avoid the display in a different color on specific screens.

COLOR_STATUS_NO_PRODUCTION

This constant controls the color for the all statuses except *production.* Standard setting is "$1090FF" (dark yellow).

COLOR_STATUS_NOT_ASSIGNED

This constant defines the color for the status "Not assigned". Standard setting is "$1010D0" (dark red).

COLOR_YIELD

This constant defines the color to display yields. Standard setting is "1077248" (dark green, $107000). With this constant, the color must be entered as a decimal value to avoid the display in a different color on specific screens.

COLOR_SCRAP

COLOR_SCRAP controls the color to display scrap. Standard setting is "$1010D0" (dark red).

COLOR_INSPECTION_DUE

This constant can set a background color to display a CAQ inspection due. Standard setting is "$1090FF" (yellow).

COLOR_INSPECTION_DONE

This background color indicates if a minimum inspection scope was reached. Standard setting is "1077248" (dark green, $107000). With this constant, the color must be entered as a decimal value to avoid the display in a different color on specific screens.

COLOR_INSPECTION_ERROR

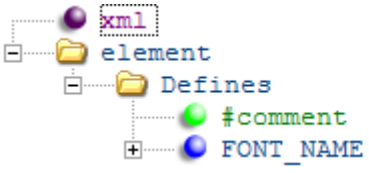
If an error occurs in the inspection planning, the relevant area is shown in the color set. Standard setting is "$1010D0" (dark red).

COLOR_MAINTENANCE_STATUS_0, …, COLOR_MAINTENANCE_STATUS_3

Color to display maintenance status of resources. Standard settings are "$FFFFFF" (white), "$873418" (blue), "$1090FF" (yellow) and "$1010D0" (red).

The name of customer specific constants must include the prefix "U". This way, they cannot be mixed up with constants of the standard.

The following example shows how to override a constant in a scope so that it is merged with the settings in the standard scope.




```
version="1.0" encoding="utf-8"


Font name
Arial
```

3.3.3 Data sources (ProviderDefinition)

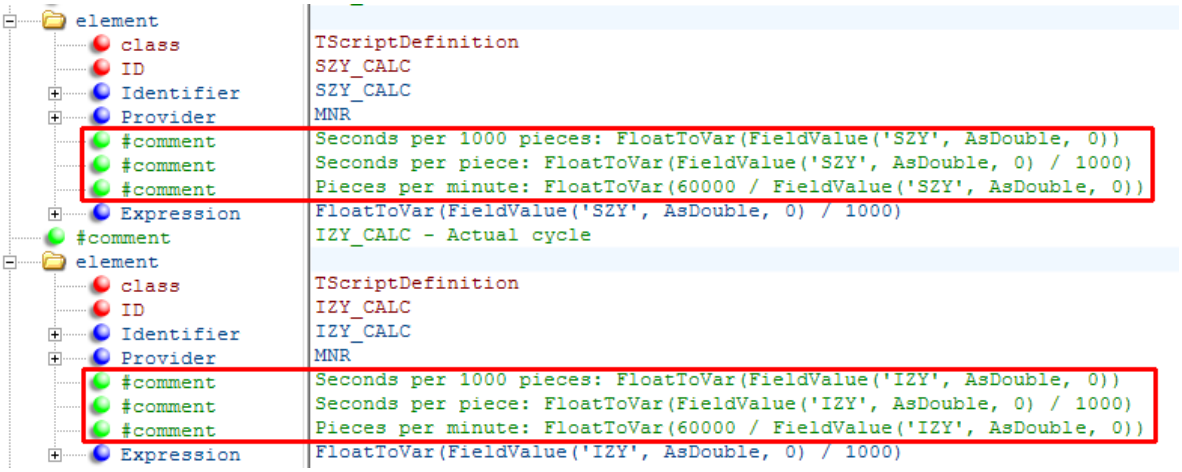
The settings define the correlation between the individual data sources and may not be changed.

3.3.4 Calculated fields

Calculated fields configure the display of the target and actual cycle. Both fields can be provided either as "time for 1000 pieces", "time for one piece" or "pieces per minute". The setting is done using the calculated fields *SZY_CALC* and *IZY_CALC*. The following formulas can be stored in the attribute *Expression* :

Field	Presentation	Formula
Target cycle	Time for 1000 pieces	FloatToVar(FieldValue('**SZY**', AsDouble, 0))
	Time for one piece	FloatToVar(FieldValue('**SZY**', AsDouble, 0) / 1000)
	Pieces per minute	FloatToVar(60000 / FieldValue('**SZY**', AsDouble, 0))
Actual cycle	Time for 1000 pieces	FloatToVar(FieldValue('**IZY**', AsDouble, 0))
	Time for one piece	FloatToVar(FieldValue('**IZY**', AsDouble, 0) / 1000)
	Pieces per minute	FloatToVar(60000 / FieldValue('**IZY**', AsDouble, 0))

Three options can be used as comments in the file globaldefines.xml:

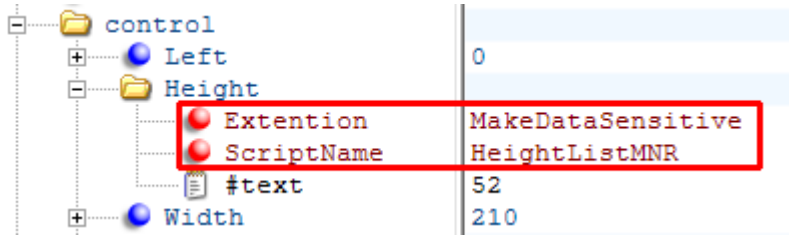


The formatting of both fields is configured using the constant FORMAT_CYCLE. Please refer to section "1.2.2 Constants (Defines)" for further information.

3.3.5 Functions (ScriptDefinitions)

The settings may not be changed during configuration of the GUI.

Fields that start a function are identified by the attributes *Extention* and *ScriptName*. The following example controls the height of the entries in the list of workplaces in the main view (a_list_mnr.xml):



The entry in the field *#text* is not active in this case. It is overwritten by the result of the previously entered function. If you want to change the height entered in this field, you must delete the two attributes *Extention* and *ScriptName.* Please note that in this case the data dependent identification of the height is disabled.

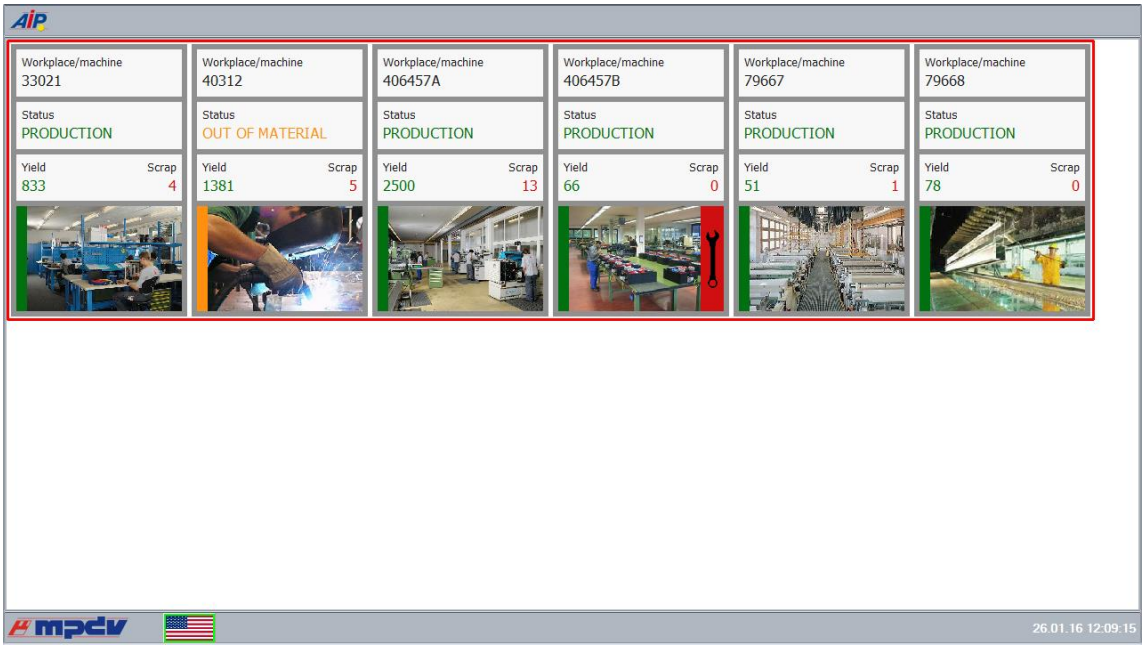
3.4 Layout definition

The definition of the layout is separated into layout files beginning with „l_“ and areas consisting of file names beginning with "a_".

There are the following layouts and areas in the standard:

l_view_mnr.xml

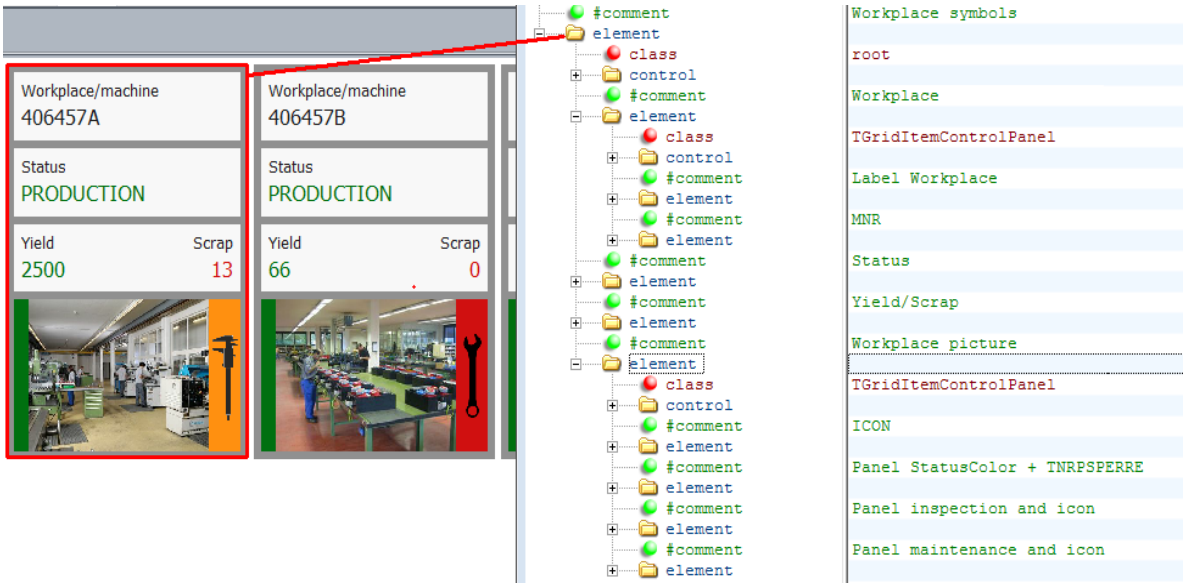
The tile view shows workplaces assigned to the terminal:



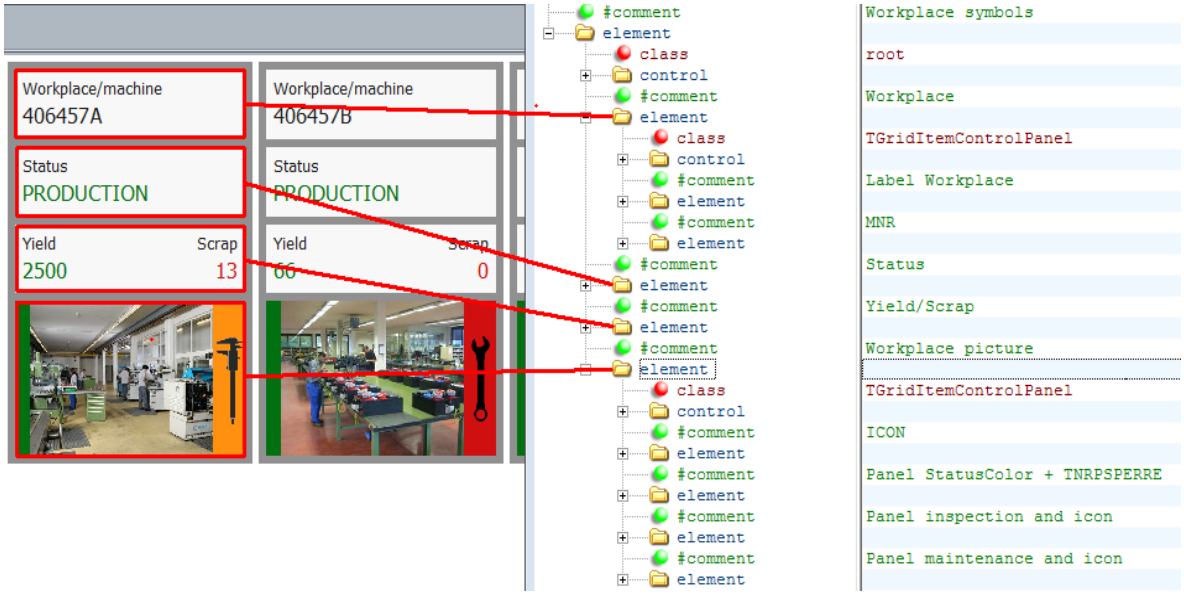
The structure for the individual workplaces is stored in the file **a_view_mnr.xml.**

The following screenshots show the assignment of the elements in the file to the objects in the GUI.

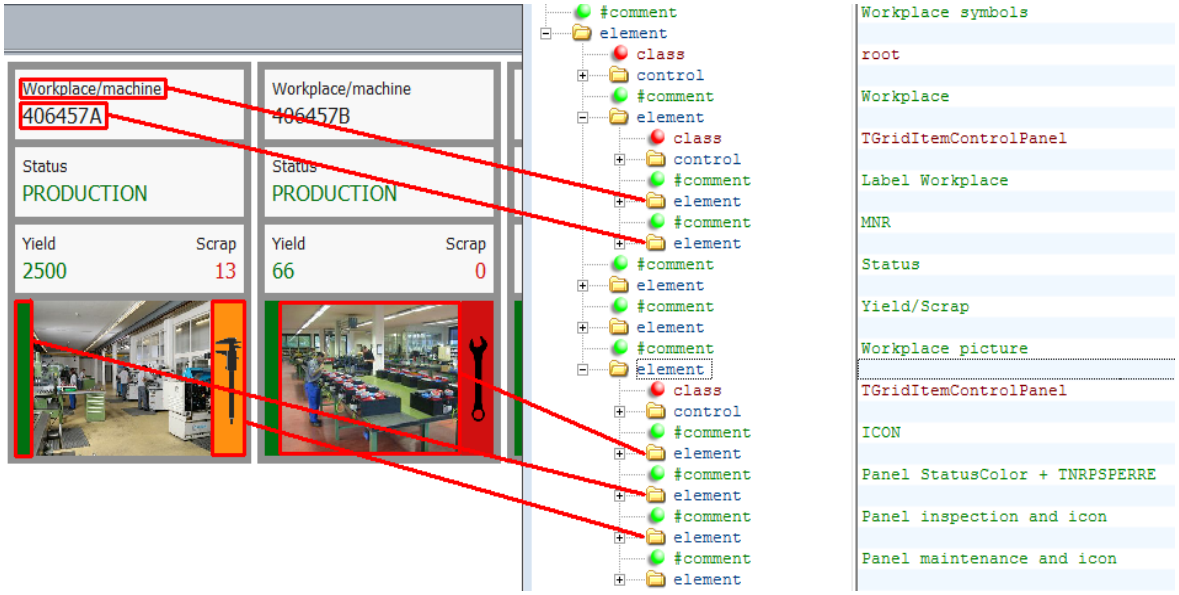
The element on the highest level of the tree structure is the big outer tile with gray frame:



The elements in the next level of the tree structure are the 3 tiles in light gray and the tile including the image:



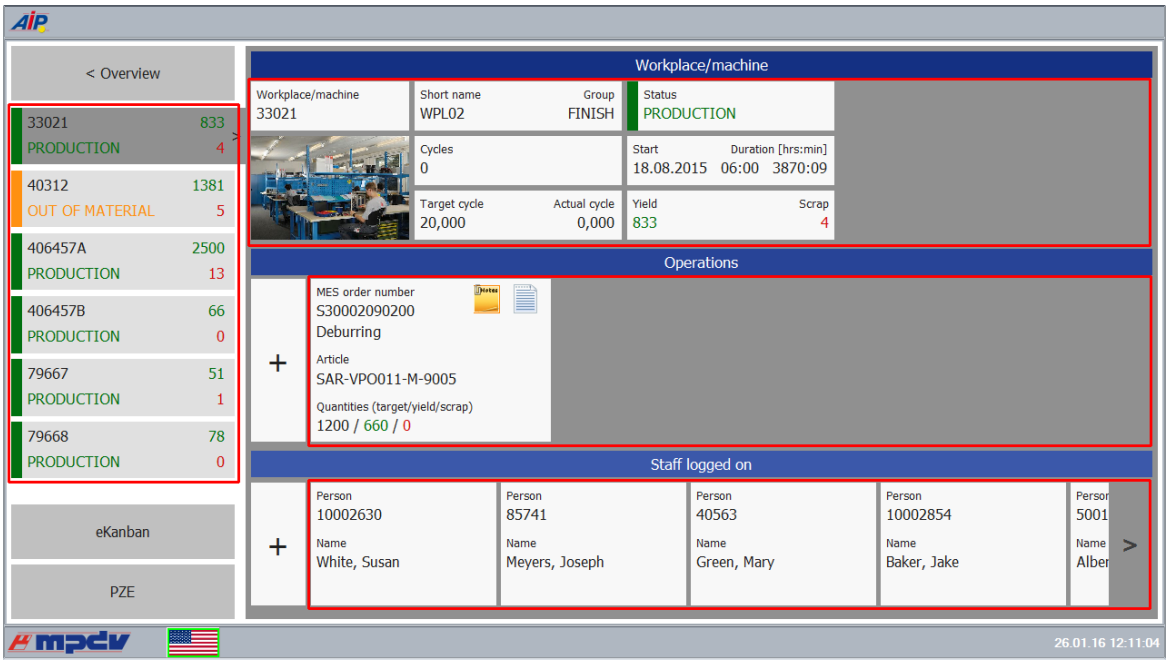
The elements in the next level of the tree structure are assigned as shown in the example of the light gray tile on top and the tile including the image at the bottom:



The lowest element in the list defines the layout of the tile including the screwdriver.

l_main.xml

Select a workplace to reach the main view:



The file **a_list_mnr.xml** contains the structure of the view of a workplace located in the list on the left hand side.

The file **a_data_mnr.xml** defines the upper area on the right hand side to display data for the workplace. This area is used for all layouts that follow.

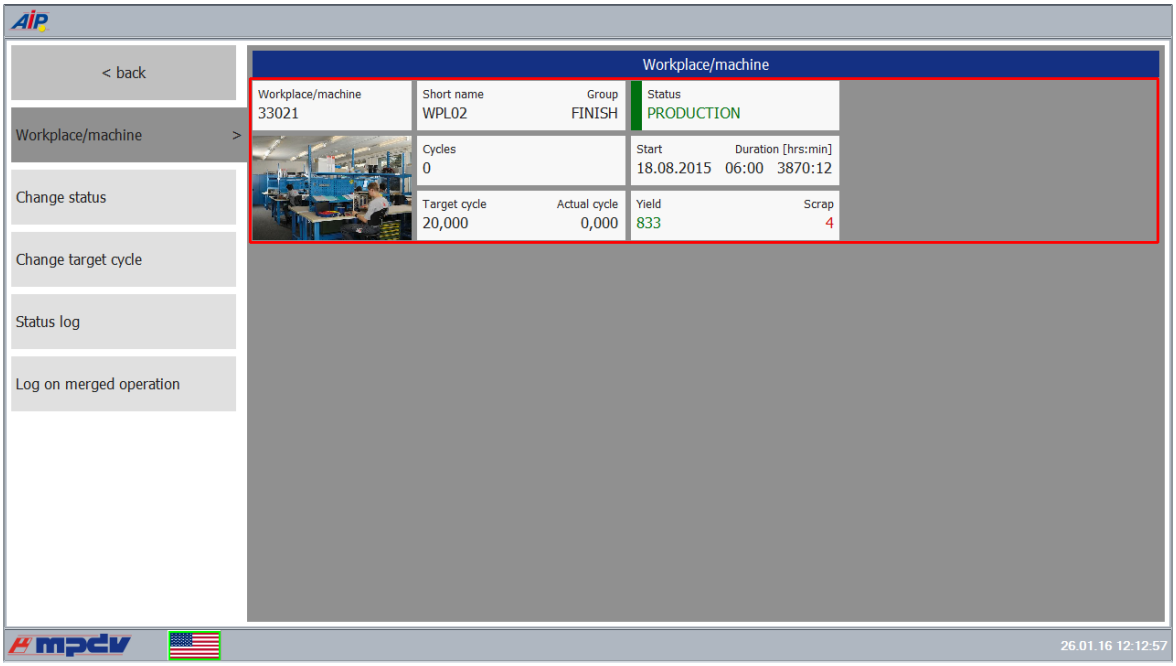
The file **a_list_anr.xml** stores the layout of an operation in the middle of the screen on the right hand side. The button to the left, which is used to log on an operation, as well as all other buttons outside a red frame are located directly in the **l_main.xml** layout.

Various lists can be displayed at the bottom on the right hand side. You can set in the workplace configuration which of the 3 lists are available at a workplace. The displayed data are defined in the following files:

- **a_list_pnr.xml**: Persons logged on
- **a_list_pnrg.xml**: Persons logged on at a group workplace
- **a_list_res.xml**: Resources logged on
- **a_list_emat.xml**: Logged on input material
- **a_list_amat.xml**: Produced output batches

l_mnr.xml

If you click on the area showing the data for the selected workplace, you will get to the workplace layout:

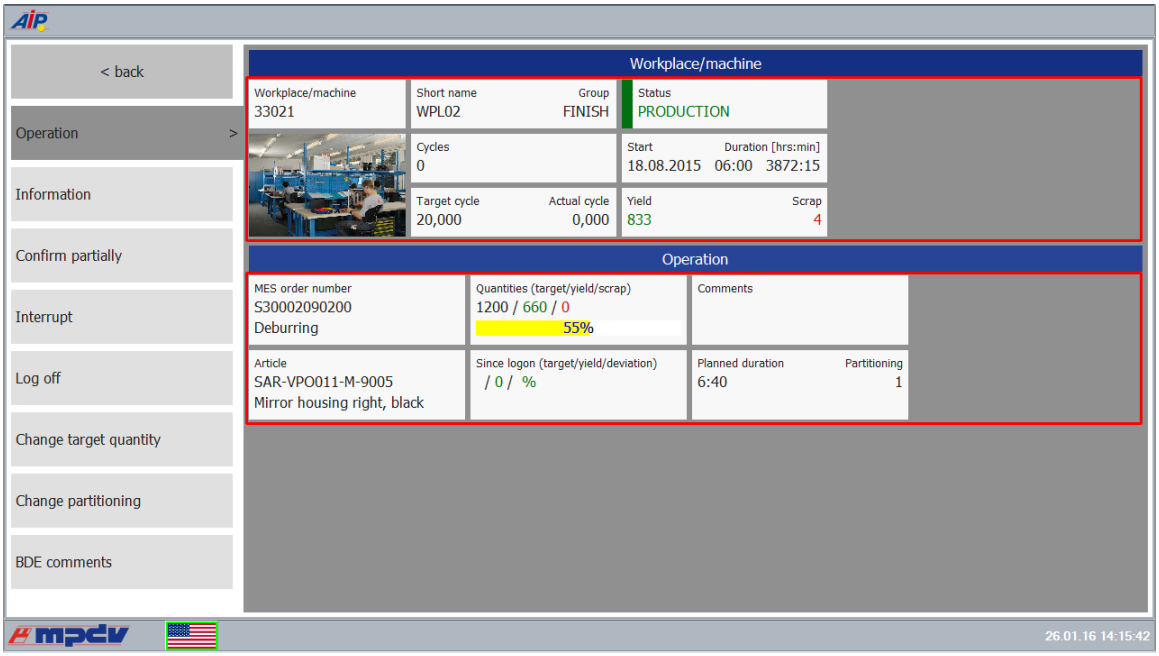


As described, the file **a_data_mnr.xml** defines the upper area on the right hand side to display data for the workplace.

The buttons on the left side are located in the layout **l_mnr.xml**.

l_anr.xml

If you click an operation in the main view, the operation layout appears:



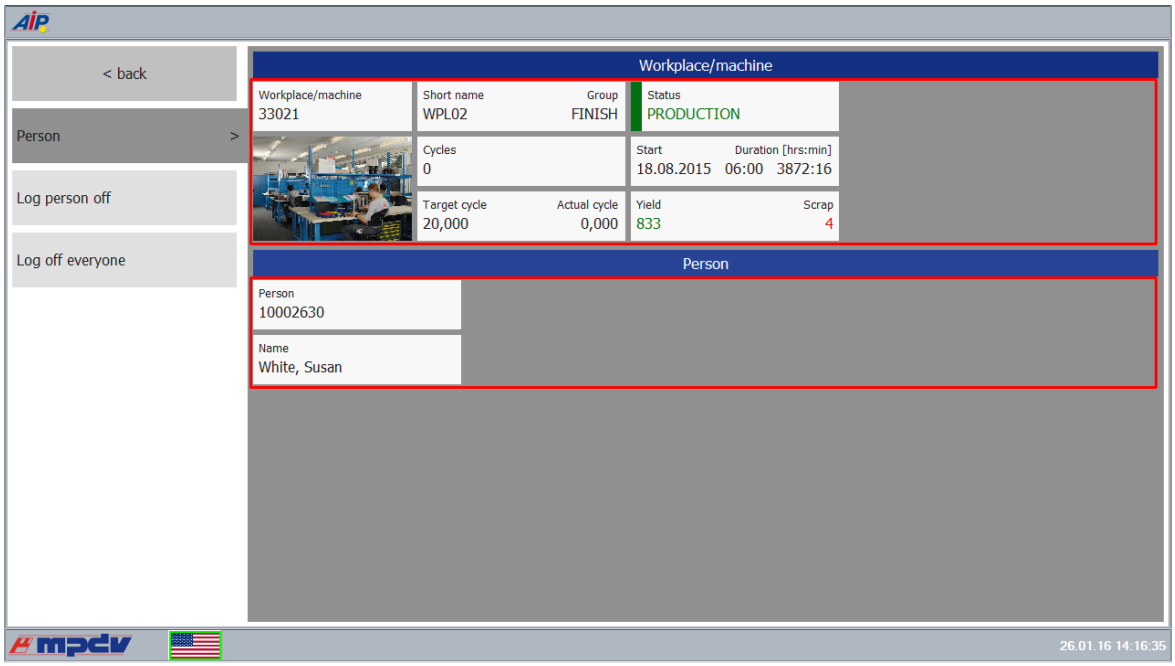
As described, the file **a_data_mnr.xml** defines the upper area on the right hand side to display data for the workplace.

The file **a_data_anr.xml** contains the definition for the area in the middle of the layout showing data for the operation.

The buttons on the left side are located in the layout **l_anr.xml**.

l_pnr.xml

The layout for staff appears if you select a person in the main view



As described, the file **a_data_mnr.xml** defines the upper area on the right hand side to display data for the workplace.

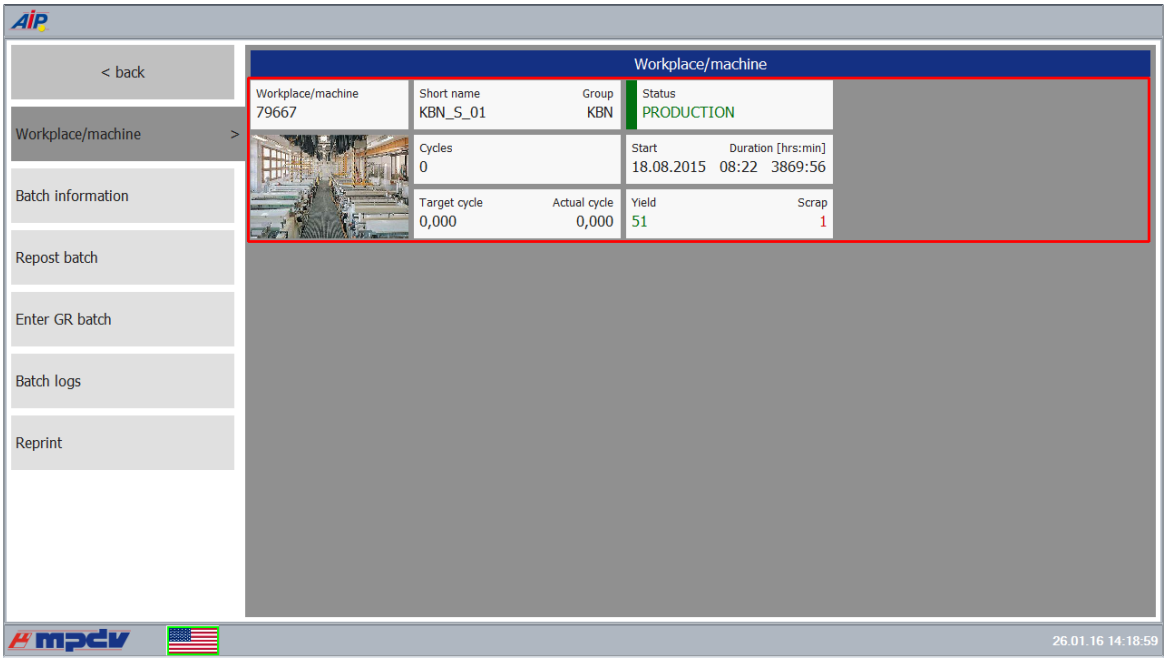
The file **a_data_pnr.xml** contains the definition for data relating to staff in the middle of the layout.

The button on the left side is located in the layout **l_pnr.xml**.

This layout is also used to display data and to request functions for staff logged on to a group workplace. When requesting the functions, an error message appears as staff and operations are logged on together to a group workplace.

l_res.xml

The resource layout opens if in the main view the third list "Resources logged on" is displayed and you click a resource:



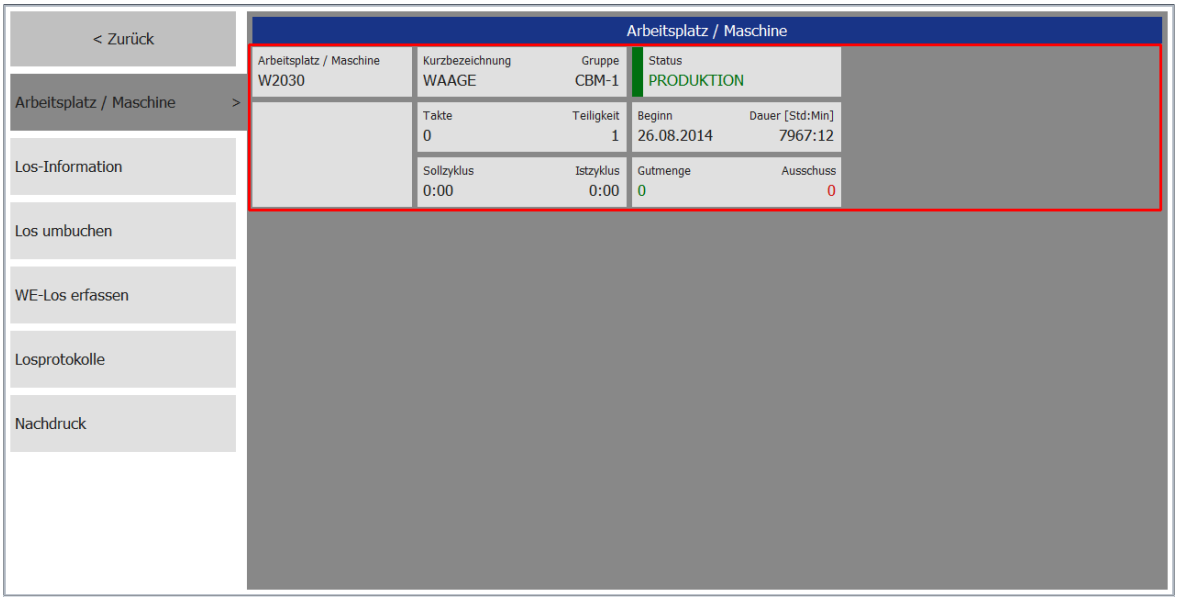
As described, the file **a_data_mnr.xml** defines the upper area on the right hand side to display data for the workplace.

The file **a_data_res.xml** contains the definition for data relating to a resource in the middle of the layout.

The buttons on the left side are located in the layout **l_res.xml**.

l_mat.xml

To request material layout, go to the main view. Click the button containing three dots to the left of the 3. lists "Input material logged on" and "Produced output batches":

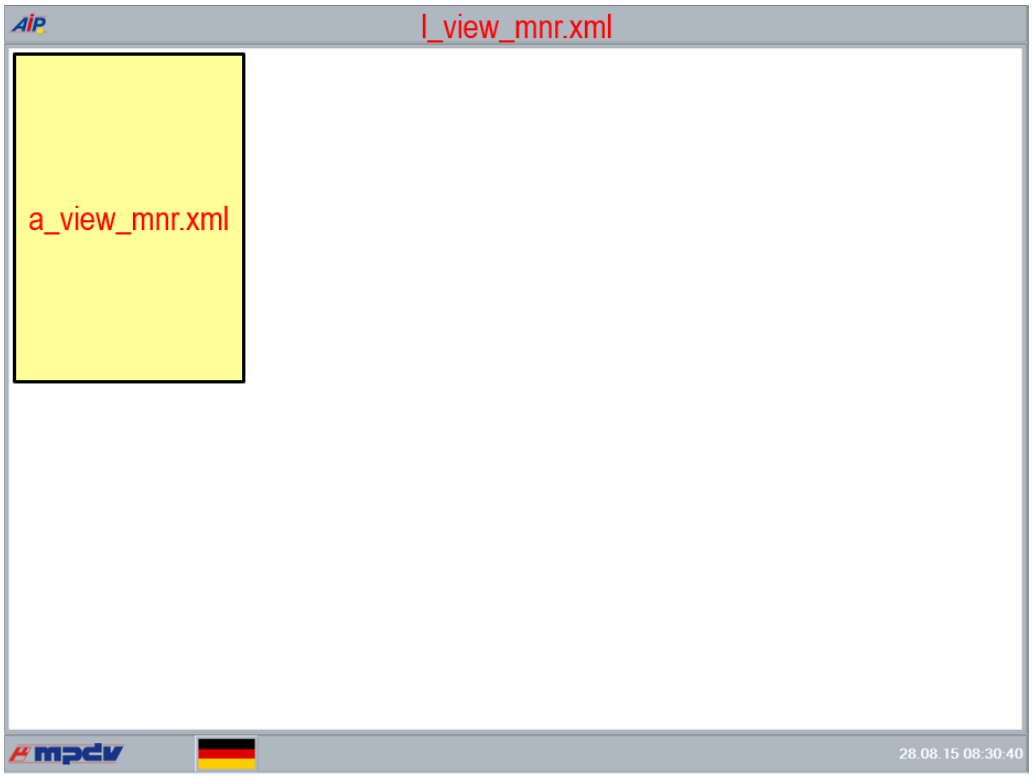


As described, the file **a_data_mnr.xml** defines the upper area on the right hand side to display data for the workplace.

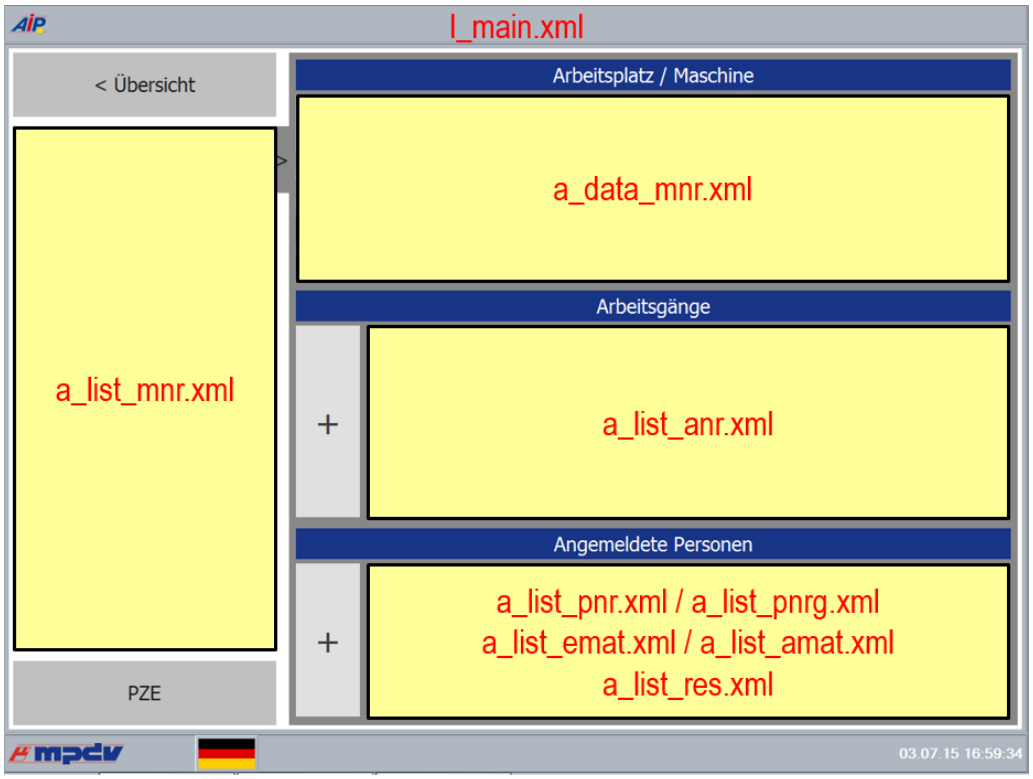
The buttons on the left side are located in the layout **l_mat.xml**.

3.4.1 Overview XML files

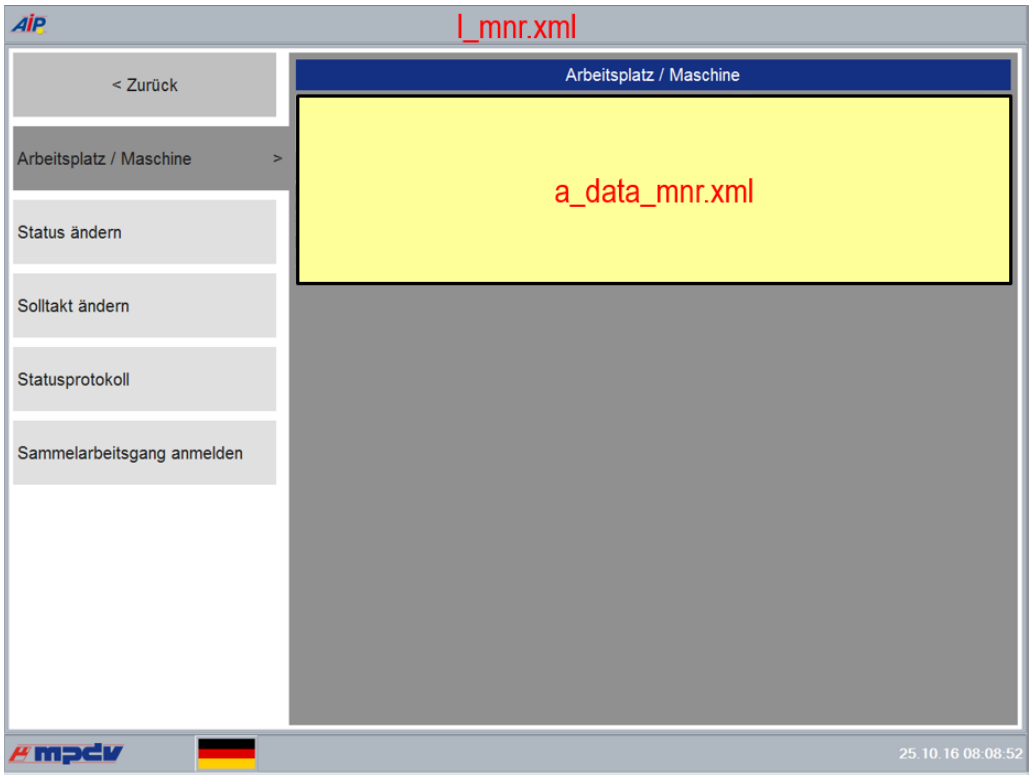
Icon **view:**



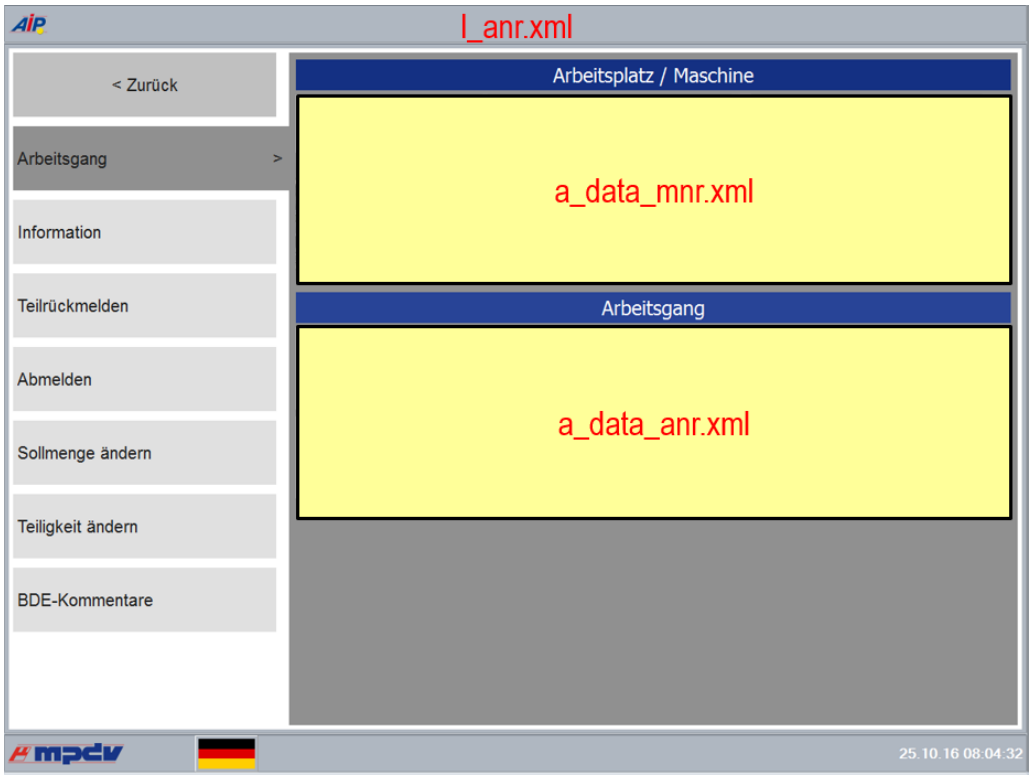
Main view:



Workplace layout:



Operation layout:



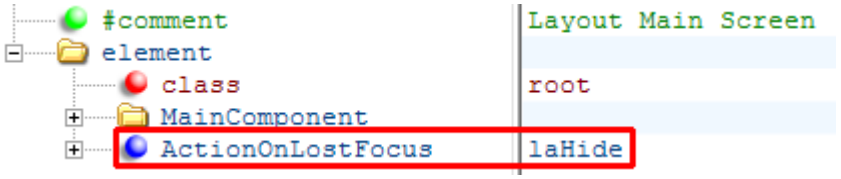
3.4.2 Layouts depending on the worplace type

You can override the layouts requested in the main view depending on the *batch management* and the *workplace type*. Both settings are stored in the dialog *Workplace and resource configuration* in the tab *Workplace configuration* and consist of one letter. If you copy a layout and both letters are written as lower case letters, are separated by an underscore ("_") and added to the file name, then this layout is used for all workplaces including batch management and used with the relevant workplace type.

For example, buttons should be made available with other dynamic dialogs for order postings at a packing station (letter "C") without batch management (letter "N"). To do so, copy the layout **l_anr.xml** onto the file name **l_anr_nc.xml**. You can then modify the buttons for the packing station in this layout.

3.4.3 Taking over changes in the layout configuration

Using the attribute „ActionOnLostFocus“, you can make the following settings per layout:



laFree

Using this setting, the displayed layout is discarded on changing to another layout and loaded anew from the XML files on the next start of this layout. Changes in the configuration of this layout, which were made in the meantime, are taken over.

laHide

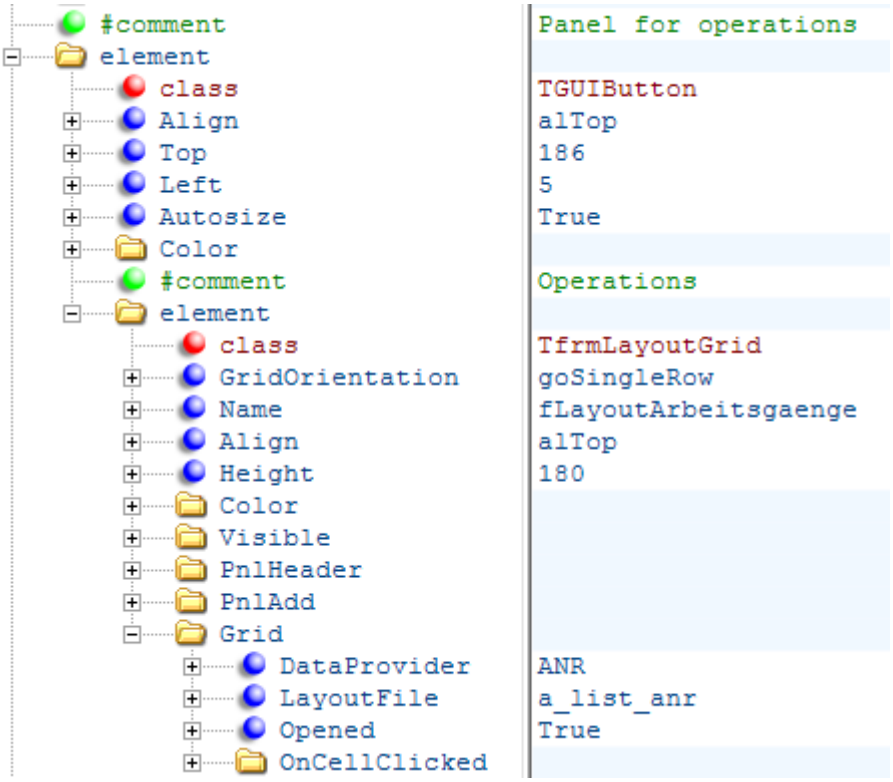
With this setting, the layout is not discarded, but stays in the background when you change to another layout. Changes of the layout configuration, that were saved after the first layout display, do not have an effect as the layout is not loaded anew from the XML files.

The setting *laHide* is applied by default in the layout of the main view (**l_main.xml**) to keep the scroll position in the lists on the right hand side when you return to this layout from another layout.

When you change the language during runtime using the flag in the status bar, the currently displayed layout is loaded. Changes of the layout in the main view are then taken over.

3.5 Configuration of lists

The configuration of lists is explained using the list of the logged on operations in the layout **l_main.xml:**



Lists of operations have their own panel in order to keep their position if operations of aggregates (a line is separated) are hidden.

The class *TfrmLayoutGrid* is responsible for the list display.

The settings below *PnlHeader* specify if and how a heading is displayed above a list.

The settings below *PnlAdd* define the button to create a new entry. In the above example, it's the button containing a "+"-symbol .

In the *Grid* area the data source (*DataProvider*) is set for the list. *LayoutFile* specifies which file defines the display of an element in the list. Below *OnCellClicked* you control what happens if you click an element in the list.

3.5.1 Filtering the displayed elements in the user interface (as of version 8.2.1.1)

The displayed lists can be filtered in the user interface. A text field must be included in the header of the list of type *TFrmLayoutGrid.*

The search syntax is equal to the search syntax of the lists in the dynamic dialogs.

A search field is integrate to a list of the type *TFrmLayoutGrid* eingebunden:

```
<element class="TfrmLayoutGrid">
        <SearchPanel>
                <Settings>
                        <EnableSearch>true</EnableSearch>
                        <SearchType></SearchType>
                        <ExecuteEvent>return</ExecuteEvent>
                        <SearchFields>ANR|AGNR</SearchFields>
                        <SearchControlWidth>50</SearchControlWidth>
                </Settings>
        </SearchPanel>
        <SearchPanelPosition>header</SearchPanelPosition>
…
</element>
```

Details on the properties

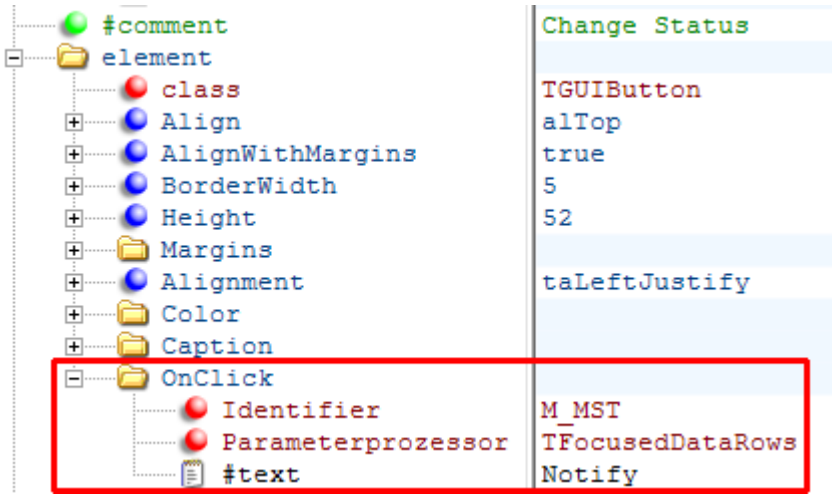
Properties	**Description**
Settings.EnableSearch	Display search panel
Settings.SearchType	Currently, only text is possible. Describes the visual search component.
Settings.ExecuteEvent	Event that should trigger the search. The following configurations are possible: 1. **KeyDown** The search is performed immediately on pressing the key. Compared to the following configurations, the above configuration costs a lot of time and shoud only be used for small data quantities. 2. **Button** The search is performed by clicking an additionally displayed button. 3. **Return** (Extension of "Button") The search can also be performed by pressing the return key.
Settings.SearchFields	Fields in the data source where you want to find the text you entered.

Settings.SearchControlWidth	Width of the input text box
TfrmLayoutGrid.SearchPanelPosition	Search panel position **row**: A row above the cells **header**: In the header of the TFrmLayoutGrids

If the search line is displayed, the list of type *TFrmLayoutGrid* requires more space in height. This space is at the expense of the tiles showing the data in the view. It is therefore recommended that you also change the height of the tiles with your data when using this functionality.

3.6 Request dynamic dialogs

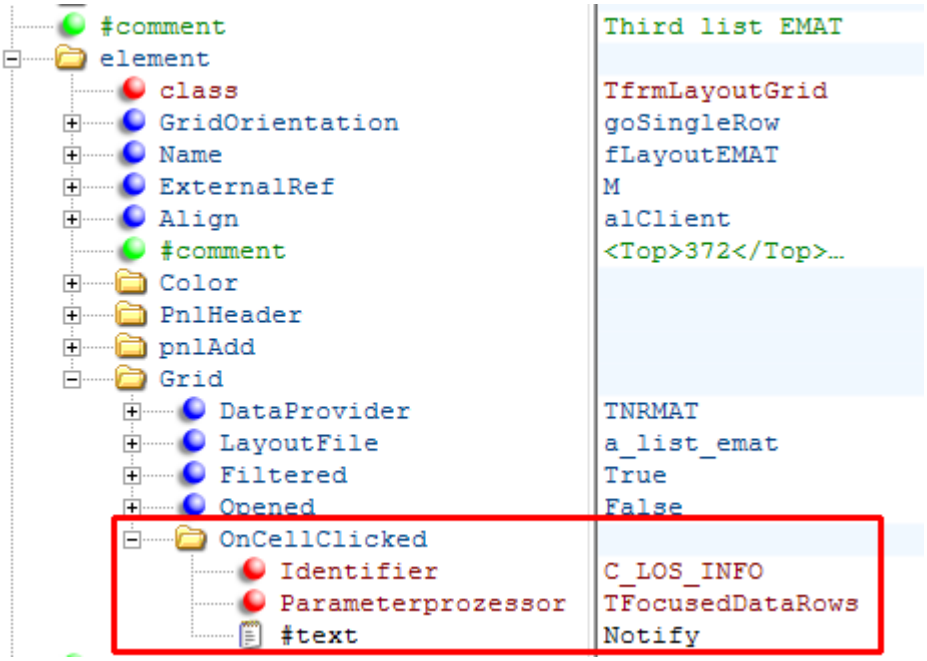
You can store an action in the individual fields and elements in order to perhaps request a dynamic dialog. In case of a button which is included on the left hand side in many layouts, you can enable this by using the entry *OnClick*:



This example shows the button "Change status" in the layout **l_mnr.xml.**

The entry *Identifier* defines the dynamic dialog to be requested. Both other entries are constant.

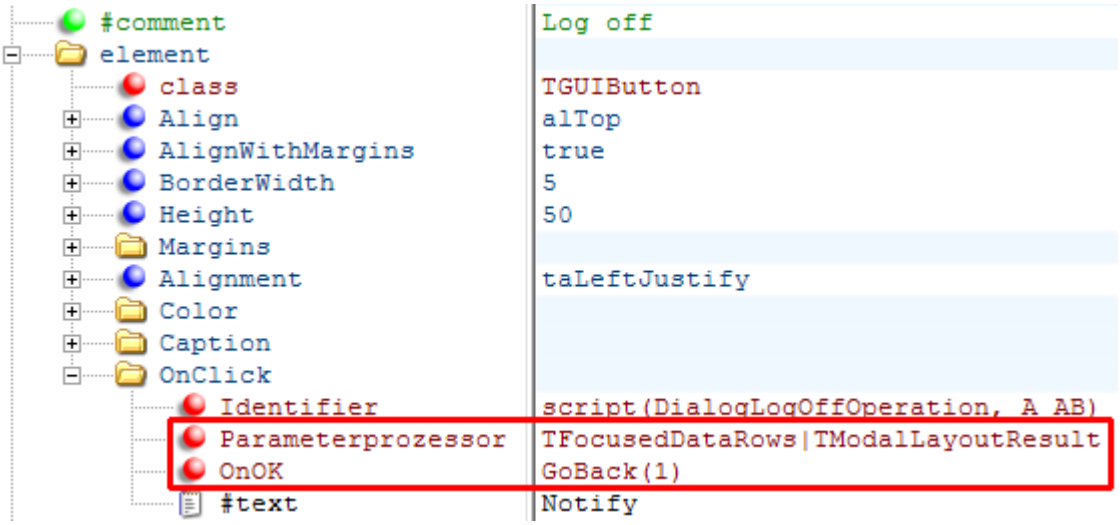
In a list of the class *TfrmLayoutGrid* containing several objects, the entry is called *OnCellClicked* and affects the elements below:



This example shows the request "MES Batch information" when selecting the input material in the layout **l_main.xml.**

3.6.1 Return after a dynamic dialog

After the execution of a dynamic dialog, you return to the layout where the dialog has been requested from. Alternatively, you can leave this layout and return to the previous layout which is normally the main view. Once an operation is logged off, it makes more sense to return to the main view than still displaying the data of a logged off operation.



A script request is stored in the setting *Identifier*. Depending on the workplace settings, the script request controls which dynamic dialog is requested. The first parameter specifies the script to be run and the second parameter the default value which is used if the script does not exist.

You then return to the previous view no matter if the dialog was completed or not, if errors occurred or if the dialog was interrupted without a change.

3.7 Positioning

The individual elements in the layout are arranged in a tree structure. The positioning of a subordinate item is always done in relation to the superior one (folder).

If a field shows a description and a corresponding value, the position of the description and the value refers to the top left corner of the field.

There are two ways to specify the position of the information. They are described in the following chapters.

3.7.1 Fixed positioning

Here, the position and the size of an element is specified with the following properties:

Top

Distance from the top

Left

Distance from the left

Height

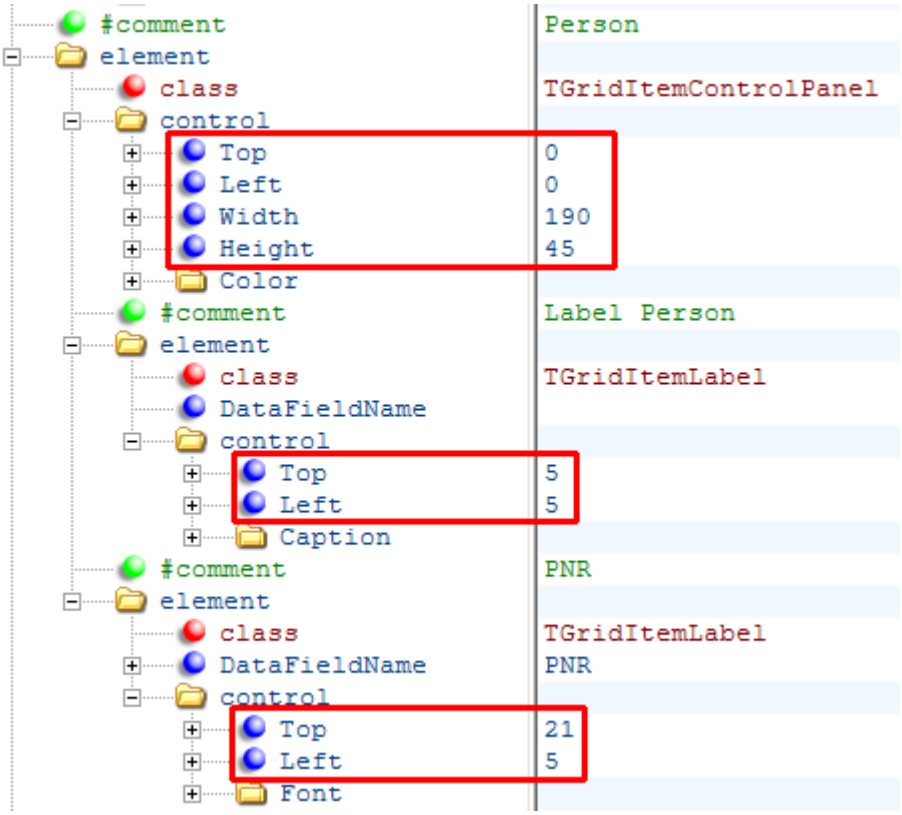
Height of the element

Width

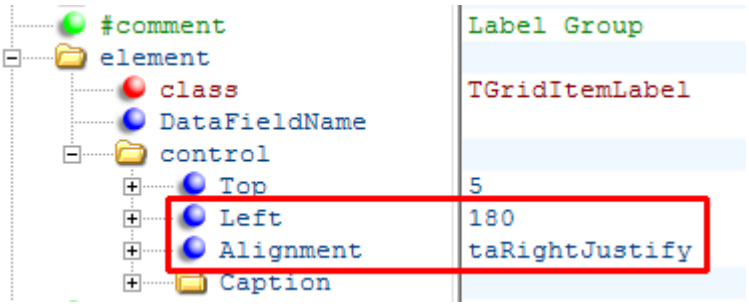
Width of the element

Properties can be found below the entry *control*.

The following example shows a logged on person for **a_data_pnr.xml**:



The property *Alignment* also specifies if the element is positioned towards the left (*taLeftJustify*), or the right (*taRightJustify*) or towards the center (*taCenter*). If no other property is explicitly set, the standard setting is left-aligned. The following example shows a position towards the right of the label *Group* of workplace data (**a_data_mnr.xml**):



3.7.2 Dynamic positioning:

If the positioning is done dynamically then the elements adapt their position and size to the one's above or next to it. The property *Align* can set the following:

Align=alTop / Align=alBottom

This element takes over the upper or lower limit and the width of the superior element. The property *Height* specifies the height.

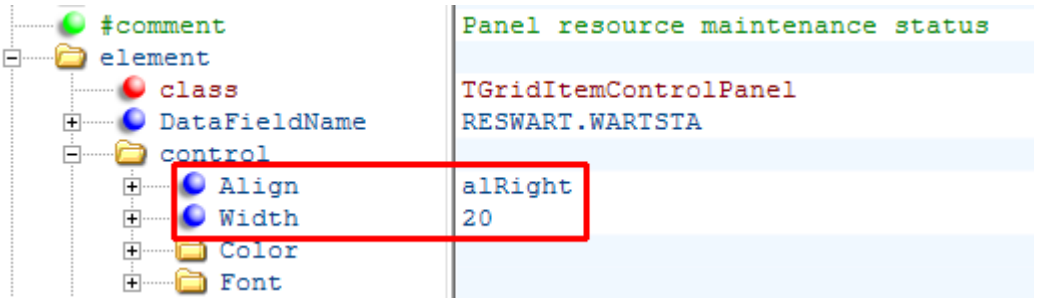
Align=alLeft / Align=alRight

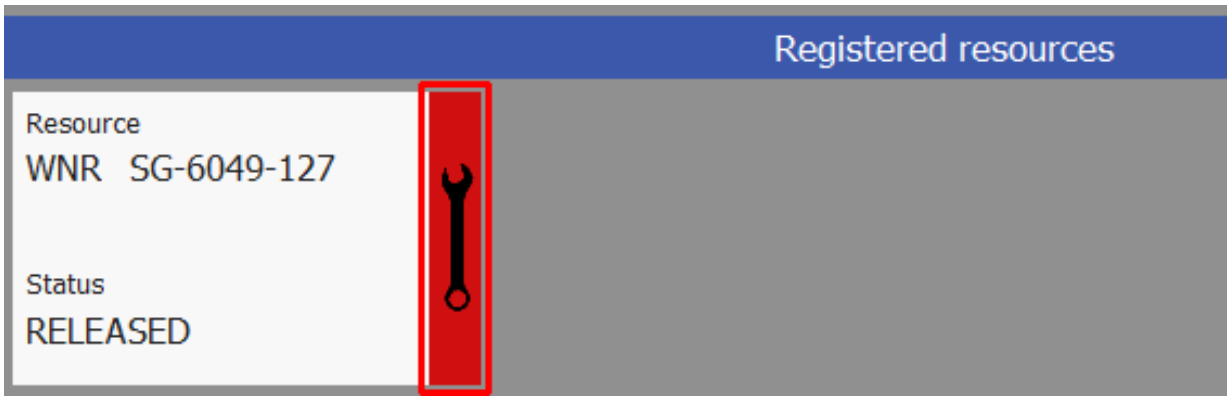
This element takes over the right or left limit and the height of the superior element. The property *Width* specifies the width.

Align=alClient

Aligns itself to the complete space of the superior element.

The following example defines the area for the color display of the maintenance status in the list of resources (**a_list_res.xml**):

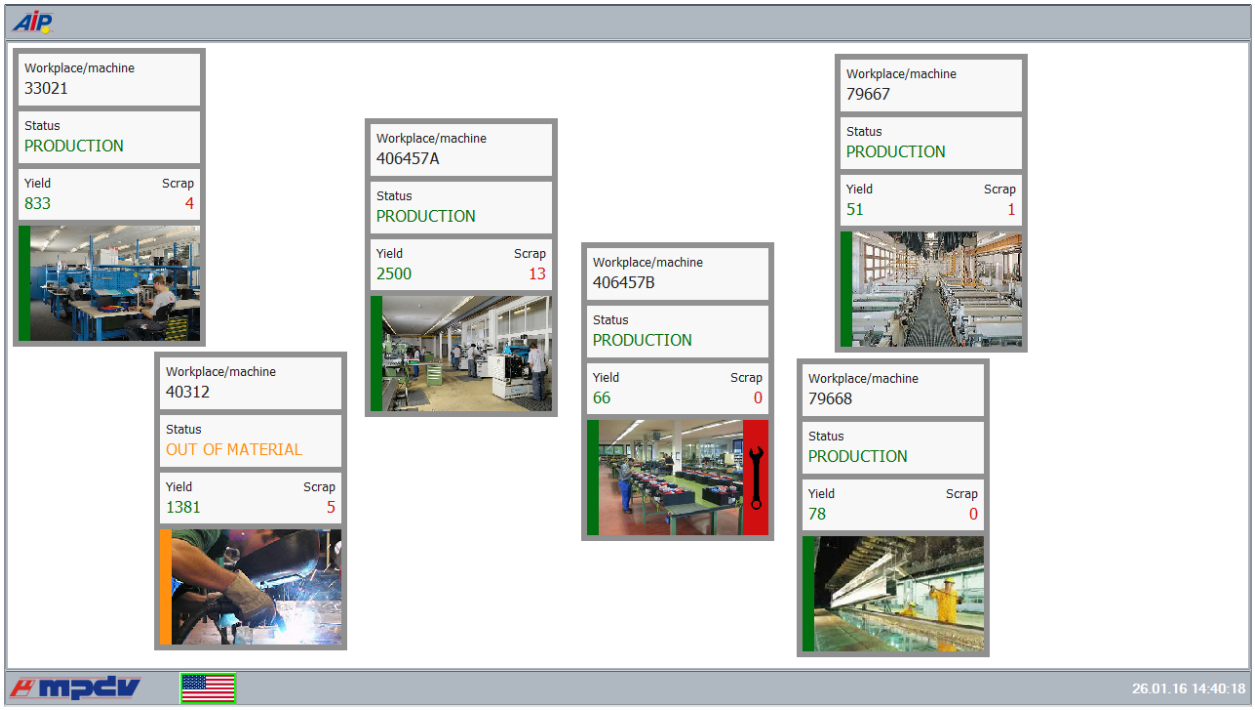




If neighboring elements have the same entry in the property *Align,* then they are displayed below or next to each other. This functionality is used in the button bars to request individual functions and ensures that there are no gaps if a function is hidden:

3.7.3 Positioning of workplaces in the icon view

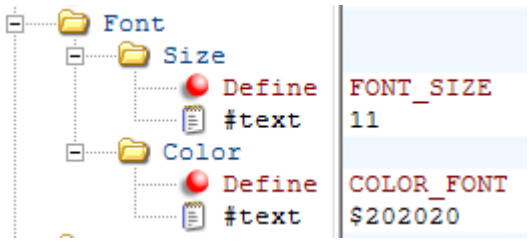
Positioning of individual workplaces in the icon view can be changed during runtime. You can start the design mode (password protected "mos6050") by double click the AIP icon in the top left corner. You can then position the workplaces by Drag&Drop. Double click the AIP icon to finish the design mode. The positioning of the workplaces is stored in the file *gui\p_view_mnr.xml.*



To reset the positioning, please delete the file *gui\p_view_mnr.xml.*

3.8 Text formatting

Text formatting in the GUI is performed using the entries below the field Font:



You can make the following settings:

Size

Set the font size

Color

Set the font color in reversed RGB notation

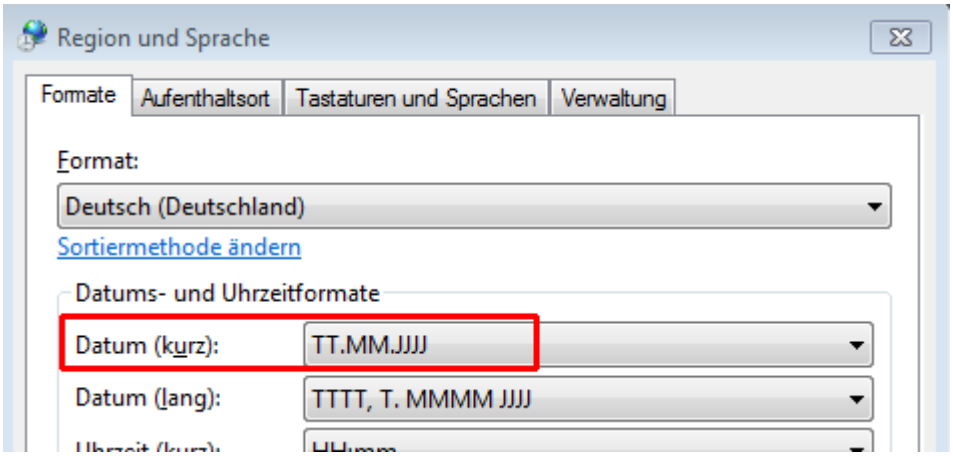
If the attribute *Define* is applied, the value entered in the field *#text* is not used. Instead the content of the entered constant is used (with both settings).

3.9 Formatting functions

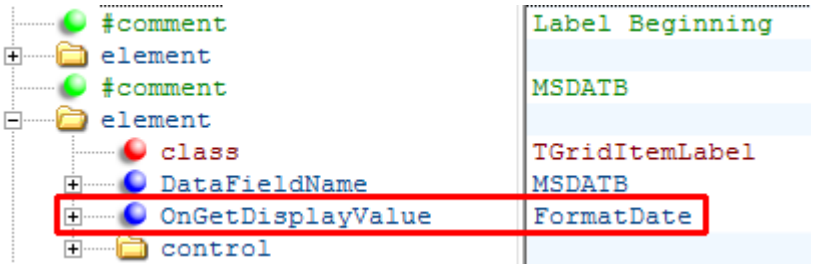
You can display data with the aid of different formatting functions:

FormatDate

This function sets a date depending on the date format *date (short)* set in the operating system:

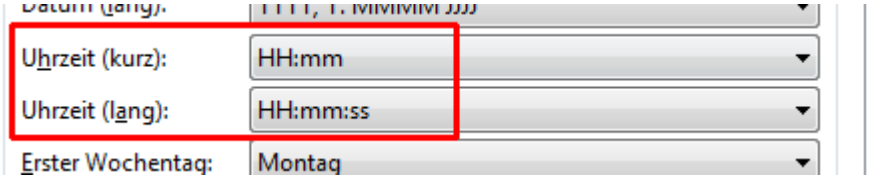


There is an example located in the workplace data (**a_data_mnr.xml**) for the start date of the current status



FormatTime / FormatTimeLong

The function *FormatTime* sets the time depending on the time format *Time (short*) set in the operating system:



FormatTimeLong uses the format *Time (long)*.

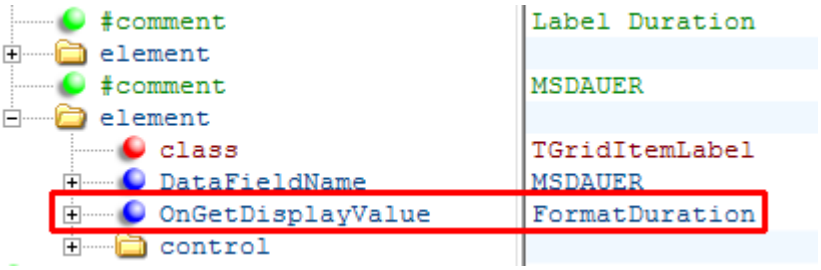
FormatDuration / …

There are various functions to display *durations* in different output formats.

Function	Format
FormatDuration	Hours:Minutes
FormatDurationMMSS	Minutes:Seconds

FormatDurationHHMMSS	Hours:Minutes:Seconds
FormatDurationHHII	Hours,decimal hours (Industrial minutes)
FormatDurationHHIII	Hours, decimal hours (3 decimal places)
FormatDurationMMII	Minutes,decimal minutes

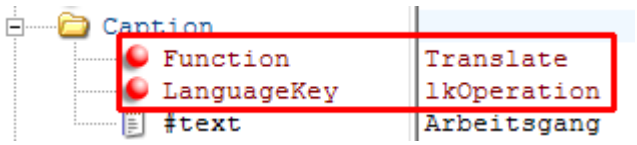
You can find an example for the output of a duration in workplace data(**a_data_mnr.xml**). It states the duration of the current status :



3.10 Multilingualism

AIP2 uses just like the AIP the Multilizer to translate text into another language. Language keys with the prefix "lk" are used for the new GUI. German texts without the prefix "lk" are also processed if they are included in the mld file.

Text for translation can be added using the function "Translate" and the entry "LanguageKey" (in accordance with the language set).



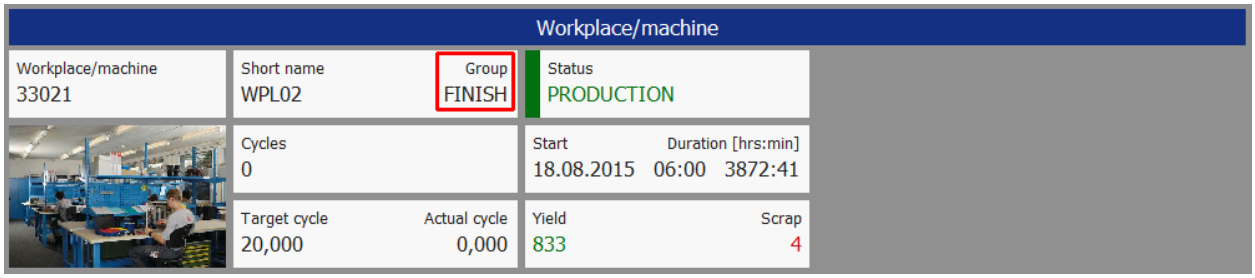
This example contains a German text "Arbeitsgang" (operation) which does not affect the processing. The text is replaced by the translated text using the language key during runtime.

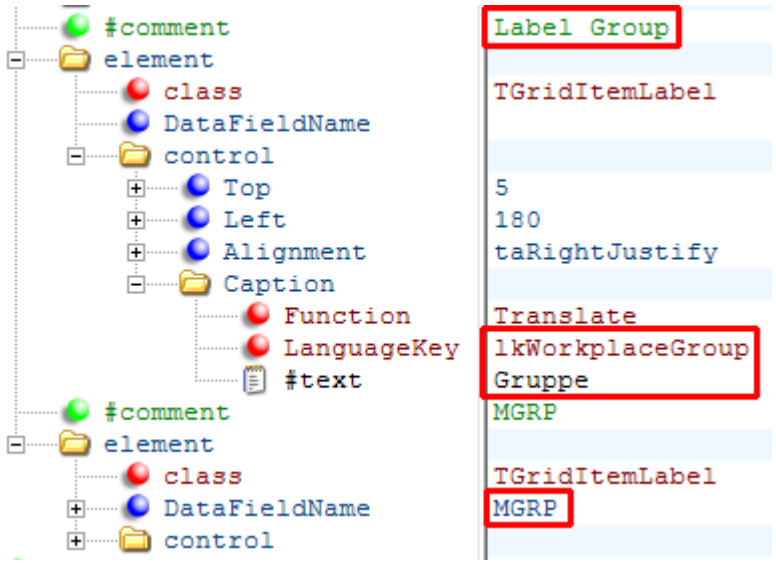
3.11 Examples / exercises

This chapter shows customization options of the layout using examples.

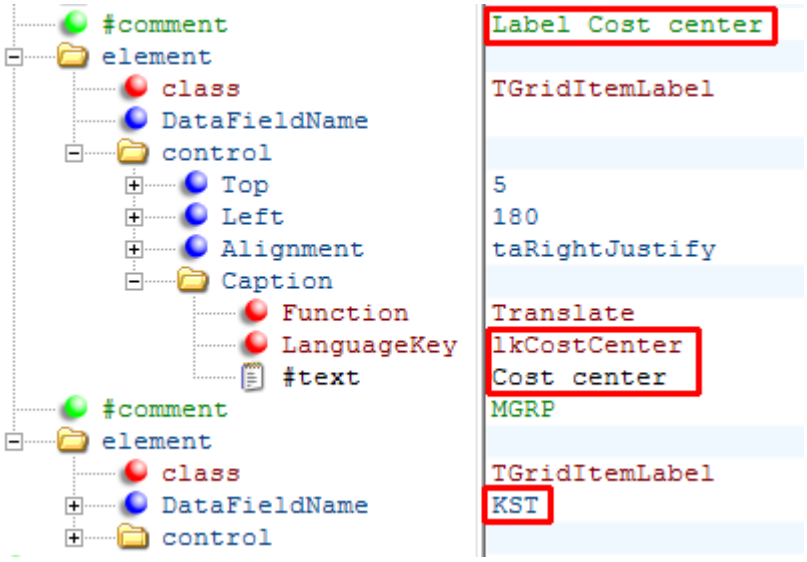
3.11.1 Change existing fields

Replace the field "group" with "cost center" in the displayed workplace data.





You need to change the entries for "label group" and "MGRP" (machine group) in the parameter **a_data_mnr.xml** as follows:

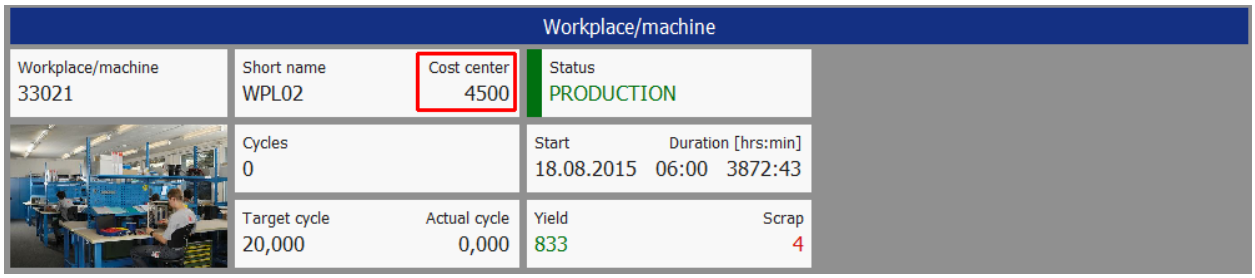


Entries with the description "#comment" are comments which do not affect processing.

The first element in the red frame shows the description above the data. The language key lkWorkplaceGroup is replaced by *IKCostCenter*. You can directly insert the text in the field "#text" if there is no language key available for the description. Both entries "Function" and "LanguageKey" must be deleted in this case.

The description is displayed towards the right hand side at position 180 ("Left": 180; "Alignment": taRightJustify).

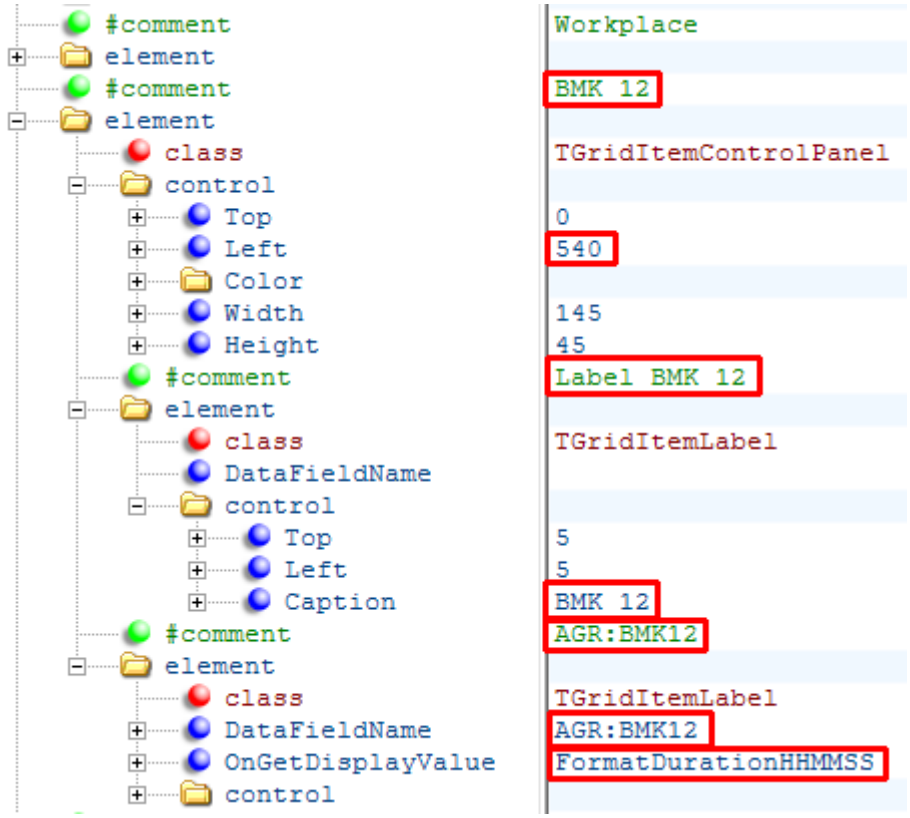
The second element is responsible for the display of the data field. Change the entry "DataFieldName" from *MGRP to KST.*



3.11.2 Add a new field

Display the duration booked on RPA 12 to the right of the status display.

This is done by copying the element with the comment *Workplace*. This element specifies the light gray space and includes 2 other elements including name and data.



The comment located above is also copied and changed on BMK12.

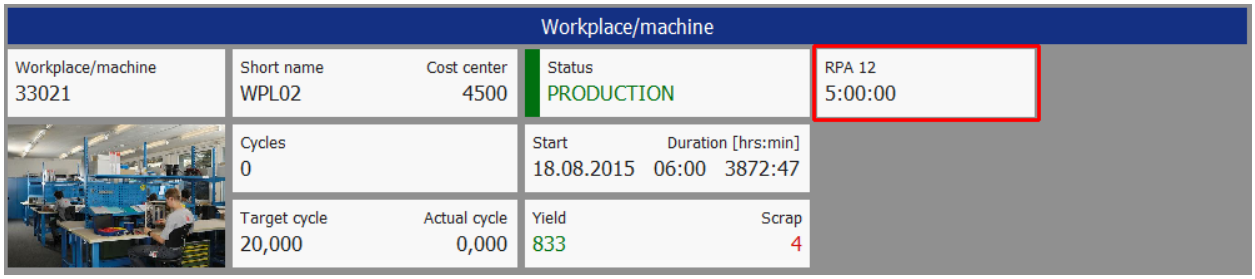
The position of the light gray area ("left") is made up of position ("left") and the width of the element *Workplace Status* plus a distance of 5 dots (345 + 190 + 5= 540). Both fields are located below the entry "Control".

The elements *Label BMK12* and *AGR:BMK12* are located on the light gray area. The position of both elements ("Top" and "Left") refer to the top left corner of the light gray space.

The entry "Caption" specifies the displayed text. Here, language keys have not been used so the text is not translated.

The entry DataFieldName below the comment *AGR:BMK12* was changed to the field name *AGR:BMK12*.

The formatting function FormatDurationHHMMSS shows the duration in hours:minutes:seconds.



3.11.3 Add user fields

Add a user field to the interface by adding a new field as described in the previous chapter. Use the the acronym of the user field (e.g. ANR_FU_65).

If you want to format user fields for dates, times, or durations, you can use the formatting functions. See section "3.9 Formatting functions".

Load user field with cataiplay.ini

Note that the AIP lists that serve as data providers do not contain user fields in the standard system. In order for the user fields to be added to the list, you must configure it in the **ctaiplay.ini** file. As long as the user fields in the **ctaiplay.ini** file are not configured correctly, they remain empty on the user interface.

In the customer-specific terminal directory (e.g. if user fields are to be added at terminal group level: \mip\<systemnr>\custom\aip2\tgrp_xxx\) a **ctaiplay.ini** is created, which contains the different section.

- Activate the additional loading of the user fields for operation- or order-related XML files in the section [Custom Userfields ANR].
- Activate the additional loading of user fields for machine-related XML files in the [Custom Userfields MNR] section.

You can find examples on how to do it further along.

The activated fields are then available in all XML files connected to the DataProvider ANR or MNR.

Available user field in machine and order lists

All identifiers of user fields and also other fields that can be reloaded are located in the headers.dat file in the "spool" directory of the terminal. It consists of four lines:

Start of the row	Content
10|…	Machine list: Fields that are always included in the list.
*10|…	Machine list: Fields that can be reloaded.
11|…	Order list: Fields that are always included in the list.
*11|…	Order list: Fields that can be reloaded.

The following user fields can be reloaded:

Machine list

FU:1 to FU:66

Machine user fields

Order list

ANR_FU_1 to ANR_FU_66

Operation user fields

AUNR_FU_1 to AUNR_FU_66

Order user fields

MNR_FU_1 up to MNR_FU_66

Machine user fields

VERARBCODE_FU_1 up to VERARBCODE_FU_66

Processing code user fields

AGR_FU_1 to AGR_FU_66

User fields of the operation status (cannot be used in the standard system, reserved for Customizing!)

Example 1: User fields in the operation list

- User field 1 of the operation should be entered in the order list with the name " Order date ".
- User field 66 of the machine with the name "My long user field" should be added to the order list.

Field definition in section [Custom user fields ANR]

```
[ Custom usernfields ANR ]
GRID_LIST_TYP=ANR
; additional fields of the order list
ANR_FU_1= ; User field 1 of operation, MyDate FU:1 [operations list]
MNR_FU_66= ; User field 66 of machine, My long user field [operations list]
```

Example 2: User field in the machine list

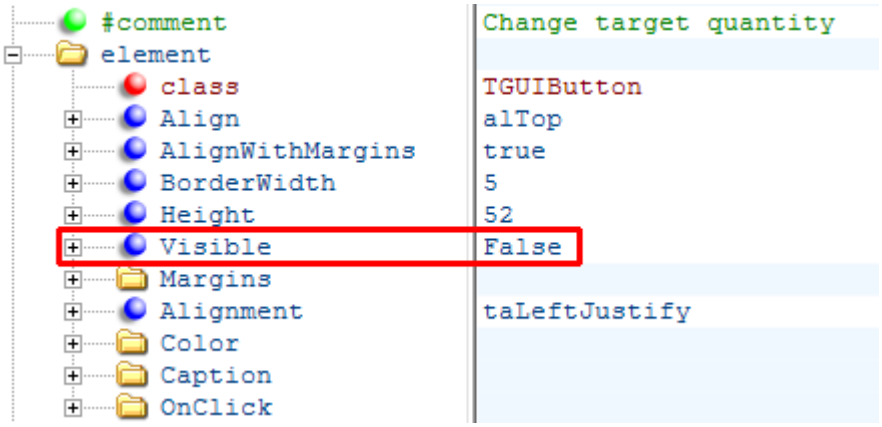
User field 66 of the machine with the name "My long user field" should be added to the machine list.

Field definition in the section [Custom user fields MNR]

```
[ Custom userbfields MNR ]
GRID_LIST_TYP=MNR
; Additional fields in the machine list
FU:66= ; User field 66 of machine, My long user field [machine list]
```

3.11.4 Remove button

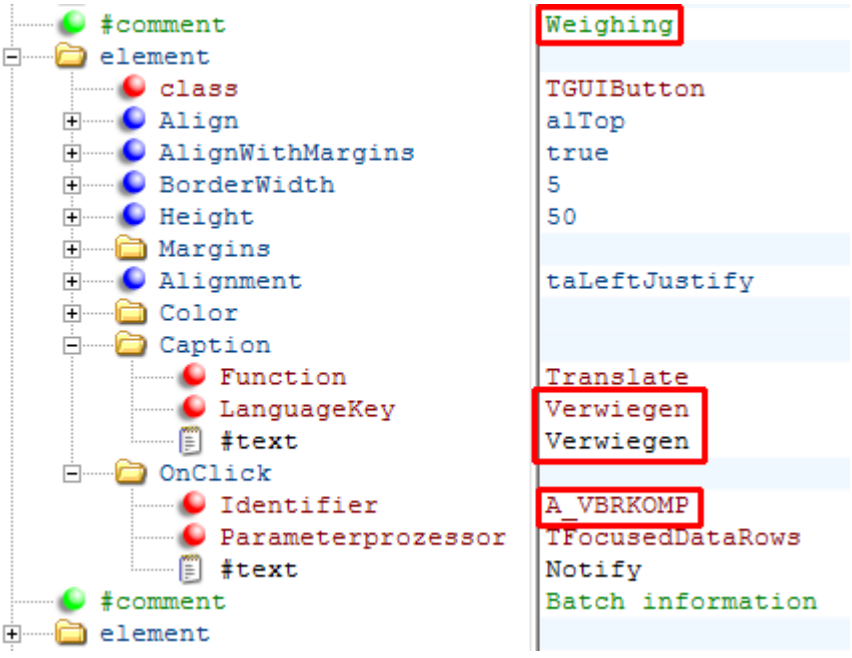
Delete the button *Change target quantity* from the operation layout **(l_anr.xml).**



The new entry Visible=False hides the button. Optionally, you can also delete the comment and the element.

3.11.5 Add button

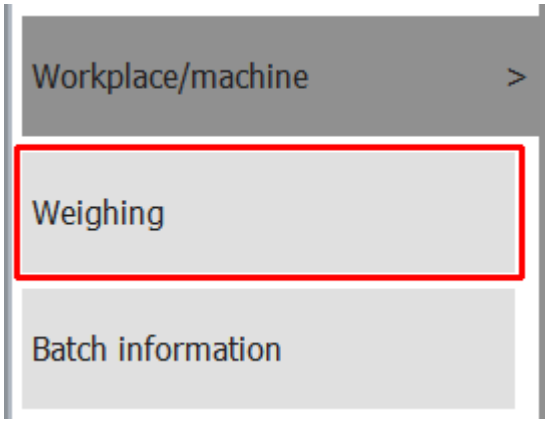
You want to add a new button "Weighing" in the layout for "input material", "output batch" and **(l_mat.xml).**



First of all, copy an existing button including the corresponding comment. In this case the button "Batch information" was copied. Change the comment in order to easily find the new button in the list of elements.

The entry "Caption" specifies the displayed text. In the example, the English text "Weigh" is used as language key.

The "Identifier" specifies which dynamic dialog is requested.



3.11.6 Integration of a picture

The task is to have a logo displayed in the main view (**l_main.xml**) below the button "PZE".

```
#comment        PZE-Button
element
#comment        Company logo
element
  class         TGUIButton
  Align         alTop
  AlignWithMargins  true
  Height        50
  Margins
  Alignment     taCenter
  Color
  #comment      Picture
  element
    class       TsImage
    control
      Top       15
      Left      44
      Width     180
      Transparent  True
      Stretch   False
      Proportional  True
      Picture
        Function    LoadPictureFromFile
        Identifier  mpdv.png
```

Copy the button "PZE" and the comment. Change the comment.

As the button has no labeling, delete the entry "Caption". Also delete the entries "Visible" and "OnClick".

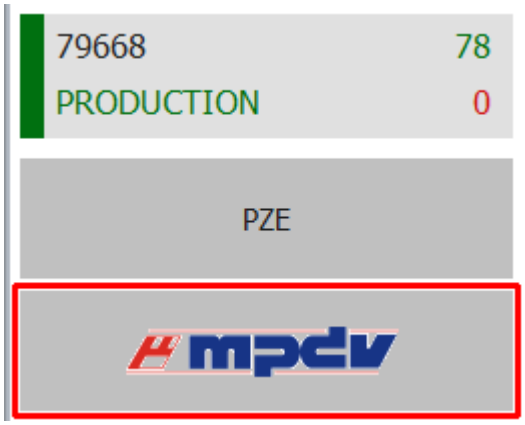
You need a new element of the class "TsImage“ in order to display the new picture. This element was copied from the staff list (a_list_pnr.xml) and has the class "TGridItemImage", as it is not located on a button but in a list. Change the class to "TsImage" after copying. Delete the entry "Visible".

The file name of the picture is entered in the field "Identifier" of the entry "Picture“. There are two options to load the picture:

- **LoadPictureFromFile** reads the file from the spool directory.
- **LoadPictureFromAIP** uses pictures included in the AIP2 in the file "pict.zip" or "pict_cust.zip". This information is more efficient as these picture are stored in a buffer. This method only supports images of type PNG and BMP.

Different settings are available to display the picture.

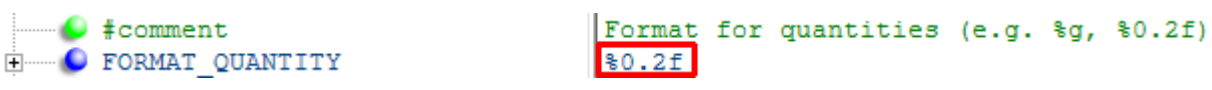
- **Transparent** – For example, PNG files support transparent areas where the background of the picture is visible. Functionality can be switched off using the value *False.*
- **Stretch** specifies if the picture is shown in its original size (value *False*) or if *Height* and *Width* are adjusted (value *True*).
- **Proportional** controls whether the ratio of the width of the image and the height of the image is maintained (value *True*) or not (value *False*) when the image size is adjusted to the specified *Height* and *Width.*



3.11.7 Change quantity format

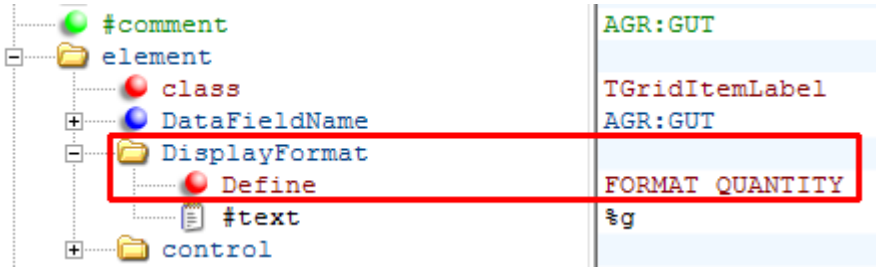
Generally show the quantity format with 2 decimal places.

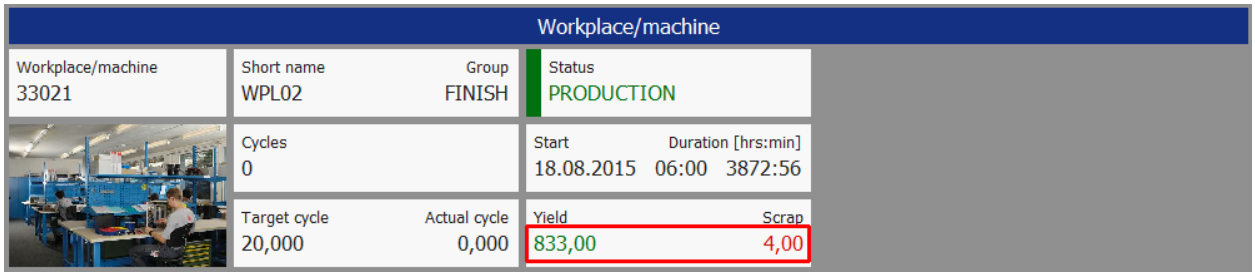
The quantity format is configured in the file **globaldefines.xml** using the constant *FORMAT_QUANTITY:*



The value "%0.2f" ensures that quantities are displayed with 2 decimal places.

You can find an example for this constant (Define) when yield is displayed for data of the workplace (**a_data_mnr.xml**):



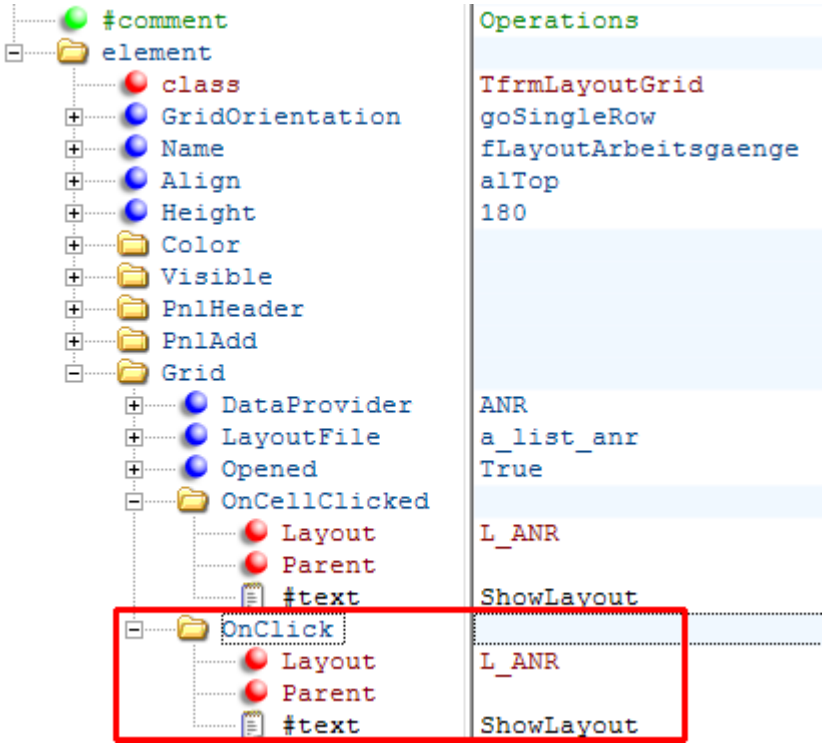


The set format only affects configured layouts and not dynamic dialogs.

3.11.8 Postings for operations not logged on

The workplace configuration in the MOC has a button called "Posting of operations not logged on". If this button is activated, you can interrupt or log off the operations not logged on (posting to the server). You have to extend the configuration if you would like to carry out these postings in the AIP2.

The operation layout only opens in the AIP by using the buttons *Interrupt* and *Logg off* if you select the logged operation. If the following extension in the file l_main.xml is carried out, this layout also opens if you click the empty space in the list of operations.



The dynamic dialogs must also be customized in order to interrupt and logg off operations. Unless so-called simple dialogs are used.

3.12 Index

4 AIP2 - Customizing of the GUI

4.1 Overview

The layout for the new GUI of the AIP2 terminal (tile design) is stored in XML files. XML files can be edited using a standard text editor. Microsoft's *XML Notepad 2007* can also be used. This XML editor provides a clearer presentation, a user-friendly copy function for entire objects and the possibility to move complete objects. *XML Notepad 2007* was used for the generation of screenshots included in this document.

This document describes the customizing options of the new GUI and is based on the documentation dealing with the Configuration of the GUI.

4.2 Settings

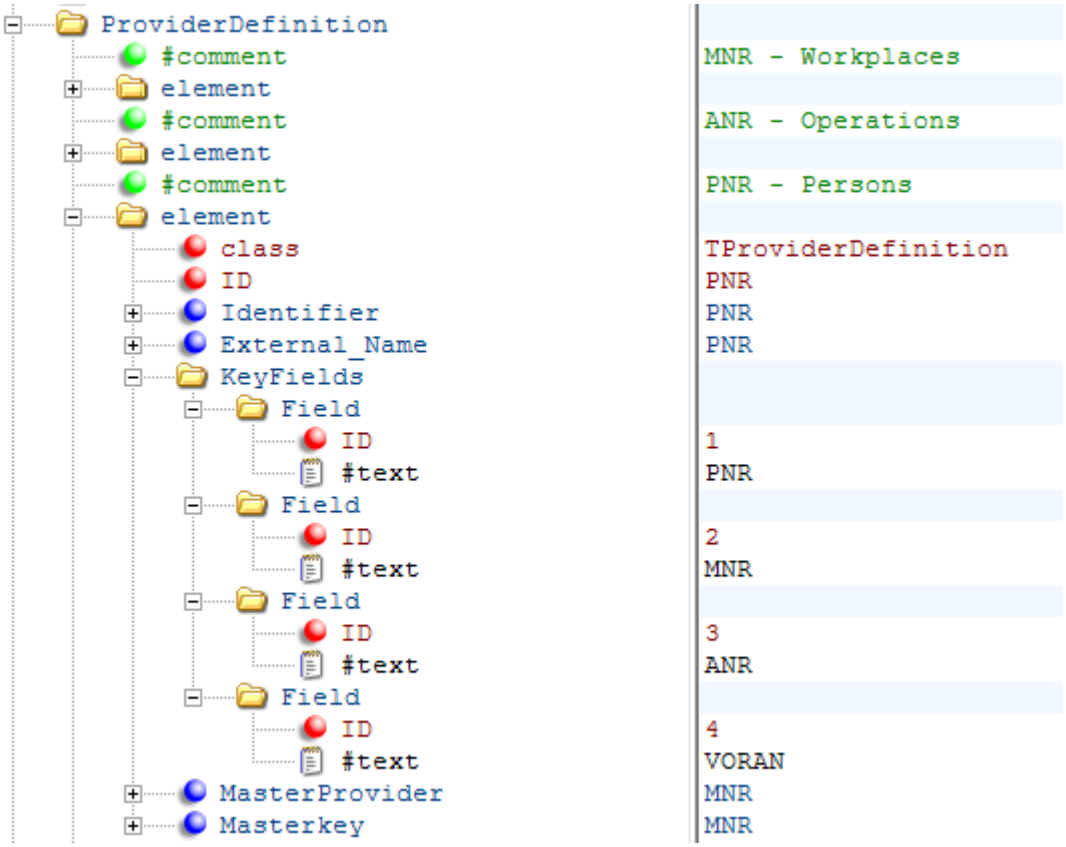
The file *globaldefines.xml* includes general settings, constants, data sources, calculated fields and functions.

Data sources can be added and calculated fields and functions can be created or changed in Customizing.

4.2.1 Data sources (ProviderDefinition)

4.2.1.1 General configurations

The AIP2 provides data as files that can be read by the interface as data sources. Data sources are defined in the section *ProviderDefinition* and you can also add further data sources:



The attributes have the following meaning:

class

The class name for the data source is always TProviderDefinition.

ID

Since the globaldefines.xml file merges the contents of the scopes, a unique ID is required for the entries in this file. Normally, the same values are used as in the following field *Identifier.*

Identifier

Name of the data source that can be used to access the data source in the calculated fields.

External_Name

File name of the data source that needs to be located in the sub directory *spools.* The extension ".lst" is automatically appended to this file name.

KeyFields

This section contains a list of the key fields that uniquely identify a record. If there are more than one record in the data source for the key fields entered here, an error message is displayed.

MasterProvider

Name of the data source that the records to be displayed depend on. For example, the PNR data source contains all persons who are logged on to this terminal. However, only those persons who are logged on to the selected workplace are displayed.

MasterKey

Name of the data field in the data source defined in the previous paragraph that is used to filter the records to be displayed. The name of the data fields must be the same in both data sources.

The name of customer-specific data sources must begin with the prefix "U_" to avoid overlaps with data sources of the standard.

4.2.1.2 Sort data sources

It is possible from terminal version 8.2.1.1 to sort data of a data source. The configuration to sort is made directly in the provider definition. You need to add a new element "SortFields" in the XML file of the provider responsible for this.

```
<SortFields>
        <Field ID="1">MNR;desc;AsString</Field>
        <Field ID="2">MSDATB;asc;AsDate</Field>
</SortFields>
```

When this configuration is made, sorting is done first by machine number in descending order (desc) and then by date in ascending order (asc).
It may be necessary to specify the data type to sort fields correctly. The following data types are available for this purpose: AsString (text), AsInteger (integer), AsDouble (floating point number), AsDate,(date) AsTime (time, hh:mm:ss)
In this case, the default sorting is ascending (asc) and based on the data type "String".

4.2.1.3 Filter data sources

It is possible from terminal version 8.2.1.1 to filter the data in a data source. The configuration to filter is made directly in the provider definition. You need to add a new element "FilterFields" in the XML file of the provider responsible for this.

```
<FilterFields>
    <Field ID="1">MNR=DREH0001|FRÄS</Field>
```

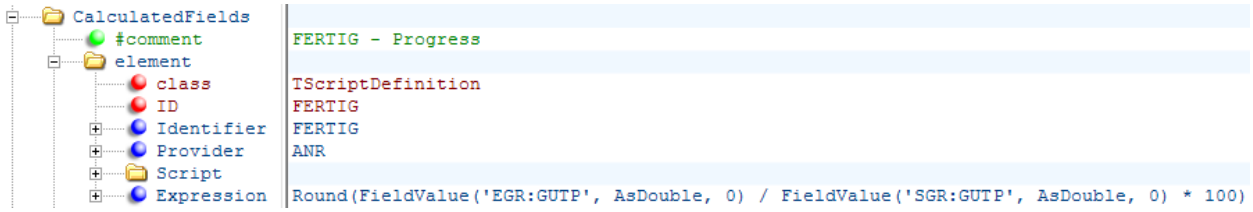
```
    <Field ID="2">MGRP=GRP1</Field>
</FilterFields>
```

This configuration enables the display of data records that have machine numbers (MNR) DREH0001 or FRAS assigned and that have a machine group (MGRP) GRP1.

In contrast to the manual filtering in the interface, no "Contains" is executed here if a WildCard is not specified, but a "=".

4.2.2 Calculated fields

New data fields can be generated from existing data using calculated fields.



This example shows how the order progress is calculated from the fields "yield" (EGR.GUTP) and "target quantity" (SGR:GUTP).

The name of customer-specific calculated fields must begin with the prefix "U_" so that there is no overlap with calculated fields in the standard.

The attributes have the following meaning:

class

The class name for calculated fields is always *TScriptDefinition*.

ID

Since the globaldefines.xml file merges the contents of the scopes, a unique ID is required for the entries in this file. Normally, the same values are used as in the following field *Identifier.*

Identifier

Name of the calculated field. Using this acronym the field can be accessed while customizing the layout.

Provider

Name of the data source from which the data fields derive.

Script

As an alternative to the attribute *Expression,* a script function consisting of one or several lines can be entered in this field. In this case the function name must be defined in the *Expression* field.

Expression

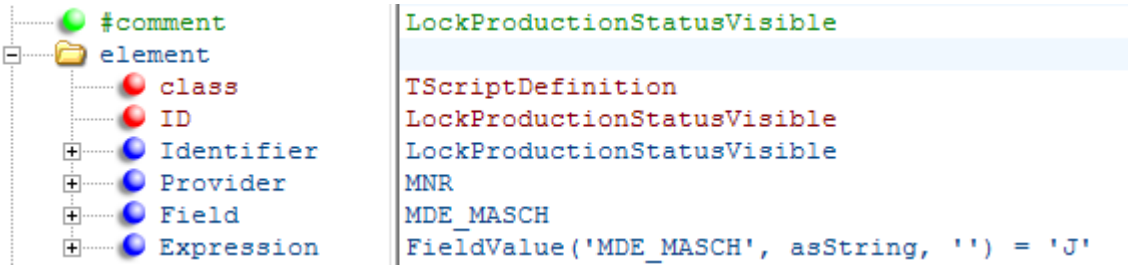
Expression calculating the field. The syntax is described in chapter 1.1.4.

Interval

Calculated field that depend from the current point in time, can be re calculated using the attribute *Interval.* The cyclic update rate in milliseconds is entered as the value of the *Interval* attribute.

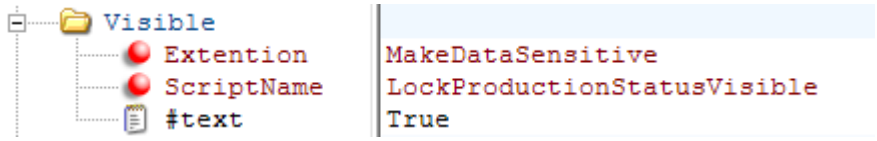
4.2.3 Functions (ScriptDefinitions)

Functions can be used, for example, to control visibility of elements:



This function controls that the button for setting the "production lock" is only visible for MDE machines.

The described function is used in the layout l_mnr.xml:



The single attributes have the same meaning as for the calculated fields.

The name of customer-specific function must begin with the prefix "U_" to avoid overlaps with functions of the standard.

4.2.4 Syntax and calculated fields and functions

Calculated fields and functions are implemented in the script language PasScript .

The following functions can also be used:

FieldValue(Fieldname, Typecast, Defaultvalue)

This function is used to access fields from the assigned data source. As parameters the field name is returned in single quotes and also the data type to be returned (asString, asInteger, asDouble, asDate).The default value is transferred in single quotes. .

ProviderValue(Provider, Fieldname, Typecast, Defaultvalue)

The function enables the access to a field from another data source. The 1st parameter transferred to this function is the name of the data source in single quotes. The remaining parameter are the same as during the function call of FieldValue().

Define(Definename)

Use this function to access a value of a constant. As parameter the name of the constant is transferred in single quotes.

ConditionIsValid(Condition)

Use this function to check fields from the terminal configuration and authorization keys. The condition is transferred as parameter in single quotes.

Example:

ConditionIsValid('%BART:CAQ=J%')	Checks if CAQ is activated in the terminal configuration.
ConditionIsValid('$ADE-SAG$')	Check if the authorization key ADE-SAG is active.

Source.FieldExists(Fieldname, Index)

Use this function to check whether the field, which is transferred as the 1st parameter in single quotes, is available in the data source. As the 2nd parameter a variable is transferred which returns the position of the field in the data source. The field is then accessed via *Source.Fields[Index]*.

Example:

```
Function FindSuffix;
var Index;
begin
        if Source.FieldExists('MNRBTN.MODUS', Index) then
                result := Source.Fields[Index].asString
end;
```

4.3 Calling external programs

In order to call up an external program, it must be entered in the section *[ext. software]* of the *ctaip.ini* file:

ProgFileName

The path and file name of the program to be requested are defined here.

WindowName

The string entered in this field is used to check in the process list if the application has already been started. If it has been started, the running application is brought to the front.

SearchParts=On

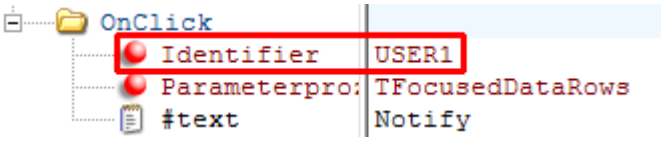
This option defines that not the entire process name must match the entered *WindowName.* It is sufficient if it includes the *WindowName.*

If you want to request further external programs, add a number (starting with 2) to the relevant entries.

Example:

```
[ext. software]
ProgFileName=c:\Windows\notepad.exe
WindowName=Notepad
SearchParts=On
ProgFileName2=c:\Windows\System32\mspaint.exe
WindowName2=Paint
SearchParts2=On
```

Starting external programs is similar to starting dynamic dialogs (see previous chapter). *Identifiers* for external programs are "USER1“ for the 1st entry and "USER2“ to "USER9“ for the entries that follow:



5 AIP2 - GUI Scripting

5.1 Overview

You use the language **PasScript** to implement calculated fields and functions used in the tile view of the AIP2.

5.2 PasScript Overview

PasScript is natively implemented Pascal like language. It is a untyped, Variant-based, dynamic scripting language. PasScript supports declaration of procedures and functions, variables, constants declaration and global statements. It also supports Delphi like true exception handling features using **raise/try/except/finally** syntax.

The language introduce a variety set of statements to control execution of the script-code. For conditional code execution the program can use conditional statements, for looping/iterating - loop statements, for exception raising and handling - exception handling statements.

PasScript is a untyped Variant-based language. This means that every variable, parameter or constant in the language is of Variant type. PasScript supports all Variant value types, supported by Delphi, including numbers, strings, booleans, dates, IDispatch objects, as well as special values **nil,** Unassigned, Null, True, False. Arrays are also supported.

5.2.1 Global declarations

At the global declaration level script source-code (unit) can contain procedure and function declarations, global variable and constant declarations. The unit may also contain global statements block at the end:

```
var
      X, Y;
      Z = 7;

const
      Pi = 3.14;

procedure P(A; B);
var
      i, j;
      k = 3;
begin
      ShowMessage(A + B + F * Pi);
end;

function F;
begin
      Result := 7;
end;
```

```
begin
      ShowMessage('This is a global statement');
end;
```

There can be more than one **var** or **const** sections, moreover they can be placed between procedures/functions. The global statement block (**begin**/**end**) should reside at the end of the unit. The global statement block is optional and can be omitted. Just like for global variables, PasScript allows to specify initializers for local variables.

Unlike Delphi, the order of procedures/functions/variables/constants declarations is not meaningful, procedures/functions can refer to each other, unrelated to the declaration order.

Since PasScript is untyped language, no type specification is allowed in variables/parameters/functions declaration.

Procedure/function parameters are declared as a list of parameter names delimited with ";". By default, parameters have by-value semantic. **var** and **out** keywords can be used to denote parameters passed by reference; there are no difference between **var** and **out** parameters, two distinct keyword are just used to help writing self-documented code. **const** keyword is used to denote read-only parameters. Parameter list can be empty. Here the examples of parameter declarations:

```
procedure P;

procedure P();

procedure P(A);

procedure P(A; B; const C);

procedure P(A; var B; out C);
```

Implicit Result variable is accessible inside a function. This variable should be used to assign returning value to the function. This variable works just as simple local variable, you can read it value, assign a value or pass the Result variable by reference.

Script code can use Exit intrinsic function to immediately exit from the parent procedure or function. Run-time error will occurs if Exit function is used outside of any function or procedure (e.g. in global code).

5.2.2 Compound Statements

Statements in PasScript should be ended with a semicolon ";" symbol. The semicolon is not allowed before **else** keyword and may be omit before **end**/**until** keywords. Just like in Pascal, PasScript uses **begin**/**end** keywords to combine several statements into single compound statement.

5.2.3 Conditional Statements (if, case)

If statement

If/then/else statement is used to execute code conditionally. The else part can be omit. Here some examples:

```
if X > 0 then
      ShowMessage('X is positive');

if S <> '' then
      ShowMessage('S is not empty')
else
      ShowMessage('S is empty');

if S <> '' then
begin
      ShowMessage('S is not empty');
      Result := True;
end
else
      ShowMessage('S is empty');
```

Case statement

Case statement is another statement for conditional code execution. Unlike Delphi, in PasScript you can specify any expressions in case labels, not only constants. Also, PasScript does not check values uniqueness. Just as in **if** statement, **else** part can be omit. Examples:

```
case x of
      0:
            ShowMessage('Zero');
      1, 3, 5, 7, 9:
            ShowMessage('Odd');
      2, 4, 6, 8, 10:
            ShowMessage('Even');
else
      ShowMessage('I''m too young to know numbers greater than 10');
end;
```

Since any expressions in case labels are allowed, you can also specify, for example, string expressions:

```
case s of
      'True':     b := True;
      'False':    b := False;
else
      ShowMessage('Invalid value');
end;
```

Value ranges are also allowed in case labels:

```
case x of
      1..7:
            x := 1;
      10, 12, 20..26:
            x := 2;
end;
```

5.2.4 Loop Statements (for, while, repeat)

For, while, repeat statements

Loop statements are used to repeatedly execute code. You can use **for**/**to** statement to iterate from low to high bounds, **for**/**downto** to iterate in reverse order, or **while** or **repeat** statements to iterate with condition. Examples:

```
for i := 0 to 10 do
      ShowMessage(i);

for i := 10 downto 0 do
      ShowMessage(i);

i := 1;
while i < 10 do
begin
      ShowMessage(i);
      i := i * 2;
end;

i := 1;
repeat
      ShowMessage(i);
      i := i * 2;
until i >= 10;
```

Break and Continue

The Break intrinsic function can be used to break loop statement. Continue intrinsic function can be used to go to the next loop iteration. Using Break/Continue outside of the loop will raise run-time error.

```
i := 1;
while i < 100 do
begin
      if i = 15 then
            Break;
      i := F(i);
end;

for i := 0 to 10 do
begin
      if (i mod 2) <> 0 then
            Continue;
      ShowMessage(i + ' is an even number');
end;
```

The Break/Continue functions always works with innermost loop:

```
for i := 0 to 10 do // outer
begin
      if i = 5 then
            Break; // Break outer loop.
      for j := 0 to 10 do // inner
      begin
            if j = 5 then
                  Break; // Break inner loop.
      end;
end;
```

5.2.5 Exception Handling Statements (raise, try)

Raise statement

PasScript provides true exceptions support through **try/except**, **try/finally** and **raise** statements. The raise statement can be used to throw exceptions, like this:

```
if X > 10 then
      raise Exception.Create('X is too big');
```

Note, that here Exception.Create(...) is just an expression that creates an instance of the exception. So, to really use the raise statement, it is required to add imported VCL unit(s) to the script-control to allow script-code to use exception types (Exception in this case). Otherwise, you will get "Undeclared identifier" run-time error.

Try/except statement

Your program can use **try/except** statement to catch raised exceptions and execute error handling code. Just like in Delphi the except part of the statement can be in a simple or complex form. The simple form example:

```
try
      DoSomething;
except
      ShowMessage('Error has occurred.');
end;
```

The **except** part in this form will catch all possible exception. The example of complex **except** form:

```
try
      DoSomething;
except
      on E: EArgumentException do
            ShowMessage('Invalid argument');
      on E: EOutOfMemory do
            ShowMessage('Out of memory');
      else
```

```
            ShowMessage('Other error.');
end;
```

Any number of **on**/**do** blocks can be specified. An **on**/**do** block catch only exception of the specified class (or a subclass of it). The **else** block catches all other exceptions. The **else** block is optional. Note, that empty **else** block also catches all possible exceptions, however, if the **else** block is omit, then all not caught exceptions will be thrown away from the current **try** statement.

The variable name (E in the example) is optional, but, if specified, the variable becomes accessible within the corresponding **on**/**do** block and holds the reference to the current exception instance:

```
try
      DoSomething;
except
      on E: EArgumentException do
            LogMessage('Error: ' + E.Message);
end;
```

Normally, if the exception is caught by **except** handler, it will not be re-raised implicitly to parent statements. If you need to re-raise the exception, you can use the **raise** statement without arguments:

```
try
      DoSomething;
except
      ShowMessage('Error');
      raise;
end;
```

This form of the **raise** statement is only allowed inside **except** handler. The run-time error will occurs otherwise.

Try/finally statement

Your program can use the try/finally statement to guarantee execution of the finalization code in both cases: when error occurred and when it is not:

```
obj := TMyObject.Create;
try
      obj.DoSomething;
finally
      obj.Free;
end;
```

The obj instance, created in above example will be freed in any case, even if DoSomething will raise an exception. **try**/**finally** statement always implicitly re-raise the current exception, so it can be caught by parent statements.

If Break, Continue or Exit intrinsic functions are used inside the **try**/**finally** block and leads outside of the **try**/**finally**, the **finally** statements will be also executed:

```
for i := 0 to 10 do
begin
	try
		Break;
	finally
		ShowMessage('In finally');
	end;
end;
```

It is illegal to use Break, Continue or Exit intrinsic functions inside the **finally** itself; run-time error will be raised:

```
for i := 0 to 10 do
begin
	try
		DoSomething;
	finally
		Break; // Illegal!
	end;
end;
```

5.2.6 Expressions

PasScipt language supports a variety set of expressions to allow script program to assign variables, call procedures and functions, accessing objects properties (including indexed properties), perform logical and math operations and more.

Literals

PasScript supports Integer, floating-point and string literals, as well as **nil**, **True**, **False**, **Unassigned** and **Null** constants. Here are some examples:

```
x := 7;
x := -7;
x := 7.0;
x := +0.25E+10;
s := 'Some string';
```

Integers in hexadecimal form are also supported with the syntax similar to Delphi:

```
x := $A23BD7;
```

String literals also has syntax close to Delphi, including escaping the quote char and specifying non-printable characters using # symbol:

```
s := 'This is a string containing a single '' symbol';
s := 'This is a '#13#10'two line string';
```

The keyword **nil** is used to denote no-object value. It is equivalent to Nothing in VBScript and represented as a Variant with VType = varDispatch and VDispatch = nil;

The Unassigned is used to denote not yet assigned (or empty) variable. It is equivalent to VBScript Empty constant and represented as a Variant with VType = varEmpty.

The Null is used to denote database NULL value, which has the meaning of unknown/unspecified value; represented as a Variant with VType = varNull.

Operators

The following operators are supported by PasScript language:

`>`	Greater than
`<`	Less than
`>=`	Greater or equal
`<=`	Less or equal
`<>`	Not equal
`=`	Equal
`in`	Value in set. Used with imported from Delphi **set** types; for example: `if fsBold in Font.Style then` See set constructors for more info.

`is`	Value is of type. Used with imported from Delphi class and record types to test whether the object or record instance is of specified type; for example:
	`if obj is TButton then`
	`if rec is TPoint then`

`+`	Addition
`-`	Substraction
`*`	Multiplication
`/`	Division
`div`	Integer division
`mod`	Integer modulus
`or`	Logical or. "Incomplite Boolean eval" logic is used.
`xor`	Logical/bitwise xor
`and`	Logical and. "Incomplite Boolean eval" logic is used.
`shl`	bitwise shift to left
`shr`	bitwise shift to right
`not`	Unary logical not
`@`	Make event handler. Look here for more info.

Incomplite Boolean evaluation

The **and** and **or** operators use incomplite Boolean evaluation logic, just like in Delphi. That is, the second operand is evaluated only when necessary. The following examples demonstrate the advantage of this evaluation strategy:

```
if (obj <> nil) and (obj.Width < 100) then
      DoSomething;
```

```
if (obj = nil) or (obj.Width >= 100) then
      DoSomething;
```

In both cases obj.Width will be evaluated only if obj is not equal to **nil**. In both cases no error is possible, because when the obj is **nil**, the second part of the expression is not evaluated.

Calling object methods and accessing properties

PasScript uses Delphi like syntax to call global procedures/function as well as object methods/properties. Parameters for procedures/functions/methods are specified in round brackets; however if there no parameters expected, the brackets can be omit. Parameters for indexed properties are specified in square brackets. Here some examples:

```
P(5, 7);
x := F1(3);
x := F2; // Brackets omit.
x := F2();
h := MyFont.Height;
s := Application.ActiveForm.Caption;
s := Memo.Lines.Items[5]; // Indexed property
```

Set constructors

PasScript supports special syntax for working with **set** types imported from Delphi. Set constructors allows to specify **set** elements in square brackets to compose a set value. Empty set value is also supported. Here are some examples:

```
Font.Style := [fsBold, fsItalic];
Font.Style := []; // Empty set.
```

Recall that to test, whether an element included in the **set** value, script program can use **in** operator:

```
if fsBold in Font.Style then
      ShowMessage('Font is bold');
```

Delphi like Include and Exclude intrinsic functions are also supported:

```
FontStyle := [];
if NeedBoldFont then
      Include(FontStyle, fsBold);
Font.Style := FontStyle;
```

Event handlers

PasScript supports special syntax for creating a references to procedures written in script-code and assigning these references to events of objects. This can be done using @ symbol:

```
procedure Button1Click(Sender);
begin
```

```
      ShowMessage('Hello from script');
end;

begin
      Button1.OnClick := @Button1Click;
end;
```

The global code from the above example assigns a procedure written in script-code to the OnClick button's event. After assignment, clicking on the button will execute Button1Click procedure.

5.2.7 Arrays

Creating arrays

PasScript allows to create single-dimensional arrays as well as multi-dimensional, allows to specify both: low bounds and high bounds of the array. The script program should use array constructor syntax to create array. For example:

```
a := array[0..5]; // Single-dimensional array.
a := array[0..7, 1..3]; // Multi-dimensional array.
```

The low bound of the array is optional and can be omit. If the code omit low bound it set to zero by default:

```
a := array[5]; // Same as [0..5]
a := array[7, 2]; // Same as [0..7, 0..2]
```

PasScript also supports creation of the arrays with elements of type other than varVariant. For example, it is possible to create an array with elements of varByte for more efficient data storage:

```
a := array[100] of Byte;
```

Following table specifies the list of type names that can be used in array constructor:

Type name	Variant value type code
Integer	varInteger
String	varOleStr
Double	varDouble
Single	varSingle
Boolean	varBoolean
Variant	varVariant
Byte	varByte
Word	varWord
LongWord	varLongWord
SmallInt	varSmallInt
Currency	varCurrency
ShortInt	varShortInt
Int64	varInt64
UInt64	varUInt64
Error	varError

Object	varDispatch

If the type name is omit, the array will be created with varVariant element type.

Using arrays

The script program should use square bracket syntax to access array elements, just like in Delphi:

```
a := array[0..10];
a[0] := 5;
a[1] := 7;

b := array[0..5, 0..7];
b[1, 3] := 11;

Y := a[0] + a[1] + b[1, 3];
```

The Length intrinsic function can be used to determine the length of the array. If the array is a multi-dimensional array, the dimension number can be specified as a second argument in a Length function call. The dimension numbers starts from 1. Here some examples:

```
a := array[0..10]
x := Length(a);

b := array[0..5, 0..7];
y := Length(b, 2);
```

The Low and High intrinsic functions can be used to determine the low and hight bounds of the array. Just like with Length function, the dimension number can be specified as a second argument:

```
a := array[0..10];
for i := Low(a) to High(a) do
      a[i] := i + 100;
```

5.2.8 Intrinsic Functions

Following is the list of intrinsic procedures, functions and constants that are directly built into PasScript language. There are no need to import Delphi units to use these functions. All these functions are almost identical to the corresponding Delphi functions:

Special branching functions

```
Exit
Break
Continue
```

Constants

```
Null
Unassigned
```

```
True
False
```

Functions

```
Include
Exclude
Beep
ArcTan
Cos
Dec
Sin
LowerCase
High
Low
Ln
AnsiCompareStr
AnsiCompareText
AnsiLowerCase
AnsiUpperCase
Abs
CompareStr
CompareText
Date
DateTimeToStr
DateToStr
DayOfWeek
DecodeDate
Exp
FloatToStr
Frac
Int
IntToHex
IntToStr
IsLeapYear
IsValidIdent
Length
Now
Odd
Pos
Random
Round
Sqr
Sqrt
StrToDate
StrToDateTime
StrToFloat
StrToInt
StrToIntDef
Time
TimeToStr
StrToTime
Trim
TrimLeft
TrimRight
Trunc
UpperCase
VarIsNull
VarToStr
```

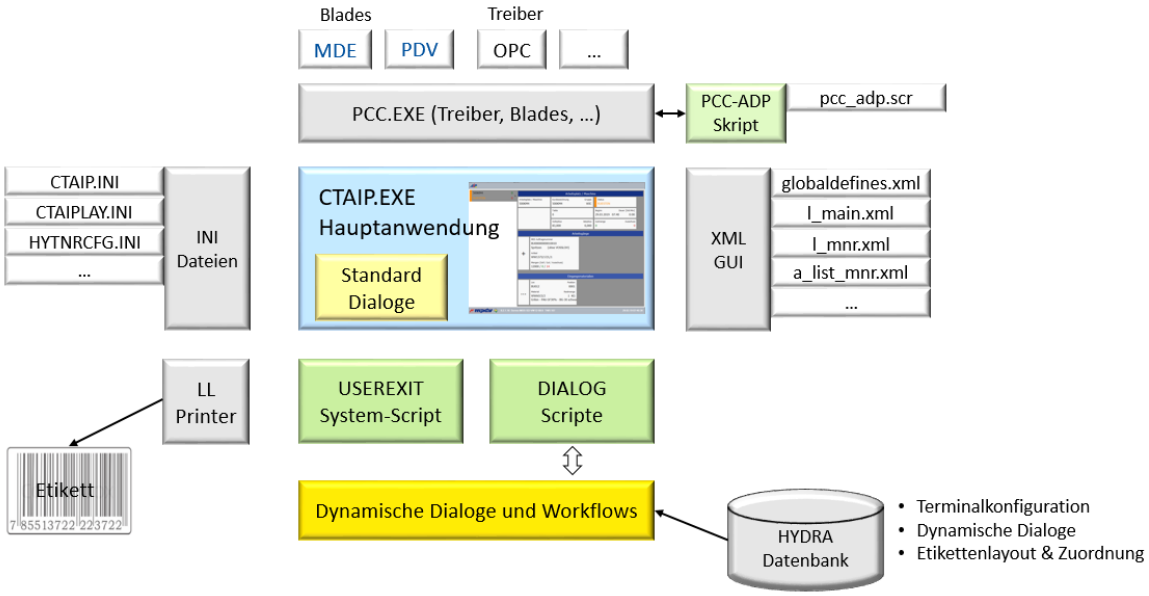
```
Assigned
ShowMessage
Insert
IncMonth
Inc
Chr
Copy
Delete
CreateOleObject
GetActiveOleObject
InputQuery
DecodeTime
EncodeDate
EncodeTime
Format
FormatFloat
FormatDateTime
Ord
```

6 AIP2 - Scripting - Reference

6.1 Features

You can use the MES Development Suite (MDS) to change and extend the data collection functions of the Acquisition Information Panel (AIP2). For specific sections, the MDS provides user exits to implement changes of the standard processing.

This section describes the AIP2 processing of terminal scripts. The diagram below provides an overview of the structure and the logic components of the AIP2.



On the AIP2, the following types of terminal scripts are available:

- USEREXIT (system script):
  USEREXITs are used to extend standard and customized functions on the AIP2.
- DIALOG – script:

  DIALOG scripts are used to extend existing standard dialog functions or to implement new customer-specific dialog functions on the AIP2.

  DIALOG scripts are used to control the configured dynamic dialogs.

- PCC scripts:

  Further script functions are available to control or connect machines via the PCC interface (e.g. pcc_adp.scr). The documentation of the PCC scripts is included in the document "CUT-PCC_81_PCC-ADP_Kurzreferenz".

You can use the terminal scripts to implement the following changes and extensions, for example:

- Changing and extending the existing displays and dialog functions
- New dialog functions
- Interfacing of external interfaces
  - Blades
  - Drivers (e.g. OPC-UA)
  - Scanner
- etc.

6.2 Programming aids

6.2.1 Visual Basic

The script language used is based on VBScript. There are also so-called callback functions that are used as interfaces to the main application. The script functions available are described in the sections that follow.

6.2.2 Naming conventions

6.2.2.1 Script files

The file names of script files on the AIP2 can be in lower or upper case letters. The file name of the ZIP container may only have lower case letters because with Linux operating systems the ZIP files having upper case letters in their file names are not loaded!

ZIP container: The download of the terminal scripts from the server to the terminal is performed using a so-called ZIP container (ZIP file with extension .zip).

Note the following for script file names:

- Script files for the AIP2 always start with "aip_".
- Customer-specific DIALOG scripts start with "aip_U_" unless the name is otherwise specified by its intended use. This ensures that customer-specific scripts are overwritten by MPDV updates.
- Customer-specific USEREXIT scripts must start with "aip_system".
- Optionally, you can extend script files via project abbreviation/customer numbers and scopes.

6.2.2.2 Includes in script files

Note: Include files are supported as of AIP2 version V# 8.2.1.11.

You use include files to integrate other files into existing terminal scripts. A better structure of the scripts is then possible.

You can use the directive"**$<include-<Name>.scr>**" to integrate files into any row of the terminal script.

For reasons of downward compatibility, the include file is integrated as comment.

Include files are only loaded if

- the directive is at the beginning of the row.
- the include file starts with "**include-**" and has the extension "**.scr**".
- the include file is stored in the local directory (.*etc, .\etc\var or .\etc\local*) of the terminal.

Recursive loading of include files is not supported.

In a terminal script, an include file is loaded only once with the terminal script.

Example: File "include-custom-utils.scr":

```
'------------------------------------------------
' $Id: include-custom-utils.scr $
'------------------------------------------------
Sub doCustomValidation
  ' define Custom Validation for using function ...
  If VVar("UE:PAR","BTN.FKT") = "A_TR" Then
    '...
  End If
End Sub 'doCustomValidation
'------------------------------------------------
```

Include into “aip_system_<project>.scr“:

```
<'-------------------------------------------------------------------------
'$<include-custom-utils.scr>
'-------------------------------------------------------------------------
Sub UserExitButtonClick '
  doCustomValidation
End Sub 'UserExitButtonClick
...
```

Only use the include files, if an organization of the scripts has advantages.

In case of a script error, a terminal script extract and the script or include file is displayed with the row of the error. (see section "Exception - Script - Dialog")

6.2.3 Scope Concept

The names of the terminal script files are based on the so-called scope concept. When names are assigned, the well-known scopes standard, custom, partner and local are supported.

MPDV standard	MPDV custom	Partner (@var)	Local (@local)
			aip2_<project>@local.zip aip2@local.zip local\aip_system@local.scr local\aip_DIALOG@local.scr
		aip2_<project>@var.zip aip2@var.zip var\aip_system@var.scr var\aip_DIALOG@var.scr	
	aip2_<project>.zip aip_system_<project>.scr aip_DIALOG_<project>.scr		
mpdv-aip.zip aip_mpdv-system.scr aip_mpdv-DIALOG.scr			

The more a scope is "special", the higher its priority. The special scope always takes priority over the general/standard scope. A file in the local scope takes priority over a file included in the standard scope.

New functions are only valid in the scope where they were implemented. They can be overridden by a scope of a higher priority.

The terminal script files are read and used in the following order/priority:

Scope	Priority	AIP 2	Description
MPDV	1	**.\aip_mpdv-system.scr** **.\aip_mpdv-<dialog>.scr**	MPDV standard
CUSTOM	2	**.\aip_system_<customer no>.scr** **.\aip_<dialog>_<customer no>.scr**	MPDV customization with customer number

Scope	Priority	AIP 2	Description
CUSTOM	3	.\aip_system_<project>.scr .\aip_<dialog>_<project>.scr	MPDV customization with project abbreviation
VAR	4	.\var\aip_system_<customer no>@var.scr .\var\aip_<dialog>_<customer no>@var.scr	Partner scripts (partner scope) for a customer project with customer number
VAR	5	.\var\aip_system@var.scr .\var\aip_<dialog>@var.scr	Partner scripts (partner scope) for standard partner software
LOCAL	6	.\local\aip_system_<customer no>@local.scr .\local\aip_<dialog>_<customer no>@local.scr	Customer scripts (local scope) of customer with project abbreviation and customer number
LOCAL	7	.\local\aip_system@local.scr .\local\aip_<dialog>@local.scr	Customer scripts (local scope) of customer

6.2.4 Storage structure of the scripts

ZIP files:

The terminal scripts are compressed and stored in a ZIP container on the server. The AIP2 downloads the ZIP files from the server and unpacks the files locally on the client under the directory .\etc. If the download was successful, the ZIP file is locally unpacked.

Locally on the AIP2:

- .\etc ; directory of extensions by MPDV (standard / custom)
- .\etc\var ; directory of partner extensions (partner)
- .\etc\local ; directory of extensions by the customer (local)

On the server:

Name and store the ZIP files for the relevant scope in the specified directory on the server as follows:

Scope	Priority	Storage structure of ZIP container on server	Description
MPDV	1	.\ctnet\win\aip2\etc\mpdv-aip.zip	MPDV scripts (standard scope)
CUSTOM	2	.\custom\userexit\aip2_<customer number>.zip	MPDV customization with customer number

Scope	Priority	Storage structure of ZIP container on server	Description
CUSTOM	3	**.\custom\userexit\aip2_<project>.zip**	MPDV scripts (custom scope) for customer project
VAR	4	**.\custom\userexit\aip2_<project>@var.zip**	Partner scripts (partner scope) for customer project
VAR	5	**.\custom\userexit\aip2@var.zip**	Partner scripts (partner scope) for standard partner software
LOCAL	6	**.\custom\userexit\aip2_<project>@local.zip**	Customer scripts (local scope) of customer with project abbreviation
LOCAL	7	**.\custom\userexit\aip2@local.zip**	Customer scripts (local scope) of customer

Note: customer number and project are specified in the basic settings for each system.

6.2.5 Program parameters for developer mode

The following useful program parameters are available on the AIP2 in developer mode:

Parameters for development (ctaip.ini: INI section [***system***])

parameters= … -AskForOverwriteScriptFiles -AlwaysReloadScript …

„-AskForOverwriteScriptFiles“

Prevents overwriting of locally changed scripts on restart. Before unpacking the ZIP files, the AIP asks whether locally changed scripts should be overwritten.

„-NeverOverwriteScriptFiles“

Prevents overwriting of locally changed scripts on restart (without confirmation).

„-AlwaysReloadScript“

Reloads terminal scripts of the file each time its called. Use the button to process changes during runtime.

„-SkipAipStartUpUpdate“

Prevents overwriting of locally changed INI, CFG, XML files and DLLs on restart (without confirmation).

„-AskForRemoveDirectory“

Prevents deleting of local partner/customer extensions when the relevant directory is deleted on restart.

The AIP asks before deleting the following directories:

.\etc\var\ ; directory for *partner extensions*

.\etc\local\ ; directory for customer extensions

„-NeverRemoveDirectory“

Similar to „-AskForRemoveDirectory“. The directories are not deleted, there is no query.

„DEMO mode“

When you start AIP2 in DEMO mode, the ZIP containers are not unpacked.

6.2.6 Communication interfaces

The AIP2 provides different communication interfaces that are used to exchange data. The most important interfaces are the following:

Interface	Description
PDM commands	The PDM interface is used to send PDM commands (e.g. DLG=A_AN) to the server. You use PDM commands to send and book postings to the server.
PDM list requests	You use list requests to the server to request data as list file (e.g. mnr.lst – list of assigned machines).
File transfer	You can use the file interface to transfer files directly to or from the server.
Port (gateway)	You can use the input port to send PDM messages to the terminal. The external PDM client requires a relevant communication interface. e.g. update of lists
PCC interface	Blades, OPC, PCC-DIF (file interface) All of these interfaces are connected via the additional program PCC.EXE. It is possible to exchange messages between the PCC and the main application CTAIP.EXE and to process them specifically.
Scanner via COM interface	The scanners are logistically assigned to a COM port. If data is read via scanner, this data is provided to the AIP2 via interrupt handler. Via user exit, you can process this data in a specified manner. You assign the COM port in the CTAIP.INI.

The results of the lists are written in files on the server. The client passes the file name with a relative path to the server. The server creates the file. The client then loads the file and can process it.

The file should be created in the spool directory on the server.

Note: The file name must be unique per client. Only then, the server will not overwrite files of another request. If unique file names are not guaranteed, processes can be blocked on the server because these processes must access the same file.

You can use the following methods to assign unique file names:

- Integrate a unique number per client in the file name (e.g. with AIP use the user number = terminal number + 2000) On AIP2, the user number is included in the script variable SYS_USR.
- Integrate the current time stamp in the file name.

Examples:

- With user number: FILE=./spool/myfile**2043**.dat|
- With time stamp (format: MonDDhhmmssMMM with milliseconds): FILE=./spool/myfile**Dec31235959999**.dat|

6.2.7 Differences of the graphical user interface with and without XML GUI

With terminal scripting, there are 3 different GUI of the AIP2. The differences are as follows:

GUI	Description of the differences / notes
CTWIN	Similar to AIP8.1. But the buttons are situated at the bottom (via ctaipbut.ini). The graphic interface is displayed without skin. For the use of terminal scripts, there are no differences to the AIP8.1.
AIP 8.1	You configure the buttons in the main view via the file ctaipbut.ini.
XML GUI	The graphic interface is configured in XML. The main view displays the data as tiles. A selection is always performed when you open a detail dialog. The main view does not show any selected entries in the lists.

6.2.8 Static and temporary lists on the AIP

The AIP2 directory .\spool includes static and temporary list files, which are usually loaded from the server. The following section describes the most important files.

For more information on the content and properties, refer to the standard PDM documentation.

See the note on unique file names for lists on the server in section "6.2.6 Communication interfaces".

6.2.8.1 Static lists

6.2.8.2 aart.lst (order types)

Server command: DLG=LIST;87|..

Includes all order types configured in the system.

6.2.8.3 agrd.lst (scrap reason list)

Server command: DLG=LIST;84|MOD=T|TNR=706|..

The scrap reason list includes reasons for scrap/yield/rework/open quantity of the machines assigned to the terminal. (ART=G,A,N,P,…). It also includes SYSTEM reasons.

6.2.8.4 anr.lst (order list)

Server command: LIST;11|MOD=L|USR=2706|..

The oder list includes all running orders of all machines, which are assigned to the terminal or logged on to the terminal.

6.2.8.5 bmk.lst (list of RPA accounts)

Server command: DLG=BMK.LIST|..

The list of the resource performance accounts (RPA) includes all RPAs configured in the system.

6.2.8.6 bpos.lst (machine operator positions)

Server command: DLG=LIST;14|USR=2706|..

The list includes all operator positions of the machines assigned to the terminal.

6.2.8.7 hztyp.lst (material types)

Server command: DLG=LIST;21|..

The list includes all configured material types in the system (is only loaded with a machine in batch mode)

6.2.8.8 lizenz.lst (licenses)

Server command: DLG=LIST;48|..

The list includes all licenses and function keys.

6.2.8.9 mnr.lst (machine list)

Server command: LIST;10|USR=2706|..

The list includes all machines assigned to the terminal or dynamically assigned machines (via logon).

6.2.8.10 mstat.lst (machine status list)

Server command: DLG=LIST;16|MOD=T|USR=2706|..

The list includes all machine statuses of the machines that are assigned to the terminal in a fixed form (=configuration) or dynamically (logon, possibly only after server update).

6.2.8.11 paths.lst (directory list)

Server command: DLG=LIST;81|..

The list includes all paths of the modules configured in the system (DNC, DOK,...)

6.2.8.12 pnr.lst (list of persons)

Server command: DLG=LIST;12|USR=2706|MOD=V|..

The list includes all persons logged on to the machines of the terminal.

6.2.8.13 qrdcfg.lst (label printing – configuration (only with active license))

Server command: DLG=SYSTEM.CALL|PROG=hyettlst.scr|USR=2222|..

Includes all labels assigned that are active on the terminal and assigned to a dialog.

Note: the label definition is included in the sub folder llprinter of the AIP2

6.2.8.14 schicht.lst (list of shifts)

Server command: DLG=LIST;38|MOD=T|TNR=706|..

The list includes the shift configurations of the MDE machines assigned to the terminal

6.2.8.15 tkenn.lst (terminal label)

Server command: DLG=LIST;45|TNR=706|..

The list includes the configured terminal label with settings from the basic settings (e.g. batch number length).

6.2.8.16 tnrmat.lst (terminal – list of input material (only loaded with machine in batch mode))

Server command: DLG=LIST;13|MOD=T|USR=2706|..

Terminal – list of input material (only loaded with machine in batch mode)). The list includes all input batches/materials logged on to the machines of the terminal.

6.2.8.17 tnrres.lst (terminal – resource list (WRM))

Server command: DLG=LIST;129|MOD=T|USR=2706|..

Terminal resource list (WRM). The list includes all active resources of all machines of the terminal

6.2.8.18 tpe.lst (transport units (MPL))

Server command: DLG=LIST;52|..

The list includes all transport units configured in the system

6.2.8.19 vlpkz.lst (list of premium indicators)

Server command: DLG=LIST;24|..

The list includes all premium indicators created for all machines of the terminal.

6.2.8.20 zloueb.lst (material buffers / target locations (MPL))

Server command: DLG=LIST;49|TNR=706|..

List of all material buffers/target locations of the system.

6.2.8.21 vlist.<MNR>.lst (order sequencing list)

Server command: DLG=LIST;11|MOD=V|MNR=DBCM1010|..

The order/sequencing list includes all operations for this machine if the status of the machine is configured to be displayed in the sequencing list (usually prepared and interrupted operations).

6.2.8.22 <MNR>_amat.lst (output batches)

Server command: --- / this list is only locally maintained by the terminal!

List of the output batch created per machine.

6.2.8.23 Temporary lists

6.2.8.24 mat.lst (list of input material)

Server command: DLG=LIST;13|MOD=M|MNR=DBCM1010|ANR=010001010010|DLG.DLGCFG=A_AN_MPL|..

This file includes the component list last loaded of an order at a machine (with logged on input batches).

NOTE: this file is only read in ONLINE mode (e.g. with OP logon or input batch change)

6.2.8.25 nanr.lst (order info)

Server command: DLG=LIST;11|MOD=A|ANR=010001010010|..

This file includes the order info last loaded of an operation.

6.2.8.26 amat.lst (list of produced output batches for an operation)

Server command: LIST;13|MOD=A|DBCM1010|ANR=010001010010

List of produced output batches for an operation

6.3 Script – functions and variables

User exits are exit point that are called when the terminal main application is running. You can use the user exits to interfere in the terminal program and implement extensions.

The script functions are callback functions that the terminal main application provides for a script to read or change information or perform functions.

An AIP terminal script includes global variables valid in the execution context and different script functions to access these variables.

6.3.1 Script variables

6.3.1.1 UE_RET (general data exchange)

When used in read access, the complete content of the script variable **[n#]UE:RET** is returned.
[n#] If functions are called recursively, a reference index is added as prefix

Read access is performed with **VVar("UE:RET","<DlgID>")**
Oo all values with **VVar("UE:RET","#GET#ALL#VALUES#")**

If used in write access, the complete content of the script variable **[n#]UE:RET** is deleted if an empty string is assigned. If you assign a DlgID with value (e.g. "TEST=1|"), an already existing entry is replaced. If the entry does not exist, it is added to the existing values.

e.g. **VVar("UE:RET","U_ERRCODE")**

6.3.1.2 UE_SND (general data exchange)

If used in read access, the complete content of the script variable **[n#]UE:SND** is returned.
[n#] If functions are called recursively, a reference index is added as prefix

Read access is performed with **VVar("UE:SND","<DlgID>")**
Or all values with **VVar("UE: SND","#GET#ALL#VALUES#")**

When used in write access, the complete content of the script variable **[n#]UE:SND** is deleted if an empty string is assigned. If you assign a DlgID with value (e.g. "TEST=1|"), an already existing entry is replaced. If the entry does not exist, it is added to the existing values.

6.3.1.3 UE_RCV (general data exchange)

If used in read access, the complete content of the script variable **[n#]UE:RCV** is returned.
[n#] If functions are called recursively, a reference index is added as prefix

Read access is performed with **VVar("UE:RCV","<DlgID>")**
Or all values with **VVar("UE: RCV","#GET#ALL#VALUES#")**

When used in write access, the complete content of the script variable **[n#]UE:RCV** is deleted if an empty string is assigned. If you assign a DlgID with value (e.g. "TEST=1|"), an already existing entry is replaced. If the entry does not exist, it is added to the existing values.

6.3.1.4 DLGVAR (general data exchange)

If used in read access, the complete content of the script variable **[n#]DLG.DLG** is returned.
[n#] If functions are called recursively, a reference index is added as prefix

Read access of a value is performed with **VDlg ("#GET#ALL#VALUES#")** or **VVar("DLG.DLG","#GET#ALL#VALUES#")**

See also **DLGOUT**

6.3.1.5 DLGSND (general data exchange)

A direct read access to this variable is not possible.

Read access of a value is performed with **VOut("<DlgID>")** or **VVar("DLG.OUT","<DlgID>")**
Read access of all values is performed with **VOut("#GET#ALL#VALUES#")** or **VVar("DLG.OUT","#GET#ALL#VALUES#")**
Everything else is identical to **DLGOUT**

Special feature: deleting is performed with **'#DELETE#ALL#VALUES#'**

```
DLGSND="#DELETE#ALL#VALUES#"
sDlg=scrDeleteItems(sDlg,"EGT:GUT|EGT:AUS|EGT:GES")
DLGSND=sDlg
```

See also **DLGOUT**

6.3.1.6 DLGOUT (general data exchange)

If used in read access, the complete content of the script variable **[n#]DLG.OUT** is returned.
[n#] If functions are called recursively, a reference index is added as prefix

Read access of a value is performed with **VOut("<DlgID>")** or **VVar("DLG.OUT","<DlgID>")**
Read access of all values is performed with **VOut("#GET#ALL#VALUES#")** or **VVar("DLG.OUT","#GET#ALL#VALUES#")**

When used in write access, the complete content of the script variable <[n#]DLG.OUT> is **not** deleted if an empty string is assigned. If you assign a DlgID with value (e.g. "TEST=1|"), an already existing entry is replaced. If the entry does not exist, it is added to the existing values.
Deleting **DlgID** with the function **EraseDlgOut("<DlgID>")**
To delete all **DlgID** using the call **EraseDlgOut("#ERASE#ALL#DLG.OUT#")**

6.3.1.7 LSTVARS (general data exchange)

Here, a direct read access is not possible (e.g. MsgBox " LSTVARS " + LSTVARS)
[n#] If functions are called recursively, a reference index is added as prefix
For example: LSTVARS = "LST.FILTER=“ + “MNR=100 & ZUMAN=J"
LSTVARS = "LST.MODE="+"COLNUMSORT=TRUE|DYNAMICFILTER= MNR,MST"

Read access of a value is performed with **VVar("LST.MODE","< COLNUMSORT >")** == “TRUE”
Read access of all values is performed with **VVar("LST.FILTER","#GET#ALL#VALUES#") == “MNR=100 & ZUMAN=J”**

When used in write access, the complete content of the script variable **[n#]LST...** is deleted if an empty string is assigned. During assignment (e.g. LSTVARS = "LST.MODE=" +"COLNUMSORT=TRUE|) the previously set value is completely replaced, i.e. there is no DlgID update.

To delete a "single entry", you use the assignment LSTVARS = "LST.MODE="
To delete all values **LST.xyz** you use LSTVARS = "" or EraseDlgVars("LST.")

Used with - scrFktList - scrFieldChange (DynDlgFieldChange_XYZ) - scrFieldList (DynDlgFieldListe_XYZ)

6.3.1.8 DD_SND (general data exchange)

If used in read access, the complete content of the script variable **[n#]UE:SND** is returned.
[n#] If functions are called recursively, a reference index is added as prefix

Read access is performed with **VVar("UE:SND","<DlgID>")**
Or all values with **VVar("UE: SND","#GET#ALL#VALUES#")**

When used in write access, the complete content of the script variable **[n#]UE:SND** is deleted if an empty string is assigned. If you assign a DlgID with value (e.g. "TEST=1|"), an already existing entry is replaced. If the entry does not exist, it is added to the existing values.

6.3.1.9 DD_RCV (general data exchange)

If used in read access, the complete content of the script variable **[n#]UE:RCV** is returned.
[n#] If functions are called recursively, a reference index is added as prefix

Read access to a value is performed with **VRcv("<DlgID>“)** or **VVar("DD.RCV","<DlgID>")**
Read access to all values is performed with **VRcv("#GET#ALL#VALUES#“)** or **VVar("DD.RCV","#GET#ALL#VALUES#")**

If used in write access, the complete content of the script variable **[n#]DD.RCV** is deleted if an empty string is assigned. If you assign a DlgID with value (e.g. "TEST=1|"), an already existing entry is replaced. If the entry does not exist, it is added to the existing values.

6.3.1.10 SCRVARS (general data exchange)

Is identical to **LSTVARS** implementation
Here, a direct read access is not possible.
[n#] If functions are called recursively, a reference index is added as prefix

For example: SCRVARS = "XXX.FILTER=“ + “MNR=100 & ZUMAN=J"
SCRVARS = "XXX.MODE=" +"COLNUMSORT=TRUE|DYNAMICFILTER= MNR,MST"

Read access to a value is performed with VVar("XXX.MODE","< COLNUMSORT >") == “TRUE”
REad access to all values with **VVar("XXX.FILTER","#GET#ALL#VALUES#") == “MNR=100 & ZUMAN=J”**

If used in write access, no script variable **[n#]xyz...** is deleted if an empty string is assigned. During assignment (e.g. SCRVARS = "XXX.MODE=" +"COLNUMSORT=TRUE|) the previously set value is completely replaced, i.e. there is no DlgID update.
To delete a "single entry", you use the assignment SCRVARS = "XXX.MODE="
To delete all values <XXX.xyz> you use EraseDlgVars(" XXX.")

6.3.1.11 GLOBALVARS (general data exchange)

When used in read access, the complete content of the global variable is returned. (if necessary several rows)
For example: GLOBALVARS = "#X#=" + Item("1","1")+ Item("2","2")
GLOBALVARS = "#Z#=" + Item("A","A")+ Item("B","B")

Read access to a value of a row **GVars("#X#","1") == “1”**
Read access to all values with **GVars ("#Z#“ ","") == “A=A|B=B|”**

If used in write access, no script variable **[n#]xyz...** is deleted if an empty string is assigned.
Seting / saving DD items GLOBALVARS = "#XXX#=" + Item("1","1")
GLOBALVARS = "#XXX#=" + Item("2","2")

Is equal to GLOBALVARS = "#XXX#=" + Item("1","1")+ Item("2","2")
Update GLOBALVARS = "#XXX#=" + Item("2","333")
To delete a DD item in a row, you use the assignment GLOBALVARS = "#XXX#=" + Item("2","")
To delete a "row", you use the assignment GLOBALVARS = "#XXX#=" + ""
!!! IMPORTANT !!! To delete the global memory, you use GLOBALVARS = "#DELETE#ALL#GLOBALVARS#"

6.3.1.12 SYS_IP (IP address of the terminal)

Only read access: IP address of the terminal (according to TNR status = variable otherwise via API function)

6.3.1.13 SYS_DDHEADER (dialog data header)

- Only read access: Dialog data header („DAT=09/17/2017|ZEI=48637|USR=2706|SWZ=S|USR=2706|ID=4|“)

6.3.1.14 SYS_USR (user number)

- Only read access: User number = terminal number (TNR) + 2000 ➔ 2001 .. 2999

6.3.1.15 SYS_TNR (terminal number)

- Only read access: Terminal number (TNR) ➔ 1 .. 999

6.3.1.16 SYS_DAT (terminal system date)

- Only read access: Terminal system date in format ("MM/DD/YYYY") ➔ "09/17/2017"

6.3.1.17 SYS_ZEI (terminal system time)

- Only read access: Terminal system time in format ("NNNNN" = seconds per day) ➔ "43200"

6.3.1.18 SYS_DT (terminal system date/time string)

- Only read access: Terminal system date/time string (current Windows setting) ➔ "17.09.2017 12:00:00"

6.3.1.19 SYS_SCRIPT_DEBUG (terminal script debug window)

- only read access: Terminal script debug window (see section "Script - Debug - Dialog")

6.3.1.20 SYS_NEW_CNR_FR (standard production batch)

- Read only: Generate a lot number for a standard production batch (not suitable for customer-specific batch number assignment).

6.3.1.21 SYS_NEW_CNR_WE (standard goods receipt batch)

- Read only: Generate a lot number for a standard goods-receipt batch (not suitable for customer-specific batch number assignment).

6.3.1.22 SYS_QUEUE_ITEMS (number of QUEUE entries)

- Only read access: Number of QUEUE entries (in spool\ddqueue.dta)

6.3.1.23 SYS_OFFLINE (check OFFLINE / ONLINE)

- Only read access: Check OFFLINE / ONLINE (using hypdm32.dll function)

6.3.1.24 SYS_DEMO (terminal demo mode)

- Only read access: Terminal demo mode active

6.3.1.25 SYS_SCRFCT (terminal script function)

- Only read access: Outputs the current terminal script function

6.3.1.26 SYS_TNRGRP (terminal group)

- Only read access: Outputs the terminal group of the terminal (0 = "" otherwise "xxx")

6.3.1.27 SYS_PING (online PDM command)

- Only read access: an online PDM command is performed "DLG=SCMD;47|" to check if the services run on the server (MIP1 MW-LANT-Server <N>)

6.3.1.28 cFF* (field attributes)

For the dialog control of the dynamic dialog fields **DLGVAR = AddIt("ANR", "" , cFFEnable)**

Note: several field attributes are added as follows **DLGVAR = AddIt("ANR", "" , cFFEnable+"#F")**

(write access is not possible)

`cFFReadOnly`	;#RO	= readonly (field → set attribute READONLY)
`cFFEnable`	;#E	= Enable (enable field)
`cFFDisable`	;#D	= Disable (disable field)
`cFFHide`	;#H	= Hide (hide field)
`cFFVisible`	;#V	= Visible (show field)
`cFFRequired`	;#R	= Required (mandatory field)
`cFFFocus`	;#F	= Focused (focus field)
`cFFBarcode`	;#B	= Barcode (field → set attribute BARCODE)
`cFFHideListBtn`	;#HL	= Hide-List-Btn (hide list List-Btn of an input field)
`other`	;#N	= Nullable (field → set attribute NULL = without input)
	;#C	= Change field caption/text

6.3.1.29 DIR_APP (application directory)

- Only read access:: Application directory (e.g. C:\MPDV\AIP2\ )

6.3.1.30 DIR_SPOOL (spool directory)

- Only read access: spool directory (e.g. C:\MPDV\AIP2\SPOOL\ )

6.3.1.31 DIR_ETC (etc directory)

- Only read access: etc directory (e.g. C:\MPDV\AIP2\ETC\ )

6.3.2 Script functions

6.3.2.1 Availability of script functions

The sections in the following show for each function where the function can be used:

- (UE) Used in user exit in the main application
- (DLG) Used in dynamic dialog

6.3.2.2 VTnr (read value from terminal label)

```
(UE) + (DLG)          VTnr("AKRONYM")
```

Available: (UE) + (DLG)

Read info from the static list of the terminal label (TKENN.LST)

6.3.2.3 VVar(variable , acronym):string

6.3.2.4 Read transfer parameters

```
(UE)                  VVar("UE:PAR","XYZ")
```

6.3.2.5 Info on current machine from list of assigned machines (MNR.LST)

```
(UE)                  VVar("UE:MNR","XYZ")
```

6.3.2.6 Info on current operation from list of running operations (ANR.LST)

```
(UE)                  VVar("UE:ANR","XYZ")
```

6.3.2.7 rsIni(inidatei , sektion, key, default):string

```
(UE) + (DLG)          rsIni("ctaiplay.ini","main","CLASSIC_ONLINE_LAMP","")
```

Read INI file (with automatic writing of <default> if entry is not available)

To delete an entry, you use **scrExecute("DeleteIniKey"**…(see below).

6.3.2.8 wsIni(inidatei , sektion, key, value):string

```
(UE) + (DLG)          wsIni("ctaiplay.ini","main","CLASSIC_ONLINE_LAMP","…")
```

Write INI file.

6.3.2.9 scrUECmd(…):string

```
(UE) + (DLG)          scrUECmd(…)
```

Execution of a PDM command with a file as result

See the note on unique file names for lists on the server in section "6.2.6 Communication interfaces".

```
'*** load cost centers
UE_SND = ""
UE_SND = Item("DLG",    "SYSTEM.CALL" )
UE_SND = Item("PROG",   "custom_list.scr" )
UE_SND = Item("AKTION","kostenst" )
UE_SND = Item("DATEI",  ".\spool\kostenst."+SYS_USR )
UE_SND = Item("FILE",   "kostenst.lst" )
' ------  UE SND  = Item("CMD:CPY", "BINARY" )   ' load binary if required
scrUECmd( UE_SND )
```

6.3.2.10 SYS_SCRIPT_DEBUG

```
(UE) + (DLG)          SYS_SCRIPT_DEBUG
```

Open script debug window dialog. Shows all available variables.

6.3.2.11 SYS_DT

```
(UE) + (DLG)          SYS_DT
```

Date/time stamp. (Example: 01/31/2020)

6.3.2.12 SYS_NEW_CNR_FR

```
(DLG)                 SYS_NEW_CNR_FR
```

Generate a new production batch number (no user exit batch number() support).

6.3.2.13 SYS_NEW_CNR_WE

```
(DLG)                 SYS_NEW_CNR_WE
```

Generate a new goods receipt batch number (no user exit batch number() support).

6.3.2.14 SYS_NEW_CNR_HU

```
(DLG)                SYS_NEW_CNR_HU
```

Generate a new packaging (handling unit) batch number (no UserExitLosnummer() support).

6.3.2.15 AddIt(id,value,attribut)

```
(UE) + (DLG)         AddIt( id,value,attribut )
```

Script function (aip_mpdv-system.scr)

A dialog item in format "ID=VALUE;ATTR" is generated (the attribute is only attached if the third parameter does not equal "").

List of the attributes (see also section on system variables, section "cFF* (field attributes)":

```
cFFReadOnly, cFFEnable, cFFDisable, cFFHide, cFFRequired, cFFFocus,
cFFBarcode, cFFHideListBtn
```

6.3.2.16 Item(id,value)

```
(UE) + (DLG)         Item( id,value )
```

Script function (from aip_mpdv-system.scr)

A dialog item in format "id=value" is generated

6.3.2.17 IncStrDec (int)

```
(UE) + (DLG)         IncStrDec ( int )
```

Script function (from aip_mpdv-system.scr)

Decimal incrementing of an integer string (note: up to 15 digits maximum) e.g. IncStrDec("100") becomes "101"

6.3.2.18 StrFmtRight(Value,Len,char)

```
(UE) + (DLG)         StrFmtRight(Value,Len,char)
```

Script function (aip_mpdv-system.scr)

Right-aligned formatting of a string with fill characters

For example:StrFmtRight("101", 5, "0") becomes "00101"

StrFmtRight("101", 2, "0") becomes "01"

6.3.2.19 MsgPopUp(msg,sec)

```
(UE) + (DLG)          MsgPopUp(msg,sec)
```

Script function (from aip_mpdv-system.scr)

MsgPopUp "Ticket [XYZ] is printed." , "3"

If parameter sec = "" the info dialog must be closed with OK

6.3.2.20 VVar(item,id)

```
(UE) + (DLG)          VVar(item,id)
```

Function to read from script VARS – Items

6.3.2.21 VTnr(id)

```
(UE) + (DLG)          VTnr(id)
```

Function to read from 'TKENN.LST' - items

6.3.2.22 VPar(id)

```
(DLG)          VPar(id)
```

Function to read 'DLG.PAR' – Items
VPar(id) can only be used in user exit **DynDlgInit**

If you open a dialog with script initialization,
REOPEN = TRUE/FALSE is set in **DLG.PAR**

FALSE = first request
TRUE = repeated opening after e.g. DB error

6.3.2.23 VMnr(id)

```
(DLG)          VMnr(id)
```

Function to read 'DLG.MNR' - Items ('DLG.PAR' takes priority)

VMnr(id) is only used in the user exit **DynDlgInit**

6.3.2.24 VAnr(id)

```
(DLG)          VAnr(id)
```

Function to read 'DLG.ANR' - Items ('DLG.PAR' takes priority)

VAnr(id) is only used in the user exit **DynDlgInit**

6.3.2.25 VVAR(„*ANR“,id); VVAR(„*MNR“,id)

```
(DLG)          VVAR(„*ANR“,id); VVAR(„*MNR“,id)
```

Direct access to “DLG.ANR” or „DLG.MNR“ → „DLG.PAR“ is bypassed!!

6.3.2.26 VDlg(id)

```
(DLG)          VDlg(id)
```

Function to read 'DLG.DLG' items

6.3.2.27 VDat(offset)

```
(UE) + (DLG)          VDat(offset)
```

Function to read to current date

in format „MM/DD/YYYY“

with <offset> = "0" = today

with <offset> = "-1" = yesterday

6.3.2.28 VZei(offset)

```
(UE) + (DLG)          VZei(offset)
```

Function to read the current time in format "NNNNN" = seconds since midnight

with <offset> = "0" = now

with <offset> = "-30" = now – 30 seconds

6.3.2.29 GStore(func,filter)

```
(DLG)          GStore(func,filter)
```

Access function to a grid (with FIRST,NEXT,ACTIVE) in 'DLG.GRD'

Note on the selection of grid rows:

Using the instruction DLGVAR = Item("GRD.ROW", “<value>”) in a “DynDlg” user exit, you can make a selection in the current dialog grid. Possible values are:

- “FIRST” first grid row is selected
- “LAST” last grid row is selected
- “PREV” Current display position – 1 (if display position > 1)
- “NEXT” Current display position + 1 (if display position < “LAST”)

With the return value of function GStore(..) you can select as follows:

- “0”.. “X” Position index – without taking into account any sorting
- “#” + “0”.. “X” Display position – taking into account a possible sorting

6.3.2.30 VStore(id)

```
(DLG)          VStore(id)
```

Function for Store **GStore(..)** to read in 'DLG.GRD'

6.3.2.31 SStore(id, value)

```
(DLG)          SStore(id, value)
```

Function for Store **GStore(..)** to write in 'DLG.GRD'

6.3.2.32 AStore(id)

```
(DLG)          AStore(id)
```

Function for Store **GStore(..)** to add (write) in 'DLG.GRD'

6.3.2.33 VOut(id)

```
(DLG)          VOut(id)
```

Function to read the dialog data transferred in user exit **DynDlgInit**

6.3.2.34 VSnd(id)

```
(UE) + (DLG)          VSnd(id)
```

Function to read the dialog data sent in the user exits **DynDlgAfterSend** and **UserExitDynDlgAfterSend**

6.3.2.35 VRcv(id)

```
(UE) + (DLG)   VRcv(id)
```

Function to read the reply returned from the server in the user exits **DynDlgAfterSend** and **UserExitDynDlgAfterSend**

6.3.2.36 EraseDlgOut(id)

```
(UE) + (DLG)   EraseDlgOut(id)
```

Function to delete individual IDs from the dialog string ('DLG.OUT'). Usually in the user exits **DynDlgBeforeSend** and **UserExitDynDlgBeforeSend**

6.3.2.37 EraseDlgVars(id)

```
(DLG)          EraseDlgVars(id)
```

Function to delete SCR-VARS in “LST.“ , “FKT.“ , “DD.“…

6.3.2.38 scrMsgBox(msg)

```
(DLG)          scrMsgBox(msg)
```

The simple message box (only single string in parameter msg) is modal. This means that the script processing stops at this place and waits until OK is pressed. In case of a message box that is automatically closed after x seconds, the script processing is continued. If the parameter "vModal" is additionally set, the script processing waits also in case of automatically closed messages until the message is confirmed or is automatically closed.

6.3.2.39 scrMsgBox(msg)

Display of an info window with text (**msg**)

6.3.2.40 scrMsgBox("3^Hallo")

3^: Display for 3 seconds; message closes automatically

6.3.2.41 scrMsgBox("3|vModal|Caption^Hallo")

Caption: text displayed in the title bar of the message

vModal: the dialog is displayed as a modal dialog window

6.3.2.42 DlgJaNein(caption,msg)

```
(DLG)          DlgJaNein(caption,msg)
```

Display of a query with the options Ja/Nein (Yes/No)

Example:

```
sRes=DlgJaNein("delete advance logon","really delete batch logged on in
advance?")
If sRes="#JA#" Then
  DeleteVLos(sMNR)
End If
```

If the user chooses "No" in the query, the function returns "#2#".

6.3.2.43 DlgJaNeinAbbruch(caption,msg)

```
(UE) + (DLG) DlgJaNeinAbbruch(caption,msg)
```

Message box from script for <Ja/Nein/Abbruch> query (Yes/No/Cancel).

The following values are returned: **#JA#**, **#NEIN#**, **#CANCEL#**

6.3.2.44 VDlg(id)

```
(DLG)          VDlg(id)
```

Function to read 'DLG.DLG' items

6.3.2.45 scrFieldChange

```
(DLG)          scrFieldChange
```

You use this function to link data and text fields.

For example: If you enter a status, the status text can be updated additionally.

For an example, refer to the description of the user exit **DynDlgFieldChange.**

6.3.2.46 scrFieldList

```
(DLG)          scrFieldList
```

You use this function to implement a list selection in the user exit **DynDlgFieldListe**.

LSTVARS LST.xxx are used

„LST.MODE=..|FORCEAREOPEN=TRUE|..“ (only with function scrFieldList)

Effect: Data is loaded from the hard disk. This way, it is possible to display a file that is already in memory after a server comparison.

For examples, refer to the user exit description.

6.3.2.47 scrFieldVAGList

```
(DLG)          scrFieldVAGList
```

deprecated / backward compatibility / use **scrFieldList**

LSTVARS LST.xxx are used

(optional with loading from server)

6.3.2.48 scrFktList

```
(DLG)          scrFktList
```

LSTVARS LST.xxx are used

Function like list selection without dialog

(optional with loading from server)

Transfer of data in **[n#]DLG.OUT** if **LST.FILTER=xyz** + **LST.RET=xyz** are set.

6.3.2.49 scrDDSndRcv(oSnd:AnsiString;var pSnd:AnsiString;var pRcv:AnsiString):integer

```
(UE) + (DLG)   scrDDSndRcv(oSnd:AnsiString;var pSnd:AnsiString;var pRcv:AnsiString):integer
```

Function to send dialog data (see < scrDDSnd[WOErr] >)

for application DLL interface!

Parameters:

oSnd	"original data sent"
var pSnd	"actual data sent" (MST without ";x")
var pRcv	"actual data received" (server result, DDQueue result, ..)

6.3.2.50 scrDDSnd

```
(DLG)          scrDDSnd
```

deprecated / backward compatibility / use **scrDDSndRcv**

The script variable **DD_SND** is used here.

Note: with **EraseDlgVars("DD.")** the old data memory of the variable **DD_SND** can be deleted.

new send parameter **PROCESSDLGEVENT=TRUE**

- Execution via **ProcessDlgEventSend** with local booking of standard events (A_AN, A_TR, A_UN, A_AB, M_MST, P_AN, P_AB, ..) and with active PCC/MDE interfacing with list notification.

New send parameter **$TNR.KEEP_DATETIME=ON**

- Transferred time stamp (DAT/ZEI) is kept

New send parameter **LST_RELOAD=OFF**

- Reload request of the server is ignored.

6.3.2.51 scrDDSndWOErr

```
(DLG)          scrDDSndWOErr
```

sends the BAPI string from DD_SND like scrDDSnd, but does not automatically issue an error that might be returned.

New send parameter <PROCESSDLGEVENT=TRUE>

- Execution via < ProcessDlgEventSend > with ScriptlokalUpdate

6.3.2.52 scrPCCValues(value)

```
(DLG)          scrPCCValues(value)
```

This function sends data to the "pccdll.dll" and therefore to the machine via the respective driver. In combination with the UserExitPccDllToTerminal the implementation of control tasks is possible.

Return of the requested values in (with "DLG=GETVAL|..")

(1) customer system script **UserExitPccDllToTerminal**

(2) In addition, all requested "V:…“ values are sent in an opened dynamic dialog as bar code.
The values transferred can be processed in the **DynDlgFieldExit_ XYZ** (XYZ=DynDlgKennung). (Can be identified via request
"FLD.MOD" = "BARCODE")

6.3.2.53 vbsGetCentralPccID(sFilter)

```
(UE) + (DLG)          vbsGetCentralPccID(sFilter)
```

This function is required if the terminal does not start the PCC.EXE and thus the MDE itself locally, but the PCC.EXE runs at a different location than central MDE.

ctaip.exe V# 8.2.1.35 / pcc.exe V# 7.2.4.3 / mpdv-aip.zip 03.12.2018 / MQTT
Configuration: ctaip.ini → [DLL] → BusDLL=CENTRAL or PCC.EXE

The function searches a PCC ID in the file "**central.tnr.lst**" that matches the filter criterion. By default, the list only includes the machines that are assigned to a PCC/TNR as MDE machine.

Syntax: "TYP=<..>&ID=<..>"

Examples: "TYP=M&ID=MDE100" = PCC ID of the entry found or ""

"FIRST", "LAST" = PCC ID of the first, last entry

6.3.2.54 vbsCentralPCCValues(sCMD,ByVal sPCCID)

```
(UE) + (DLG)          vbsCentralPCCValues(sCMD,ByVal sPCCID)
```

This function is required if the terminal does not start the PCC.EXE and thus the MDE itself locally, but the PCC.EXE runs at a different location than central MDE. In this case, the vbsCentralPCCValues function must be used instead of the scrPCValues function.

ctaip.exe V# 8.2.1.35 / pcc.exe V# 7.2.4.3 / mpdv-aip.zip 03.12.2018 / MQTT
Configuration: ctaip.ini → [DLL] → BusDLL=CENTRAL or PCC.EXE

The function dispatches the command transferred <sCMD> (GETVAL,SETVAL) to the stand-alone-PCC/MDE terminal in combined operation with the PCC/terminal number <sPCCID>.
You can identify the <sPCCID> of an MDE machine using the function "vbsGetCentralPccID("TYP=M&ID=<MNR>")".

6.3.2.55 scrExecDynDlg(dlg,ret,values)

```
(DLG)          scrExecDynDlg(dlg,ret,values)
```

Function for DynDlg Aufrufe (without dialog script)

Parameter **ret** = RETURN is set, if DLG is not available.

6.3.2.56 rsCfg(Sektion,Key,Value)

```
(UE) + (DLG)          rsCfg(Section,Key,Value)
```

Function to read an entry from the file **HyTnrCfg.ini as string.**

The configuration file HyTnrCfg.ini contains an additional 0 for all terminals or 2000+terminal number in the section if the section is to apply to only one terminal.

Example:

```
[Konfiguration 0]
Value = 10

[Konfiguration 2100]
Value = 20
```

The query rsCfg("Configuration", "Value","") at terminal 100 returns the result 20. The result is 10 for all other terminals. A terminal group specific configuration is possible by storing the HyTnrCfg.ini in the terminal group specific subdirectory (e.g. .\hydra\<1>\custom\aip2\tgrp_901\). The 0 must then be used in the section so that all terminals in the group are addressed.

A default value can be transferred in the function parameter "Value", which is returned if the configuration does not exist in the file.

6.3.2.57 scrFileExists(file)

```
(UE) + (DLG)          scrFileExists(file)
```

Cheks if a file is available. If the file exists, the function returns 0.

Example:

```
If scrFileExists(DIR_SPOOL+"test.txt")="0" Then
      'File exists
  End If
```

6.3.2.58 scrFileDelete(file)

```
(UE) + (DLG)          scrFileDelete(file)
```

Delete file (OK ➔ „0“)

Example:

```
scrFileDelete(DIR_SPOOL+"test.txt")
```

6.3.2.59 scrFileCopy(file,newfile)

```
(UE) + (DLG)          scrFileCopy(file,newfile)
```

Copy file (OK ➔ "0")

Example:

```
rc=scrFileCopy(DIR_SPOOL+"mat.lst",DIR_SPOOL+"mat.tmp")
```

6.3.2.60 scrFileRename(file,newfile)

```
(UE) + (DLG)          scrFileRename(file,newfile)
```

Rename file (OK ➔ "0“)

Example:

```
rc=scrFileRename(DIR_SPOOL+"mat.lst",DIR_SPOOL+"mat.tmp")
```

6.3.2.61 GSrce(sFct,sParam)

```
(DLG)          GSrce(sFct,sParam)
```

Access to static DD lists

Example:

```
Dim rc
rc=GSrce("LOAD","FILE="+DIR_SPOOL+"mstat.lst")
rc=GSrce("FIRST","MNR=110")
While rc<>"#EOF#STORE#"
  If bActive Then
    rc=SSrce("HARC:ID","1")
  Else
    rc=SSrce("HARC:ID","0")
  End If
            'Read access: sMST=VSrce("MST")
  rc=GSrce("NEXT","MNR=110"))
  Wend
rc=GSrce("CLOSE","SAVE=TRUE")
```

You will find further information in chapter „6.7.3 How to use the functions GSrce, VSrce“.

6.3.2.62 VSrce(sID)

```
(DLG)          VSrce(sID)
```

Read access to DD list

6.3.2.63 SSrce(sID,sValue)

```
(DLG)          SSrce(sID,sValue)
```

Write access (update)

6.3.2.64 ASrce(sID,sValue)

```
(DLG)          ASrce(sID,sValue)
```

Write access (add)

Example:

```
rc=ASrce("EGR:GUTP","5")
```

6.3.2.65 scrStatusBarMsg(sMsg,sMode,sSec)

```
(UE+DLG)              scrStatusBarMsg(sMsg,sMode,sSec)
```

Output of messages via status bar

6.3.2.66 scrLog(sLine)

```
(DLG)          scrLog(sLine)
```

Write to the log file (spool\script.txt) An exact time stamp is automatically set at the beginning of each line. The call stack of the user exit is at the end of the row.

6.3.2.67 scrReadRemoteFile(remote,local,params)

```
(DLG)          scrReadRemoteFile(remote,local,params)
```

Function to read a file from the server

Example:

```
r1 = scrReadRemoteFile("./spool/tnr"+SYS_USR+".rld",DIR_SPOOL+"tnr"+SYS_USR+".rld",
"CMD:LST=DELETE|CMD:CPY=BINARY|")
```

See the note on unique file names for lists on the server in section "6.2.6 Communication interfaces".

6.3.2.68 scrReadCfgFile(ssLstCmd,ssLstFile)

```
(DLG)          scrReadCfgFile(ssLstCmd,ssLstFile)
```

Function to create (DLG=LIST;..) and read a file from the server

Example:

```
rc=scrReadCfgFile("DLG=LIST;11|MOD=A|MNR=M100|ANR=123450010",DIR_SPOOL+"nanr.lst")
```

See the note on unique file names for lists on the server in section "6.2.6 Communication interfaces".

6.3.2.69 scrDeleteItemsInDlgLstFileWithFilter (sFileName,sFilter,sParam)

```
(UE+DLG)              scrDeleteItemsInDlgLstFileWithFilter (sFileName,sFilter,sParam)
```

Deletes entries in a DD list file that match the filtering

Example:

```
scrDeleteItemsInDlgLstFileWithFilter DIR_SPOOL+"lokvlist.lst","ANR="+sAnr,""
```

6.3.2.70 scrMergeDlgLstFileIntoFile (NewItemFile,SourceFile)

```
(UE+DLG)              scrMergeDlgLstFileIntoFile (NewItemFile,SourceFile)
```

Merging of a DD list file to a target file.
New items are created in the target file.

Example:

```
rc=scrMergeDlgLstFileIntoFile(DIR_SPOOL+"u_l_seqlist.lst",DIR_SPOOL+"u_scrap_op.lst")
```

6.3.2.71 scrQuickSearch(sFilename,sFilter)

```
(UE+DLG)              scrQuickSearch(sFilename,sFilter)
```

Searches in a DD list file for the first entry matching the filter.
If the value FIRST is passed as filter, then the first row is returned.

Example:

```
asMnr=scrQuickSearch(DIR_SPOOL+"mnr.lst","MNR="+sMnr)
```

6.3.2.72 scrClearDlgLstFile(sFilename)

```
(UE+DLG)              scrClearDlgLstFile(sFilename)
```

Deletes all rows in a DD list file except the header

Example:

```
scrClearDlgLstFile(DIR_SPOOL+"anr_x.lst")
```

6.3.2.73 scrCreateEmptyFile(sFilename)

```
(UE+DLG)              scrCreateEmptyFile(sFilename)
```

Creates an empty file with size 0 byte

Example:

```
rc=scrCreateEmptyFile(DIR_SPOOL+"data.lst")
```

6.3.2.74 scrMergeDataIntoDlgLstFile(asData, asDlgLstFile,"TRUE")

```
(UE+DLG)              scrMergeDataIntoDlgLstFile(asData, asDlgLstFile,"TRUE")
```

Generates a DD List file or adds a data line with all IDs contained in the DD List header. TRUE forces a reload of the file (e.g. ANR.LST) otherwise only the file itself is extended.

Example:

```
rc=scrMergeDataIntoDlgLstFile("CNR=1234|ATK=100|SLP=5|",DIR_SPOOL+"xmat.lst","FALSE")
```

6.3.2.75 scrGetDlgLstLine(sFilename,sLine)

```
(UE+DLG)              scrGetDlgLstLine(sFilename,sLine)
```

Reads any row in a DD list file (sLine=„1"/ „2".. or „FIRST"/„LAST")

Example:

```
asCnr=scrGetDlgLstLine(DIR_SPOOL+"mat.lst","3")
```

6.3.2.76 scrStr2Real(value):real

```
(DLG)          scrStr2Real(value):real
```

Converts a string 123.256 into a real value.

Background: The values in a list created by the HYDRA server use a decimal point. In a list created by the HYDRA server, floating point values are always formatted with the period as decimal separator. The VBA functions for type conversion use the decimal separator set in the operating system, for example, in Germany, the comma. Therefore, use the function scrStr2Real() to convert the floating point numbers from a list works independently of the operating system settings. Likewise, you should use the function scrReal2Str() described below when writing information in lists or dialog strings. Therefore the function scrStr2Real() should be used when reading values from a list. Conversely, the function scrReal2Str() described below should be used when writing.

Example:

```
rValue=scrStr2Real(VStore("EGR:GUTP"))
```

6.3.2.77 scrReal2Str(value:real):string

```
(DLG)          scrReal2Str(value:real):string
```

Converts a real value 123.256 into a string

Example:

```
rc=SSrce("EGG:GUTS",scrReal2Str(rValue))
```

6.3.2.78 scrDDItem(sID,Values):string

```
(DLG)          scrDDItem(sID,Values):string
```

Identifies an item from a DD string

Example:

```
sCnr=scrDDItem("CNR",asCnr)
```

6.3.2.79 scrStrReplace(Value,OldPattern,NewPattern):string

```
(DLG)          scrStrReplace(Value,OldPattern,NewPattern):string
```

Replaces <old strings> with <new strings> in a string

Example:

```
sNewString=scrStrReplace("Auftrag <ANR> nicht gefunden","<ANR>",sAnr)
```

6.3.2.80 scrEraseDDItem(sID,Values) :string

```
(DLG)          scrEraseDDItem(sID,Values) :string
```

Deletes a DD item from a DD string

Example:

```
sNewString=scrEraseDDItem(asCnr,"DLL")
```

6.3.2.81 scrReplaceDDItem(sID, sItem,Values):string

```
(DLG)          scrReplaceDDItem(sID, sItem,Values):string
```

Replaces a DD item <sID> by <sItem> in a DD string <values>

Example:

```
sNewString=scrReplaceDDItem(asDat,"CNR",SYS_NEW_CNR_FR)
```

6.3.2.82 scrReplaceAllDDKennung(sVor,sNach,sValues,sNoCnvIDs)

```
(DLG)          scrReplaceAllDDKennung(sVor,sNach,sValues,sNoCnvIDs)
```

Replaces all DD items of <sValues> mit Präfix <sVor> and Suffix <sNach> and leaves <sNoCnvIds>

Example:

```
asDat=scrReplaceAllDDKennung("V.",".N","DLG=XX|MNR=1|X=3|ID=5|","DLG|ID")
```
result string = "DLG=XX|V.MNR.N=1|V.X.N=3|ID=5|"

6.3.2.83 scrGetPart(sString,sSeparator,sIndex)

```
(DLG)          scrGetPart(sString,sSeparator,sIndex)
```

Returns a substring of <sString> with separator <sSeparator> with index <sIndex>

Examples:

```
scrGetPart("R|W|Q","|","1")   ➔  "R"

scrGetPart("R|W|Q","|","3")   ➔  "Q"

scrGetPart("R|W=T|Q=Z","=","2")     ➔  " T|Q"
```

6.3.2.84 scrPosStr(ssSubString,ssString)

```
(DLG)          scrPosStr(ssSubString,ssString)
```

Checks if a substring **ssSubString** is contained in **ssString** .

Example:

```
sItem=scrPosStr("DLG=","XXX=100|DLG=12|..")
```
➔ “DLG=12|..“

6.3.2.85 scrLosnummer(sParam,sMnr,sAnr)

```
(DLG)          scrLosnummer(sParam,sMnr,sAnr)
```

Function for the customer-specific generation of batch numbers
Internally calls the user exit **UserExitLosnummer.**

Param:

CNR->TYP= **cmFertigung**,cmWarenEingang,cmVerpackung

Example:

```
sCnr=scrLosnummer("CNR->TYP=cmVerpackung","","")
```

6.3.2.86 scrStoreUpdate(sMode,sID,sValue)

```
(DLG)          scrStoreUpdate(sMode,sID,sValue)
```

Fuction for a local update of the list files ANR.LST and MNR.LST in user exit **UserExitLocalMnrAnrUpdate**

Explanation <sMode> = “READ“ <sID> = “XYZ“
→ reads value from DD list

<sMode> = “ADD“ <sID> = “XYZ“ <sValue> = “10“
→ adds <10> to value in DD list

<sMode> = “UPDATE“ <sID> = “XYZ“ <sValue> = “ABC“
→ updates value in DD list to <ABC>

Examples and further information, see user exit **UserExitLocalMnrAnrUpdate**

6.3.2.87 scrTranslate(Text,Data)

```
(DLG)          scrTranslate(Text,Data)
```

Function to translate texts to other languages.

Example:

Text: "The password of the person [<PNR>]<n>runs on<PWD:VALIDTG>. day(s)."

Data: “RET=0|KT=|LT=|INFO=3645|PNR=44444444|PWD:VALIDTG=1|ID=152|“

Notes: The placeholders <XYZ> are replaced from "Data".
<n> = line feed + <t> = tabulator

6.3.2.88 scrWriteRemoteFile(local,remote)

```
(DLG)          scrWriteRemoteFile(local,remote)
```

Function to write a local file to the server

See the note on unique file names for lists on the server in section "6.2.6 Communication interfaces".

Example:

```
Dim sDBFileName,sLocalFileName,asRet
sDBFileName=scrGetInfo("HydraPath","")+"spool\barcodes."+SYS_TNR
sLocalFileName=DIR_SPOOL+"barcodes.lst"
asRet=scrWriteRemoteFile(sLocalFileName,sDBFileName)
If scrDDItem("RET.OK",asRet)="TRUE" Then
  ' File was successfully transferred
End If
```

6.3.2.89 scrProcessQuickReportPrinterForDialog (dlg,data)

```
(DLG)          scrProcessQuickReportPrinterForDialog (dlg,data)
```

Enables printing a label without sending the assigned dialog

Example:

```
rc=scrProcessQuickReportPrinterForDialog("U_ETK",asPrint)
```

6.3.2.90 scrProcessQuickReportPrinter (dlg,data,ret)

```
(DLG)          scrProcessQuickReportPrinter (dlg,data,ret)
```

Function to print a configured label with transfer of the return value of the PDM command <RET>

6.3.2.91 scrExecuteQuickReportPrinter(params,file)

```
(DLG)          scrExecuteQuickReportPrinter(params,file
```

Function to print a file in RPB format. This function is used by default in the function "Label reprint (EV_NDRUCK)".

6.3.2.92 vbsFolderExists(sFolder)

```
(DLG)          vbsFolderExists( sFolder )
```

Example see <aip_mpdv-system.scr>

Checks if the directory exists -> OK = "0"

6.3.2.93 vbsFolderCreate(sFolder)

```
(DLG)          vbsFolderCreate( sFolder )
```

Creates directory -> OK = "0"(exists) or "1"(has been created)

6.3.2.94 vbsFileExists(sFile)

```
(DLG)          vbsFileExists( sFile )
```

Checks if file exists -> OK = "0"

6.3.2.95 vbsCreateFolderTree(sFolder)

```
(DLG)          vbsCreateFolderTree(sFolder)
```

Creates a directory tree

6.3.2.96 vbsValidateFolder(sFolder)

```
(DLG)          vbsValidateFolder(sFolder)
```

Identify directory string with closing "\" and placeholders <DIR_APP> + <DIR_SPOOL>.

6.3.2.97 scrWriteDataIntoFile(asData,asFile)

```
(DLG)          scrWriteDataIntoFile(asData,asFile)
```

Attaches a data string to a file or creates it if it does not exist.

Example:

```
Dim sHeader,rc
sHeader="MNR=Maschine|ANR=Auftrag|KNR=Kartennummer|"
rc=scrFileDelete(DIR_SPOOL+"u_pnr.lst")
rc=scrWriteDataIntoFile(sHeader,DIR_SPOOL+"u_pnr.lst")
```

6.3.2.98 scrAddAction(sAction,Param,Data)

```
(UE) + (DLG)          scrAddAction(sAction,Param,Data)
```

Saves an action that is processed in the main loop of the application (CTAIP).

Example:

```
rc=scrAddAction("mtaDIALOG","DLG=AUTO:CA_WL_RS|..","MNR=100|..")
```

➔ saves an action to automatically open a dialog

Further notes:

- with Data = Item("ANR+MNR","RELOADED.WITH.VALUES")

the start dates ANR/MNR row are identified using the parameters

In the example above, MNR=100 is used irrespective of the currently selected machine row to start the script dialog.

Note: this function is only possible when you use script dialogs!

- scrAddAction("#STATE#","#BASIC#","") returns the number of actions and the execution status

- „0|0“ = no action / no action / dialog open

- „1|1“ = 1 action / Action/Dialog open

IMPORTANT: You can only transfer data to the dialog if the user exit **USEREXITButtonClick** is implemented in the script.

6.3.2.99 GVars(id,item)

```
(UE) + (DLG)          GVars(id,item)

Saving data in the script with GLOBALVARS = „ABC=XYZ=1|…“
```

Global buffer of variables

Example:

```
GLOBALVARS="DATA=DLG=A_AN|MNR=M100|ANR=123450010|KNR=9999|"
sDlg=GVars("DATA","DLG")
sAnr=GVars("DATA","ANR")
```

Every time a dialog is opened via script, the call parameters "#{DLG-ID}#PAR#" are saved.

Additional „#{DLG-ID}#ANRR#“ , „#{DLG-ID}#MNR#“, ..

e.g. xRowID = GVars("#CE_WL_RF#PAR#","ID#ROW")

6.3.2.100 scrEvaluateDuration(sDatB,sZeiB)

```
(UE)          scrEvaluateDuration(sDatB,sZeiB)
```

Creation of a continuous string in the configured format to be displayed in the OP info.

The parameters sDatB and sZeiB are passed in MPDV format (MM/DD/YYYY, Sec. since midnight). The duration since this point in time is calculated.

Example:

```
s=scrEvaluateDuration(scrDDItem("DAT",asDat),scrDDItem("ZEI",asDat))
```

6.3.2.101 scrFormatDuration(sSeconds)

```
(UE)          scrFormatDuration(sSeconds)
```

Formats a duration in seconds and uses the specified format (industrial time unit, if required) for the display in the OP info

6.3.2.102 scrFormatTimeStamp(sDat,sZei)

```
(UE)          scrFormatTimeStamp(sDat,sZei)
```

Formats a time stamp in MPDV format for the display in the OP info (hh:mm dd.mm.yyyy)

6.3.2.103 scrUrlDownload (scheme,user,password,host,port,url_path,loc_path,prot_path:AnsiString):integer;

```
(UE) + (DLG)          scrUrlDownload
(scheme,user,password,host,port,url_path,loc_path,prot_path:AnsiString):integer;
```

Load files via URL download

Example:

```
iRes=scrUrlDownload("hydra","hydadm","hydadm","win2003-3","10403",
"\hydra724\dncfiles\H10007410750.pdf",".\spool\Temp.pdf",".\spool\prot_ev.txt")
```

6.3.2.104 Ret=scrUrlDownload2(Path,FileName)

```
(UE) + (DLG)          Ret=scrUrlDownload2(Path,FileName)
```

Simplified call of URL download:

Path: using this string, the download parameters are read from „Paths.lst“

6.3.2.105 scrUrlUpload (scheme,user,password,host,port,url_path,loc_path,prot_path:AnsiString):integer;

```
(UE) + (DLG)          scrUrlUpload
(scheme,user,password,host,port,url_path,loc_path,prot_path:AnsiString):integer;
```

Copy files via URL upload (same syntax as scrURLDownload)

6.3.2.106 scrSplitOrder(sAuftrag)

```
(UE) + (DLG)          scrSplitOrder(sAuftrag)
```

Returns all separate IDs for an order number (ANR, AUNR, AFOLG, AGNR, UAGNR, SPLNR)

Example:

```
asDat=scrSplitOrder("123450010")
```

ANR=123450010|AUNR=12345|AFOLG=|AGNR=0010|UAGNR=|SPLNR=

6.3.2.107 scrDateTime(Mode:Ansistring):double

```
(UE) + (DLG)          scrDateTime(Mode:Ansistring):double
```

Function to read the TickCount/Now ➔ Result = DOUBLE

- "TC" = provides the current time in seconds since the program has been started.
- "TCMS“ = provides the current time in milliseconds since the program has been started.
- "TCSYS" = provides the current time in seconds since the computer has been started.
- "TCSYSMS" = provides the current time in milliseconds since the computer has been started.
- "DTMS" = Time in milliseconds since 30 December 1899
- "", "DT" = Time in seconds since 30 December 1899

6.3.2.108 scrGetInfo(Fkt,Param:string):string;

```
scrGetInfo(Fkt,Param:string):string;
```

```
(UE) + (DLG)          scrGetInfo(Fkt,Param:string):string;
```

This function requests data from the terminal.

6.3.2.109 GetPLock

```
scrGetInfo("GetPLock","MASCH100")= "J"/"N"
```

Prod. lock active?

6.3.2.110 HasShift

```
scrGetInfo("HasShift","MASCH100")= "J"/"N"
```

Machine has shift?

6.3.2.111 GetAllOrdersOfMachine

```
sAnrLst=scrGetInfo("GetAllOrdersOfMachine",<Maschine>)
```

Returns orders logged on to machine separated by commas.

6.3.2.112 GetParallelOrders

```
sAnrLst=scrGetInfo("GetParallelOrders",<order>)
```

Returns all orders logged on in parallel to the same machine including the order transferred (separated by comma).

6.3.2.113 GetDefaultPerson

```
scrGetInfo("GetDefaultPerson","MNR=4711")
```

Specified person to get dialogs (only if HoldPersonInfo=on is set)

6.3.2.114 GetDlgBufferValue

```
scrGetInfo("GetDlgBufferValue","DLG=@ACTIVE|AKRO=ANR")
```

Read value from dialog (set values with scrSetData("SetField"..)

6.3.2.115 GetPathData

```
scrGetInfo("GetPathData","PATH="+sPath+"|FILE="sLoadDateiName+"|EXT=")
```

Query of path data → SCHEME=FTP|USR=TK|PWD=123...

6.3.2.116 GetSelected

```
scrGetInfo("GetSelected","")
```

Returns selected machine and order → MNR=..|ANR=..

6.3.2.117 GetProductionLock

```
scrGetInfo("GetProductionLock","4711")
```

Query production lock of a machine → „J“/„N“

6.3.2.118 GetGridLineWithFilter

```
scrGetInfo("GetGridLineWithFilter","ATK=12345")
```

```
Only (DLG)
```

Read a line of the dialog grid specified by the filter.

6.3.2.119 GetGridData

```
scrGetInfo("GetGridData","-1")
```

```
Only (DLG)
```

Read active row of the dialog grid

```
asGrid=scrGetInfo("GetGridData","DLG=@FIL=DLG=A_AN|LINE=-1")
```

Extension by filter dialog

6.3.2.120 GetBatchMode

```
scrGetInfo("GetBatchMode","MNR=4711")
```

Read batch mode of a machine →

lmNormal,lmLos,lmDurchlauflos,lmChargenVerw,lmUser1,lmUnknown

6.3.2.121 IsDlgFieldVisible

```
scrGetInfo("IsDlgFieldVisible","DLG=@ACTIVE|AKRO=MNR")
```

Only (DLG)

Query if a field is visible in the dialog → Y/N

6.3.2.122 GetDDlgEntries

```
scrGetInfo("GetDDlgEntries","TNR=100|DLG=GEB_DRU|TGRP=42")
```

Read all configured dialog fields of a dialog

6.3.2.123 GetOrderData

```
scrGetInfo("GetOrderData","ANR=TK0000000010")
```

Reads order row from file anr.lst, vlist.lst or from server (nanr.lst)

6.3.2.124 GetButtonCaption

```
scrGetInfo("GetButtonCaption","DLG=@ACTIVE|FKT=FKT=WG1")
```

Read button text from a dynamic dialog (reference via FKT)

6.3.2.125 GetTimeDiff

```
scrGetInfo("GetTimeDiff","D1=11/04/2018|T1=48744|D2=11/04/2018|T2=5200
0")
```

Specification of the number of seconds between two time stamps

6.3.2.126 GetTimeStamp

```
scrGetInfo("GetTimeStamp","")
```

Returns the time stamp of the last MDE query in the format yyyymmddhhnnss

6.3.2.127 GetMachineData

```
scrGetInfo("GetMachineData","MNR=4711")
```

Reads the complete row from the machine list

→ MNR=4711|MGRP=122|MBEZK=..

```
scrGetInfo("GetMachineData","MNR=4711|AKRO=MST")
```

Reads a specific value from the machine list

6.3.2.128 GetSelectedGridData

```
scrGetInfo("GetSelectedGridData","LIST3")
```

Reads the complete data row of the selected row from the local lists.

MNR – selected machine

ANR – selected order

LIST3 – 3. list (material, person, resource)

6.3.2.129 IniSectionExists

```
scrGetInfo("IniSectionExists","INIFILE=ctaiplay.ini|SECTION=CE-Scan-
Liste")
```

Check if a section in the INI file is included (“0”→Yes / “-1”→No)

6.3.2.130 GetScriptStack

```
scrGetInfo("GetScriptStack","")
```

Returns stack of user exits - to analyze recursive calls

6.3.2.131 scrSetData (set data)

```
scrSetData( funktion, params):string
```

These function sets data in the terminal or in the dialogs.

6.3.2.132 SetFocusToField

```
scrSetData("SetFocusToField","DLG=@ACTIVE|AKRO=EGR:PRB|RED=1")
```

Set focus in the active dialog to the field "EGR:PRB" and color the field red

6.3.2.133 PressButton

```
scrSetData("PressButton","DLG=@ACTIVE|RCODE=0")
```

Press keys of dialog from script ' RCODE=0->OK 1->Cancel

```
scrSetData("PressButton","DLG=@ACTIVE|AKRO=NEXT")
```

Press keys of dialog from script → Selection of button via ID

6.3.2.134 SetField

Setting content of an input field

```
scrSetData("SetField","DLG=@ACTIVE|AKRO=CNR|VALUE=1000000000")
```

Setting colors

```
rc=scrSetData("SetField","DLG=@ACTIVE|AKRO=CNR|FONT.COLOR=clLime|CAPTI
ON.FONT.COLOR=clLime")
```

FONT.COLOR: Font color of the field content

CAPTION.FONT.COLOR: Color of description (Caption)

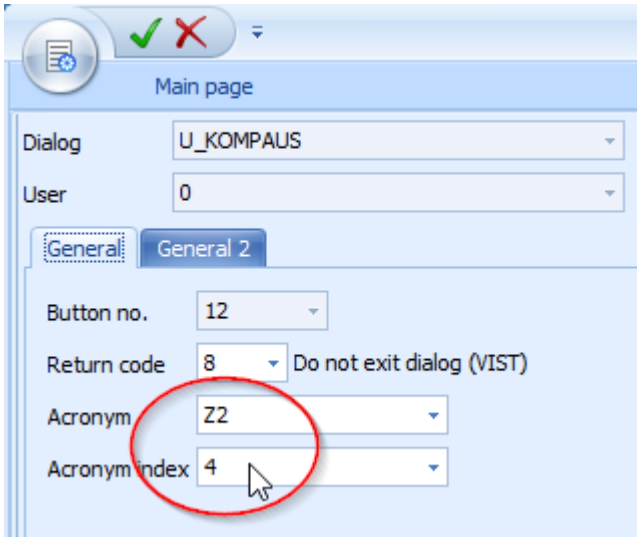
Note: The colors cannot be set with all field types.

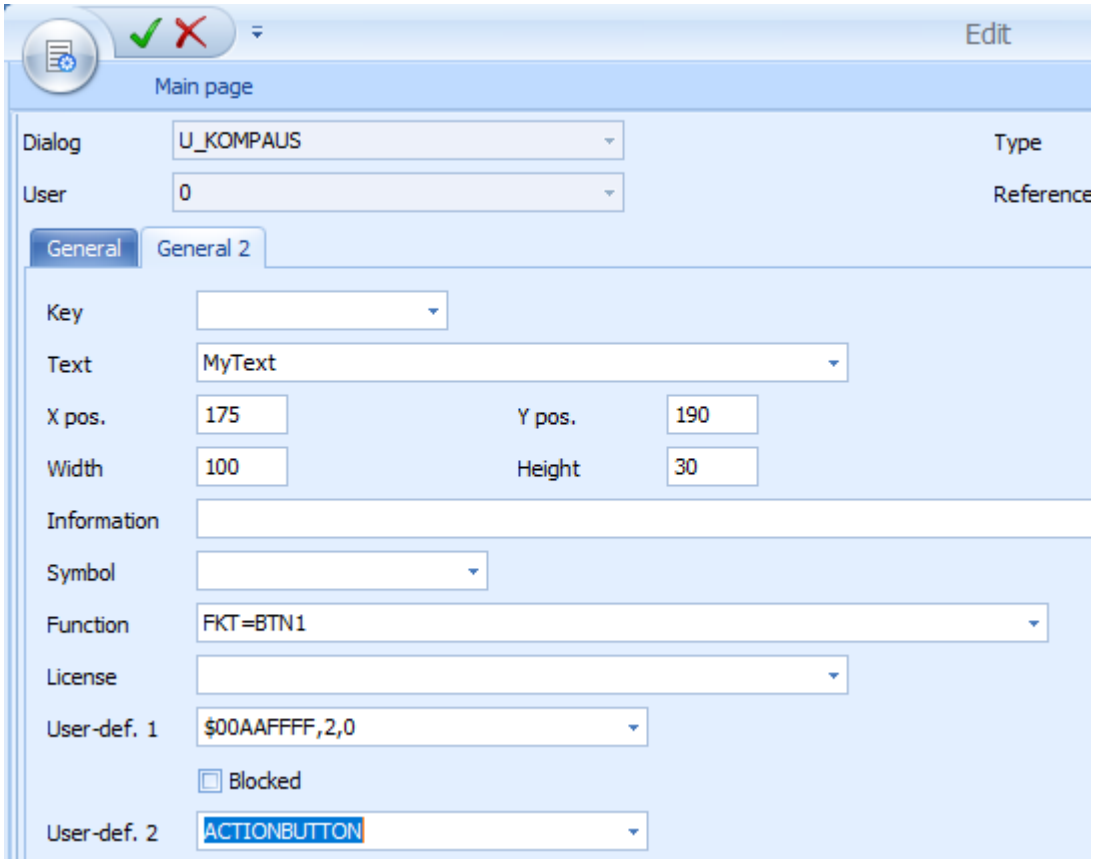
Setting labeling of function keys

For buttons of type "ACTIONBUTTON" in the configuration.

```
rc=scrSetData("SetField","DLG=@ACTIVE|AKRO={ButtonAcronym}[:{ButtonAcr
onymIndex}|BUTTON.CAPTION=XXX")
```

Example:






```
rc=scrSetData("SetField","DLG=@ACTIVE|AKRO=Z2:4|BUTTON.CAPTION=MyNewCaption")
```

Using this function, you can also change buttons in the button bar of the dialog (as of AIP version 8.2.0.36).

6.3.2.135 SetFocusToButton

```
scrSetData("SetFocusToButton","DLG=@ACTIVE|RCODE=0")
```

Set focus to a key that produces the specified return code.

Standard return codes:

- RCODE=0=>OK
- RCODE=1 => Cancel

6.3.2.136 SetButtonVisible

```
scrSetData("SetButtonVisible","DLG=@ACTIVE|FKT=OK|ACTION=HIDE")
```

Show/hide button, ACTION=SHOW/HIDE/READ/TGL

ACTION=DISABLE/ENABLE is also possible.

6.3.2.137 ButtonClick

```
scrSetData("ButtonClick","CA_WL")
```

Triggers clicking a button in main program (ctaip) (the acronym transferred is identical to the entry in ctaipbut.ini or to the identifier in the XML GUI).

6.3.2.138 ProtIntoFile

```
rc=scrSetData("ProtIntoFile","PROTFILE="+sProtDatei+"|MSG="+sMessage)
```

Logging of messages in any file in the directory spool (time stamp is automatically put in front).

6.3.2.139 SelectData

```
rc=scrSetData("SelectData","MNR=4711|ANR=12345")
```

Select machine and/or order on the terminal

6.3.2.140 SetProductionLock

```
rc=scrSetData("SetProductionLock","MNR=4711|ACTIVE=1|")
```

Set/release production lock of a machine

6.3.2.141 DelayedButtonClick

```
rc=scrSetData("DelayedButtonClick","CA_WL")
```

Delayed triggering of a button click. The button is clicked when the timer event is released in the main timer.

Advantage: If the button click opens a dialog, the script processing continues in the background.

```
rc=scrSetData("DelayedButtonClick","CA_WL|FORCEDIALOG=ON")
```

The dialog is repeated until "OK" is pressed.

```
rc=scrSetData("DelayedButtonClick","DLG=@ACTIVE|RCODE=0")
```

Click button with delay in the dialog via RCODE=0

```
rc=scrSetData("DelayedButtonClick","DLG=@ACTIVE|FKT=FKT=SEND")
```

Triggers specified function of the dialog with delay

```
rc=scrSetData("DelayedButtonClick","CA_WL|CLOSE_ALL_DLG=ON|")
```

Before triggering the dialog, all opened dynamic dialogs are closed

```
rc=scrSetData("DelayedButtonClick","DLG=@AINFO|BTN.FKT=AI_CLOSE")
```

Click button in the OP info

(also for DLG=@MINFO)

```
rc=scrSetData("DelayedButtonClick","DLG=@ACTIVE|KENN=LOS_MELDEN")
```

The button with KENNUNG=LOS_MELDEN in the dialog is clicked with delay.

6.3.2.142 UpdateGrid

```
rc=scrSetData("UpdateGrid","DLG=@ACTIVE|GRD.ACCESS=TRUE|DLG.GRID=REOPE
N")
```

Reread grid of a dynamic dialog

```
rc=scrSetData("UpdateGrid","DLG=@ACTIVE|GRD.ACCESS=TRUE|DLG.GRID=RELOA
D")
```

Updates list of server and rereads the grid of a dynamic dialog (is only supported if it is a SCRIPT-GRID or WF-GRID and is configured "CMD".

```
rc=scrSetData("UpdateGrid","DLG=@ACTIVE|GRD.ACCESS=TRUE|GRD.FILTER=LEV
EL=1|GRD.ORDER=ART|")
```

Dynamic dialog with grid: reset filter

- GRD.FILTER=<ALL> (no filtering)

- GRD.ORDER=ART (specify sorting)

```
rc=scrSetData("UpdateGrid","DLG=@FIL=DLG=CE_ASW_RF|GRD.ACCESS=TRUE|GRD
.FILTER=ABKZ=N & ATK="+VDlg("ATK")+" & SLP=0002;00001")
```

SLP=0002;00001 → Semicolon is an OR conjunction

```
rc=scrSetData("UpdateGrid","DLG=@ACTIVE|SHOW=0")
```

Here, the grid of a dynamic dialog can be hidden (SHOW=1 to show)

(as of AIP version 8.2.0.35)

6.3.2.143 AddListFileLine

```
rc=scrSetData("AddListFileLine","FILE=mat.lst|CNR=123|ZLO=....")
```

Entry of a new row in list file

6.3.2.144 DelayedDialogFunction

```
rc=scrSetData("DelayedDialogFunction","DLG=@ACTIVE|FLD.COL=CNR,clRed")
```

Color field with delay.

Advantage: If the field is directly colored, it could be overwritten during event processing.

```
rc=scrSetData("DelayedDialogFunction","DLG=@ACTIVE|FLD.VAL=CNR,100")
```

Set field value with delay.

```
rc=scrSetData("DelayedDialogFunction","DLG=@FIL=DLG=CA_WL_MPL|DLG.FOCUSED.FLD=ATTR:10")
```

Focus field with delay.

```
rc=scrSetData("DelayedDialogFunction","DLG=@ACTIVE|SCFKT=DNC_REOPEN_GRID")
```

Delayed call of a script function (scrSetData("ExecFunction",...))

6.3.2.145 LocalUpdate

```
rc=scrSetData("LocalUpdate","TYP=ANR,MNR|DLG=A_TR|MNR=4711|ANR=TK11111
10010|EGR:GUT=5|EGR:AUS=1")
```

The tables (ANR, MNR) specified under TYPE are locally updated using the event transferred.

6.3.2.146 TriggerLoopStop

```
rc=scrSetData("TriggerLoopStop","MODE=ONETIME")
```

Execute UserExitMainInitLoopStop once

6.3.2.147 DisableReload

```
rc=scrSetData("DisableReload","PNR,ANR")
```

Deactivate cyclic loading of lists

→MNR, ANR, PNR, MSTAT, HZTYP, AGRD, LPKZ, BPOS, NCOM, LICENSE, ZLO, TPE, PATHS, DNC_FAM, IOP_RQ, AART, MAT, RES, TNRDATA

6.3.2.148 DialogStartTime

```
rc=scrSetData("DialogStartTime","Elapsed")
```

prevents message 'Dialog is open for more than 5 min'

6.3.2.149 ActivateSetupFunction

```
rc=scrSetData("ActivateSetupFunction","DisableAllOperationFilters")
```

Deactivate filter of order list in the main view (all OPs of all machines are then displayed that are included in the list anr.lst) (functionality is only available in old AIP GUI).

6.3.2.150 DeleteLine

```
rc=scrSetData("DeleteLine","File=List.lst|Line=5")
```

Deletes a row in a list. Only the file operation is performed. The GUI is not updated.

6.3.2.151 ResetBatch

```
rc=scrSetData("ResetBatch","CNR=PR..")
```

Resetting of the last batch number that has been generated automatically using the function SYS_NEW_CNR_FR

6.3.2.152 SetMaxParallelOrders

```
rc=scrSetData("SetMaxParallelOrders","20")
```

Increasing the maximum number of OPs that are permitted to be run at the same time at a machine with parallel make-to-order production (OPs with different partitioning).

If you use this function, errors might occur because of data strings that are too long!!

6.3.2.153 AddListFileColumn

```
rc=scrSetData("AddListFileColumn","FILE=vlist.lst|AKRO=SEL|VALUE=")
```

Add column to a list file

6.3.2.154 ExecFunction

```
rc=scrSetData("ExecFunction","DLG=@ACTIVE|FKT=REFRESH")
```

Call a script function of a dialog (in DynDlgFunctions_.. the function should be implemented)

6.3.2.155 UpdateTextView

```
rc=scrSetData("UpdateTextView","DLG=@ACTIVE|AKRO=LOC:NOTE|ACTION=CLEAR
")
```

Access to a TextView of a dynamic dialog via its ID. Possible calls: ACTION=REOPEN/ RELOAD/ CLEAR

6.3.2.156 SetFieldVisible

```
rc=scrSetData("SetFieldVisible","DLG=@ACTIVE|AKRO=EGI:GUT|FKT=HIDE")
```

Hide, show, enable, etc. field of a dialog.

Values for FKT:

SHOW → Field is visible

HIDE → Field is not visible

TOGGLE → Toggle visible<->invisible

ENABLE → Field allows input

DISABLE → Field becomes ReadOnly

6.3.2.157 DeleteListFileLine

```
rc=scrSetData("DeleteListFileLine","FILE=mat.lst|FILTER=ATK=1234")
```

Deletes all rows in a LST file that match the filter criterion.

Return values: 0:OK / -1:file not found /

-2:no data rows available / -3:no data matching the filter

6.3.2.158 DelayedDlgSelectLine

```
rc=scrSetData("DelayedDlgSelectLine","DLG=@ACTIVE|AKRO=ATK|VALUE="+sAT
K)
```

Selects the first row in the dialog grid that matches the filter criterion. The function is performed with the next timer run, also if the current script function has been completed.

```
rc=scrSetData("DelayedDlgSelectLine","DLG=@ACTIVE|AKRO=EINTNR|VALUE=00
044|SWITCH_ALWAYS=TRUE")
```

SWITCH_ALWAYS=TRUE → if the row that is already active is found during selection, then the active column is at least changed to trigger a CellChange event.

NOTFOUND=SELECTFIRST / NOTFOUND=SELECTLAST

Selects the first or the last row if the filter has no result. If the first or the last row of the grid is generally selected, then the function is faster if AKRO is empty.

6.3.2.159 DeleteGridLine

```
rc=scrSetData("DeleteGridLine","DLG=@ACTIVE")
```

delete current row in the dialog grid

6.3.2.160 ReopenMainGrid

```
rc=scrSetData("ReopenMainGrid","MNR,ANR,LIST3")
```

Locally reread the specified grids in the main view (no reload from server).
This function is only required in mode XML-GUI=OFF

6.3.2.161 ProcessMessage

```
rc=scrSetData("ProcessMessage","INIT=300|TEXT=Ausgangsloswechsel")
```

Display of SplashScreen that is also shown when the terminal program is booted or when lists are reloaded.

INIT=xxx: opening the window with the specified height

```
rc=scrSetData("ProcessMessage","TEXT=-----------------")
```

TEXT=...: adding a text row (the rows are added at the bottom and disappear at the top edge of the window)

```
rc=scrSetData("ProcessMessage","INIT=END")
```

INIT=END: closes dialog.

6.3.2.162 SetGridAutofilter

```
rc=scrSetData("SetGridAutofilter","DLG=@ACTIVE|CAPTION=Strukturfilter|
FONT=ARIAL|SIZE=8|FOCUS=1")
```

Configuration of the auto filter field in the dynamic dialog

CAPTION: Alternative text for “Filter”

FONT/SIZE: setting for label and edit field

FOCUS=1: set focus of dialog on the auto filter field in the grid

```
rc=scrSetData("SetGridAutofilter","DLG=@ACTIVE|TEXT=Artikel-Filter")
```

The text of the auto filter field of a grid in the dynamic dialog can be overwritten.

```
rc=scrSetData("SetGridAutofilter","DLG=@ACTIVE|ACRO=AUNR")
```

The acronym (filter field) of the auto filter field of a grid in the dynamic dialog is reset

6.3.2.163 SetMessageDelayTime

```
rc=scrSetData("SetMessageDelayTime","TIME=1")
```

On the AIP, the default display time of a message in the status row (top right) is 10 sec. Use this command to change the time. The previously valid default time is returned. It is best to use this time for reset directly after the message.

Example:

```
Function StatusBarTimedMsg(sMsg,sTyp,sTime)
  Dim rc,sDelay
  sDelay=scrSetData("SetMessageDelayTime","TIME="+sTime)
  rc=scrStatusBarMsg(sMsg,sTyp,"1")
  sDelay=scrSetData("SetMessageDelayTime","TIME="+sDelay)
  StatusBarTimedMsg=sDelay
End Function
```

6.3.2.164 ActivateMainButton

```
rc=scrSetData("ActivateMainButton","PANEL=MNR|BTN=VNR_AN,VNR_AB|ACTIVE
=-1")
```

Activate/deactivate keys in the main view of the AIP

(only in mode XML-GUI=OFF)

Values for ACTIVE:

Value	Meaning
-1	disable
1	enable
-2	hide
2	show

6.3.2.165 SetFocusToGrid

```
rc=scrSetData("SetFocusToGrid","DLG=@ACTIVE|FOCUS=FILTER")
```

Set focus of dialog on the grid

FOCUS=FILTER → Filter field

FOCUS=GRID → Table area of the grid

6.3.2.166 scrExecute(...)

6.3.2.167 WinExec

```
rc=scrExecute("WinExec","SW_SHOWNORMAL|""c:\Programme\TextPad
4\TextPad.exe"" """+DIR_SPOOL+"druck.000""")
```

Start an external application program

Show parameter: https://docs.microsoft.com/de-de/windows/win32/api/winuser/nf-winuser-showwindow

6.3.2.168 WriteBufferToFile

```
rc=scrExecute("WriteBufferToFile",sPrnFileName+"|"+asDat)
```

Writing an ansi string in a file

6.3.2.169 CloseDynamicForm

```
rc=scrExecute("CloseDynamicForm","DLG=TA_CAB_SOND|MNR=ENTGRAT5|CloseAc
tive=1")
```

Close a dynamic dialog

6.3.2.170 RequestReload

```
rc=scrExecute("RequestReload","MNR,ANR,PNR")
```

Request reloading of lists.

The list is only reloaded during the next run of the timer. In the script, the result cannot be waited for.

Possible parameters:
MNR,ANR,PNR,MSTAT,HZTYP,AGRD,LPKZ,BPOS,NCOM,PAINT,DLOSE,PPARAM,LOKVLIST,YSR,LICENSE,ZLO,TPE,CAQ_SEND,CAQ_RECV,QMS_TIMER,PROC_INT,PATHS,DNC_FAM,IOP_RQ,AART,SKAL,MAT,RES

6.3.2.171 ResetRequestReload

```
rc=scrExecute("ResetRequestReload","MNR,ANR,PNR")
```

Reset reload request of lists

6.3.2.172 RunWithAttachedPrg

```
rc=scrExecute("RunWithAttachedPrg","FILE=c:\mpdv\aip2\spool\Infos.doc"
)
```

The file transferred is started using the application that is linked to the file extension in Windows.

The complete path of the target file must be specified. Also network paths are allowed. Internet links do not work.

6.3.2.173 RunWithAttachedPrg2

```
rc=scrExecute("RunWithAttachedPrg2","FILE=c:\mpdv\aip2\spool\Infos.doc
|OPERATION=open")
```

Alternative function that also works on the terminal server. All internet links are here possible.

In the case of printable files, "OPERATION=print" triggers immediate printing on the default printer.

(uses ShellExecute)

6.3.2.174 ShowVirtKeys

```
rc=scrExecute("ShowVirtKeys","DLG=@ACTIVE|VISIBLE=0")
```

Hide/show virtual keyboard

6.3.2.175 CheckQueue

```
rc=scrExecute("CheckQueue","")
```

This function tries to empty the queue. In the offline case, the offline timeout is not waited for. The terminal tries to send all records one after the other. If the function is successful, the function returns the value "0". If records remain in the queue, the return value matches the number of records with a minus sign put in front.

6.3.2.176 ChangeExtension

```
rc=scrExecute("ChangeExtension","FILE=C:\data\file.ctw|NEW=.dat|Change
File=1|DeleteExisting=1")
```

Change file extension (e.g. from **file.ctw** to **file.dat**)

ChangeFile=1: file is changed (otherwise only the changed name is returned)

DeleteExisting=1: if the target file already exists, this file is replaced.

6.3.2.177 DeleteIniKey

```
rc=scrExecute("DeleteIniKey","FILE="+DIR_APP+"hcc_data.ini|SECTION=xxx
xxx|IDENT=xxxx")
```

Deleting a key in an INI file

6.3.2.178 GetQuotedDDItem

```
xV=scrExecute("GetQuotedDDItem",<ID>+"|"+<VALUES>)
```

Identifies an item from a dialog string with masked DD items.

Example: VALUES = „<MNR=m1|PARAM=MNR=pM1\|ANR=pA1|ANR=a1>“

Call <ANR|…> == „a1“

Call <PARAM|..> == „MNR=pM1|ANR=pA1|“

6.3.2.179 MakeQuotedDDItem

```
xV=scrExecute("MakeQuotedDDItem",<xID>+"|"+<xVALUE>)
```

Creates a masked dialog string item

Example:

```
scrExecute("MakeQuotedDDItem","DATA|DLG=A_AN|MNR=M100|ANR=123450010|KNR=9999")
```

➔ `DATA=DLG=A_AN\|MNR=M100\|ANR=123450010\|KNR=9999\||`

6.3.2.180 scrDeleteItems(asData,asAkros:string):string

```
(UE) + (DLG)          scrDeleteItems(asData,asAkros:string):string
```

Deleting acronyms from a dialog string

Example: (UserExitDynDlgBeforeSend)

```
Dim sDlg
sDlg=VDlg("#GET#ALL#VALUES#")
DLGSND="#DELETE#ALL#VALUES#"
sDlg=scrDeleteItems(sDlg,"EGT:GUT|EGT:AUS|EGT:GES")
DLGSND=sDlg
```

6.3.2.181 vbsTranslateDataValues(Items , Values)

```
(UE) + (DLG)          vbsTranslateDataValues( Items , Values )
```

Implementing in aip_mpdv-system.scr

Translates the values of the passed <items> into a dialog data string <values>

Examples:

```
s=vbsTranslateDataValues("MSTTXT","..|MSTTXT=#MST1|..")
      -> "..|MSTTXT=PRODUKTION|.."

s=vbsTranslateDataValues("S1|S2","..|S1=#S1|S2=#S2|.." )
      -> "..|S1=Spalte1|S2=Spalte2|.."
```

6.3.2.182 scrComportDataWrite(string):string

```
(UE) + (DLG)          scrComportDataWrite(string):string
```

With <HYREADER.DLL>, connection of external reader to write data to external reader with the ID <DATA2WRITE>. Relevant IDs to identify external readers are <COM> and <TYP>

- <COM> is preferred -> Data is only written using the specified COMPORT

- if only <TYP> is specified, the data is transferred to all instances of the <TYP> to write.

Example:

```
rc=scrComportDataWrite("TYP=DRV_CX_CVERIFY|COM=4|VERIFY=ON|CQ.MINPASS=3|")
```

6.3.2.183 scrComportEventResult(string):string

```
(UE) + (DLG)          scrComportEventResult(string):string
```

With <HYREADER.DLL>, connection of external reader to write an event result to an external reader with the ID <RET> and <RET.TXT> including a description in text form. Relevant IDs to identify external readers are <COM> and <TYP>

- <COM> is preferred -> Data is only written using the specified COMPORT

- if only <TYP> is specified, the data is transferred to all instances of the <TYP> to write.

6.3.2.184 scrGWCUpdateResult(string):string

```
(UE) + (DLG)          scrGWCUpdateResult(string):string
```

Only available for **UserExitOnGatewayData**

If the DD value <FT_ERROR> is set, the respective result is set for the calling external program.

See section (Using < scrGWCUpdateResult > / < UserExitOnGatewayData >)

6.3.2.185 scrForceDirectories(DIR_SPOOL+"prnlay\")

```
(UE) + (DLG)          scrForceDirectories(DIR_SPOOL+"prnlay\")
```

Create a directory structure

6.3.2.186 scrLizenz(lizenz:String):boolean

```
(UE) + (DLG)          scrLizenz("AIP-MF")
```

Function to test a license

Return values: true / if license is active
false / if license is not active

Example:

```
If scrLizenz("MPL-SNR") Then
   '..
  Else
   '..
End If
```

6.3.2.187 Notes on the script functions

Using < scrGWCUpdateResult(..) / UserExitOnGatewayData

You may only execute this application callback function in the user exit **UserExitOnGatewayData.** You use this function with customer-specific gateway events. The function passes the result of events with the ID **FT_ERROR** and sometimes also additional information on the error with the ID **FT_ERROR_TXT** to the calling external program.

The standard processing of a gateway event is that the event is sent to the server with the addition <..|EVCOM=J|..>.

Possible error codes **FT_ERROR** with the default error description **FT_ERROR_TXT**:

FT_ERROR String identifier	**Integer value**	**Default description of FT_ERROR_TXT**
fteOK	0	OK
fteTnrTmOt_NO_PLAUSI	1	NO_PLAUSI (TimeOut: TNR <-> DB)
fteDB_PLAUSERROR	2	PLAUSERROR (DB)

fteTNR_OFFLINE	3	OFFLINE (TNR-QUEUE)
fteTNR_Busy	4	TNR_BUSY (Process runs)
fteTIMEOUT_GW_TNR	5	TIMEOUT (GateWay <=> TNR)
fteGW_TNR_notINIT	7	CFG: GateWay -> TNR not init
fteTNRnotREADY	8	TNR not Ready for Process
fteTNR_GW_notCONFIG	9	CFG: TNR -> GateWay not init
fteDEFAULT_FT_ERROR	50	DEFAULT FT ERROR (is set if value of <FT_ERROR> cannot be identified)
fteDLG_UNDEF	90	unknown dialog
fteNO_DATA_TO_SND	99	No data sent available
fteNO_VALID_DATA_FORMAT	100	Invalid data format
fteCLIENTSOCKET_GETDATA_EXCEPTION	101	FCT_EXCEPT (exception in processing function)
fteCLIENTSOCKET_GETDATA_UNDEF	102	FCT_UNDEF (undefined processing function)
fteTNR_MNR_NOT_CFG	900	MDE->MNR not config
fteWNR_NOT_CFG	901	ANR->WNR not runs
fteMNR_MST_NOT_CFG	902	MNR->MST not config
fteANR_MNR_NOT_RUNS	903	MNR->ANR not runs
fteAUSGRD_NOT_CFG	904	EGG:AUS not config or empty
fteBARCODE_LEN_ERROR	905	Undefined Barcode length [valid: 0,13,16]
fteMNR_MST_NOT_POSSIBLE_PSP_ACTIVE	906	MNR->MST not possible - PSPerre active
ftePLOCK_COUNT_OFF	907	P-Lock active without counting
fteMNR_KEINE_SCHICHT	950	Machine "no shift"
#DONE#	---	After calling the callback function **scrGWCUpdateResult(..)** in user exit **UserExitOnGatewayData**, you must use the instruction UE_RET = Item("FT_ERROR", "#DONE#") to set **FT_ERROR** to the value **#DONE#** so that the standard processing is not run. The notify events are still sent to the modules –DLL's (caq72.dll, pzezks72.dll).

If **FT_ERROR** has been set using the string identifier or the respective integer value in the user exit **UserExitOnGatewayData** ,the standard processing is not run any more (exception: the notification events of the modules).

(1) Example ➔ setting negative <FT_ERROR> via application (not with application callback scrGWCUpdateResult())

>>> Send string:

<DLG=EV_MST|MELDZEI=43200|MELDDAT=03/05/2018|BEARB=KFS|MNR=M000002|MST=1|CLI.SND.T=10:51:35.746|>

<<<< Receive string:

<DT:TNR=2,7970000170|FT_ERROR=906|FT_ERROR_TXT=MNR->MST not possible - PSPerre active (.An MDE->MNR >M000002< production lock is active. Maschine status change is not permitted / UserExitOnGatewayData) [21]|COM.ID=4@|DLG=EV_MST|

MELDZEI=43200|MELDDAT=03/05/2018|BEARB=KFS|MNR=M000002|MST=1|CLI.SND.T=10:51:35.746|TNR=17|DT:CLI=3,0000000726|>

(2) Example ➔ Set positiver FT_ERROR=0 via application callback scrGWCUpdateResult())

>>> Send string:

<DLG=EV_MST|MELDZEI=43200|MELDDAT=03/05/2018|BEARB=EVCOM|MNR=M000002|MST=1|CLI.SND.T=12:07:45.011|>

<<< Receive string:

<DT:TNR=0,2499999013|FT_ERROR=0|FT_ERROR_TXT=OK (.EV_MST verarbeitet / CallBack / scrGWCUpdateResult) [0]|COM.ID=5@|

DLG=EV_MST|MELDZEI=43200|MELDDAT=03/05/2018|BEARB=EVCOM|MNR=M000002|MST=1|CLI.SND.T=12:07:45.011|FT_MODE=WAIT;2; 150|TNR=17|DT:CLI=0,3590002656|>

Other control identifier and variable that is relevant for the gateway processing: **FT_MODE:**

Empty „“	is equal to <NORMAL> ➔ after setting of result, there is not break in the processing. → The ClientServerThread sends the result, but the GateWay event processing in the terminal is not completed until the Windows queue of the application has been run through. → For example, if after the call < scrGWCUpdateResult > a server communication is performed, the terminal cannot receive a new GateWay command until this action is completed.
SLEEP;2;150	➔ after setting the result, the application is stopped **2** times for **150** msec. (with **2** MessageBeep) → Default for the number <**2**> is 1. The default for the duration <**150**> is 200 MSec. → For notes on the processing, see < **Empty „“** >
WAIT;2;150	➔ after setting the result, the application is stopped **2** times for **150** msec. (with **2** MessageBeep and processing of the Windows queue using *ProcessMessages*) → The ClientServerThread sends the result and the GateWay processing in the terminal is completed. The terminal is therefore available for a new GateWay event.

General notes:

- Only one gateway event can be processed. (display in application)

<<< Result in calling program **FT_ERROR=4|FT_ERROR_TXT=TNR_BUSY (Process runs) [Client Event just runs] [4]|….**

6.3.3 Working with numbers

The following must be observed when working with numbers in a terminal script:

There are different VB script engines of the Internet Explorer (that we use). When comparing <, >, … , some script engines identify whether a variable is integer/floats, which variables are to be compared, and then convert them automatically.

Some versions do not identify this, and may compare the variable contents as strings. For this reason, it is always recommended to make an explicit type cast in case of comparisons.

Using conversion functions:

from / to	Int	String	Real
Int	-	**CStr(i)**	-
String	**vbsIntDef(s,"0")**	-	**scrStr2Real(s)**
Real	*Rounded:* **CInt(r)** *Truncated:* **Fix(r)**	**scrReal2Str(r)**	-

6.3.4 Using "IF" queries

The terminal scripts execute all comparisons with "IF" queries including "AND" conditions.

For example, this leads to a runtime error in the following instruction if the variable sInt is an empty string or is not convertible:

Example:

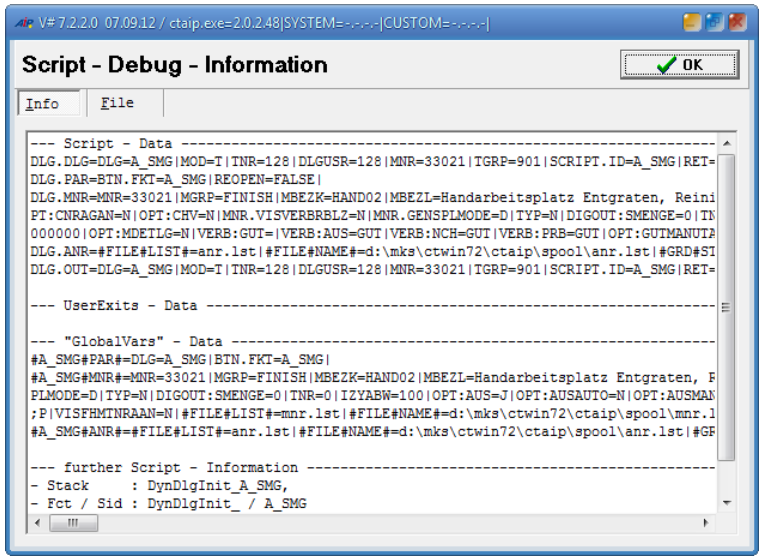
```
Sub ...
  Dim sInt
  ...
  sInt = VDlg("FU:32")     ' *** identify string
  If IsNumeric( sInt ) And Int( sInt ) = 4 Then
    ...
  Else
    ...
  End If
```

```
' *** possible procedure without run time error
  If IsNumeric(sInt) Then
    If Int(sInt) = 4 Then
      ...
    End If
  Else
    ...
  End If
  ...
```

6.3.5 Debugging

6.3.5.1 Script - Debug – Dialog

Using the function **SYS_SCRIPT_DEBUG,** the script debug information is displayed as follows at runtime in an additional dialog.



Tab "**Info**" includes the following information.

- „**Script - Data**“: Data/variables of the current/last DIALOG
- „**UserExits - Data**“: Data/variables of the current/last user exit
- „**GlobalVars - Data**“: Global variables of the application
- „**further Script - Information**“: Information on the current script status.
  - Script call stack, i.e. name of the function that is executed
  - Script function + Dialog ID
  - Counter for active dialog and user exit functions

Tab "**File**" includes the following information:

- „**Script - File - Infos**“: Information in tabular form on the scripts currently loaded

- „**Script - File - Overview**“: Short information on the scripts loaded + zip files.

- „**Script - Methods - Overview**“: Information on the functions available in the system + dialog – script files of the scripts currently loaded.

6.3.5.2 Exception - Script – Dialog

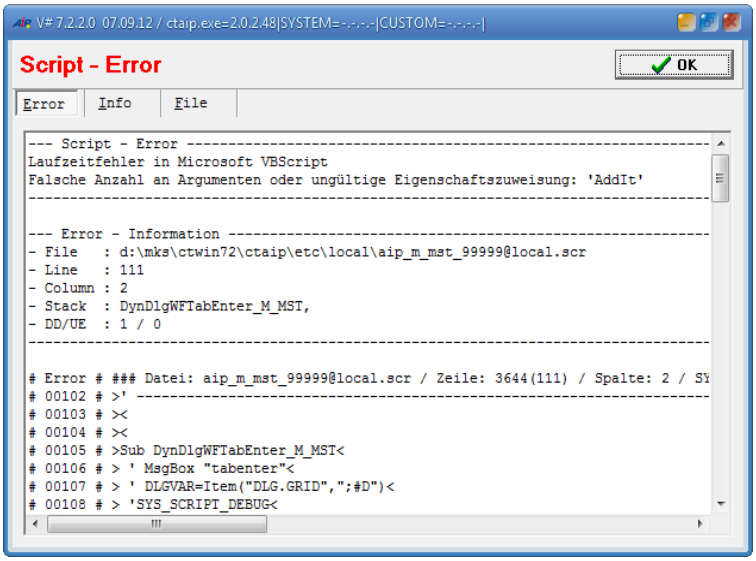
In addition to the information described in section "**Script - Debug – Dialog**", this dialog also includes the tab "**Error**" that displays the following data:

- **Script-Error**: Description of the error occurred

- Error information: extended information on the error:
  - Script file (with error)
  - Row in script file (with error)
  - Column in script file (with error)
  - Script call stack, i.e. name of the function that is executed
  - Counter for active dialog and user exit functions

- A script excerpt where the error is marked.

- The script file that includes the error.



6.4 USEREXIT in the system script

Die Implementierung von kundenspezifischen Userexits wird in einer kundenspezifischen „System“-Script-Datei (aip_system_<KD>.scr) durchgeführt. Im kundenspezifischen Script können Script-Funktionen aus dem Standard System-Script (aip-mpdv-system.scr) verwendet werden.

For information on the storage and naming of the system script, refer to the sections "Storage structure of scripts" and "Naming conventions".

6.4.1 UserExitInitLosnummer

Functionality:

You use this user exit to change the length of the standard batch number locally on the AIP2.

Implementation notes:

If the value **LEN:CNR** is not set, the batch number length specified in the basic settings is used and not changed. Using the user exit, the default batch number lenth is changed for production batches (e.g. PRxxxxxxxx), goods receipt batches (e.g. WExxxxxxxx) and handling units (HUxxxxxxxx).

Note: The batch number length in the basic settings should always be set to a value greater than the length used in the user exit so that the input fields on the client are displayed with the appropriate length.

Example: the batch number length is fixed and set to 10 digits.

```
Sub UserExitInitLosnummer
  UE_RET = Item("LEN:CNR", "10")
End Sub
```

6.4.2 UserExitLosnummer

Functionality:

This user exit implements a customer-specific generation of batch numbers.

Implementation notes:

Observe the handling of dynamic dialogs and the abort functionality (e.g. in case of an output batch change).

If a dialog function is canceled, an UNDO is usually performed for the number range. The currently assigned batch number is reset to its original value. This can only work if the dialog is closed and then sent, and if send is not performed in the dialog script itself (e.g. using OK). To avoid the problem, the mode UNDO can be used in this user exit (see example below).

Available functions	Description
`VTnr("XYZ")`	Info from list TKENN.LST
`VVar("UE:PAR","XYZ")`	Transfer parameters
`VVar("UE:MNR","XYZ")`	Info from list MNR.LST for the current machine
`VVar("UE:ANR","XYZ")`	Info from list ANR.LST for the current order
`rsIni(ini,Sektion,Key,Default)`	Read INI file (with automatic writing of <default> if entry is not available)
`wsIni(ini,Sektion,Key,Value)`	Write INI file

Input parameters:

Parameter	Value	Description
MODE	GENERATE	Generate batch number
CNR.FIX.TYP	cmWarenEingang	Create goods receipt batch
CNR.FIX.TYP	cmFertigung	Create production batch
MODE	UNDO	Perform Undo. With RET=0, the standard undo function is skipped.

Return parameters:

Parameter	Value	Description
UE_RET	CNR	Generated batch number (e.g. CNR=NNNNNNNNN)

Example: generation of a customer-specific batch number for production batches. The elements of the number are read from an INI file.

```
Sub UserExitLosnummer
  Select Case VVar("UE:PAR","MODE")
    Case "GENERATE"
      '-------------------------------------------------------------------------
      '-- Modi = "sCnr.FIX.TYP" = "cmWarenEingang", "cmFertigung"
      '-- if no  "sCNR=NNNNNNNNN" in <UE_RET> is defined
      '-- the standard batch number generation is used
      '-------------------------------------------------------------------------
      OnGenerate
    Case "UNDO"
      '-------------------------------------------------------------------------
      '      '-- if a "RET=0" in <UE_RET> is set
      '-- the standard batch number undo function is skipped
      '-------------------------------------------------------------------------
      UE_RET = Item("RET", "0")
  End Select
End Sub

Sub OnGenerate
  Dim sLfd, sCnr
  If VVar("UE:PAR","sCnr.FIX.TYP") = "cmWarenEingang" Then
    UE_RET = Item("RET", "DEFAULT")
  Else
    sLfd = IncStrDec( sLfd )
    sLfd = rsIni( "u_losnr.ini", "Losnummer", VTnr("TNR")+"->sLfd", "0" )
    sLfd = wsIni( "u_losnr.ini", "Losnummer", VTnr("TNR")+"->sLfd", sLfd )
    sLfd = wsIni( "u_losnr.ini", "Losnummer", "sCnr->UNDO->TNR->sLfd", sLfd )
    sCnr = "2"+VTnr("TNR") + StrFmtRight( sLfd, 5, "0" )
    sCnr = wsIni( "u_losnr.ini", "Losnummer", "sCnr->UNDO->TNR->sCnr", sCnr )
    UE_RET = Item("sCnr", sCnr )
    UE_RET = Item("RET", "0" )
  End If
End Sub
```

6.4.3 UserExitMainInitLoopStop

Functionality:

This user exit is run when the main application is started after initialization (before the terminal switches to the Run - Mode Mainloop), and if necessary in the MainLoop (see example for explanation) and when the terminal is closed.

Input parameters:

Parameter	Value	Description
UE:PAR	Input parameters	Input parameter (VVar("UE:PAR","XXX"))
MODE	INIT	Request after terminal restart
MODE	LOOP	Cyclic request if < LOOPTIME=X > if X > 0 has been set.
MODE	STOP	Is called when the terminal program is closed (manually or remote via terminal status)
LOOPTIME=0		Current LOOPTIME (Default = 0)
MINSTEP=5		Minimum cycle (default=5)
ONETIME=FALSE		Unique <LOOP> call if in debug screen “Reload-Status” (Ctrl+Alt+T) **UserExitMainInitLoopStop** has been activated.

Return parameters:

Parameter	Value	Description
UE_RET	MODE=INIT, LOOP	LOOPTIME=<> Call LOOP after 10 seconds

Example: set loop timer to x seconds

```
Sub UserExitMainInitLoopStop
  Select Case VVar("UE:PAR","MODE")
    Case "INIT"
      scrLog(" UserExitMainInitLoopStop = PAR ( "+VVar("UE:PAR","#GET#ALL#VALUES#")+" )")
      ' --- after program start -> call LOOP after 5 seconds
      UE_RET = Item("LOOPTIME", "5")
      ' --- NOTE: if <LOOPTIME> is not set in "INIT"
      scrLog(" UserExitMainInitLoopStop = RET ( "+VVar("UE:RET","#GET#ALL#VALUES#")+" )")
    Case "LOOP"
      ' scrLog(" UserExitMainInitLoopStop = RET ( "+VVar("UE:RET","#GET#ALL#VALUES#")+" )")
      ' --- then -> call LOOP after 10 seconds
      UE_RET = Item("LOOPTIME", "10")
    Case "STOP"
      scrLog(" UserExitMainInitLoopStop = PAR ( "+VVar("UE:PAR","#GET#ALL#VALUES#")+" )")
  End Select
End Sub
```

Note:

The main timer of the terminal program waits in "LOOP" mode until the UserExitMainInitLoopStop is processed. If a longer action is started from here or a message is displayed, the clock stops at the bottom right of the terminal. This also means that the terminal background processes are not processed. User actions should therefore be processed via "DelayedButtonClick", as the processing of this function does not stop the main timer.

6.4.4 UserExitButtonClick

Functionality:

This user exit is used to check the plausibility of a button from the button bar (lower part of the screen, possibly configured via ctaipbut.ini) or the identifier of an OnClick event in the XML interface.

The current machine and order rows are transferred to the user exit.

Implementation notes:

`VVar("UE:MNR","<ID>")`	Access to all data in the machine list mnr.lst
`VVar("UE:ANR","<ID>")`	Access to all data in the order list anr.lst
`UE_RET=Item("BTN.FKT","#FKT#->#EXIT#")`	Terminates functions (no default processing in the terminal programm)
`UE_RET=Item("BTN.FKT","A_UN")`	Overwrites the original function.
`UE_RET=Item("ANR+MNR","RELOADED")`	Before starting a script dialog, order and machine data are re-read.
`UE_RET=Item("BTN.FKT", "SCRIPT->A_UN")`	Instead of the standard implementation in the terminal program a script (here aip_mpdv-A_UN.scr) is used.
`UE_RET=Item("BTN.RET","1")`	Error message "Function not implemented".
`@<function>`	A prefixed @ means that a dynamic dialog is not sought. Instead implementation is performed in the terminal script.
`@@<function>`	@@ at the beginning of the function does not check whether the function was processed in the UserExitButtonClick. Therefore, a return value must not be set → `Item("BTN.FKT","#FKT#->#EXIT#")`

Example:

```
Sub UserExitButtonClick
  Select Case VVar("UE:PAR","BTN.FKT")
    Case "A UN"
      If VVar("UE:ANR","AGNR") = "0051" Then
        scrMsgBox( " (A_UN) bei AGNR = 0051 über Script !\n Script: [ A_UN ] ausführen." )
        UE_RET   = Item("BTN.FKT", "SCRIPT->A_UN")
        UE_RET   = Item("ANR+MNR", "RELOADED" )
      End If
    Case "A AB"
      If VVar("UE:ANR","AGNR") = "0052" Then
        scrMsgBox( " (A_AB) with AGNR = 0052 not possible !\n Function: [ A_UN ] execute." )
        UE_RET   = Item("BTN.FKT", "A_UN")
      End If
    Case "A TR"
      If VVar("UE:ANR","AGNR") = "0053" Then
        scrMsgBox( " (A_TR) with AGNR = 0053 not possible !\n Function is canceled." )
        UE_RET   = Item("BTN.FKT", "#FKT#->#EXIT#")
      End If
    Case "@@TEST"
      ' do something / testing during development
  End Select
End Sub
```

6.4.5 UserExitDynDlgBeforeInitialize

Functionality:

This user exit is called before a dynamic dialog is initialized.

This user exit makes it possible to assign a terminal script for a programmed dialog to map a customer-specific extension.

Example: customizations "Machine-related preassignment of badge number" or "Recording of material consumption".

The user exit also allows you to carry out initializations for all dynamic dialogs, since it is always run through. That means there is no need to create a script for each individual dialog.

Input parameters:

Parameter	Value	Description
DLG.DLG	Dialog data	Complete dialog data for the calling dialog

Return parameters:

Parameter	Value	Description
DLG.OUT	Return data	If you set the following return value, you can use this user exit to prevent that the dialog is opened: **DLGVAR=Item("RET","#CANCEL#")**

Beispiel 1: Dialogsteuerung der Standarddialoge <A_TR> , <A_UN> , <A_AB> mit dem Script <A_VERB_WZB>.

```
Sub UserExitDynDlgBeforeInitialize
    Select Case VOut("DLG")
    ' ----- Recording of material consumption in toolmaking ----
      Case "A_UN","A_AB","A_TR"
        DLGVAR = Item("SCRIPT.ID","A_MENGE_WZB")
    End Select
End Sub
```

Example 2: customer-specific preassignment of the badge number.

```
Sub UserExitDynDlgBeforeInitialize
  DLGVAR = Item("KNR",GVars("SYSTEM","KNR"))
End Sub
```

6.4.6 UserExitDynDlgBeforeSend

Functionality:

This user exit is used to complete or suppress all PDM postings that are not processed in a DIALOG script **DynDlgBeforeSend_XYZ**.

Implementation notes:

You can prevent a PDM posting from being sent by using the following line in the script.

```
DLGSND=Item("EVENT","EVENT_DIALOG_OHNE_SENDEN")
```

Example:

```
Sub UserExitDynDlgBeforeSend
  Select Case VDlg("DLG")
    Case "U_MTZ"
      DLGSND = Item("EVENT","EVENT_ONLINE_OHNE_AUTO_MENGEN")
      DLGSND = Item("MNRTNR.TNR",VDlg("TNR"))
      DLGSND = Item("MNRTNR.MNR",VDlg("MNR"))
      DLGSND = Item("MNRTNR.OPT:TMP","J")
      If VDlg("MODUS") = "Z" Then
      ' -- create new dynamic terminal machine assignment -------
      '  -- DLG=MNRTNR.INSERT|MNR=xxxx|TNR=xxx|KNR=xxxx|OPT:TMP=J ------
        DLGSND = Item("DLG", "MNRTNR.INSERT" )
      Else
      ' -- delete dynamic terminal machine assignment ------------
      '  -- DLG=MNRTNR.DELETE|MNR=xxxx|TNR=xxx|KNR=xxxx ----------------
        DLGSND = Item("DLG", "MNRTNR.DELETE" )
      End If
    Case "U_XYZ"
      ' *** Implementation of further customer-specific actions
  End Select
End Sub
```

6.4.7 UserExitDynDlgAfterSend

Functionality:

You use this user exit to execute customer-specific requirements after a successful PDM posting (see also section "*DynDlgAfterSend_XYZ*").

Example: list update (in main view) after a successful PDM posting.

Input parameters:

Parameter	Value	Description
UE:SND		Complete send string in PDM format
UE:RCV		Return value of the PDM command sent

Return parameters:

Parameter	Value	Description
DD_RCV		Extend return value e.g. reload lists

```
Sub UserExitDynDlgAfterSend
      Select Case VSnd("DLG")
    Case "U_MTZ"
       ' ----- Ex. "Dynamic machine terminal assignment -------------
      DD_RCV  = Item( "LOAD", "MNR,MST,ANR,PNR,"+ VRcv("LOAD") )
    Case "U_XYZ"
      ' *** Implementation of customer-specific actions
  End Select
End Sub
```

6.4.8 UserExitAfterSendError

Functionality:

This user exit is called if the server refuses a posting. The call is made before the error message is displayed on the terminal.

Input parameters:

Parameter	Value	Description
UE:PAR	VVar("UE:PAR","XYZ")	Send string that the terminal has sent to the server
UE:RET	VVar("UE:RET","<ID>")	Return string of the server (error number, error text...) The error number is requested with VVar("UE:RET",“RCV.RET“)

Return parameters:

Parameter	Value	Description
UE_RET	VIEWERROR=FALSE	Can be used to stop display of error message
	#REPEAT_SND#=J	Resend dialog message

Implementation notes:

Here, you can store data in global variables (→GLOBALVARS) that are used when the dialog is reopened.

Example: catch error code from server and open dialog

```
Sub UserExitAfterSendError
  scrLog("UserExitAfterSendError|"+VVar("UE:PAR","#GET#ALL#VALUES#"))
  Select Case VVar("UE:PAR","DLG")
    Case "CA_WL_PA"
      AfterSendError_CA_WL_PA
  End Select
End Sub

Sub AfterSendError_CA_WL_PA
  Dim sRET,sVal
  'sVal=VVar("UE:PAR","*JA_NEIN_CHECK")
  'sRET=VVar("UE:RET","RCV.RET")
  If VVar("UE:RET","U_RET")="7013" Then
    ' Can be used to stop error message
    UE_RET=Item("VIEWERROR","FALSE")
  End If
End Sub
```

Example 2: using the data when a dialog is reopened:

```
Sub DynDlgInit_U_CA_WL
  If VOut("REOPEN")="J" Then
    s=GVars("#RCV#AFTER#SEND#ERROR#","RCV.RET")
  ' ...
```

Example 3: remove a field and immediately re-send data

```
Sub UserExitAfterSendError
  Select Case VVar("UE:PAR","DLG")
    Case "A_P_AN","A_AN"
      AfterSendError_A_AN
    End Select
End Sub

Sub AfterSendError_A_AN
  Dim sRet,sData,sMATCHECK,sDialogText
  sData = VVar("UE:PAR","#GET#ALL#VALUES#")
  sMATCHECK = VVar("UE:RET","MATCHECK")
  If VVar("UE:RET","RCV.RET")="424" and (sMATCHECK = "FALSE") Then
```

```
    sDialogText=rsCfg("DIALOG->TEXT","MATCHECK_TEXT","Nummer speichern?")
    sRet=DlgJaNein(scrTranslate("Auswahl",""),scrTranslate(sDialogText,""))
    If sRet = "#JA#" Then ' JA: Daten erneut senden
          ' here, the items that are to be deleted (U MATCHECK,HALLO,CHECK)
      UE_RCV=Item("#DELETE_ITEM#","U_MATCHECK,HALLO,CHECK")
      UE_RET=Item("VIEWERROR","FALSE") + Item("#REPEAT_SND#","J")
    Else
      UE_RET = Item("VIEWERROR","FALSE")
    End If
  End If
End Sub
```

6.4.9 UserExitLocalMnrAnrUpdate

Functionality:

You use this user exit to update the local MNR.LST + ANR.LST after a successfully performed posting (= event).

Implementation notes:

This user exit is only executed if the ID MNR or ANR exists in the send string and if these are included in the MNR/ANR list.

Available functions	**Description**
`VVar("UE:SND","<ID>")`	You use this function to access the values of the PDM send string sent. Example: PDM send string: DLG=M_MST|MST=2|..| VVar("UE:SND","MST") returns the value "2"
`scrStoreUpdate(sMode,sID,sValue)`	Function to read, write, add up values in DD lists. See also section "Script functions".

Input parameters:

If required, this user exit is requested several times. This depends on the data of the dialog string sent. The different requests have different parameters `UE:PAR=MODE` and are performed in the order specified in the table:

Sequence	Parameter	Value	Description
1	`UE:PAR=MODE`	`MNR->UPDATE`	This user exit is only called with mode MNR->UPDATE if the requested dialog includes the ID "MNR".
2	`UE:PAR=MODE`	`ANR->UPDATE->LAUFEND`	This user exit is only called with mode ANR->UPDATE->RUNNING if the dialog requested includes the IDs "MNR" and "ANR" and if the operation has the status "running" at the machine. (AST_OPT_PKENN=L).
3	`UE:PAR=MODE`	`ANR->UPDATE`	This user exit is only called with mode ANR->UPDATE if the requested dialog includes the ID "ANR".

Example: When an order-related PDM posting has been performed, the ANP list moves to the next item.

```
Sub UserExitLocalMnrAnrUpdate
  Dim sDlgID,sMode,rc
  sMode = VVar("UE:PAR","MODE")
  sDlgID = VVar("UE:SND","DLG.DLGCFG")
  If sDlgID = "" Then sDlgID = VVar("UE:SND","DLG")
  Select Case sDlgID
    Case "U_CA_WL_RF"
      Select Case sMode
        Case "MNR->UPDATE"
          scrLog( " UserExitLocalMnrAnrUpdate ( "+sMode+" )"  )
        Case "ANR->UPDATE->LAUFEND"
          scrLog( " UserExitLocalMnrAnrUpdate ( "+sMode+" )"  )
          If VVar("UE:SND","CNR") <> "" Then
            rc = scrStoreUpdate( "UPDATE","CNR",VVar("UE:SND","CNR") )
            scrLog( " UserExitLocalMnrAnrUpdate ( CNR = "+rc+" )"  )
          End If
        Case "ANR->UPDATE"
          scrLog( " UserExitLocalMnrAnrUpdate ( "+sMode+" )"  )
          If VVar("UE:SND","KDCNR") <> "" Then
            rc = scrStoreUpdate( "ADD","AGR_FU_11","1" )
            scrLog( " UserExitLocalMnrAnrUpdate ( AGR_FU_11 "+rc+" (+1) )"  )
          End If
      End Select
    Case "U_A_UN_RF"
      Select Case sMode
        Case "MNR->UPDATE"
          scrLog( " UserExitLocalMnrAnrUpdate ( "+sMode+" )"  )
        Case "ANR->UPDATE->LAUFEND"
          scrLog( " UserExitLocalMnrAnrUpdate ( "+sMode+" )"  )
          rc = scrStoreUpdate( "UPDATE","AST_OPT_PKENN","U" )
        Case "ANR->UPDATE"
          scrLog( " UserExitLocalMnrAnrUpdate ( "+sMode+" )"  )
          If VVar("UE:SND","KDCNR") <> "" Then
            rc = scrStoreUpdate( "ADD","AGR_FU_11","1" )
          End If
      End Select
    Case "U_A_AB_RF"
      Select Case sMode
        Case "MNR->UPDATE"
          scrLog( " UserExitLocalMnrAnrUpdate ( "+sMode+" )"  )
        Case "ANR->UPDATE->LAUFEND"
          scrLog( " UserExitLocalMnrAnrUpdate ( "+sMode+" )"  )
          rc = scrStoreUpdate( "UPDATE","AST_OPT_PKENN","E" )
        Case "ANR->UPDATE"
          scrLog( " UserExitLocalMnrAnrUpdate ( "+sMode+" )"  )
          If VVar("UE:SND","KDCNR") <> "" Then
            rc = scrStoreUpdate( "ADD","AGR_FU_11","1" )
          End If
      End Select
  End Select
End Sub
```

6.4.10 UserExitEventFinished

Functionality:

You use this user exit to execute customer-specific requirements after a successful PDM posting (= event).

Input parameters:

Parameter	Value	Description
DD.SND	Transmit data	PDM event DLG=A_UN|MNR=<..>|ANR=<..>|…
DD.RCV	Receive data	PDM result RET=0|KT=<..>|LT=<..>|…

Return parameters:

Parameter	Value	Description
UE_RET	---	without processing

Implementation notes:

The script processing of a DB event is structured as follows.

(e.g. for dialog „DLG=A_XYZ|MNR=M100|ANR=1A007..|DLG.DLGCFG=XYZ|…“)

To identify the dialog, you mainly use the item **DLG.DLGCFG.** If **DLG.DLGCFG** is not included, the item **DLG** is used.

The DM event identified may only include the following characters "_" , "A" .. "Z" , "0" .."9" !

Other characters are replaced with the character "_". (e.g. DLG=ADEPRO.ADD ➔ „ADEPRO_ADD“)

1. *DynDlgBeforeSend_XYZ *1*
   UserExitDynDlgBeforeSend ➔ *Case „XYZ“*
   *(if *1 does not exist or with background event)*
2. *DynDlgAfterSend_XYZ *2*
   UserExitDynDlgAfterSend ➔ *Case „XYZ“*
   (if *2 does not exist or with background event)
3. Here, the label printing is performed
4. *UserExitLocalMnrAnrUpdate* ➔ *Case „XYZ“*
   (if available and <MNR>/<ANR> included in DB event)
5. *UserExitEventFinished_XYZ* ➔ *Customer-specific implementation /*
   (if available) / ***aip_system_<project>.scr***
6. *UserExitEventFinished__XYZ__* ➔ *Standard processing*
   (if available) / ***aip_mpdv-system.scr***

This user exit has been realized for the standard MPL processing of coil cutting processes, for example. Here, you can delete the cutting plans once the coil cutting OPs have been logged off or interrupted.

Example:

```
Sub UserExitEventFinished_U_CAWL_RS
  Dim rc,sSnd,sRcv,sCALT20,rc
  scrLog("UserExitEventFinished_U_CAWL_RS")
       sSnd = VSnd("#GET#ALL#VALUES#")
       sSnd = scrReplaceDDItem("DLG","U_CA_WL",sSnd)
       ' set CALT20 (KFB) from received result in send string
       sRcv = VRcv("#GET#ALL#VALUES#")
       sCALT20 = scrDDItem("RET.CALT20",sRcv)
       sSnd = scrReplaceDDItem("CALT20",sCALT20,sSnd)
       rc = vbsUpdateMnrALosListe(sSnd,VRcv("#GET#ALL#VALUES#"))
End Sub
```

6.4.11 UserExitPccDllToTerminal

Functionality:

You use this user exit to process PCCDLL events (e.g. counter, V variables,...)

Input parameters:

Parameter	Value	Description
UE:DAT	PCCDLL-Event-Daten	Complete event data in PDM format. Events of the MDE blade are transferred with the prefix "PCC".

Return parameters:

Parameter	Value	Description
UE_RET	PCCDLL event return data	If you set the acronym **#PCCDATA-MODE#** with the value **#NEW#,** then you can change the return string.

Implementation notes:

Using the function **VVar("UE:DAT","<ID>")** you can access any field of the events.

Using the function **VVar("UE:DAT","","#GET#ALL#VALUES#"),** the PCCDLL event is read.

The data received from the driver can be changed in this user exit before the terminal program processes the data. The following must be set for this:

```
UE_RET = Item("#PCCDATA-MODE#","#NEW#")
```

Additional functions:

```
UE_RET = Item("#PCCDATA-MODE#","#CLEAR#")
```

➔ deletes the data

```
UE_RET = Item("#PCCDATA-MODE#","#EXIT#")
```

➔ exits the distribution function (no processing of the data in the terminal program)

Events of the MDE blade are transferred to this user exit with the prefix "PCC". The following events are transferred from the MDE blade:

Shift change:

PCC.TID=<>|DLG=PCC.A_ASW|MNR=<>|DAT=<>|ZEI=<>|MST=<>|AGR:HUB=..|

Beginning of shift:

PCC.TID=<>|DLG=PCC.A_AAN|MNR=<>|DAT=<>|ZEI=<>|MST=<>|AGR:HUB=..|

End of shift:

PCC.TID=<>|DLG=PCC.A_AUN|MNR=<>|DAT=<>|ZEI=<>|MST=<>|AGR:HUB=..|

Cyclic quantities/status update:

PCC.TID=<>|DLG=PCC.M_AST|MNR=<>|DAT=<>|ZEI=<>|MST=<>|AGR:HUB=..|

Update of counter and display:

PCC.TID=<>|DLG=PCC.COUNTER.UPDATE|MNR=<>|AGR.C:5=<>|AGG.C:5=<>|AGB.C:5=<>|..|
..|AGR:HUB=<>|IZY=<>|PSPERRE=<>|#3
DLG=LIST.UPDATE| #9 FILE@MNR.LST #9 FILTER@MNR=<> #9 ADD@AGR:GUTP=<>|
AGR:GUT=<>|AGR:HUB=<>|SET@IZY=<>| #9 FILE@ANR.LST #9 FILTER@MNR=<>&ANR=<>
ADD@EGS:GUT=<> #9 FILTER@ANR=<> #9 ADD@EGR:GUTP=<>| #9 |TICKCOUNT=<>|

Automatic machine status change

PCC.TID=<>|DLG=PCC.M_MST|MNR=<>|MST=<>|PSPERRE=<>|DAT=<>|ZEI=<>|DT=<>|TICKCOUNT=<>|

Important note: you must always change these automatic events in the MDE blade. You can guarantee a correct data collection with a correct GUI update by doing it this way.

Example 1: catch digital input I:I301 and perform customer-specific action.

```
Sub UserExitPccDllToTerminal
  scrLog("UserExitPccDllToTerminal: "+VVar("UE:DAT","#GET#ALL#VALUES#"))
  If scrPosStr("|I:I301=1|",VVar("UE:DAT","#GET#ALL#VALUES#")) <> "" Then
    '...
  End If
End Sub
```

Example 2: customer-specific integration of 2 balances ("Waage") using number 1/2

```
Sub UserExitPccDllToTerminal
  If VVar("UE:DAT","V:WAAGENR")<>"" Then HandleScales
End Sub

Sub HandleScales
  Dim sWaageNr,sWaageValue
  '-------------------------------------------------------------------------------------------
  ' From the scales, it is always V:BRUTTO=xxxx
' only the scales number changes if the balance changes V:WAAGENR=1 or V:WAAGENR=2
  ' Value of scales 1 (scale 1) writes
  ' Convert value of scale 2 to the correct input field in the dialog
  '-------------------------------------------------------------------------------------------
  sWaageNr  = VVar("UE:DAT","V:WAAGENR")
  sWaageValue = VVar("UE:DAT","V:BRUTTO")
  If sWaageNr="2" Then
    UE_RET = ""
    UE_RET = Item("DLG","GETVAL")
    ' To take over changed data from the script
    UE_RET = Item("#PCCDATA-MODE#","#NEW#")
    ' Scale value, if balance number=2 is set to dialog field of balance 2
    UE_RET = Item("V:WAAGENR",sWaageNr)
    UE_RET = Item("V:EGR:GUT",sWaageValue)
    ' so that the field of scale 1 is not also filled
  End If
End Sub
```

6.4.12 UserExitAutomaticQuantities

This user exit only exist to guarantee downward compatibility.

Note:

You cannot use this user exit to change automatic quantities because the MDE has been moved to a blade and the GUI is updated using the calculated quantities of the blade. Always change automatic quantities in the blade. Otherwise, you cannot ensure that the blade has the current/valid counter readings to monitor functions like "target quantity reached", etc.

6.4.13 UserExitExternalReaderEvent

Functionality:

You use this user exit to process the external ID/bar code readers integrated via < HYREADER.DLL >.

Implementation notes:

The following two callback functions are supported:

`scrComportDataWrite(string):string`

- to write data to external reader

`scrComportEventResult(string):string`

- to write processing result to external reader (e.g. "..|RET=1|RET.TXT=Error->Firmennr|..)

For further information on the structure of bar codes and on bar codes with prefixes, refer to the documentation of the "AIP functions shop floor/machine data".

Available functions	**Description**
`VVar("UE:PAR","<ID>")`	Call parameter with MODE and system/company number
`VVar("UE:BAR","<ID>")`	Bar code data from bar code dispatcher
`VVar("UE:RET","<ID>")`	Return: Processed bar code

Input parameters:

Parameter	Value	Description
`UE:PAR->MODE`	`CALLBACKEVENT`	Call mode with ID/bar code events e.g. VVar("UE:PAR","MODE")
`UE:PAR->MODE`	`CALLBACKSTATE`	Call mode with INFO/WARNING/ERROR messages
`UE:BAR`	`<>`	Complete bar code data string e.g. request with VVar("UE:BAR","#GET#ALL#VALUES#")
`UE:BAR`	`RAWDATA`	Original data string of reader

Return parameters:

Parameter	Value	Description
`UE_RET = Item("RESULT","-1")`	`RESULT=-1`	Bar code is processed, do not pass to application
`UE_RET = Item("RESULT","1")`	`RESULT=1`	(DEFAULT with **special case**) Bar code is processed, do not pass to application Using <HYREADER.DLL> function < ComportEventResult() >, data is written on external reader → value of <RESULT> is copied to <RET> **Special case:** If the ID <KNR> is included in ID/bar code event and no <IDCODE> is included. If the ID <FIR> does **not** match the configured <company number> (<SYSNR> from TKENN.LST), then <IDCODE> is internally set using ID/bar code event IDs <FIR>+<KNR> +<PZ>.
`UE_RET = Item("RESULT","0")`	`RESULT=0`	(DEFAULT) Bar code is passed to application **Special case:** If the ID <KNR> is included in ID/bar code event and no <IDCODE> is included. If the ID <FIR> matches the configured <company number>

		(<SYSNR> from TKENN.LST), then <IDCODE> is internally set using ID/bar code event IDs <FIR>+<KNR> +<PZ>.
UE_RET=Item("SEND-AS-BARCODE","FALSE")	False	(DEFAULT) Standard transfer of the data ➔ value is transferred in a dialog into an active field, for example.
UE_RET=Item("SEND-AS-BARCODE","TRUE")	TRUE	Transfer of data as STD-BARCODE (identification of length, acronym,...)

Example:

```
Sub UserExitExternalReaderEvent
  Select Case VVar("UE:PAR","MODE")
    Case "CALLBACKEVENT"
      OnReaderCallbachEvent
    Case "CALLBACKSTATE"
    '--- Here, the messages INFO/WARNING/ERROR are processed
      ' !!! Implementation !!!
    Case Else
      UE_RET = Item("ACTION", "### "+VVar("UE:PAR","MODE")+" ###")
  End Select
End Sub

Sub OnReaderCallbachEvent
  Dim sEvent,sData
    '--- Here, the ID/bar code events are processed
  sEvent = VVar("UE:BAR","#GET#ALL#VALUES#")
  sData = VVar("UE:BAR","RAWDATA")
  ' !!! Implementation !!! (siehe HINWEISE)
  ' *** !!! bar code processed
  UE_RET = Item("RESULT","-1") ' Barcode verarbeitet->keine weitere Verarbeitung
End Sub
```

6.4.14 UserExitBarcodeToMain

Functionality:

If a barcode is scanned while the terminal is in the basic mask, this user exit is called.

If the termnial program receives a barcode when it is in the basic mask, the barcode is interpreted as machine status. The Change Status dialog opens and the status is preset if the barcode has the appropriate format.

You can use this user exit to call another dialog instead of the Change Status dialog. This could be the dialog *Log Person.*

For further information on the structure of bar codes and on bar codes with prefixes, refer to the documentation of the "AIP functions shop floor/machine data".

Input parameters:

Parameter	Value	Description
UE:BAR	Bar code data	Raw data (bar code as scanned) For example: UE:BAR=BAR=PR3X58G112|LESERTYP=BAR|COM=1|DL

		G=MAIN->FORM|BAR.DLGID=CNR|BAR.VALUE=PR3X58G112|MNR=RW10|ANR=080006830290|
BAR.DLGID	Field ID	Field ID identified via bar code length (e.g. KNR)
BAR.VALUE	Value bar code	Bar code (perhaps without check digit with KNR) – Value passed to the field

Return parameters:

Parameter	Value	Description
UE_RET	Return data	UE:RET=RET=#FKT#->#EXIT#|..“ ➔ prevents the standard processing in the terminal program (processing of the barcode only by the script) ➔ using the function **scrAddAction(),** you can open a dynamic dialog, for example. No return value: The standard processing in the terminal program opens the dialog "Change status" (M_MST)

Example: Open a customer-specific dialog when a batch number has been scanned.

```
Sub UserExitBarcodeToMain
  Dim rc
  ' MsgBox "UserExitBarcodeToMain = "+VVar("UE:BAR","#GET#ALL#VALUES#")
  If VVar("UE:BAR","BAR.DLGID")="CNR" Or Len(VVar("UE:BAR","BAR"))>= 10 Then
    rc = scrAddAction("mtaDIALOG","DLG=U_PACK|",Item("CNR",VVar("UE:BAR","BAR")))
    UE_RET = Item("RET","#FKT#->#EXIT#")
  End If
End Sub
```

6.4.15 UserExitDynDlgBarcode

Functionality:

You can use this user exit to implement or manipulate a customer-specific bar code processing. The call occurs when a barcode is scanned while the dialog is open.

Das Terminalprogramm entscheidet anhand der Länge des Barcodes und anhand der im geöffneten Dialog vorhandenen Felder, zu welchem Feld der Barcode passen könnte. This only works for fields defined in the standard system. If customer-specific fields are to be scanned, a corresponding implementation is required here.

Implementation notes:

Can only be used if the scanner is connected via COM port. A scanner that is looped into the keyboard does not trigger this user exit!

If a bar code must be processed in the main application (without dynamic dialog), you must use the USEREXIT BarcodeToMain.

You only use this user exit to manipulate bar codes in dynamic dialogs.

The bar code passed to the dialog can be manipulated.

For further information on the structure of bar codes and on bar codes with prefixes, refer to the documentation of the "AIP functions shop floor/machine data".

Available functions	Description
`VVar("UE:BAR")`	Transfer parameter from bar code dispatcher (interrupt handler)
`VVar("UE:RET")`	Return: Processed bar code

Available parameters	Description
`VVar("UE:BAR","DLG")`	Dialog ID
`VVar("UE:BAR","LESERTYP")`	Reader type (LESERTYP=BAR,.....)
`VVar("UE:BAR","COM")`	Comport of reader
`VVar("UE:BAR"," BAR.DLGID")`	Input field ID **??=** if no input field can be assigned Otherwise the ID of the field is transferred (e.g. KNR)
`VVar("UE:BAR","BAR.VALUE")`	Bar code read without prefix (e.g. a badge number)
`VVar("UE:BAR","DLG.ALL.FLD")`	All dialog input field IDs separated by semicolon
`VVar("UE:BAR","DLG.FLD")`	ID of the target field identified in the dialog using the standard bar code identification. If no standard bar code is identified (BAR.DLGID=?), then the ID includes the currently focused dialog field.
`VVar("UE:BAR","BAR")`	Default value of the interpreted bar code Ex. CNR=xxxxxxxxx or ??=xxxxxxxxxx
`VVar("UE:BAR","BAR.RAWDATA")`	Original bar code string that has been read
`VVar("UE.DLG","...)`	Complete dialog data of the dynamic dialog before bar code processing (e.g. DLG.FOCUSED.FLD is the focused dialog field)

Return parameters:

Parameter	Value	Description

UE_RET		Return data from user exit
		Example:
		`UE_RET = ""`
		`UE_RET = Item("CNR" , barval )`

Example:

```
Function UserExitDynDlgBarcode
  Select Case VVar("UE:BAR","DLG")
    Case "A_P_AN_MPL","A_AN_MPL","CE_WL_MPL"
      OnBarcode_MPL
    End Select
End Function

Sub OnBarcode_MPL
  Select Case VVar("UE:BAR","BAR.DLGID")     ' fokusiertes Feld
    Case "DLL","CNR","??"
      ' gescannten Wert in das Dialogfeld CNR eintragen
      UE_RET=Item("CNR",VVar("UE:BAR","BAR.VALUE"))
  End Select
End Sub
```

6.4.16 UserExitOnExternOrderListChange

Functionality:

This user exit is called if entries are added or removed when the order list is reloaded from the server. This is the case if the orders are logged on or off from the server by another terminal, MOC or by an automatism.

Input parameters:

Parameter	Value	Description
UE:DAT	<MNR1>=<ANR1>| <MNR2>=<ANR2>| …	Added operations
UE:PAR	<MNR1>=<ANR1>| <MNR2>=<ANR2>| …	Removed operations

Return parameters:

Parameter	Value	Description
UE_RET	---	without processing

Implementation notes:

For these orders, the system can perform customer-specific actions that are also performed if the order postings are directly made on the terminal. (Examples: setting an output signal, sending order data to a machine connection,...).

Example:

```
Function UserExitOnExternOrderListChange
  Dim asLostAGData,res
  If IsCustom_ErfassungOFF Then Exit Function
  ' UE:DAT: OPs added
  ' UE:PAR: OPs removed
  ' Format: <MNR1>=<ANR1>|<MNR1>=<ANR2>|<MNR2>=<ANR3>|<MNR3>=<ANR4>|
  asLostAGData=VVar("UE:PAR","#GET#ALL#VALUES#")
  If asLostAGDat<>"" Then
    res=DeleteOpFiles(asLostAGDat)
  End If
End Function 'UserExitOnExternOrderListChange
```

Reading data can be implemented as follows:

```
  Dim iPos,sEntry,sMnr,sAnr
  iPos=1
  Do
    sEntry=scrGetPart(asLostAGDat,"|",iPos)
      If sEntry="" Then Exit Do
      sMnr=scrGetPart(sEntry,"=",1)
      sAnr=scrGetPart(sEntry,"=",2)
      '*** bearbeite sMaschine, sAuftrag
    End If
    iPos=iPos+1
  Loop
```

6.4.17 UserExitOnGatewayData

Functionality:

This user exit is called with each message that is received via gateway port. There are two types of gateway messages (events):

1) **Notify GateWay-Events:** these messages are immediately identified as received and processed for the calling program. The processing is performed asynchronously in the main timer of the main program.
Do not write any interface results here because a reception confirmation has already been sent.

2) **Standard Gateway-Events:** These messages are processed as soon as the queue of the main program is processed.
Here, a result must be confirmed if the command is not for an active module.

Input parameters:

Parameter	Value	Description
UE:DAT	Gateway Event-Daten	Complete message in PDM format. For example: COM.ID=2@|DLG=KFS_MST|MELDZEI=43200| MELDDAT=03/05/2018|BEARB=KFS|MNR=M00000 2| MST=1|CLI.SND.T=10:03:59.983|
UE_RET	Gateway Event-Return-Daten	Copy of the complete message in PDM format with attached ..|RET=*| for internal processing. For example: COM.ID=2@|DLG=KFS_MST|MELDZEI=43200| MELDDAT=03/05/2018|BEARB=KFS|MNR=M00000 2| MST=1|CLI.SND.T=10:03:59.983|RET=*|

Return parameters:

Parameter	Value	Description
UE_RET	Gateway Event-Return-Daten	If you set the acronym **#DATA#UPDATE#** to the value **TRUE,** then you can change the return string.

Implementation notes:

You can change the return string of the event processing using the following assignment.

UE_RET = Item("#DATA#UPDATE#","TRUE")

As part of a PCC_ADP interfacing, this interface supports the following further options:

- With the parameter "..|**NOTIFY.ERROR.TO.PCC=TRUE|..**", a notification (answer) is sent to the PCC also in case of a server error if configured accordingly.
- With the parameter " ..|**EVENT=EXECUTE-AS-LIST|..**" the command is processed as list request.
- With the parameter " ..|**DATAFORMAT=ANSI|..**" the file loaded from AIP2 in UTF8 format is converted into "ANSI" for the PCC. (if <ANSIFILE=..> has not been specified, the file is converted into <FILE=..> or into ".\evcom.lst" without specification)

- With prefix "**DLG=STORED-EXECUTION:<CMD>|..** " the command is performed asynchronously in AIP2, i.e. the PCC does not wait for the result of the command.

e.g.

„DLG=ABC.REQUEST|FILE=c:\mpdv\aip2\spool\u_data.lst|EVENT=EXECUTE-AS-LIST|NOTIFY.ERROR.TO.PCC=TRUE|DATAFORMAT=ANSI|“

Performs a list request with notification also with server error and converts the requested list into ANSI.

With **DLG= STORED-EXECUTION:ABC.REQUEST|..“** the execution is performed asynchronously in AIP2

For further information, refer to the description of **scrGWCUpdateResult**

Example:

```
Sub UserExitOnGatewayData
  Dim sDlgID,sDATA,sTCMS,rc,sMsg
  sDlgID = VVar("UE:RET","DLG")
  Select Case sDlgID
    Case "U_MST","U_STK","U_HUB"
      sMsg = "#  Event-Verarbeitung für Dialog [ <DLG> ] läuft! Bitte warten ... #"
      sMsg = scrTranslate(sMsg,Item("DLG",sDlgID))
      rc = scrStatusBarMsg(sMsg,"EVMsg","-1")
      sTCMS = scrDateTime("TCMS")
      sDATA = VVar("UE:RET","#GET#ALL#VALUES#")
      If vbsExecuteEvent(sDATA) Then
        scrLog("vbsExecuteEvent(TRUE) "+StrFmtRight(CStr(scrDateTime("TCMS")-sTCMS),8,"0") _
               +" msec<"+sDlgID+">"+sDATA+"<")
        sMsg = "#  Event-Verarbeitung für Dialog [ <DLG> ] beendet! #"
        sMsg = scrTranslate(sMsg,Item("DLG",sDlgID))
        rc = scrStatusBarMsg(sMsg,"EVMsg","1")
      Else
        scrLog("vbsExecuteEvent(FALSE) "+StrFmtRight(CStr(scrDateTime("TCMS")-sTCMS),8,"0") _
               +" msec<"+sDlgID+">"+sDATA+"<")
        sMsg = "#  Abbruch der Event-Verarbeitung für Dialog [ <DLG> ] ! #"
        sMsg = scrTranslate(sMsg,Item("DLG",sDlgID))
        rc = scrStatusBarMsg(sMsg,"EVMsg","1")
      End If
  End Select
End Sub
```

6.4.18 UserExitModifyListCmd

Functionality:

This user exit is called when the terminal requests lists from the server. Here, you can change the PDM command to load a list (e.g. DLG=LIST;74).

Input parameters:

Parameter	Value	Description
UE:PAR	VVar("UE:PAR","XYZ")	Load command that the terminal has sent to the server

Return parameters:

Parameter	Value	Description

UE_RET		In UE_RET the changed send data can be returned. If you want to change the send string, the complete string must be transferred!

Example: If the list 74 is requested, the ID TEST=XYZ is added

```
Sub UserExitModifyListCmd
  UE_RET = ""
  Select Case VVar("UE:PAR","DLG")
    Case "LIST;74"
      '     ' If the list 74 is requested, the ID TEST=XYZ is added
    ' all data and the additional items must be returned
      UE_RET = VVar("UE:PAR","#GET#ALL#VALUES#") + Item("TEST","XYZ") + Item("XXX","XYZ")
  End Select
End Sub
```

6.4.19 UserExitSysReadFile

Functionality:

This user exit is called, if a file has been loaded using the basic function of the main program (sys_read_file).

Input parameters:

Parameter	Value	Description
UE:PAR UE:DAT	Infostring	Information on the lists loaded including number of files

Return parameters:

Parameter	Value	Description
UE_RET	---	without processing

Implementation notes:

The loaded files are inserted in the list using lower case letters.

The files can be read using `VVar("UE:PAR","mnr.lst") or VVar("UE:PAR","mnr.lst").`

The data is structured as follows:

```
FILE:COUNT=1|anr.lst=c:\mpdv\aip2\anr.lst;8132;2011-05-27;09:24:45.036;1;|…
<FILE:COUNT>        =       <number of loaded files>
<file name>         =       <path+file name>        ;
                            <file size in bytes>    ;
                            <date> (Format YYYY-MM-DD) ;
                            <time>          (Format HH:MM:SS.ZZZ) ;
                            <transfer format> (0=binary,1=text)
```

Example: A customer-specific action is performed when the machine list has been loaded (file "mnr.lst").

```
Sub UserExitSysReadFile
```

```
  If VVar("UE:PAR","mnr.lst") <> "" Then
    ' DO CUSTOM ACTION AFTER READ <MNR.LST>
    doCustomActionAfterReadFileMNR
  End If
End Sub
```

6.4.20 UserExitAfterListLoaded

Functionality:

Using this user exit, the developer can perform standard and custom extensions in the user exit after request of lists.

This user exit is therefore run twice to develop the custom and the standard extensions each.

1. *UserExitAfterListLoaded_LIST_13* ➔ *Custom implementation / (if available) / **aip_system_<project>.scr***
2. *UserExitAfterListLoaded__LIST_13__* ➔ *Standard processing (if available) / **aip_mpdv-system.scr***

Input parameters:

Parameter	Value	Description
UE:LST.PAR		Input parameter of list
UE:LST.SND		Send parameter for list command

Return parameters:

Parameter	Value	Description
UE_RET		

Example: Extension of list DLG=LIST;13|MOD=P|..

```
Sub UserExitAfterListLoaded_LIST_10
  Dim sFileName,rc
  sFileName=VVar("UE:LST.PAR","FILE")
  If Right(sFileName,7)="mnr.lst" Then
    rc=scrSetData("AddListFileColumn","FILE="+sFileName+"|AKRO=SEL|VALUE=")
  End If
End Sub
```

6.4.21 UserExitGetCellData

Functionality:

This user exit is used for the free programming of a field content in a grid.

To do so, you must define a field name in the grid configuration in the file ctaiplay.ini, which starts with the character "@".

Example for ctaiplay.ini:

```
[order                                                    list]
...
@PAL.CNT=N10.0,60,R,A.S.a.P.     ; number of parts on pallet
```

Input parameters:

Parameter	Value	Description
UE:RET	@GRD.ITMFLD	Acronym of the field (→@PAL.CNT)
	@GRD.ITMVAL	Previous value
	@GRD.ROWNUM	Row in the grid
	@GRD.COLNUM	Column in the grid
	@GRD.ACTROW	Active (selected) row of the grid
	@GRD.TABLENAME	Section in ctaiplay.ini → 'order list'
	@GRD.FILENAME	List file with path
	@GRD.EXTFILENAME	List file → "anr.lst"
	@GRD.INIFILE	Actual name of layout file
	@GRD.FILTER	Filter of grid (e.g. "MNR=4711“)
	@GRD.ORDER	Sorting of grid
UE:GRD		The complete data row of the grid that is to be drawn

Return parameters:

Parameter	Value	Description
UE_RET	@GRD.ITMVAL	The value that is identified is returned in UE_RET using the ID "@GRD.ITMVAL".

Example:

```
Function UserExitGetCellData
  Dim sFile,sAuftrag,sAcro,sValue
  sFile=VVar("UE:RET","@GRD.EXTFILENAME")
  If sFile="anr.lst" Then
    sAuftrag=VVar("UE:GRD","ANR")
    If IsNumeric(sAuftrag) Then
      sAkro=VVar("UE:RET","@GRD.ITMFLD")
      If sAkro="@PAL.CNT" Then
        sMaschine=VVar("UE:GRD","MNR")
        sValue=CStr(iSchlagPalette("Read",sMaschine,0))
        UE_RET=Item("@GRD.ITMVAL",sValue)
      End If
    End If
  End If
End Function 'UserExitGetCellData
```

6.4.22 UserExitPzeCfgLoad

Functionality:

This user exit is called with the cyclic loading of the PZE configuration. You use this user exit to load additional customer-specific files.

Input parameters:

Parameter	Value	Description
UE:PAR	Terminal label (configuration)	Includes the data of the terminal label (configuration) Example: UE:PAR=TNR=826|TYP=830|CFG:1=1|HWADR=10.10.62.163|TZ=|..|PORT=|'

Return parameters:

Parameter	Value	Description
UE_RET	---	Return value is not evaluated in the terminal program. Example: UE:RET=RET=*|..|CNT=< number of files loaded >|..|

```
Sub UserExitPzeCfgLoad
  Dim cnt
  cnt = "0"
  If LoadWageType Then cnt=IncStrDec(cnt)    '*** Lohnarten laden
  If LoadCostCenter Then cnt=IncStrDec(cnt)  '*** Kostenstellen laden
  UE_RET = Item("CNT",cnt)
End Sub

Function LoadWageType
  UE_SND = ""
  UE_SND = Item("DLG",     "SYSTEM.CALL" )
  UE_SND = Item("PROG",    "custom_list.scr" )
  UE_SND = Item("AKTION", "lohnart" )
  UE_SND = Item("DATEI",   ".\spool\lohnart."+SYS_USR )
  UE_SND = Item("FILE",    "lohnart.lst" )
  ' ------  UE_SND  = Item("CMD:CPY", "BINARY" )  ' load binary if required
  scrUECmd(UE_SND)
  LoadWageType=(VVar("UE:RCV","RET")="0"))
End Function

Function LoadCostCenter
  UE_SND = ""
  UE_SND = Item("DLG",     "SYSTEM.CALL" )
  UE_SND = Item("PROG",    "custom_list.scr" )
  UE_SND = Item("AKTION", "kostenst" )
  UE_SND = Item("DATEI",   ".\spool\kostenst."+SYS_USR )
  UE_SND = Item("FILE",    "kostenst.lst" )
  ' ------  UE_SND  = Item("CMD:CPY", "BINARY" )  ' load binary if required
  scrUECmd(UE_SND)
  LoadCostCenter=(VVar("UE:RCV","RET")="0"))
End Function
```

6.4.23 UserExitAGInfoGetCaption

Functionality:

You use this user exit to customize the AIP dialogs MINFO(MMINFO) and AINFO(MAINFO).

Input parameters:

Parameter	Value	Description
UE:PAR	MODE	The dialod used to run the function is specified via the ID MODE="MINFO" or "AINFO"
	MNR.MNR	Machine number

Return parameters:

Parameter	Value	Description
UE_RET	Field ID	As return, the field IDs can be set in the info dialog.

Implementation notes:

The dialogs "MMINFO" and "MAINFO" are only available in the classic main view without XML GUI on AIP2.

It is possible to change the output or make changes for a dialog field.

You can also describe an added field in the dyn. dialog configuration

Example:

```
Sub UserExitAGInfoGetCaption
  Dim sShowMode, s, s1, sHub, sMnr, sSzy, rSzy, r
  On Error Resume Next
  s = VVar("UE:PAR","#GET#ALL#VALUES#")
  scrLog(s)
  sMnr = VVar("UE:PAR","MNR.MNR")
  sShowMode=VVar("UE:PAR","MODE")      ' MINFO / AINFO
  'Ex.: Output CLOCK/MIN in MINFO
  'Query:  MINFO=Machine info data
  'Query:  AINFO=Order infor data
  If sShowMode = "MINFO" Then    '// or respectively AINFO
    sMnr = VVar("UE:PAR","MNR.MNR")
    mData = scrGetInfo("GetMachineData","MNR="+sMnr)
    sSzy = scrDDitem("SZY",mData)
    rSzy = scrStr2Real(sSzy) / 1000
    If rSzy <> 0 Then
      r = 60 / rSzy
        s = RealToStrNK(r,2)
        s1 = RealToStrNK(rSzy,2)
      UE_RET=Item("HUB",s) ' + Item("MNR.SZY",s1)
    End If
  End If
  If Err.Number <> 0 Then
    scrLog("Error:UserExitAGInfoGetCaption|ERR.Number:"&CStr(Err.Number) _
          & "|Source:"&Err.Source &"|Description:"&Err.Description&"|")
  End If
  On Error Goto 0
End Sub
```

6.4.24 UserExitCAQChangeImageTreeView

Functionality:

This user exit is used to change the standard image in the CAQ tree view. To this end, add images to the file pict_cust.zip.

To change data, call the user exit in a node of the tree.

Input parameters:

Parameter	Value	Description
UE:SND	NODEDATA	Here, the data of the internal tree node is available. The data is separated by the character chr(8) (backspace). The separator must then be converted to a pipe character.
	CHARACTERISTICDATA	Here, the characteristic data of the internal tree node is available. The data is separated by the character chr(8) (backspace). The separator must then be converted to a pipe character.
	IMAGEINDEX	Default ImageIndex of the application
	COLUMN	Column in tree (in the tree, 4 columns are available)
	QUALITYSTATE (numeric)	Current quality status of the node 0 = QS_IO 1 = QS_NIO 2 = QS_BEDINGT_IO 3 = QS_UNKOWN 4 = QS_CALC
	ERFASSUNGSSTATUS (numeric)	Current collection status of the node 0 = ESTA_NOETIG 1 = ESTA_MOEGLICH 2 = ESTA_FERTIG 3 = ESTA_ABGESCHLOSSEN 4 = ESTA_FEHLER 5 = ESTA_UNDEF 6 = ESTA_NULL
	ERFASSUNGSSTATUSTEXT	Current collection status as text
	TOLERANCELIMITREACHED (numeric)	Is tolerance of node respected 0 = TL_UNKNOWN 1 = TL_UPPER

		2 = TL_LOWER
	TOLERANCELIMITREACHEDTEXT	Is tolerance of node respected
	INTERVENTIONLIMITREACHED (numeric)	Is action limit of node respected 0 = IL_UNKNOWN 1 = IL_UPPER 2 = IL_LOWER
	INTERVENTIONLIMITREACHEDTEXT	Is action limit of node respected

Return parameters:

Parameter	Value	Description
UE_RET	RET	To accept the new ImageIndex, RET=0 must be returned.
	IMAGEINDEX	The customer-specific images start from 20. By default, 5 images are read from the zip files.

Implementation notes:

Standard images with index

Customer-specific images

The customer-specific images are only loaded if the user exit is defined.

The files with the name "caq_image_[index].png" are then loaded. The images must have the format 24 x 24 pixels. The indexes for customer-specific images are always from 20 onwards.

In the standard configuration, 5 customer-specific images are loaded from the indexes 20 to 24. If you require more than 5 images, you can increase the number of possible customer-specific images in the file caq_dc_t.ini.

Example for caq_dc_t.ini:

```
[OPTIONS]
…
NUMBER_OF_ADDITIONAL_IMAGES=10
…
```

The images must be available in the file pict_cust.zip.

Example:

```
Sub UserExitCAQChangeImageTreeView
  scrLog VVar("UE:SND","#GET#ALL#VALUES#")
  nodeData = scrStrReplace( VVar("UE:SND","NODEDATA") ,Chr(8), "|")
  merkmalData = scrStrReplace(VVar("UE:SND","CHARACTERISTICDATA"),Chr(8), "|")
  scrLog "nodeData:"  +  nodeData
  scrLog "merkmalData:" + merkmalData
  if (VVar("UE:SND","COLUMN")) = "1" then
    'attributive without cavity (data collection based on characteristics)
    if ((scrDDItem("NODE:PREFIX",nodeData)) = "PPKT_MM") AND _
       ((scrDDItem("BEURTBASIS",nodeData) = "STICHPR_MSTP") OR _
      (scrDDItem("BEURTBASIS",nodeData) = "STICHPR_ESTP")) AND _
      (scrDDItem("KEINNEST",nodeData) = "1") then
        if (VVar("UE:SND","NEWDATA") = "TRUE") then
            UE_RET = Item("RET","0") + Item("IMAGEINDEX","20")
        else
            'scrMsgBox VVar("UE:SND","QUALITYSTATE")
            if (VVar("UE:SND","QUALITYSTATE") = "0") then
                UE_RET = Item("RET","0") + Item("IMAGEINDEX","0")
            end if
            if (VVar("UE:SND","QUALITYSTATE") = "1") then
                UE_RET = Item("RET","0") + Item("IMAGEINDEX","21")
            end if
        end if
    end if
    'attributive without cavity (data collection based on characteristics)
    if ((scrDDItem("NODE:PREFIX",merkmalData)) = "PPKT_MM") AND _
       ((scrDDItem("BEURTBASIS",merkmalData) = "STICHPR_MSTP") OR _
      (scrDDItem("BEURTBASIS",merkmalData) = "STICHPR_ESTP")) AND _
      (scrDDItem("KEINNEST",merkmalData) = "0") then
        if (VVar("UE:SND","NEWDATA") = "TRUE") then
            UE_RET = Item("RET","0") + Item("IMAGEINDEX","20")
        else
            'scrMsgBox VVar("UE:SND","QUALITYSTATE")
            if (VVar("UE:SND","QUALITYSTATE") = "0") then
                UE_RET = Item("RET","0") + Item("IMAGEINDEX","0")
            end if
            if (VVar("UE:SND","QUALITYSTATE") = "1") then
                UE_RET = Item("RET","0") + Item("IMAGEINDEX","21")
            end if
        end if
    end if
  end if
End Sub
```

6.5 DIALOG scripts

The script dialog processing has been implemented for the initialization and the dialog control of new dynamic dialogs (that are not implemented in the source code).

This kind of dialog is configured or called via entry in the file "***ctaipbut.ini***" in the main view. In the new GUI, the dialog must be configured in a layout file such as e.g. "***l_anr.xml"***, "***l_mnr.xml"***, "***l_pnr.xml"*** or "***l_res.xml"***.

For information on the storage and naming of dialog scripts, refer to section "1.1.1 Storage". For notes on the processing, refer to section "1.1.2 Processing".

Currently, the following dialog user exits are implemented or defined:

Dialog "user exits"	Script description
DynDlgInit_[DIALOG-ID]	Dyn. dialog (initialization)
DynDlgGridInit_[DIALOG-ID]	Dyn. dialog (grid initialization)
DynDlgFieldChange_[DIALOG-ID]	Dyn. dialog (input field - change/bar code)
DynDlgFieldExit_[DIALOG-ID]	Dyn. dialog (input field - exit)
DynDlgFieldListe_[DIALOG-ID]	Dyn. dialog (input field - attached list)
DynDlgFunctions_[DIALOG-ID]	Dyn. dialog (button - function)
DynDlgBeforeSend_[DIALOG-ID]	Dyn. dialog (before DB posting)
DynDlgAfterSend_[DIALOG-ID]	Dyn. dialog (after DB posting with <RET=0|..>)
DynDlgTimer_[DIALOG-ID]	Dyn. dialog (timer for cyclic processings)
DynDlgFormValidationBeforeFunction _[DIALOG-ID]	Dyn. dialog (entry of validation before execution of function)
DynDlgKeyDown_[DIALOG-ID]	Dyn. dialog (keyboard - events)
DynDlgPluginCreate_[DIALOG-ID]	Dyn. dialog (plug-in - initialization)
DynDlgWFTabEnter_[DIALOG-ID]	Dyn. dialog (display before workflow)
DynDlgWFTabExit_[DIALOG-ID]	Dyn. dialog (exit before workflow)

The following examples for the DIALOG "XYZ" explain the functions provided by the dialog user exits.

6.5.1 DynDlgInit_XYZDynDlgInit_XYZ

Functionality:

This user exit is called when a script dialog is initialized.

Available functions	Description
`VVar("DLG.DLG","XYZ")` oder `VDlg("XYZ")`	Basic initialization of the dyn. dialog (e.g. DLG=M_MST|…)
`VPar("XYZ")`	Parameter of the dialog call (or the dialog data of the calling dialog)
`VMnr("XYZ")`	Current machine info from MNR.LST for the machine selected in the main view. If the acronym is included in in **`VPar("XYZ")`**, this acronym is used.
`VAnr("XYZ")`	current order info from ANR.LST for the order selected in the main view. If the acronym is included in in **`VPar("XYZ")`**, this acronym is used.
`VVar("DLG.CGD","XYZ")`	includes (if available) the current row of the third grid of the main view or the selected row if the call has been made using a button in a dynamic dialog with grid.

The functions: VMnr(), VAnr() und VVar("DLG.CGD") include the following additional information:

<#FILE#LIST#> → includes the file name without path

<#FILE#NAME#> → includes file name with path

the values of these fields are in lower case letters

For the active grid of the main view, the following information is additionally passed:

➔ "..|#GRD#STATE#=FOCUS|.."

From a script dialog with grid, the following value is passed

➔ "..|#GRD#STATE#=DIALOG|.."

Implementation notes:

(1) If a value is entered in the dialog, the information is not updated, i.e. if you change the machine or the order in the dialog, these values are not changed.

(2) Also if you access the **DynDlg…_** user exit that follows, the variable content might not be available or correct.

The values required for the processing should be included in STATUS or in hidden dialog fields.

You can create a hidden field in the dialog in each **DynDlg…** user exit sing **DLGVAR = Item("*MNR" , VMnr("MNR"))**.

You can also save values using **GLOBALVARS = "#XXX #PAR#=WAAGENTERMINAL=1".** The developer is responsible for editing the contents and deleting after use.

(3) Mind the note in section "Dynamic dialog/workflow with a WF step"

```
Sub DynDlgInit_XYZ
  If VOut("REOPEN") = "J" Then
' ----- repeated opening of the dialog, e.g. after DB plausibility error <RET=..|KT=..|LT=..|>
  Else
' ----- Plausibility checks if authorization for opening dialog exists
    If "X" <> "X" Then
     '  scrMsgBox(" Dialog -> Plaus. error  [ "+VOut("DLG")+" / "+VOut("ScriptID")+" ]")
       DLGVAR = Item("RET", "$>"+VOut("DLG")+"<")
       DLGVAR = Item("KT", "(Kurztext)")
```

```
      DLGVAR = Item("LT", "(Langtext)")
      '' alternativ: Dialog nicht öffnen - ohne Fehlermeldung:
      'DLGVAR = Item("RET","#INVISIBLE#MSG#")
    Else
' ----- opening the dialog e.g. via <ButtonClick()> or via <Remote-Dialog-Call()>
    '  scrMsgBox(" Dialog -> Init [ "+VOut("DLG")+" / "+VOut("ScriptID")+" ]")
      DLGVAR = Item("DT", SYS_DT,"")
      DLGVAR = Item("MNR", VMnr("MNR"))
      DLGVAR = Item("ANR", VAnr("ANR"))
      'DLGVAR = AddIt("CNR","",cFFDisable)
    End If
  End If
End Sub
```

6.5.2 DynDlgGridInit_XYZ

Functionality:

If the dialog includes a grid, you can initialize it using this user exit. Condition: The grid must be configured with the "field attribute" SCRIPT_GRID.

Input parameters:

Parameter	Description
GRD.CMD	Command to load the list from the server If value is set, the list is loaded on opening of dialog.
GRD.FILE	The data of this file (in the subdirectory "spool") is displayed.
GRD.INI	Configuration file including the layout configuration (default: ctaiplay.ini).
GRD.SECTION	Section in the configuration file that includes the layout.
GRD.FILTER	Filter criterion to show only part of the data records of the list.
GRD.ORDER	Sorting criterion – you can specify several field IDs separated by "|". The first criterion has the highest priority. Example for descending sorting: GRID_ORDER=MSDAUER=-

Implementation notes:

1. With the following entry, you can also use the < GRID_ORDER > entry of a < SCRIPT_GRID > that is included in the configured INI section of the INI file.

   ```
   SCRVARS = "GRD.ORDER="   + "#USE#INI#ITEM#"
   ```

2. If you want to reload the grid after having changed the file, you can set the following value in a dialog script (e.g. DynDlgFieldListe_xx):

   ```
   DLGVAR = Item("DLG.GRID","RELOAD","")
   ```

Example:

```
Sub DynDlgGridInit XYZ
  SCRVARS = "GRD.CMD="    + "DLG=LIST;u_l_list|MOD=U|MNR=<MNR>|ANR=<ANR>|"
  SCRVARS = "GRD.FILE="   + "u_list.lst"
  SCRVARS = "GRD.INI="    + "hytnrcfg.ini"
```

```
  SCRVARS = "GRD.SECTION="+ "Layout->U_LIST"
  SCRVARS = "GRD.FILTER=" + ""
  SCRVARS = "GRD.ORDER="  + "CNR"
End Sub
```

6.5.3 DynDlgFieldChange_XYZ

Functionality:

After execution of the function **scrFieldChange**, the result configured with "LST.RET" is passed to <[n#]DLG.OUT>.

If no entry is found, the specified fields are deleted and the input field is colored in magenta.

Important!

In the user exit itself *-identifiers cannot be set (DLGVAR = Item("*ABC", ... is ignored).

Input parameters:

Parameter	Value	Description
DLG.DLG VDlg("...")	Dialog data	All dialog data in the dynamic dialog as PDM string. Note: the data changed is not yet set here.
	DLG.FLD	ID of the changed field (e.g. MST)
	DLG.VAL	Value of the changed field (e.g. MST)
	DLG.GRD	Selecting a row in the grid
	DLG.GRD.DBLCLK	Selecting a row in the grid via double-click
VStore("...")	Data row	Selected row in the grid with all data
	DLG.GRD.ROWCOUNT	Returns number of rows in the grid
	DLG.GRD.REOPEN	Is called when the grid is reopened (reloaded) If you set the value "DLGVAR=DLG.PROCESS.RESULT=TRUE" the assigned values are processed.

Return parameters:

Parameter	Value	Description
DLG.OUT	Field ID	As return, the field IDs can be set in the dialog.

Example 1: If you manually enter a machine status in field "U_MST", the relevant machine status text is identified and entered in the dialog field "U_MSTTXT". If the machine status text is not found, the field "U_MST" is colored in magenta.

```
Sub DynDlgFieldChange_XYZ
```

```
  Select Case VDlg("DLG.FLD")
    Case "U_MST"
      LSTVARS = "LST.FILE="    + "mstat.lst"
      LSTVARS = "LST.FILTER=" + "MNR="+VDlg("MNR")+" & "+"MST="+VDlg("FLD.VAL")
      LSTVARS = "LST.RET="     + "U_MSTTXT=MSTTXT"
      scrFieldChange
  End Select
End Sub
```

Example 2: If you select a row, data of this row should be passed to the dialog fields.

```
Sub DynDlgFieldChange RES AB
  Select Case VDlg("DLG.FLD")
    Case "DLG.GRD", "DLG.GRD.DBLCLK"
      If VStore("RES") <> "" Then
        DLGVAR=Item("RES",VStore("RES"))
        DLGVAR=Item("RESTYP", VStore("RESTYP"))
      End If
  End Select
End Sub
```

Example 3: When the file is opened, the value of the first row of column "POS" is passed to the field "POS" and focused.

Via double-click, the value changes between blank ("") and "X" in the column "SELECT" of the currently selected row. These implementations are often used if a multiple selection is implemented.

```
Sub DynDlgFieldChange XYZ
  Select Case VDlg("DLG.FLD")
    Case "DLG.GRD.REOPEN"
      ' after reading the grid, the field <POS> is filled with the value
      ' of the first row and focused.
      DLGVAR = Item("DLG.PROCESS.RESULT","TRUE" )+Item("POS", VStore("POS")+";#F" )
    Case "DLG.GRD.DBLCLK"
      'select row in grid
      If VStore("SELECT") = "X" Then
        tmp = SStore("SELECT","")
      Else
        tmp = SStore("SELECT","X")
      End If
  End Select
End Sub
```

Note when implementing the event "DLG.GRD.DBLCLK" that the AIP is primarily intended for use with a touch screen. A double click is not a practical on a touch screen. It is better to make a selection using an additional button in the dialog.

6.5.4 DynDlgFieldExit_XYZ

Functionality:

This user exit is called if an input field in the dialog is exited or if a field has obtained a bar code. The bar code event can be identified via the request:

If VDlg("FLD.MOD")="BARCODE" Then

Implementation notes:

The dialog data is available in VDlg(„XYZ“). Use the function DLGVAR to pass values to the dialog.

Input parameters:

Parameter	Value	Description
DLG.DLG VDlg("...")	Dialog data	All dialog data in the dynamic dialog as PDM string.
	DLG.FLD	ID of the changed field (e.g. MST:1) • "FLD.MOD" = "FLDEXIT" Field that has been exited • "FLD.MOD" = " BARCODE“ target field of bar code
	FLD.MOD=MOUSEDOWN	Mouse button has been pressed

Return parameters:

Parameter	Value	Description
DLGVAR	Dialog data	As return, the field IDs can be set in the dialog via DLGVAR.

Example 1: After having exited a field with the ID "MST:1", the status text is read from the list and entered in the dialog in field "MSTTXT:1".

```
Sub DynDlgFieldExit XYZ
  Select Case VDlg("DLG.FLD")
    Case "MST:1"
      LSTVARS = "LST.FILE="      + "mstat.lst"
      LSTVARS = "LST.FILTER="    + "MNR="+VDlg("MNR")+" & MST="+VDlg("MST:1")
      LSTVARS = "LST.RET="       + "MSTTXT:1=MSTTXT"
scrFktList
  End Select
End Sub
```

Example 2: Using the extension < VDlg("FLD.MOD") = "MOUSEDOWN" >, you can implement a "localization grid" in a configured <IMAGE> using the field attribute <MOUSEDOWN>. For example, in this "grid recording", a grid is placed over an article image in order to be able to record the precise position of a defect.

```
Sub DynDlgFieldExit_XYZ
  Dim x,y
  If VDlg("FLD.MOD") = "MOUSEDOWN" Then
    Select Case VDlg("DLG.FLD")
      Case "ATKIMG"
        x = Int( CInt(VDlg("ATKIMG@XPOS")) / Int(CInt(VDlg("ATKIMG@WIDTH")) / 10))+1
        y = Int( CInt(VDlg("ATKIMG@YPOS")) / Int(CInt(VDlg("ATKIMG@HEIGHT")) / 5))+1
        If x > 10 Then x = 10
        If y > 5 Then x = 5
        DLGVAR = Item("RASTER:X",CStr(x))+ Item("RASTER:Y",CStr(y))
      Case Else
    End Select
  Else
    ...
  End Select
End Sub
```

Example 3: When a field with the ID "ABC" is left, the column "2" in the dialog grid is selected. Using the optional parameter [;LOCK] you can prevent that the processing "DynDlgFieldChange_XYZ" is executed. These implementations ensure that in case of a later selection, also in the current grid row a selection is still possible. Note: If a dialog is opened with a grid, the column "2" is always opened initially. With a later selection, the column "2" should therefore be configured to be hidden (display width of 0 pixels, e.g. "DMY1=C3,0,Z").

```
Sub DynDlgFieldExit XYZ
  Select Case VDlg("DLG.FLD")
    Case "ABC"
      DLGVAR=Item("GRD.SETCOL","2;LOCK")
    End Select
End Sub
```

6.5.5 DynDlgFieldListe_XYZ

Functionality:

You can use this user exit to implement a list selection for any field.

Input parameters:

`VDlg(„DLG.FLD")` includes the ID of the field whose list button has been pressed.

The field attribute "DIALOGLIST" is set in the dialog configuration to show a list button. The value "SCRIPT_LIST" is also set in "Dialog list function".

Example for a field with a list button: 

The function LSTVARS is filled with the parameters for the list:

LST.CMD	Command to request the list from the server (optional).
LST.FILE	File name (the local directory "spool" is always put in front).
LST.CAPTION	Window caption of the selection dialog
LST.FILTER	Filter for the list to be displayed (e.g. "MNR=100 & ZUMAN=J|N")
LST.SORT	List sorting
LST.INI	INI file where the <section> is read (""=ctaiplay.ini)
LST.SECTION	INI section including the layout definition of the list to be displayed
LST.RET	Configuration of the values from the list that are transferred into the calling dialog e.g. < MST:1=MST"+" & "+"MSTTXT:1=MSTTXT" > copies the values of columns <MST> and <MSTTXT> of the selected entry of the list in the dynamic dialog fields <MST:1> und <MSTTXT:1>.
LST.MODE	Additional processing modes (configurations separated by "|") "COLNUMSORT=TRUE" (or in INI section GRID_COLNUMSORT) "DYNAMICFILTER=MGRP,MNR,MST" (or in INI section

	<GRID_DYNAMICFILTER) "PAGESCROLLING=TRUE" (or in INI section GRID_PAGESCROLLING) "WILDCARD=+" "FILTERSENSITIVE=TRUE" (or in INI section GRID_ FILTERSENSITIVE)

Implementation notes:

The result string configured in LST.RET is transferred into the global variable <[n#]DLG.OUT> (is equal to <DLGVAR>). Additionally, the complete row selected is stored in <[n#]LST.VALUES>. If the list has been read by the server (→LST.CMD), the result of the server request is saved in <[n#]LST.CMD:RET>.

In general, only a static list should be specified as **LST.FILE** for the display without previous update (`LST.CMD=""`).

Example: Selection of a local list

Example : Input of an additional MST in a dialog (this selection is not realizable in the standard system, i.e. a realization is only possible with script).

Using the general selection list **scrFieldList:**

```
Sub DynDlgFieldListe XYZ
  Select Case VDlg("DLG.FLD")
    ' ---- Input field with dialog list button
    Case "MST:1"
          ' ---- Initialization of the <LSTVARS>
      LSTVARS = ""
          ' ---- File name (local spool directory is always put in front)
      LSTVARS = "LST.FILE="     + "mstat.lst"
          ' ---- Window caption of the selection dialog
      LSTVARS = "LST.CAPTION="  + "machine status list [ <DLG> ]"
      ' ---- Filter auf die anzuzeigende Liste ( z.B. "MNR=100 & ZUMAN=J|N" )
      LSTVARS = "LST.FILTER="   + "MNR="+VDlg("MNR")
      ' ---- Sortierung der Liste
      LSTVARS = "LST.SORT="     + "MNR|MST"
       ' ---- Ini file where the <section> is read (""=ctaiplay.ini)
      LSTVARS = "LST.INI="      + ""
      ' ---- Ini-Section mit der Layoutdefinition der anzuzeigenden Liste
      LSTVARS = "LST.SECTION="  + "Maschinenstatusliste"
          ' ---- Configuration of the values from the list that are transferred into the calling
dialog
      ' ---- - z.B. < MST:1=MST"+" & "+"MSTTXT:1=MSTTXT" >
      ' ---- - kopiert die Werte der Spalten <MST> und <MSTTXT> des selektierten Eintrag
      ' ---- - der Liste in die dynamischen Dialogfelder <MST:1> und <MSTTXT:1>
      ' ---- - bei < MST "+" & "+" MSTTXT" > erfolgt keine  DlgID - Umsetzung
      LSTVARS = "LST.RET="      + "MST:1=MST"+" & "+"MSTTXT:1=MSTTXT"
     ' ---- Additional processing modes (configurations separated by "|")      ' ---- zusätzliche
Verarbeitung-Modi  (mit "|" getrennte Konfigurationen)
          ' ---- - "COLNUMSORT=TRUE"            (or in Ini-Section <GRID COLNUMSORT)
      ' ---- - "DYNAMICFILTER=MGRP,MNR,MST" (or in Ini-Section <GRID DYNAMICFILTER)
      ' ---- - "PAGESCROLLING=TRUE"         (or in Ini-Section <GRID_PAGESCROLLING)
      ' ---- - "WILDCARD=+"
      ' ---- - "FILTERSENSITIVE=TRUE"       (or in Ini-Section <GRID  FILTERSENSITIVE)
      LSTVARS = "LST.MODE="     + ""
      ' ---- further Ini-Section-configurations / internal note
      ' ---- - GRID MAXIMIZE LIST=TRUE          (displays the selection list maximized)
     ' ---- - the field contents of the calling dialogs are also transferred
scrFieldList
  End Select
End Sub
```

Example: Selection of an ONLINE/server list

Example: Implementation of a module or a customer-specific sequencing list (user exit to server)

Verwendung der Auswahlliste **scrFieldList**.

INI section [sequencing list] of customer-specific global INI file **hytnrcfg.ini.**

```
Sub DynDlgFieldListe_A_AN_RS
  Select Case VDlg("DLG.FLD")
    case "ANR"
           ' ---- Initialization of the <LSTVARS>
      LSTVARS = ""
          ' ---- File name (local spool directory is always put in front)
      LSTVARS = "LST.FILE="     +  "vrslst.lst"
         ' ---- Server command to request file
      LSTVARS = "LST.CMD="      +  "DLG=LIST;11|MOD=V|MNR="+VDlg("MNR")+"|MOD3=M|"
          ' ---- Window caption of the selection dialog
      LSTVARS = "LST.CAPTION="  + "Vorgabeliste [ <MNR> -> <DLG> ]"
       ' ---- Ini file where the <section> is read (""=ctaiplay.ini)
      LSTVARS = "LST.INI="      + "hytnrcfg.ini"
       ' ---- Ini file where the <section> is read (""=ctaiplay.ini)
      LSTVARS = "LST.SECTION="  + "sequencing list"
           ' ---- Configuration of the values from the list that are transferred into the calling
dialog
      LSTVARS = "LST.RET="      +  "ANR"+" & "+"ATK"+" & "+"ABEZ=AGBEZ"
     ' ---- Additional processing modes (configurations separated by "|")
      ' ---- - "COLNUMSORT=TRUE"           (or in Ini-Section <GRID COLNUMSORT)
      LSTVARS = "LST.MODE="     + ""
scrFieldList
    Case Else
    ' scrMsgBox ( "FLD.LISTE = "+VDlg("DLG.FLD") )
  End Select
End Sub
```

6.5.6 DynDlgFormValidationBeforeFunction_XYZ

Functionality:

You use this DIALOG user exit to check the dialog entries before the user exit "DynDlgFunctions_XYZ" is executed.

DLG.RESTYP in DLGVAR wird hier nicht verarbeitet.

`DLGVAR=Item("DLG.PROCESS.RESULT","FALSE")` → DLGVAR wird nicht übernommen

`DLGVAR=Item("DLG.FORM.VALIDATION","TRUE")` → FormValidation is executed before the configured button function.

`DLGVAR=Item("DLG.SET.FORM.VALIDATION.ERROR","XXX")` → Function is not executed; the field with the identifier XXX is selected and red

Example:

```
Sub DynDlgFormValidationBeforeFunction_XYZ
  If VDlg("DLG.FKT") <> "" Then
  ' --- <FormValidationBeforeFunction> activate because of defined <Button> function
    Select Case VDlg("DLG.FKT")
      Case "DLG=V_BLZ"
          ' --- Transfer of  <DLGVAR> before <FORMVALIDATION> ---
        DLGVAR = Item("DLG.PROCESS.RESULT","TRUE")
        DLGVAR = Item("DLG.RESTYP","9")
           ' --- perform / activate <FORMVALIDATION> before <Button> function ---
        DLGVAR = Item("DLG.FORM.VALIDATION","TRUE")
```

```
        If VDlg("XXX") = "" Then
                  ' --- Setting a <FORM.VALIDATION.ERROR> independent of DynDlg configuration ---
          DLGVAR = Item("DLG.SET.FORM.VALIDATION.ERROR","XXX")
          '  MsgPopUp scrTranslate("Wert für Feld <XXX> erforderlich","") , "3"
        End If
      Case Else ' DEFAULT [ VTST, .. ]
        DLGVAR = Item("DLG.FORM.VALIDATION","FALSE")
    End Select
  Else
    ' --- <FormValidationBeforeFunction> aktivieren Aufgrund von <Button>-<RCODE>
    Select Case VDlg("DLG.RESTYP")
      Case "0","7"
        ' OK
        ' DLGVAR = Item("DLG.FORM.VALIDATION","TRUE")
      Case "1"
        ' CANCEL
    End Select
  End If
End Sub
```

6.5.7 DynDlgFunctions_XYZ

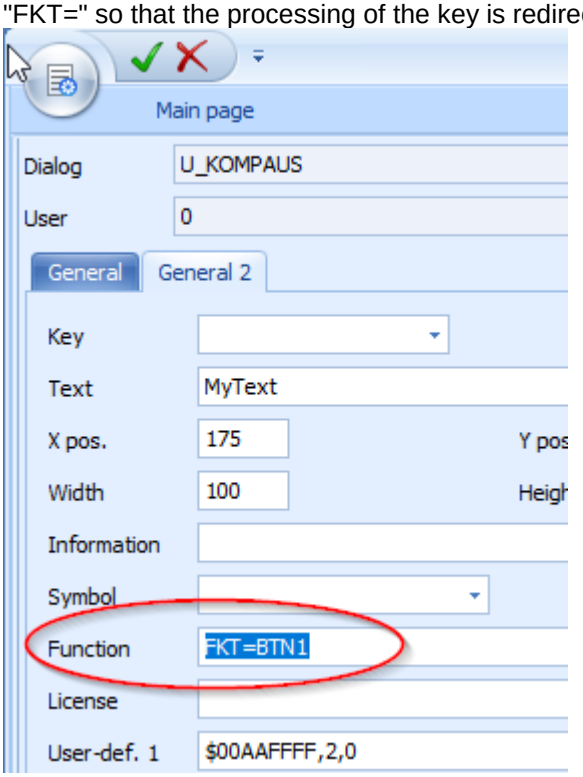
Functionality:

You use this user exit to implement function keys of the dialog. It is also called if the dialog is exited via OK or CANCEL and if the processing returns from a dialog called.

Implementation notes:

Note the following for function keys:

- The assignment of the function to the function key in the dialog configuration must start with "FKT=" so that the processing of the key is redirected to the terminal script.



- For the example above, the request would be "BTN!" in the select-instruction for VDlg(„DLG.FKT“).

- A value must be returned to the function. The terminal program then knows that the function key has been processed in the terminal script. Otherwise an error message is displayed. Example: DLGVAR=“RET=0“

The return from a called dialog (e.g. CASE "DLG=U_FILTER") occurs if the call was made via the function key configuration (also with "DLG=U_FILTER").

The data of the dialog called (and now closed) is then available in VVar("DLG.RET","XYZ"). You can access the dialog called as usual via VDLG(„XYZ“).

If you leave the dialog, the case "DLG.CLOSE=TRUE" is run. The query <<VDlg("DLG.RESTYP")="1">> specifies if the request was started by clicking OK or CANCEL.

```
Sub DynDlgFunctions_XYZ
  Select Case VDlg("DLG.FKT")
    Case "MST:BTN:1"
   ' *** Example for a selection list -> unusual see < DynDlgFieldList_... >
      DLGVAR  = "RET=0"
      LSTVARS = "LST.FILE="     + "mstat.lst"
      LSTVARS = "LST.CAPTION="  + "machine status list [ <DLG> ]"
      LSTVARS = "LST.FILTER="   + "MNR="+VDlg("MNR")
      LSTVARS = "LST.SORT="     + "MNR|MST"
      LSTVARS = "LST.SECTION="  + "Multi->Maschinenliste"
      LSTVARS = "LST.RET="      + "MST:1=MST"+" & "+"MSTTXT:1=MSTTXT"
scrFieldList
    Case "DLG=U_XYZ"
         ' *** Example of a return to the calling dialog/script
          ' *** after execution of a script dialog with the
          ' *** Condition: button with function „DLG=U_XYZ“ must have RCode (7,8,9)
    ' *** Purpose: take over values
    ' *** VDlg("<>") -> data of calling dialog
         ' *** VVar("DLG.RET","<>") -> data of dialog called
      If VVar("DLG.RET", "DLG.RESTYP") = "7" Then
        DLGVAR = Item("FAKTOR", VVar("DLG.RET", "FAKTOR") )
      End If
    Case "DLG=U_ABC"
         ' *** Example of a return to the calling dialog/script
          ' *** after execution of a script dialog with the
          ' *** Condition: button with function „DLG=U_XYZ“ must have RCode (7,8,9)
        ' *** Purpose: process control
      If VVar("DLG.RET", "DLG.RESTYP") = "1" Then
        DLGVAR = Item("DLG.RESTYP", "9")     ' dialog remains open
      Else
        DLGVAR = Item("DLG.RESTYP", "0")     ' dialog is closed and sent
      End If
    Case "DLG.CLOSE=TRUE"
         ' *** To prevent that dialog can be closed if a condition is met
          ' *** Ex.: If the dialog field has the value <> „0“ the dialog must not be
       ' ***       closed via ESC / virtual key "Cancel" or button with RCode (1)
      Select Case VDlg("DLG.RESTYP")
        Case "1"     ' CANCEL
          If VDlg("NUM") <> "0" Then
            DLGVAR = Item("DLG.CLOSE","FALSE")
          End If
        Case "0"     ' OK
          '
      End If
    Case "OUT:1#2"
         ' *** Script function <FKT=OUT:1#2> to set an output with PCCDLL connection
```

```
        ' set output 1 (=channel 301) and remove output 3 (=channel 303)
      scrPCCValues("DLG=SETVAL|O:O301=1|O:O303=0|")
  End Select
End Sub
```

6.5.8 DynDlgBeforeSend_XYZ

Functionality:

Just like the „UserExitDynDlgBeforeSend“, the dialog-specific user exit „DynDlgBeforeSend“ is called before a posting is sent to the server. The call is only performed if the dialog script is loaded - i.e. if the dialog is open or has just been closed with this posting. If a posting with identical dialog ID (DLG=XYZ) is sent in the background when the dialog is closed, the dialog script does not work.

Implementation notes:

If „DynDlgBeforeSend“ is loaded, then „UserExitDynDlgBeforeSend“ is not run!

Especially with complex project, it is therefore recommended to use only "UserExitDynDlgBeforeSend" in the system script. It is also possible to call "UserExitDynDlgBeforeSend" directly from " DynDlgBeforeSend", so that the functions implemented are effective.

You can use the function DLGSND to change the send string as shown in the example below. The processing of the posting is controlled via the ID "EVENT=EVENT_...":

EVENT_DIALOG_OHNE_SENDEN:
: If you press OK, the dialog is not sent. Is used, if the dialog is only meant to display data, or if the actual posting is explicitly sent by a script function (e.g. scrDDSndRcv()) when the dialog is open.

EVENT_OHNE_AUTO_MENGEN:
: No automatic quantities of the machine are added to the posting. You should use this setting for customer-specific postings because the server does not process automatic quantities by default. This way, quantities can be lost.

EVENT_MIT_AUTO_MENGEN:
: This is the default behavior.

EVENT_QUEUE_OHNE_AUTO_MENGEN, EVENT_QUEUE_MIT_AUTO_MENGEN:
: This posting is first set in the queue of the terminal and is then issued with delay. The same behavior is used in the standard system for shift changes and for PZE COME/GEHT postings.

EVENT_ONLINE_OHNE_AUTO_MENGEN, EVENT_ONLINE_MIT_AUTO_MENGEN:
: The posting may only be sent online. If an immediate posting cannot be sent to the server, the data record is not added to the queue. Instead the message is rejected with an error code. This variant is used if it is important for further processing that the posting has been booked on the server. This way, the server can perform processing steps that are not known to the terminal. After confirmation of the booking, the terminal can load lists that include the result of the processing.

Example:

```
Sub DynDlgBeforeSend_XYZ
  DLGSND = Item("DT", SYS_DT)
```

```
  DLGSND = Item("EVENT", "EVENT_ONLINE_OHNE_AUTO_MENGEN")
 ' *** Here, you can change / correct the dialog send ID / field ID if required
  DLGSND = Item("DLG", "A_TR")
  DLGSND = Item("MST", VDlg("MST:1"))
End Sub
```

6.5.9 DynDlgAfterSend_XYZ

Functionality:

This user exit is called, once a posting has been transferred successfully to the server. It is an alternative option to the UserExitDynDlgAfterSend in the system script. The same rules apply with respect to the processing at the same time as for DynDlgBeforeSend and UserExitDynDlgBeforeSend.

Implementation notes:

With script dialogs, the main lists (MNR,ANR,PNR,TNRMAT) are loaded by default after a posting. If you set the item < LOAD >, the standard update is not performed. In the example below, the MNR.LST and MSTAT.LST are reloaded in addition to the reloads (VRcv("LOAD")) set by the server.

If you add <RES> for the resource list (as of MDE/WRM >= 7.2.1), the system ignores this, if the product version <WRM> is smaller than <7.2.1> or if no active resource list display is configured for this terminal (<MNR.VISLIST3> does not include „R“).

Example:

```
Sub DynDlgAfterSend_XYZ
 ' *** for DD-LIST-Reload [ ANR,MNR,PNR,MAT,MST,RES ""=no DD-Lst-Updates] **************
  ' *** <RES> for resource list (as of WRM/MDE > 7.2)
 ' *** => this row updates the MNR.LST + MSTAT.LST on the terminal
 ' *** => with „VRcv("LOAD")” the DD-List-Reloads are added by the “Server”
  DD_RCV = Item( "LOAD", "MNR,MST," +VRcv("LOAD") )
End Sub
```

Tip 1: reopen dialog after posting until "Cancel" is pressed:

```
Sub DynDlgAfterSend_RES_AN
  rc=scrSetData("DelayedButtonClick","RES_AN")
End Sub
```

Tip 2: reopen dialog after posting until sending is successful (prevent "Cancel"):

```
  rc=scrSetData("DelayedButtonClick","CA_WL|FORCEDIALOG=ON")
```

6.5.10 DynDlgWFTabEnter_XYZ

Functionality:

This DynDlg user exit is executed before the display of a dialog or a workflow tab. You can use this user exit to perform an initialization similar to the configured dialog function (1) (for example, "FKT=DLGSHOW") in user exit <DynDlgFunctions_XYZ> without changing the dynamic dialog configuration.

Example:

```
Sub DynDlgWFTabEnter_XYZ
  If VDlg("*XYZ") = "" Then
    DLGVAR = Item("*XYZ","1")
  End If
End Sub
```

6.5.11 DynDlgWFTabExit_XYZ

Functionality:

This DynDlg user exit is executed before a dialog or a workflow tab is exited. You can use this user exit to perform an initialization similar to the configured dialog function (2) (for example, "FKT=DLGEXIT") in user exit <DynDlgFunctions_XYZ> without changing the dynamic dialog configuration.

Implementation notes:

If you set the item <RESULT> to the value <FALSE>, the dialog cannot be closed or the workflow tab cannot be exited.

Example: If the field "ABC" is empty, the workflow tab is not left. The field turns to red.

```
Sub DynDlgWFTabExit XYZ
  Dim rc
  If VDlg("ABC") <> "" Then
    DLGVAR = Item("RESULT","FALSE")
    rc=scrSetData("SetFocusToField","DLG=@ACTIVE|AKRO=ABC|RED=1")
  End If
End Sub
```

6.5.12 DynDlgTimer_XYZ

Functionality:

Besides the timer function available in the system script via UserExitMainLoopStop, "DynDlgTimer" can also be used to implement a cyclic call within the dialog.

Implementation notes:

The timer is activated, if you set the interval in ms in the UserExit DynDlgInit_XYZ:

```
DLGVAR=Item("DYNDLG.TIMER","100")
```

The timer event is only triggered, if the terminal is running in the foreground.

You can change the interval in the timer. You deactivate the timer if you pass "0".

Example:

```
Sub DynDlgTimer_XYZ
' *** DLG.RESTYP is not processed in the result <DLGVAR>
' *** the following row displays the date and the current time in the dialog field <DT>
  DLGVAR =  Item("DT",Cstr(now))
End Sub
```

6.5.13 DynDlgKeyDown_XYZ

Functionality:

You use this DynDlg user exit for the dialog-specific processing of keyboard events. In general, you can use this user exit to react in the script to each single key pressed. But because the focus changes each time, this can have the result that you must reprogram basic editing functions in the script. It is therefore recommended to use the function only for the ENTER key (13) as in the example.

```
Sub DynDlgKeyDown_XYZ
  If VDlg("DLG.FLD") = "KNR" Then
    If VVar("DLG.PAR", "KEY") = "13" Then
      ' ... <Action> ...
      DLGVAR = AddIt("KNR", "", cFFFocus)
    End If
  End If
End Sub
```

6.5.14 DynDlgPluginCreate_XYZ

Functionality:

You use this DynDlg user exit to initialize a dialog / workflow plug-in.

The processing is equal to the processing of the DynDlg user exit <*DynDlgInit_XYZ*>.

This user exit is only used with CAQ dialogs.

6.6 Porting notes from CTWIN/AIP to AIP2

Find below in the following sections some notes if you change from the programs CTWIN or AIP to AIP2 and if you want to port functions.

6.6.1 Dynamic dialog/workflow with one WF step

The difference in the terminal script processing in AIP and AIP2 in case of a workflow with one workflow step is as follows:

Example: Dialog/workflow: [U_TST] DialogTab: [WF_U_TST]

On the AIP, all DynDlg-UEs were executed as "workflow script" ("U_TST").

On AIP2 and independent of the dynamic workflow/dialog configuration:

- the UE "DynDlgInit_" is always executed as "workflow script" ("U_TST").
- all other DynDlg-UEs are called in dialog TabScript ("WF_U_TST").

As of AIP2 version 8.2.1.10, the processing is performed with the WorkFlow configuration

"Step 1“ "WF_U_TST“ (STEP:1= WF_U_TST)

"Script“ "W“ (WFSCR:1=W)

analogous to the processing on AIP in the "workflow script" ("U_TST").

6.6.2 Porting of customer-specific terminal scripts

VB script does not support the required UTF-8 format when normal files are written.

To write data, you should use callback functions. The following code examples illustrate the use of the VB script function on AIP and its implementation on AIP2.

```
'-------------------------------------------------------------------------
'- AIP: Function <ViewSelectedInfo> aus < aip_mpdv-AINFO_NOTES.scr>
'-------------------------------------------------------------------------
Sub ViewSelectedInfo
  ' write the note selected in the grid into the memo
  '      raw data from server: notes.lst
  ' processed for grid: notes_gr.lst
  '  text for TextViewer: notes.txt (-> is created here)
  Const ForReading=1, ForWriting=2, ForAppending = 8
  Dim asGrid,sKey1,rc,fso,sDatFileName,sTxtFileName,f,ts
  asGrid=scrGetInfo("GetGridData","DLG=@FIL=DLG=AINFO_NOTES|LINE=-1")
  sKey1=scrDDItem("SUBKEY:1",asGrid)
  sDatFileName=DIR_SPOOL+"notes.lst"
  sTxtFileName=DIR_SPOOL+"notes.txt"
  Set fso=CreateObject("Scripting.FileSystemObject")
  If scrFileExists(sTxtFileName)<>"0" Then
    scrFileDelete(sTxtFileName)
  End If
  fso.CreateTextFile sTxtFileName
```

```
  Set f=fso.GetFile(sTxtFileName)
  Set ts=f.OpenAsTextStream(ForWriting,TristateUseDefault)
  rc=GSrce("LOAD","FILE="+sDatFileName)
  rc=GSrce("FIRST","")
  While rc<>"#EOF#STORE#"
    ' show info if ANR|SUBKEY:1|INFO.OPT:INFO=J
    If VSrce("SUBKEY:1")=sKey1 And VSrce("INFO.OPT:INFO")="J" Then
      For i=1 To 10
        ts.WriteLine VSrce("INFO.INFO:"+CStr(i))
      Next
    End If
    rc=GSrce("NEXT","")
  Wend
  rc=GSrce("CLOSE","")
  ts.Close
  ' invite file notes.txt in TextView!
  ' --> ctaiplay.ini->[TV@NOTES]->TEXTFILE=notes.txt
  DLGVAR="LOC:NOTE=#REOPEN#"
End Sub 'ViewSelectedInfo


'------------------------------------------------------------------------
'- AIP2: Function <ViewSelectedInfo> from < aip_mpdv-AINFO_NOTES.scr>
'------------------------------------------------------------------------
Sub ViewSelectedInfo
  '---------------------------------------------------------------------
  ' write the note selected in the grid into the memo
  '      raw data from server: notes.lst
  ' processed for grid: notes_gr.lst
  '  text for TextViewer: notes.txt (-> is created here)
  '---------------------------------------------------------------------
  Dim asGrid,sKey1,rc,ss,sDatFileName,sTxtFileName
  asGrid=scrGetInfo("GetGridData","DLG=@FIL=DLG=AINFO_NOTES|LINE=-1")
  sKey1=scrDDItem("SUBKEY:1",asGrid)
  sDatFileName=DIR_SPOOL+"notes.lst"
  sTxtFileName=DIR_SPOOL+"notes.txt"
  If scrFileExists(sTxtFileName)="0" Then
    rc=scrFileDelete(sTxtFileName)
  End If
  rc=GSrce("LOAD","FILE="+sDatFileName)
  rc=GSrce("FIRST","")
  While rc<>"#EOF#STORE#"
    ' show info if ANR|SUBKEY:1|INFO.OPT:INFO=J
    If VSrce("SUBKEY:1")=sKey1 And VSrce("INFO.OPT:INFO")="J" Then
      For i=1 To 10
        ss = VSrce("INFO.INFO:"+CStr(i))
        rc = scrWriteDataIntoFile(ss,sTxtFileName)
      Next
    End If
    rc=GSrce("NEXT","")
  Wend
  rc=GSrce("CLOSE","")
  ' invite file notes.txt in TextView!
  ' --> ctaiplay.ini->[TV@NOTES]->TEXTFILE=notes.txt
  DLGVAR="LOC:NOTE=#REOPEN#"
End Sub 'ViewSelectedInfo
```

6.7 Special Fields of Application

6.7.1 Tips and tricks with the dialog control

This section describes tips and tricks to control dialogs.

- Starting a dialog timer for monitoring

  (Example: Cycle 500 msec

  ➔ Processing in <DynDlgTimer_XYZ >)

  ➔ DLGVAR = Item("DYNDLG.TIMER","500")

- Starting a dialog autoclose timer

  (Example: run time <7> seconds + return code <1> = CANCEL

  ➔ Processing in < DynDlgFunctions_ _XYZ >)

  ➔ DLGVAR = Item("DLG.TIMER","7^1")

  (Example: run time <10> seconds + return code <0> = OK

  ➔ Processing in < DynDlgFunctions_ _XYZ >)

  ➔ DLGVAR = Item("DLG.TIMER","10^0")

  (Example: Timer remains active also without dialog focus

  Default is „..^..^1“ / Timer stops if dialog is not active/focused

  Run time <10> seconds + return code <1> = CANCEL

  ➔ Processing in < DynDlgFunctions_ _XYZ >)

  ➔ DLGVAR = Item("DLG.TIMER","10^1^0")

- Creating temporary dynamic dialog variables

  (Example: Variable <*XXX> with value <1>

  ➔ Processing in all < DynDlg.._XYZ > functions)

  ➔ DLGVAR = Item("*XXX","1")

- Controlling of dialog buttons with ID <> “”

  (Example: Button (BTN.<CANCEL>) with text <ESC> and font color <clRed>

  ➔ Processing in all < DynDlgFunctions_XYZ >)

  ➔ DLGVAR = AddIt("BTN.CANCEL","ESC,clRed",cFFEnable)

- Color text field in the dialog red → UserExitDynDlgBeforeInitialize

  → does not work if the field has the attribute "STATUS"

  ➔ If VDlg("DLG")=”…” Then

  DLGVAR = AddIt("INFO","",cFFVisible+"#COL-clRed")

- Prevent opening the dialog → UserExitDynDlgBeforeInitialize

  ➔ If VDlg("DLG")=”…” Then

  DLGVAR=Item("RET","#CANCEL#")

- In the dialog script DynDlgFunctions_XYZ, one can react to escaping the selection list (e.g. scrap reason) via "ESC" :

  ➔ Case "@@LIST_CANCEL"

  OnListCancel

6.7.2 Assignment of a script function to a key without DDLG

In **ctaipbut.ini,** the ID must start with '@':

```
F8=@WKP_CNR_VA_DEL,Voranmld.   Delete
```

Verarbeitung im Skript **aip_system_<Projekt>.scr**:

```
Sub UserExitButtonClick
  Select Case VVar("UE:PAR","BTN.FKT")
    Case "@WKP_CNR_VA_DEL"
      OnButton_WKP
      UE_RET=Item("BTN.FKT","#FKT#->#EXIT#")
  End Select
End Sub 'UserExitButtonClick

Sub OnButton_WKP
  Dim sMnr,sCnr,sRes
  sMnr=VVar("UE:MNR","MNR")
  sCnr=GetVLos(sMnr)
  if sCnr="" Then
    scrMsgBox("kein Los vorangemeldet")
  Else
   sRes=DlgJaNein("delete advance logon","really delete batch logged on in advance?")
    If sRes="#JA#" Then
      DeleteVLos(sMnr)
    End If
  End If
End Sub
```

It is important to set the return value „BTN.FKT=#FKT#->#EXIT#“. Otherwise the error message "unknown button ID..." is displayed.

If the identifier starts with "@@" instead of "@", you do not need to set a return value.

6.7.3 How to use the functions GSrce, VSrce

You can use **GScre()** to access a list file.

The following parameters are possible:
- GSrce (''LOAD'', ''FILE=XXX'') 'XXX is the file that is loaded including directory
- GSrce (''FIRST'', ''XXX'') 'XXX is an optional filter like e.g. MNR=102030
- GSrce (''NEXT'', ''XXX'') 'XXX is an optional filter like e.g. MNR=102030

FIRST and NEXT provide a return code. If the return code is <> "#EOF#STORE#", then a further row has been found

VSrce() is then used to access the current row, e.g. VSrce(''MNR'') is used to read the machine number of the current row.

- GSrce ("CLOSE", "SAVE=TRUE") ' SAVE=TRUE is only set if the file is to be saved.

- sLine=GSrce("GETLINE","") - read row number
- rc=GSrce("SELECTLINE",sLine) - select row („0" – first riw)
- rc=GSrce("DELETELINE","") - delete current row
- rc=GSrce("DELETELINE",sLine) - delete specific row

An example is included in the description of the GSrce() function (chapter „6.3.2.61 GSrce(sFct,sParam)")

6.7.4 Update grid at the push of a button

A grid can be updated by calling DLGVAR=Item("DLG.GRID", "RELOAD"). Requirement: A command has already been assigned to the GRD.CMD to get the list. This can be done, e.g. in the user exit DynDlgGridInit

```
…
  SCRVARS = "GRD.CMD="     + "DLG=LIST;104|MOD=U|MNR=<MNR>|ANR=<ANR>|"
…
```

6.7.5 Read first row from list file

The script function scrQuickSearch can process the parameter "FIRST" instead of the filter:

```
  asAuftrag=scrQuickSearch(DIR_SPOOL+"anr.lst","FIRST")
```

It is not necessary to set an explicit filter after loading a single-row info list (e.g. nanr.lst, lnr.lst).

6.7.6 Script event when changing cell in the machine list

In the old GUI (AIP8.1, CTWIN), you can use the event "@@MNR.CELLCHANGE" to enable/disable buttons with reference to the selected machine.

```
Sub UserExitButtonClick
  Select Case VVar("UE:PAR","BTN.FKT")
    Case "@@MNR.CELLCHANGE"
      CheckBlockButtonsActivation
  End Select
End Sub
```

```
Sub CheckBlockButtonsActivation
  Dim rc
  If GetFu29="J" Then
    rc=scrSetData("ActivateMainButton","PANEL=MNR|BTN=VNR_AN,VNR_AB|ACTIVE=-1")
  Else
    rc=scrSetData("ActivateMainButton","PANEL=MNR|BTN=VNR_AN,VNR_AB|ACTIVE=1")
  End If
End Sub
```

6.7.7 Script event when loading additional info

An event is triggered after the operation additional information has been loaded. Here, the file can be manipulated from the script before it is read by the terminal program. The type of the additional info and the file name are passed in the global variable "#AINFO#".

```
Sub UserExitButtonClick
  Select Case VVar("UE:PAR","BTN.FKT")
    Case "@@AINFO.LOADED"
      If GVars("#AINFO#","TYPE")="AI" Then
        If scrFileExists(GVars("#AINFO#","FILE"))="0" Then
          ' ... change the file …
        End If
      End If
  End Select
End Sub
```

6.7.8 Extended customizing with label printing

The parameter "**PRN->PARAM**" is used for an extended customizing of a configured label with a posting event/dialog. Using this parameter, you can control if the "print order" is completely stopped or if only printing is stopped.

Parameter	Description
„**PRN->PARAM=SKIP PRINTJOB**“	Print order is completely stopped.
„**PRN->PARAM=SKIP PRINTING**“	Label printing is stopped. A server script configured in the label and a configured logging are performed. (Function available as of AIP V# 8.2.0.40)

Example: Label printing is stopped or cancelled for the customer-specific posting event/dialog "Entry of quantities (U_MENGE)".

```
Sub DynDlgBeforeSend_U_MENGE
  Select Case VDlg("PRNMODE")
    Case "L"
      DLGSND=Item("PRN->PARAM", "SKIP PRINTING")
    Case "N"
      DLGSND=Item("PRN->PARAM", "SKIP PRINTJOB")
    Case Else
  End Select
End Sub
```

6.7.9 Notes on the centralized MDE

When using the central MDE, the first step in accessing the controller is to specify which PCC is responsible for MDE processing on the machine. To do so and to access the control (machine control), the following functions are available in the terminal scripts:

- `vbsGetCentralPccID(sFilter)`
- `vbsCentralPCCValues(sCMD,sPCCID)`
- `scrPCCValues(sValue)`

For a detailed description of the functions, refer to section *"Script functions".*
You can use the functions to access the control 8machine control) via the functions **GETVAL** and **SETVAL** for e.g.:

- setting outputs
- customer-specific connection of balances
- transfer of setting data to a machine with operation logon

Requirements:

- Installed SP 13 and included hotfixes
- MQTT – must be installed and activated
- The following licenses are required --> Licenses (authorization keys)
  - AIP-EBM#8.2, SCS-PCB (MDE-NOTIFICATION)
  - PDV-RPM#8.2, PDV-RPM#8.3 (PDV-RPM)
- The following program versions are required at least:
  - ctaip.exe - 8.2.2.6
  - pcc.exe - 7.2.4.6
  - hymwmde72.dll/.so - 8.1.1.144

6.7.10 Function to identify an order info

You can use the function below to read the order information of an order transferred.

The data can be identified and used via the following command:

```
  asAnr = sys_GetAGDataAnywhere(sAnr,sMnr)

Function sys_GetAGDataAnywhere(sAnr,sMnr)
  Dim asAnr
  sys_GetAGDataAnywhere=""
  If sAnr="" Then Exit Function
  asAnr=scrQuickSearch(DIR_SPOOL+"anr.lst","ANR="+sAnr)
  If asAnr="" Then asAnr=scrQuickSearch(DIR_SPOOL+"vlist."+sMnr+".lst","ANR="+sAnr)
  If asAnr="" Then asAnr=scrQuickSearch(DIR_SPOOL+"nanr.lst","ANR="+sAnr)
  If asAnr="" Then
    '****************************
    asAnr=sys_GetAGDataFromDB(sAnr)
    '****************************
  End If
  sys_GetAGDataAnywhere=asAnr
```

```
End Function


Function sys_GetAGDataFromDB(sAnr)
  sys_GetAGDataFromDB=""
  LSTVARS=""
  LSTVARS="LST.FILE="+"nanr.lst"
  LSTVARS="LST.CMD="+"DLG=LIST;11|MOD=A|ANR="+sAnr+"|"
  '*********
scrFktList
  '*********
  If VVar("LST.CMD:RET","RET")="0" Then
    sys_GetAGDataFromDB=scrQuickSearch(DIR_SPOOL+"nanr.lst","ANR="+sAnr)
  End If
End Function
```

The function is fast because it first searches the local lists for the operation. Only if the operation is not found locally, the data will be requested from HYDRA Server.

6.7.11 Correct use of the component list with/without resources in the function "Log operation on"

Depending on the machine configuration, either the mat.lst or the combined resource/material list (fhm.lst) is active when logging on an operation. You can use the following function to read the fields of the correct list.

Example to read articles:

```
Function GetVISFHMTNRAAN(sMnr)
  Dim asMnr, sAtk, sFilter, rc
  asMnr=scrQuickSearch(DIR_SPOOL+"mnr.lst","MNR="+sMnr)
  If scrDDItem("VISFHMTNRAAN",asMnr) = "J" Then
    FileName="fhm.lst"
  Else
    FileName="mat.lst"
  End If
  sFilter=""
  rc=GSrce("LOAD","FILE="+DIR_SPOOL+FileName)
  rc=GSrce("FIRST",sFilter)
  While rc<>"#EOF#STORE#"
    If VSrce("DLL")="" and VSrce("ART")="M" Then
      sAtk=sAtk+VSrce("ATK")+"|"
      rc=GSrce("NEXT",sFilter)
    End if
  Wend
  rc=GSrce("CLOSE","SAVE=FALSE")
End Function
```

6.7.12 Staff badge number with leading zeros

You can change the badge number transferred and extend it to the badge number length defined in the basic settings. If required, the badge number is filled with leading zeros.

```
Function sys_fillKnr(sKnr)
  sys_fillKnr = StrFmtRight(sKnr,vbsIntDef(VTnr("LEN:KNR"),0),"0")
End Function
```

6.7.13 Change XML layout of script

After completing a function it may be necessary to switch to a specific XML page.

Request the current XML page: `VVar("UE:DAT","XML-GUI")`

All active XML pages are displayed separated by "comma".

Result:

L_VIEW_MNR -> icon view

L_VIEW_MNR,L_MAIN -> main view /overview

L_VIEW_MNR,L_MAIN,L_ANR -> detail view ANR

You can change to an XML page via `rc=scrSetData("XML.ShowLayout","LAYOUT=L_VIEW_MNR")`.

Example:

```
Function UserExitDynDlgAfterSend
Dim sXml, iDlgRuns
  Select Case VSnd("DLG")
    Case "A_TR"
      sXml = VVar("UE:DAT","XML-GUI")
      iDlgRuns = CInt(VVar("UE:DAT","DLG-RUNS"))
      If scrGetPart(sXml,",",2)<>"" Then
        If iDlgRuns=0 Then
          ' if no dialog is open, change layout to main screen
          rc = scrSetData("XML.ShowLayout","LAYOUT="+scrGetPart(sXml,",",1))
        End If
      End If
  End Select
End Function
```

Note: Changing the XML layout triggers the function "@@XML.LayoutChanged" in UserExitButtonClick.

In order to react to the button "Register PLC" from within the script, the global variable GVars("$XMLGUI$PAR", "CAPTION") can be read.

Example:

```
If GVars("$XMLGUI$PAR","CAPTION") = "+" Then
  DLGVAR=Item("MNR",VMnr("MNR"))
End If
```

7 AIP2 - Local Configurations File ctaipbut.ini

Buttons are configured for specific terminals in the file ctaipbut.ini stored in the terminal directory c:\MPDV\AIP2.

The button pages of the main view and the OP info dialog may be configured in the configuration file ctaipbut.ini.

The buttons can only be configured like this in the main view if the new design of the AIP2 has been deactivated.

The server directory \<serverDir>\ctnet\win\aip2 contains the complete INI files of the standard. Deviations from this are created in the customer-specific directories provided for this purpose, e.g. \<serverDir>\1\custom\aip2\tgrp_901.

Create the corresponding, empty file (e.g.: ctaipbut.ini) in this directory. Copy all sections e.g. [ANR-ALL-Page1] to this file. The configuration is performed in this file.

After the terminal restart a merge (summary) of the files from the root directory \<serverDir>\ctnet\win\aip2 with the files of the custom directory \<serverDir>\1\custom\aip2\tgrp_901 takes place, which are transferred locally to the terminal in the directory c:\MPDV\AIP2.

All sections including the string "-Page" are imported.

Entry	Comment
Definition of sections [LST-MODUS-PageX.]	**General schematic structure of a button page** **Definition of a section** LST = List identifier of the button page (MNR, ANR, LIST3) MODUS = Mode of the machine LN = MPL – Mode DN = DLL – Mode LR = RF – Mode (reel-based manufacturing) LS = RS – Mode (cutting reels) LC = Handling unit (packing station) or XX = <MPL_MOD>[1] + <TYPE>[] YY = Value from the MNR.LST column <MNRBTN.MODUS> Otherwise if no applicable entry is found for the machine mode, the section ...ALL will be used (if available) X = Button page The definition specifying the mode a machine is running on is implemented in the AIP application program.
Sample configuration [MNR-...-Page1] 1=A_AN, L, log on OP 2=BLANK, L 3=$MPL-PAL$PAL_AN,L,log on pallet, 4=%BART:PZE=J%PZE,R,PZE,PZE.PNG Note: In one section numbering of entries must be consecutive from 1...n. A gap in numbering indicates the completion of a page!	General structure: x=<Function>,<Alignment>,<ButtonName>,<Icon> For example: 1=A_AN,L,log OP on,AGAN.PNG - Function A_AN - Alignment L or R (from the first "R“ on always "R“) - ButtonName Log on OP - Icon optional icon name (PNG, resolution 24x24 px) Special functions: $...$ (e.g. $MPL-PAL$) License check fails → Button is deleted %...% (e.g. %BART:PZE=J% .) Check field with value in (T)terminal (K) label ➔ only show if they match BLANK Insert distance between buttons

Entry	Comment
Configuration of functions using wildcards x=A_AN*,L, log on OP x=A_UN*,R, interrupt OP	The dialog to be opened is located as described below if buttons are configured using wildcards ID A_AN* - Calling dialog: A_AN - Identification of the machine type - Supplementing the dialog based on the machine type - Check whether or not the dialog is available if this is the case - calling dialog: A_AN_MPL - Evaluation of the posting type (only with A_AN) - Supplementing the dialog based on the *posting type* - Check whether or not the dialog is available if this is the case - calling dialog: A_*P*_AN_MPL - Calling up the located workflow or dialog If the function is <A_UN*> or <A_AB*>, it will be checked whether or not the OP to be logged off is a merged OP. - If this is the case, <A_UN*> is changed into <SA_UN> or <A_AB*> into <SA_AB> If the virtual column <MNRDLG.SUFFIX> includes a value, it will always be used (if available). ButtonFkt MNRDLG.SUFFIX Dialog e.g. A_AN* <XYZ> A(_*P*)_AN_XYZ A_TR* <ABC> A_TR_ABC

Further standard buttons:
`P_SPERRE`
`VIEW`
`ICON`
`PDV_ISTW`
`WF_BDE_KOM`
`SA_AN`
`DNC`
`MINIMIZE`
	Lock status "production"
	Switching of the basic view:
	List view ←→ presentation of individual machines
	Calling up icon view (only possible if configured in the machine configuration)
	Calling up the actual value view of PDV
	Input of BDE comments
	Log on merged operation
	Calling up the DNC startup screen
	Minimizing of the terminal program → Windows 7 requires the compatibility mode XP

`USER1…USER9`	User-defined buttons showing and starting external software The programs are configured in the section [ext. software] of the ctaip.ini file

Button IDs for the machine info
`[MNR-ALL-Page2]`
`1=M_INFO.INFO,L,show information`
`2=M_INFO.PERS,L,staff`
`3=M_INFO.MSPROT,L,machine status log`
	Consequently, the relevant info dialog including the selected page is opened in the foreground. Switching to other pages is allowed.
	M_INFO may be used to show the info page in the foreground:
	M_INFO=M_INFO.INFO

Entry	Comment
Button IDs for OP info: [ANR-ALL-Page3] 1=A_INFO.DOKU,L,documents 2=A_INFO.HILF,L,production resources and tools 3=A_INFO.KOMP,L,components 4=A_INFO.BMK,L,RPA 5=A_INFO.FORT,L,progress 6=A_INFO.NOTE,L,notes A_INFO.TEXT1,L,User text1 A_INFO.PICT1,L,User image1 A_INFO.SCRINF1,L,User scriptInfo1 A_INFO.DIALOG1,L,User dialogs	Consequently, the relevant info dialog including the selected page is opened in the foreground. Switching to other pages is allowed. A_INFO may be used to show the information page in the foreground: A_INFO= A_INFO.INFO Direct call of user-defined pages configured in the section [OP info] of the ctaiplay.ini file. Example: Dialog1=WF_BDE_KOM_LIST,BDE comments → A_INFO.DIALOG1,L,BDE comments
Configuration of a function with different modes Just as it is the case for the configuration of the ANR/MNR info tabs, a function can be configured with different modes. 1=RES_WART.MNR,L,machine maintenance 2=RES_WART.RES,L,resource maintenance 3=RES_WART,L,other maintenance	In the configured examples, the dialog < RES_WART > is requested with the below-mentioned modes. 1 < MNR > 2 < RES > 3 without mode The values can be read out as follows in the terminal script. VPar(“BTN.FKT“) Function + mode VPar(“BTN.FUNC“) Function VPar(“BTN.MODE“) Mode

Entry	Comment

Available button sections and buttons for pages of the OP info dialog:
 	
`[A_INFO-Page1]`
 	
`[A_INFO.DOKU-Page1]`
`3=AI_VIEW,R,open document`
`4=AI_VIEW_CLOSE,R,close document`
 	
`[A_INFO.HILF-Page1]`
 	
`[A_INFO.KOMP-Page1]`
 	
`[A_INFO.BMK-Page1]`
 	
`[A_INFO.FORT-Page1]`
 	
`[A_INFO.NOTIZ]`
 	
`[A_INFO.DEFAULT-Page1]`
 	
Recommended for all pages:
`1=AI_CLOSE,L,close OP information`
	Overview
	
	Document view
	
	Production resources and tools
	
	Components
	
	Resource Performance Accounts (RPA)
	
	Progress bar
	
	Notes
	
	Configuration of a default page (used if no section is defined for the tab).
	
	The IDs may also be used for the keys in the dynamic dialog (field "function").

Available button sections and buttons for pages of the machine info dialog:
 	
`[M_INFO-Page1]`
 	
`[M_INFO.PERS-Page1]`
`2=P_AN,R,log person on`
`3=P_AB,R,log person off`
`4=P_AAB,R,log everyone off`
 	
`[M_INFO.MSPROT-Page1]`
 	
`[M_INFO.DEFAULT-Page1]`
 	
For all pages:
`1=MI_CLOSE,L,close machine information`
	Overview
	
	Staff logged on
	
	Machine status log
	
	Configuration of a default page (used if no section is defined for the tab).

Entry	Comment
Definition of sections **[ButtonPanel]**	**General section for the configuration of global settings for all used button panels** **→ 2 in main view (MNR , ANR)** **→ (W)ork(F)low**
functionkey_visible=on	Shows function keys (e.g. "F3") in button panels in order for the selection to be made using function keys (by default = off).
radiobuttonkey_visible=on	Presentation of function keys in radio group boxes of a workflow (by default = off).
functionkey_pze_visible=on	Display of function keys in PZE module (by default = off).
Definition of sections **[LIST3-ALL-Page1]** **as of CTAIP V# 2.0.2.33** **..=~<VISLIST-ID>~,L,,<PNG-File>** The characters "~" (or previously "§", should no longer be used) have been designed to identify third list buttons. Correct processing/updating (disabled/enabled) is only possible in the third grid list of the main screen.	**General section configuring functions of the configurable third list of the main screen.** INFO: The different types of the "3rd list" are configured in the machine label. The layout of a "3rd list" is defined in the "hytnrcfg.ini" file. ➔ All used lists have to be configured with their identifier „“ as follows. ➔ When changing machines, the "3rd list" is hidden/shown and buttons for "3rd lists" that are not configured are disabled.
1=~**M**~,L,,**PALETTE20x20.PNG**	Entry for "material list" ➔ "[VISLIST3(**M**)]“ from "hytnrcfg.ini“
2=~**P**~,L,,**PERSON20x20.PNG**	Entry for "list of persons“ ➔ "[VISLIST3(**P**)]“ from "hytnrcfg.ini“
3=~**R**~,L,,**RESS20x20.PNG**	Entry for "MNR_AMAT.LST“ ➔ "[VISLIST3(**R**)]“ from "hytnrcfg.ini“ with the configured Bitmap „“
4=~**A**~,L,,**NUM.PNG**	Entry for "material list" ➔ "[VISLIST3(**A**)]“ from "hytnrcfg.ini“
5=~**G**~,L,,**PERSON20x20.PNG**	Entry for "list of persons GWP“ ➔ "[VISLIST3(**G**)]“ from "hytnrcfg.ini“

8 AIP2 - Local Configuration File ctaiplay.ini

The layout is configured for specific terminals in the file ctaiplay.ini stored in the terminal directory C:\MPDV\AIP2.

The layout is configured for specific terminals in the file ctaiplay.ini stored in the terminal directory C:\MPDV\AIP2.

This file is basically used for the configuration of grids in AIP2.

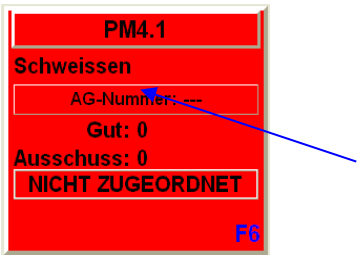
The complete standard INI files are located on the server directory \mip\ctnet\win\aip2 Any deviations from the standard are created in the customer-specific directories provided for this purpose, e.g. \mip\1\custom\aip2\tgrp_901.

Create the corresponding, empty file (e.g. ctaiplay.ini) in this directory. Modified sections are copied to this file. Make the respective configurations in this file.

After restarting the terminal, files from the main directory \mip\ctnet\win\aip2 are merged with files from the customized directory \mip\1\custom\aip2\tgrp_901. Then the merged file is transferred to the local terminal directory C:\MPDV\AIP2.

Changes to the configuration file ctaiplay.ini will not take effect until the terminal software has been restarted.

Entry	Comment
Section [OP info]	
Deaktiviert=AG_Bmk,AG_Fort	- indicated info pages are not shown - AG_Info (OP info) cannot be disabled - Entries affected by sorting are not disabled.
Sortierung=AG_TechInfo,*	- Order of info pages in the icon list - if the list ends with " ,* " the non-listed standard pages are added at the end. - Standard pages: AG_Info,AG_ZuInfo,AG_Bmk,AG_TechInfo,AG_Fort,AG_FertPap
Section [main]	
Nachkommastellen=0	Decimal places for quantities in the order/machine overview
Repaint_time=60	Cycle for updating the view (for machine list and machine info)
PopupSize->EmptyQueue=300	Empty popup window size for quick queue
PopupSize->ReloadPze=200	Reload popup window size for PZE configuration
SymbolSubstDesignation=MBEZK	The specified field replaces the machine number in the icon view.

Entry	Comment
SymbolAdditionalInfo=MBEZK	Display of any field from the machine list in the icon view between machine number and operation number:  The configured field replaces the machine number for lines and aggregates. Please note: Only in the design/GUI of AIP 8.1
MaxExpressions=50	For the configurations used in the list layouts for coloring rows or cells, 20 entries can be made by default. e.g.EXAMINE_CELLBKCOLOR20=.. The MaxExpressions setting can be used to increase the number of entries. → EXAMINE_CELLBKCOLOR50=.. This maximum index applies to all EXAMINE configurations in all grids. Internally, a corresponding amount of memory is always reserved for each grid, even if no EXAMINE configuration is used. (from AIP 8.2.1.12)
Sections for list layouts	
[Personenliste]	List displayed when staff is logged on
[Bedienposition]	Predefined list of "operator positions"
[Maschinenstatusliste]	Predefined list of "machine statuses"
[Ausschussgruende]	Predefined list of "scrap reasons"
[Abweichungsgruende]	Predefined list of "deviation reasons"
[Auftragsliste]	Lower list in the main view
[Vorgabeliste]	Order sequencing list
[Schichtinfo]	List of shift info
[Maschinenliste]	Upper list in the main view
[Eingangslosliste] input batch list	List of input batches, e.g. when "logging on the OP" in batch mode
[Ausgangslosliste]	List of preceding batches when "changing output batches"
Syntax of table formatting	
GRID_FONT=Arial	Font type
GRID_FONTSIZE=9	Font size
GRID_COLOR=clBlack	Font Color
GRID_BACKGROUND=clWhite	Background color
GRID_FIXCOLS=0	Number of fixed columns

Entry	Comment
GRID_ORDER=MSZEIB GRID_ORDER=MSZEIB=-	Sorting in ascending order Sorting in descending order Sorting is executed according to the formatting of the column. Examples: ANR_DATB=**C10**,65,L, planned start ➢ Alphanumeric sorting (the date is provided in format MM/DD/YYYYY) ANR_DATB=**dd.mm.yyyy**,65,L,planned start ➢ Sorting by date If several criteria are indicated (separated by |) only the first criterion can be sorted in descending order. All other criteria are sorted in ascending order. The following entry must be set in the configuration for the section so that the sorting is used in the display: ORDER=#USE#INI#ITEM# Example for the section *Sequencing List (Auto)* *[WF@ANR]* *CMD=DLG=LIST;11|MOD=V|MNR=<MNR>|* *…..* *SECTION=Sequencing List (Auto)* *…..* *ORDER=#USE#INI#ITEM#*
GRID_LIST_TYP=MNR GRID_LIST_TYP=ANR	The list type of the section is indicated with this entry, if fields are displayed that need to be loaded additionally. This entry also enables the search when starting. The entry has to be entered above the IDs to be reloaded!!! All identifiers to be reloaded can be found in the file **headers.dat** in the "spool" directory of the terminal. It consists of four lines: 1. 10|…: Fields that are always included in the machinery list 2. *10|…: Fields that can be reloaded for the machine list 3. 11|…: Fields that are always included in the order list 4. *11|…: Fields that can be reloaded for the order list
EXAMINE_CONTENTS_CHANGE=MGRP EXAMINE_COLOR_C1=clWhite EXAMINE_COLOR_C2=clSilver	The font color switches from clWhite to clSilver every time the MGRP value changes. Up to 8 colors can be defined.
EXAMINE_SCANEXPR1=MGRP=71|72|73 EXAMINE_SCANEXPR2=MGRP=96|97|101 EXAMINE_SCANCOLOR1=ClGreen EXAMINE_SCANCOLOR2=ClRed	The machine groups 71/72/73 are presented in green font color; the groups 96/97/101 are displayed in red font color. Up to 8 colors each can be defined.

Entry	Comment
EXAMINE_SCANBKEXPR1=BATTRIB=1 EXAMINE_SCANBKEXPR2=BATTRIB=2 EXAMINE_SCANBKCOLOR1=clBlue EXAMINE_SCANBKCOLOR2=clLime	All lines with BATTRIB=1 are shown in blue background color; rows with BATTRIB=2 are displayed in lime. Up to 8 colors each can be defined.
EXAMINE_COLOR=TEXTCOLOR EXAMINE_BKCOLOR=HGRCOLOR	Specification of a column that includes the color value for the row (e.g.: 0-Black; 255-Red, 16777215-White)
EXAMINE_CELLBKLEVEL2=EGR:AUSP,EGR:AUSP,<1*clLime|<=5*clYellow|>15*clRed	Setting of the background color depends on whether the field value reaches different threshold values.
EXAMINE_ROWBKLEVEL1=SGR:REST, SGR:REST,<=0*clRed|<=5*clLime|>15*clYellow	Setting of the background color depends on whether the field value reaches different threshold values. The entire row is colored because of the threshold value.
Syntax of column definitions	
MNR=C8,80,R MGRP=N6,60,R AGR:AUS=N10.2,125,R MSDATB=dd.mm.yy,70,L MSZEIB=hh:mm,70,L SKDATB=dd.mm.yyyy,90,L SKZEIB=hh:mm:ss,80,L MSDAUER=ddd.iii,60,R	Alpha-numeric, 8 characters, 80 pixels, right-aligned Numeric, 6 characters, 60 pixels, right-aligned Decimal, 10 digits, 2 decimal places Displayed in the form "23.03.98", (left-aligned) Displayed in the form "08:24" Displayed in the form "23.03.1998" Displayed in the form "08:24:39" Displayed in industrial time unit " 22,982"
AGR:BMK11=hhh:mm:ss,80,R,TESTHEADER	TESTHEADER: new column caption
ALIAS KOPIE=MNR=N8,120,R,TITEL	ALIAS new name is being introduced KOPIE= new identification MNR= ID in data file N8,120,R, Formatting TITEL column caption in table
ALIAS AKA=MNR[1..3]=N8,120,R,ARRAY[1..3]	The first three characters from MNR are displayed.
ALIAS ATTR=MNR(2)=N8,120,R,PARAMETER(2)	The second part separated by „ ; “ is displayed. Example: „ 12;20;130 “ ➔ „20“
ALIAS U=(R*6.2831)=N10.3,60,L,Scope	Conversion of a value Syntax: see below
ALIAS SOLLZ={_INT((_DATETIME(|SKDATE|,|SKZEIE|)-_DATETIME(|SKDATB|,|SKZEIB|))*86400)}=hh:mm:ss,60,R,SOLLZ ;target time of shift	Complex calculations relating to several fields Syntax: see below
GRID_BROWSEROW=0	only the active row is colored yellow
GRID_CELLPAINT=ON	Requirements for coloring rows column by column
GRID_REFRESH=5000	Cycle for updating the display [ms] → lists are not reloaded from the server! Recommended if a constantly changing value is calculated using an ALIAS function.

Entry	Comment
GRID_POSITION=ON	Display of the grid position  Limited/no support when scrolling using scroll bars and page scrolling
GRID_CELLPAINT=ON EXAMINE_CELLBKCOLOR=WTK:STA,WTK:STA,0-clGreen|1-clBlue|2-clYellow|3-clRed can also be used with index: EXAMINE_CELLBKCOLOR1..8 EXAMINE_CELLBKCOLOR1..20	One single column is colored in every cell subject to a value. 1st value: ID of the column to be colored. 2nd value: ID of the reference column 3rd value: Configuration (color for possible values) Notes: – The reference column MUST be shown in the list, if required with length 0 – The values are converted into capital letters when being compared.
EXAMINE_CELLBKCOLOR=DMY,COLOR	Take over the color directly from the "color" column. The column <DMY> is shown in the color defined in the column <COLOR>
; Definition virt. column(1) EXAMINE_CELLVALUE1=CV1,REF1,S=DAT|P=ZEI|A=REST|N=INFO ; Definition virt. column(2) EXAMINE_CELLVALUE2=CV2,REF2,10=MGRP|20=MNR|30=COLOR|40=MSTTXT ... ; Layout/Position virt. column(1) CV1=CELLVALUE,150,Z,Data ... ; Layout/Position virt. column(2) CV2=CELLVALUE,150,Z,M/C/T	Filling of a virtual "Case" column with values from different columns subject to the value of a reference column. 1rd value: Identification of the virtual "case" column 2rd value: Identification of the reference column 3rd value: Configuration Reference value + ‚=' + display column Please note: the virtual "case" column needs to be configured as follows <Identifier>=<Key word>,<Width>,<Alignment>,<Caption> e.g. CV1=CELLVALUE,150,Z,YST
GRID_RANDOMSORT=ON	This options randomly sorts the list. Please note: If this option is active, any configured sorting will be ignored.
GRID_CLIPBOARD=<BUTTON>@<SELECT>@<DATA>	This option copies data from a table/grid into the clipboard.
Special entries	
[Maschinenliste] ALIAS StkProMin=IZYSM= N8,48,R,Stk/min	Activation of calculation & display of the produced pieces per minute

Entry	Comment
[layout pze]	Configuration of the PZE terminal
KundenBitmap=kunde.bmp	**"Kundenbitmap=<File name>**“ file with customer logo When restarting the terminal, this file is copied from the server directory ".\ctnet\win\aip2\etc\“ into the application directory ".\etc\“.
DienstGangTaste=1,3	„**DienstGangTaste=1,3“** → Default [empty] By entering the function key numbers (1...4), a check specifying if the person is allowed to go on a business trip is performed during the posting.
StdSchrift=Arial StdDateSize=30 StdStatusSize=26 StdSpdBttnSize=16 InfoSchrift=Courier New InfoSchriftSize=20 SmallStatusFontSize=16 DateTimeLayout=dd.mm.yyy hh:mm:ss	Configuration of the used font types/font sizes as well as the layout of the date and time display.

8.1 Formulas used in grid layout

Simple conversion of a value

```
Syntax:      ALIAS <Alias>=(<formula>)=formatting
  <formula>: [1/]<ID>[<Operator><Value2>]
    <ID>: ID from list (The current value from the list is entered here
          in the formula)
<Operator>: + | - | * | / | ^
  <Value2>: 2nd Operand
```

Extensive formulas:

Formulas that can also relate to several table fields can be recognized by braces.

```
 Syntax: ALIAS <Alias>={<Formula>}
 <Formula>: (<Operand1>[<Operator><Operand2>])
 <Operand>: <Value> / <Function> / <Formula> / |KENN|
<Operator>: |+|-|*|/|^|
   <Value>: Constant ('0'..'9','e','E','.','-')
    |Kenn|: reads out a value from the table
<Function>: _<Fname>(<Operand>[,<Operand>[,...]])
           _DATETIME(<Date>,<Time>)
             <Date>: mm/dd/yyyy
              <Time>: ssss (Seconds of the day (0..86400))
             ==> Real value as TDateTime
           _INT(<Operand>)
```

```
          returns a value without decimal places
        _REAL(<Operand>)
          returns a value with decimal places
        _ROUND(<Operand>)
          returns rounded value without decimal places
        _MOD(<Operand1>,<Operand2>)
          ==> Operand1 mod Operand2
        _ABS(<Operand>)
          absolute amount of a figure
        _EXP(<Operand>)
          e raised to the power of X (e: basis of the natural logarithm)
        _LN(<Operand>)
          natural logarithm (Ln(e) = 1)
        _FRAC(<Operand>)
          proportion of decimal places
        _LOG(<Operand1>,<Operand2>)
          LOG(N,X): logarithm to base N of X
        _MAX(<Operand1>,<Operand2>)
          the greater value of two values
        _MIN(<Operand1>,<Operand2>)
          the lesser value of two values
        _SQRT(<Operand>)
          ==> Square root of Operand1
        _PI()
          ==> 3.14151926535...
```

Examples:

Calculation of the target time of a shift (ZEISS):

ALIAS SOLLZ={_INT((_DATETIME(|SKDATE|,|SKZEIE|)-
*_DATETIME(|SKDATB|,|SKZEIB|))*86400)}=hh:mm:ss,60,R,SOLLZ*

ALIAS TEST={_INT(|AGR:GUT|/|TLG|)},N3,30,R,Test

ALIAS test1={_LN(2.7182818)}=C8,80,L,Test1

New (V7.2.3.74): Utilization of intermediate variables in ALIAS functions:

ALIAS **U_Brutto**={|AGR:GUT|+|AGR:AUS|}=N8,40,Z,Brutto

ALIAS **U_BPMN**={_REAL(60000/|SZY|)*|TLG|}=N8,35,Z,BpmN

ALIAS TK_TEST={|***U_Brutto**|+|***U_BPMN**|}=N8,35,Z,TK*

Setting of the background color depends on whether the field value reaches different threshold values.

```
Syntax: EXAMINE_CELLBKLEVEL<i>=<Akro>,<Akro_ref>,
                               <Comp1><Val1>*<Col1>|
                               <Comp2><Val2>*<Col2>...
       <i>: Index 1..8
    <Akro>: Identification of the field to be colored
<Akro_ref>: Identification of the reference column
    <Comp>: Limiting characters (<,>,<=,>=)
    <Val1>: Limit value (integer or Real or ID
            of a reference value for comparison purposes)
            alternative: (Akro) - column of limit value
    <Col1>: Color (Delphi name)
```

Threshold values are searched from the left to the right. If a "<“ or a "<=“ – criterion is met, the corresponding color is set and the evaluation/report is finished. If a ">“ or ">=“ criterion is met, it will first be checked whether or not the condition that follows is also met.

The direct comparison with "=“ is not allowed. But the same function can be achieved by processing the comparisons relating to "<“..„<=“ or „>“…“>=“.

An identification put in parentheses may also be indicated instead of the limit value. During the comparison, the current field content including the specified ID is read out from the same row as the limit value.

All three fields (field to be colored, reference field and limit value field, if required) must be configured as fields to be displayed. The field width can be set to zero if one of these fields should not be visible.

The color value clWhite may be entered to prevent sections from being colored.

The values are compared as they are displayed. The actual values 0.5 and 1 are considered being equal if displayed values are to be rounded to integer values.

Coloring of the field only works if the option "GRID_CELLPAINT=ON“ is set.

The option "GRID_BROWSEROW=0“ should also be set in order for the coloring to be recognized even if the row is selected.

```
Examples:
EXAMINE_CELLBKLEVEL1=MNR,MST,<=1*clLime|<=2*clYellow|>2*clRed
EXAMINE_CELLBKLEVEL2=FS,FS,<90*clLime|>=90*clYellow|>=100*clRed
EXAMINE_CELLBKLEVEL3=EGR:GUT,EGR:GUT,<(SGR:GUT)*clLime|>=(SGR:GUT)*clYellow
```

8.2 Translations in grid layout

Column contents can be configured to be translated and displayed by entering e.g. the configuration <XYZ=T10,100,L> instead of < XYZ=C10,100,L> in the configured grid columns. A <#> character must be prefixed for these "resource strings" to provide for better classification. This modification can be used in every INI file (hytnrcfg.ini,..) where grid layouts are configured.

Please note: The data do not include any translated values. In order for them to be displayed in e.g. dynamic dialog fields, an explicit translation must be performed using the VB script function < vbsTranslateDataValues(“<columns>“ , “<data row>“) >.

Column contents can be configured to be translated and displayed by entering e.g. the configuration <PSPERRE=U1,100,L> instead of <PSPERRE=C1,100,L> in the configured grid columns. The entry for the "resource string" that depends on the field has the following structure:

	„#<Acronym>#<Value>“	
e.g.	**„#PSPERRE#J“**	**"production lock enabled“**
	„#PSPERRE#N“	**„ “ (blank character)**

This modification can be used in every INI file (hytnrcfg.ini,..) where grid layouts are configured.

Please note: The data do not include any translated values. In order for them to be displayed in e.g. dynamic dialog fields, an explicit translation must be performed using the VB script function vbsTranslateDataFields(“<columns>“ , “<data row>“) >.

8.3 Table of color values

Farbe	Name	Color value
	clWhite	$FFFFFF
	clBlack	$000000
	clBlue	$FF0000
	clLime	$00FF00
	clRed	$0000FF
	clYellow	$00FFFF
	clFuchsia	$FF00FF
	clAqua	$FFFF00
	clOrange	$0080FF
		$8000FF
		$FF8000
		$FF0080
		$80FF00
		$00FF80
		$808080

8.4 Modifications to GRID configuration / clipboard

The AIP2 provides for the configuration of copying values from the table into the clipboard.

Data can be copied into the clipboard using the shortcut "Ctrl + C", by right clicking with the mouse or an optionally configured button.

The copied values are transmitted as string in the internal format.

- Date columns as "MM/DD/YYYY“
- Time in "seconds after midnight"
- Durations in "seconds"
- Quantities with a dot as decimal separator

Data is copied including a header into the clipboard. The columns of the header and the corresponding values are separated by <TAB>. Lines are completed with <CR> <LF>.

The configuration is as follows:

GRID_CLIPBOARD=<BUTTON>@<SELECT>@<DATA>@<HEADER>

<BUTTON>	Optionally, using **"Y"** a button can be shown in the top right margin of the table. This button copies the selected data into the clipboard.
<SELECT>	Optional configuration of one or several selection criteria. Selection criteria are separated and/or linked with "|“. **GRID_CLIPBOARD=..@SELECT=X|*@..** The default selection criterion is "X“ (e.g. **@SELECT@ becomes @SELECT=X@)**
<DATA>	The data to be copied into the clipboard can be configured here. - <ALL> All columns of the line - <VISIBLE> Visible columns (Pixel>0) - <COL1|COL2|COL3|…> configured columns For the configuration options <ALL> + <VISIBLE> the selection column is removed automatically from the columns to be copied if only one selection criterion is indicated. In case no selection criterion is stated, the selected line is copied into the clipboard according to configuration.
<HEADER>	As of CTAIP V# 2.0.3.35 **"N"** can be used to prevent the header from being displayed in the clipboard.

The following example shows the machine status list including multiple selection and copy button for the clipboard.

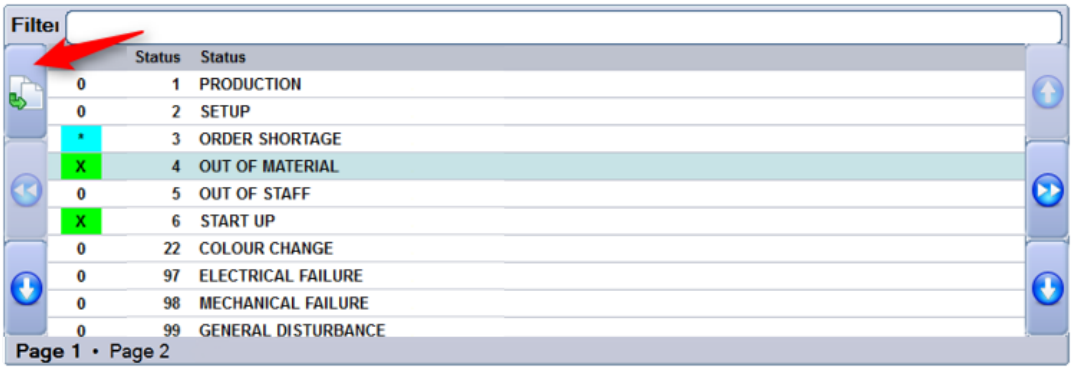


Fig. Configuration with button (red arrow) and multiple selection

```
[Maschinenstatusliste]
GRID_FONT=Arial
GRID_FONTSIZE=9
GRID_COLOR=clBlack
GRID_BACKGROUND=clWhite
GRID_CLIPBOARD=Y@KSTART=X|*@KSTART|MNR|MST|MSTTXT
GRID_CELLPAINT=ON
EXAMINE_CELLBKCOLOR=KSTART,KSTART,X-clLime|*-clAqua

ALIAS LEER1=(DUMMY1)=C1,10,L
KSTART=C1,30,Z,*
MST=N8,60,R,
DUMMY=C3,10,R
MSTTXT=C70,150,L,Status
ALIAS LEER2=(DUMMY2)=C1,475,L
```

The data selected in the screenshot have been copied into Excel using the above-described configuration for the clipboard.

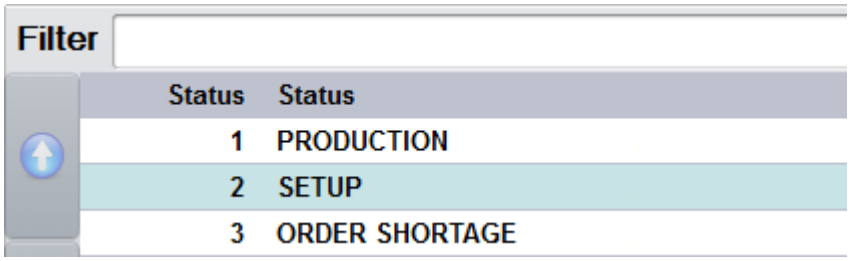
	A	B	C	D
1	KSTART	MNR	MST	MSTTXT
2	*	40312MPL	3	ORDER SHORTAGE
3	X	40312MPL	4	OUT OF MATERIAL
4	X	40312MPL	6	START UP
5				

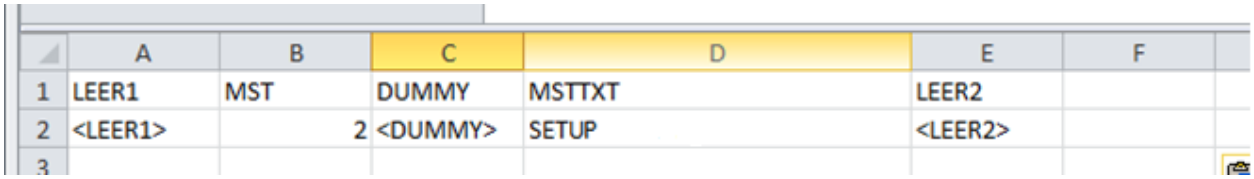
The modified configuration

```
[Maschinenstatusliste]
GRID_FONT=Arial
GRID_FONTSIZE=9
GRID_COLOR=clBlack
GRID_BACKGROUND=clWhite
GRID_CLIPBOARD=@@<VISIBLE>

ALIAS LEER1=(DUMMY1)=C1,10,L
MST=N8,60,R,
DUMMY=C3,10,R
MSTTXT=C70,150,L,Status
ALIAS LEER2=(DUMMY2)=C1,475,L
```

copies data of visible columns (pixel > 0) of the selected line into the clipboard





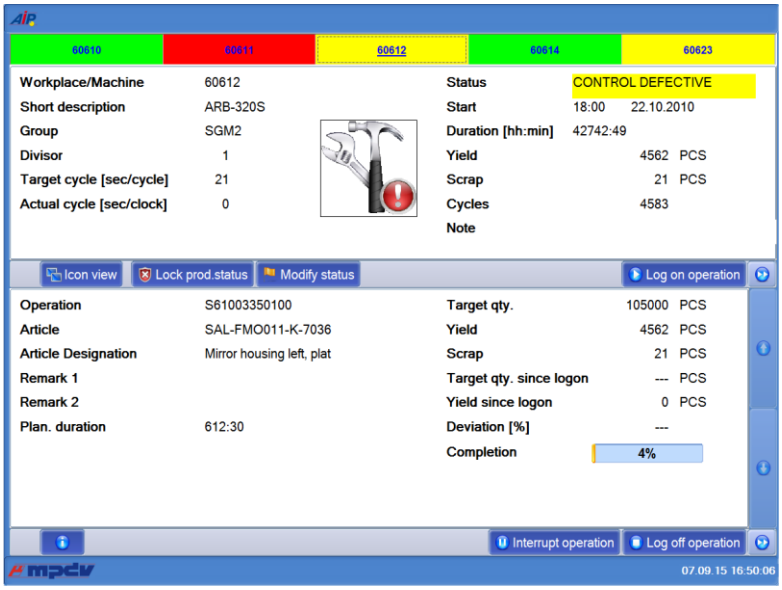
	A	B	C	D	E	F
1	LEER1	MST	DUMMY	MSTTXT	LEER2	
2	<LEER1>	2	<DUMMY>	SETUP	<LEER2>	
3						

8.5 Configuration of basic screens

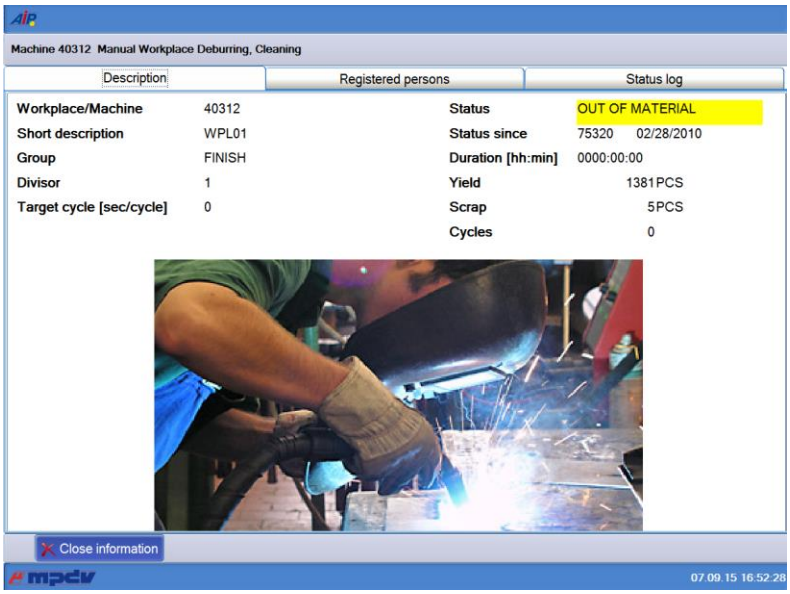
The dialogs/screens are configured using dynamic dialogs. For this reason, the following dialogs are always required:

MMINFO → Section referring to machines in the single machine view

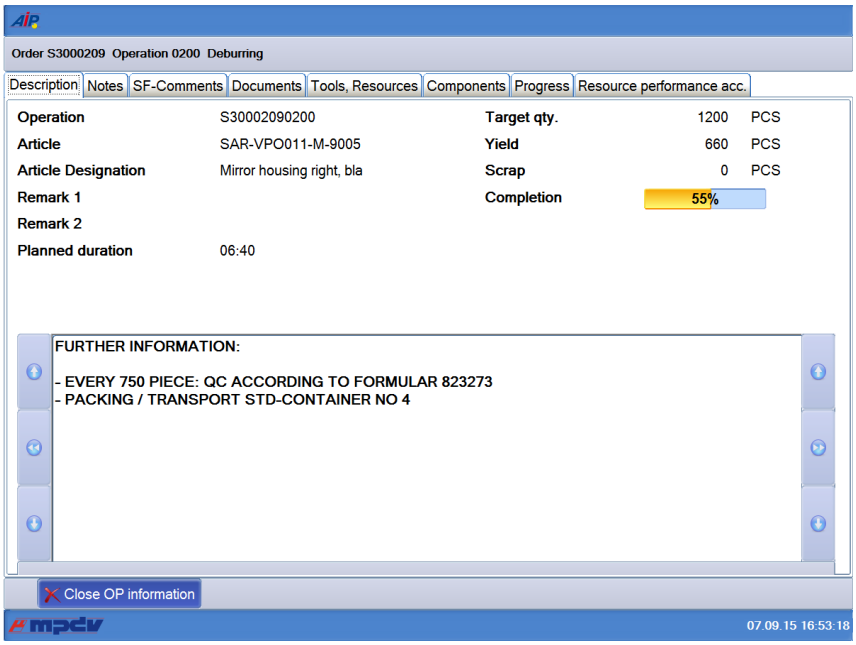
MAINFO → Section referring to orders in the single machine view



MINFO → Description of the machine information



AINFO → Description of the order information



The heights of the individual components of the basic screens and, as a result, the positions of the button bar are configured in the *ctaiplay.ini* file using the below-mentioned parameters:

Section [MainView1]	Configuration of the basic screen
MachineGridHeight=415 OrderGridHeight=500 ButtonBarHeight=50	Height configuration of components for the basic screen (machines, order grid, button bar) The configured heights are scaled to the current height. Consequently, the total sum of entered heights does not play a role.
Section [MainView2]	Configuration of the single machine view
MachineGridHeight=50 MachineInfoHeight=415 OrderInfoHeight=355 ButtonBarHeight=50	Single-row grid to select the machine Information on the machine Information on the order Height of both button bars The configured heights are scaled to the current height. Consequently, the total sum of entered heights does not play a role.

8.5.1 Available fields for the dialog configuration of basic screens

A script function completing the fields according to the customer's requirements is not available .

In general, the fields of the machine list and the order list are available. "MNR." or "ANR." must be prefixed for identification purposes.

Known quantity fields are formatted to match the configured number of decimal places.

Some fields are calculated. The following fields are additionally available:

Identification	Description
ANR.SOLL_SEIT	Target quantity since login The value is determined locally at the terminal. This is only useful for MDE machines. However, the order must be logged on locally after restarting the terminal.
ANR.ABWEICH	Deviation [%] Comparison of "target quantity since logon" and "actual quantity since logon“
MNR.SZY	Target cycle Field is transferred including "internal decimal places". The number of characters displayed is determined by the field of the dialog configuration.
MDE.IZY	Actual cycle The machine's current actual cycle - only if MDE processing is active for the machine at this terminal.
MNR.MSZEIB	Start time of the current status
MNR.MSDATB	Start time of the current status
MNR.MSDAUER	Duration of the current status
ANR.BEARBZ	Planned duration
MNR.MSTTXT	Status text
MNR.TLG	Partitioning Calculated based on the orders running at the machine.
ANR.FERTIG	Progress bar
TNRPSPERRE	Translated text for the production lock (corresponds to the configuration "TNRPSPERRE=U1,150,L,Hinweis“ in ctaiplay.ini) (the value J/N from the list can be found in MNR.TNRPSPERRE)

9 AIP2 - Central Configuration File hytnrcfg.ini

You can use this file as central place to store different configurations for all or for separate terminals.

For each section, a general version is available

[section 0].

The entries included in this section can be overwritten by entries in a terminal-specific section

[section <TNR-USER>]

<TNR-USER> = UserNo = terminal number + 2000 e.g. 2010,2101,..) for exactly one terminal/UserNo

The file hytnrcfg.ini is loaded from the server on every terminal start.

Section / Entry	Comment
	→→→
[Tnr Konfiguration 0]	
`FollowExternStatus=on`	Transfer of machine statuses when reloading machine list. Useful if status change is set by PDM or another terminal
[Terminal->Installation 0]	
`InstallFonts=on`	If set to "off", fonts are not installed during restart. ON=DEFAULT
`OnlyInstallFontsAfterDownload=false`	If "`InstallFonts=on`": If true, then fonts are only installed directly after a download. If false, then fonts are installed every time the terminal is restarted. (false = DEFAULT)
[Terminal->USR 0]	

Section / Entry	Comment
AttachedApplication=First	Displaying documents of OP info: With this configuration, the system first checks whether or not an application is linked in Windows that matches the file extension of the document. This application is then used to display the document. If no link is available, the viewers configured in ctaip.ini (→ [ext. software]) and internal viewers are used. If an extension is completely unknown, the system tries to display the document as text . Different settings are possible: **First** → search for linked application first **AfterUserViewer** → If a UserViewer is configured, this one overrides the linked application (also applies to ExcelViewer, WordViewer and PowerpointViewer) **Last** → Only if no ctaip.ini assignment is found for the file extension, then the system searches for a linked application (**default**). **Off** → The system does not search for a linked application.
HTTPBrowser=standard	Display of documents (via OP info): If documents are configured with a path of schema "http", the file is not downloaded to the terminal, but the link is transferred to a browser. The default browser for the terminal is htmview3.exe, as this one can be operated by touchscreen. If this entry is set, the default browser configured in Windows is used.
SupressErrorMessage=70012	Suppress message "material is not planned"
MSS_DIALOG=10	If the terminal is switched off longer than 15 minutes, a dialog is displayed on terminal restart. The user must then decide whether the counter pulses, which were recorded when the terminal was closed, are posted or discarded. After a configurable period of time, the dialog closes automatically with "Yes" (Yes, posting of pulses). This value configures the time in seconds the dialog is open.
MSS_FILEAGE_MIN=5 MSS_FILEAGE_OVERTIME=delete	If the backup file for counter pulses on the terminal is older than 5 minutes, then no dialog is opened and the backup file deleted. Quantities recorded at the time when the terminal was closed are not used/posted.
[SignatureRecording->User 0]	
ManualBadgeInput=true	This configuration specifies whether or not the field *User* can be edited on the terminal (by default: no editing) **true** → activates keyboard input for field *User* on the terminal

Section / Entry	Comment
Transparency=255	The display of the signature dialog can also be transparent. **255** → Signature dialog is 0 % transparent (not transparent) **1** → Signature dialog is 99% transparent (maximum transparency) (Default = 155)
ShowPosition=TR	You can change the place of the signature dialog: **TL** Top – Left **TM** Top – Middle **TR** Top – Right **ML** Middle – Left **MM** Middle – Middle (Default) **MR** Middle – Right **BL** Bottom – Left **BM** Bottom – Middle **BR** Bottom – Right
USE_SERVICE_ACCOUNT=1	**0** (default) SSO: ServiceAccount is not used (requirement: the terminal must be started with the domain "user" (SSO)). **Note:** ServiceAccount=1 can only be used if all users are in the "root" domain. SubDomain users are not supported.
SIGNATURE_1_USER_TYPE=REPORTING_USER_READONLY	**REPORTING_USER_READONLY** The user identification using the Windows user is activated. The Windows user is then preassigned in field *User*. The *User* field is read-only. Requirement: The "SSO" option must be enabled for all reporting users. Otherwise, successful authentication is not possible. **REPORTING_USER_CHANGEABLE** The user identification using the Windows user is activated. The Windows user is then preassigned in field User. The *User* field can be edited. Requirement: The "SSO" option must be enabled for all reporting users. Otherwise, successful authentication is not possible.

Section / Entry	Comment
SIGNATURE_1_LOGON_TYPE=HYDRA	**"" / Not set / "EMPTY"** There is also an alternative login procedure. **HYDRA** The user identification using the Windows user is locked. You must enter a user for identification. Requirement: All reporting persons must have been created as users and the option "SSO" must not be enabled for the users. Otherwise, successful authentication is not possible. **ACTIVEDIRECTORY** The TAB for the user identification via *User* is locked. The Windows user must be used for identification purposes. Requirement: The "SSO" option must be enabled for all reporting users. Otherwise, successful authentication is not possible. **MIXED_BUT_UNIQUE** The setting of option "SSO" specifies whether the user login or the Windows login is used. "SSO" enabled → Windows only "SSO" disabled → user only
SIGNATURE_2_LOGON_TYPE=HYDRA	Identical to SIGNATURE_1_LOGON_TYPE (see above)
ExtendedSignatureRecording=true	Used for signatures on the terminal with quality data collection.
[MDE/Blade Configuration 0]	
CONVERT-TO-ANSI-FILE=<list1|list2>	Configuration of the files that are provided from the AIP to the MDEB2 blade in ANSI format when a combined operation is available. The following lists are transferred by default if the entry is not available. counters.lst|schicht.lst|mnr.lst|mstat.lst|anr.lst|pnr.lst If you want to transfer further lists, you must specify the standard lists and the additional lists.

9.1 Layout configuration

Entry	Comment
Section [terminal configuration 0] and/or [terminal configuration 2XXX];	**(general configuration) (2XXX terminal-specific configuration)**
AUTO-CONFIRM-UHR-ERROR-MESSAGE=TRUE	This setting specifies that in case of an error that occurred reading the clock (e.g. when activated after standby

Entry	Comment
	mode), the time is transferred without confirmation dialog and the terminal time is later synchronized with the server time via PDM command.
SUPPRESS-MAXIMUM-NUMBER-OF-MACHINES-WARNING=ON	Suppresses the warning after restart of terminal if more than 32 machines are assigned to the terminal (static/dynamic). (Default=OFF)
CalcTargetYieldSinceLogon=2	CalcTargetYieldSinceLogon=**1** The duration is calculated from the total runtime since login (all statuses) minus the configured shift breaks. CalcTargetYieldSinceLogon=**2** The duration is calculated from the total runtime since login (all statuses). Defined breaks are not used and are not deducted. (default value)
Section [QRD-PRINTER->TICKET 0] ;**(general configuration)** [QRD-PRINTER->TICKET 2xxx] ;**(2XXX configuration for a specific terminal)**	
COMPLETE-ABSENCE-OF-LOCAL-MNR-DATA-FOR-EVENT=< Events >	Reloads the machine row for the configured **<Events>,** if it is not available locally ➔ This configuration might be required for a group workplace without machine assignment.
COMPLETE-ABSENCE-OF-LOCAL-ANR-DATA-FOR-EVENT=< Events >	Reloads the order row for the configured **<Events>,** if it is not available locally ➔ This option has been implemented to access order data in the master data, e.g. when logging on orders.
COMPLETE-..-EVENT=< Events > COMPLETE-..-EVENT=#ALL# COMPLETE-..-EVENT=A_AN|A_P_AN	Explanation on the configuration of <Events> ➔ Using <#ALL#> the row (ANR/MNR) that is not available is reloaded for any event. ➔ <A_AN|A_P_AN> restricts reloading of information to specified events. The ID <DLGFAM> is preferred to the ID <DLG> in order to identify the <Event>.
Section **[AIP2 Initialization 0]**	
XML-GUI=OFF	Disables the new AIP2 design and uses the AIP 8.1 design.
CTWIN-STYLE=ON	Activates the GUI that is similar to CTWIN on the AIP2. The two button bars are shown below the two lists just like on the AIP 8.1.
CTWIN-BUTTON-LAYOUT=ON	If the option CTWIN-STYLE=ON is additionally set, the two button bars are displayed at the bottom of the screen.
AUTOMATIC-CHANGE-TO-START-DISPLAY=30	As of AIP 8.2.2.28: If this option is set, the display automatically changes to the main view after the configured time if no other interaction was performed in the meantime. The changing display is configured via the option *Show machine/OP* in the *Terminal configuration*, tab *MF functions*. • List: ○ Change from the detail views or function menus (operation, person, resource, etc.)

Entry	Comment
	○ Change to the main view with "tiles" • Icons: ○ Change from the detail views or function menus or from the main view with tiles ○ Change to the icon view of workplaces The configuration is specified in seconds. Do not specify less than 10 seconds. An automatic change to the start screen is not made if an input dialog is open.

10 AIP2 - Local Configuration File keyboard.ini

You configure the virtual keyboard of the AIP2 terminal in the keyboard.ini file in the directory c:\mpdv\aip2 for the specific terminal.

To activate the changes in the configuration file, you must restart the terminal software.

Logic enabling the virtual keyboard:

The AIP2 terminal shows the keyboard if an input field is focused. The keyboard is placed with reference to the field as described below:

Logic for placing the virtual keyboard:

It is tried to place the keyboard directly below the input field. If there is not enough space to the bottom of the screen, it is tried to place the keyboard directly above the input field. If the space above the control element is not sufficient for the keyboard, the keyboard is placed at the bottom of the screen.

These are the priorities for horizontal alignment:

- to the right of the control
- to the left of the control
- to the edge of the screen that is further away from the control

If the “VirtScreenSize“ option is enabled, the virtual keyboard is not aligned on the virtual screen but still on the real screen. Consequently, the keyboard may also reach beyond the terminal program.

The virtual keyboard can be configured in the local keyboard.ini file on the terminal. Example:

```
[Keyboard]
HideTime=10
ScaleMultiplier=0.9
FixNumbers=ON
Configuration=ON
;Logging=ALL
;Processes=ctaip.exe
;ClassesForLetters=TVtEdit
;ClassesForNumbers=TMPDVSimpleNumericField
```

HideTime

The set value specifies for how many seconds the keyboard is invisible if you click on the key showing the icon  on the left hand side. This key is not visible if the value "0" is entered.

ScaleMultiplier

The keyboard size can be reduced and increased. The value range is between 0.9 and 4.0. A dot is used as decimal separator.

The default value is 1.0.

FixNumbers

Allowed values: ON|OFF

If FixNumbers=On is set, the number keys located in the top row of the virtual keyboard remain visible even if the Shift key or CapsLock key is pressed. ON is set by default.

Configuration

Allowed values: ON|OFF

The keyboard layout, which is installed and activated in the Windows language settings, specifies the layout of the virtual keyboard. You can activate different keyboards in the operating system. For the virtual keyboard, you can then switch between the different activated keyboards.

The entry *Configuration=ON* activates the button  . Use this button to open the dialog to select one of the keyboards activated in the operating system.

Default is OFF.

Logging

Allowed values: OFF|ON|ALL

Logging can be enabled using this entry. The advanced logging is configured by setting ALL.

OFF is set by default.

Processes

The entry "Processes" specifies for which additional processes the virtual keyboard will be used. The separate entries are separated by comma (e.g. processes=notepad.exe.explorer.exe). If this entry is not included, the keyboard for these processes is available in *ctaip.exe* und *iniedit.exe.*

ClassesForLetters

This entry defines for which additional classes the alpha-numeric keyboard should be displayed. The current classes for AIP2 (TMPDVSimpleField, TsEdit, TsMemo, TMPDVTypEdit, TMPDVSimpleEditField, TMPDVPictureField,TEdit, TButtonedEdit, TEditControl) are fixed in the source code. This entry can be used to extend the list.

ClassesForNumbers

This entry defines for which additional classes the numeric keyboard should be displayed. The current classes applicable for the AIP2 (TMPDVNumericField, TPagerNumField, TMPDVSimpleNumericField, TVTEdit) are fixed in the source code. This entry can be used to extend the list.

The classes that are fixed in the source code cannot be overridden using a different entry in the configuration file. If you want to display the other keyboard for a field, you can change the input type of the field in Dialog Configuration.

Dialog-specific configuration

There is the option from version 1.6.0.0 of the keyboard.exe to configure the location of the virtual keyboard per dynamic dialog. The user has to extend the configuration file keyboard.ini accordingly.

Sample configuration:

[WF_AA_QUA]	=> Name of the dynamic dialogs
X-Position=50	=> Distance in pixels from the left edge of the screen
Y-Position=50	=> Distance in pixels from the top edge of the screen

- Specifying the X- and Y-position is mandatory.
- The configuration is only available for dynamic dialogs that are configured on the MOC.

The virtual keyboard can also be switched off if the terminal is connected to a real keyboard. This can be configured in section [SYSTEM] of the local ctaip.ini file.
Example:

```
[SYSTEM]
Parameters=-t
```

Syntax:

+t/-t --> enables/disables the virtual keyboard; irrespective of the terminal type

11 AIP2 - local configuration file ctlisten.cfg/.ini

11.1 Overview

You can load any PDM lists from the server at regular intervals if you customize the files ctlisten.cfg and ctlisten.ini. You can use the data of the lists for displaying the third list or in dynamic dialogs, for example.

File ctlisten.cfg

The file ctlisten.cfg contains the definition of the server-based PDM lists. The definition of the lists must be identical in the entire system. Do not store this file for a terminal or terminal group only to avoid redundant customization.

File ctlisten.ini

Use the file ctlisten.ini to activate the list. You can perform the activation for single terminals, for terminal groups or for the entire system. You can also use the file ctlisten.ini to overwrite selected settings of the definition file ctlisten.cfg.

You can manage the files for all terminals, terminal groups or for a single terminal as follows:

The server directory \<systemDir>\ctnet\win\aip2 contains the complete CFG+INI standard files.

Any deviations are developed in specific, customized directories:

e.g.	for all terminals	.\1\custom\aip2\.
e.g.	for the terminal group 901	.\1\custom\aip2\tgrp_901\.
e.g.	for the terminal 127	.\1\custom\aip2\tnr_127\.

11.2 List definition in ctlisten.cfg

The file ctlisten.cfg contains the definition of the server-based PDM lists. The definition of the lists must be identical in the entire system. Do not store this file for a terminal or terminal group only to avoid redundant customization.

Section / Entry	Comment
[#LIST#<List-ID>] [#LIST#ABC]	Section for list "ABC"

Section / Entry	Comment
CMD=..	Server command for the request of the PDM lists **CMD=DLG=LIST;21|** (LIST;21 is the list of the material types) Possible placeholders are: <TNR> = terminal number (e.g. 90) <USR> = UserNo e.g. 2090) <SPOOL> = <APPDIR>+"spool\
MSG=..	*Optional:* Message text when list is loaded **MSG=File abc.lst is loaded.** Default: **- Load [<local file name>] ...** The configured message text / the default text is output in the update window when the list is loaded.
FILE=..	*Optional:* Local file name of PDM list **FILE=abc.lst ;(Default: ListID+„.lst")** Possible placeholders are: <TNR> = terminal number (e.g. 90) <USR> = UserNo e.g. 2090) <SPOOL> = <APPDIR>+"spool\
AKRONYME=..	*Optional:* Request of user fields with standard lists that support user fields **AKRONYME=FU:23|FU:25** With the above configuration, the user fields FU:23 and FU:25 are added to the standard list.
TEMPCOLS=..	*Optional:* Extension of the list by columns that can be used for the local processing. **TEMPCOLS=H#01|H#02|H#03** With the above configuration, the columns H#01, H#02 and H#03 are put in front in the loaded list.
CHANGEKEYS=..	*Optional:* Key fields of a list row for the processing of "**TRANSFERCOLS**". **CHANGEKEYS=HZTYP[|ID:2|..|ID:N]** The configuration above defines the column HZTYP as key field of a list row. Optionally, you can configure further key fields separated by "|" (pipe).
TRANSFERCOLS=..	*Optional:* Configuration of columns that are automatically transferred after loading of list. Only with "**CHANGEKEYS**". **TRANSFERCOLS=H#01|H#02** The above configuration defines the columns H#01 and H#02 for the automatic transfer after loading of list.

Section / Entry	Comment
LOADCYCLE=..	*Optional:* Load cycle of list **LOADCYCLE=3600 ; (Default: 7200)** The above configuration defines a load cycle of 3600 seconds.
REMOTEFILE=..	*Available as of ctaip.exe V# 8.2.2.19* Configuration of a fixed list file that is loaded at the configured cycle. **REMOTEFILE=system_mpl_atk.lst** This configuration is used to load lists that are created on the server for all terminals. The configuration must not be combined with a "CMD=.." configuration.
QUEUEEMPTY=..	*Optional:* Condition that specifies whether the list may only be updated when the QUEUE on the terminal is empty. **QUEUEEMPTY=TRUE ; (Default: FALSE)** The above configuration specifies that the list is only reloaded when the QUEUE on the terminal is empty. Set this option if terminal postings influence the content of the list.
ACTIONBEFORE=..	*Optional:* Condition that specifies whether the automatic quantities must be transferred for each MDE machine with an M_AST before the list is updated. **ACTIONBEFORE=TRUE ; (Default: FALSE)** The above configuration specifies the following: Before the list is loaded, automatic quantities of MDE machines configured on the terminal are transferred if available. This ensures that when requested the list includes the current automatic quantities that are locally processed. Only use this option in combination with "**QUEUEEMPTY=TRUE**".

Example:

```
[#LIST#ABC]
CMD=DLG=LIST;21|
AKRONYME=FU:23|FU:25
TEMPCOLS=H#01|H#02|H#03
LOADCYCLE=0
CHANGEKEYS=HZTYP
TRANSFERCOLS=H#01|H#02
```

11.3 Activating lists in ctlisten.ini

The file ctlisten.ini contains the activation and the settings for the server-based list, which deviate from the basic configuration.

If required, configure these files

- for all terminals
- for specific terminal groups

- or for a single terminal.

Entry	Comment
[#LIST#<List-ID>] [#LIST#ABC]	Section for list "ABC"
ENABLED=..	*Available as of ctaip.exe V# 8.2.2.15* Activate list via configuration ENABLED=TRUE ; (Default: FALSE) The above configuration activates the PDM list. The list is loaded on restart of the terminal and cyclically as configured.
LOADCYCLE=..	Optional: Deviating load cycle of list LOADCYCLE=900 The above configuration replaces the cycle specified in the basic configuration and specifies a cycle of 900 seconds.

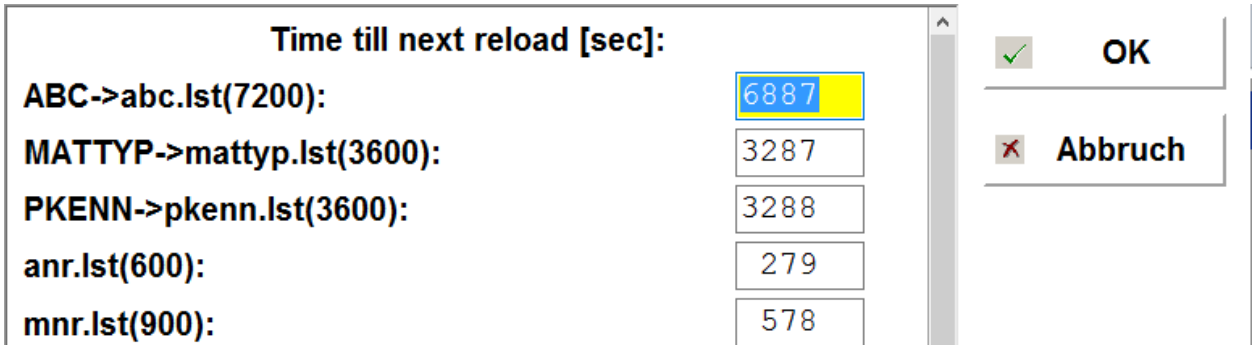
Example:

```
[#LIST#ABC]
LOADCYCLE=900
ENABLED=TRUE
```

11.4 Debugging

Active PDM lists are included in the debug timer display.

The display is performed with LIST-ID->FILE(LOADCYCLE):



The entry "PKENN->pkenn.lst(3600)" has a similar effect than the lists you configured yourself. The entry "PKENN->pkenn.lst(3600)" is automatically inserted if the PZE module is activated on the terminal via configuration in the terminal label.

12 Dynamic Dialogs - Workflow

Overview

Menu	System administration → Terminals → Workflow
Transaction code	ddconfw
Function authorization	ddconfw

Purpose

You can use the dialog configuration to change the AIP input dialogs in a quick and efficient manner according to the user's requirements.

The system delivery includes a basic dialog configuration. You can edit and change this configuration using the functions described in the following.

The function "Dynamic Dialogs - Workflow" defines the order of tabs in a complex dynamic dialog.

The complete functionality of the dialog configuration includes a lot of options, but it is also very complex. For this reason, we recommend to change the dialogs only after consultation with MPDV and only by experts.

Integration

To define the tab order, the fields and the buttons of dialogs, you must not only use the application "Dynamic dialogs - Workflow", but also the applications "Dynamic dialogs", "Dynamic dialogs - Fields" and "Dynamic dialogs - Function keys".

Requirements

The dialog that you want to configure must already exist in the "Dynamic dialogs".

Some of the functions require the development license MDS-AIS to be fully available. The restricted functions are marked in the document using "(*)".

Basics of the functions without development license:

- Existing data can be changed. Only data for default dialogs with user 0 must not be changed without development license.
- You cannot create new data without development license, except fields in existing dialogs.

- You can copy the existing data to terminal groups or terminals. Without development license, you cannot copy to default dialogs with user 0.

Selection criteria

The application provides the following selection criteria:

Workflow

Selection by workflow

Type

You can select from different dialog types:

- AIPDEF – default dialog
- AIPTNR – terminal dialog
- AIPTGRP – dialog for terminal group

Dlg user

Selection by terminal number or terminal group

Toolbar

Insert (*)

Creating a new workflow

Edit (*)

Editing a workflow

(*) Without development license, you cannot edit workflows. Editing a workflow is the same as creating a new workflow. You need a developer license to create new workflows.

Delete

Deleting a workflow

Copy

Copying a workflow

In addition to the standard function calls, the following function calls are available:

Dynamic dialogs

Function authorization: ddconf.*

The function "Dynamic Dialogs" calls the application Dynamic dialogs.

Test dialog

Function authorization: ddconf.test

The function "Test dialog" opens the selected Workflow to test all settings and the functionality.

Save dialog configuration

Function authorization: none

The function "Save dialog configuration" saves the current dialog configuration.

Activate dialogs

Function authorization: ddconf.activate

The function *Activate dialogs* activates the dynamic dialogs. The dialogs are then available on the terminals.

Field description

General

Workflow

Identifier of the workflow

The identifier can be assigned to a button on the AIP, for example. This button then opens the respective workflow dialog.

Type

AIPDEF:	default workflow
AIPTNR:	terminal workflow
AIPTGRP:	workflow for terminal group

User

User number restrictions

The default workflows have Dlg user "0" and type "AIPDEF".

Dialog

If you perform the function in the workflow using the server, the dialog identifier (DLG) specified in this field is transferred to the dialog data.

By specifying the dialog identifier via the "Dialog" field, you can define customer-specific workflows that send default dialogs to the server.

Title

Title of the workflow dialog. Using the notation <XXX>, you can define placeholders for important local information on the AIP. The title is then displayed as header in the workflow dialog.

Keep forced dialog sequence

If the field "Keep forced dialog sequence" is enabled, the order of the dialog steps is set and cannot be changed, i.e. you can only go to the next dialog step (tab).

User defined 1

Additional configuration options:

BUTTONHEIGHT=50

In case of workflows with several tabs, this configuration specifies the initial height of the button bar before scaling.

If you configure an alternative button height, the dialog field positions do not change automatically.

User defined 2...3

The fields "User defined 2...3" are currently not used.

Comment

You can use the field "Comment" to configure a description of the workflow.

Steps

Step 1...10

Name of the dialog configuration for dialog step 1...10. The dialog steps are displayed and performed in the specified order. You can use the different dialog steps in any workflow.

Script 1...10

W: The script of the workflow is run (and not the script of the dialog step).

S: The script of the dialog step is run (and not the script of the workflow).

Copying dynamic dialogs (workflow)

You can use the function "Copy" to copy complete workflow configurations of a dialog.

Function selection

Copy entire configuration

You can use the function "Copy entire configuration" to copy the complete workflow configuration, e.g. from the default configuration (AIPDEF 0) to a terminal (e.g. TNR nnn, with nnn= terminal number). If you copy the workflow configuration, the default configuration is still used for all terminals that do not have an own configuration. And later on you can change the custom configuration without affecting other users. If you use the mode "Copy entire configuration", the input fields "Workflow from" and "Workflow to" are hidden.

Copy workflow

If you use the function "Copy workflow", only the workflow entries for a selected dialog (workflow) are copied.

(*) Without a development license, you can only copy entire workflows or the entire configuration. You cannot copy to default workflows for user 0 without a development license.

Deleting dynamic dialogs (workflow)

If you use the function "Delete", you can delete the entries that include workflow data. The dialogs themselves are not deleted.

(*) Without a development license, you cannot delete default dialogs for user 0.

Testing dynamic dialogs

Use the function "Test dialog" to call the dialog. You can test its functionality or the separate steps.

Activate dialogs

If you use the function *Activate dialog*, the configured dynamic dialogs are available on the server and can then be downloaded to the terminals. Without activation, the dialogs and the possible changes are saved on the system, but the terminals still download the version activated last.

You can activate dialogs for single terminals or for terminal groups. When you activate a dialog, you specify the activation type and the user number that can be a terminal or a terminal group.

Type	**Value of field *User***	**Description**
AIPTNR, TNR	Terminal number	Activates all dialogs for the terminal. If a dialog is not explicitly configured for the terminal number, the dialog AIPDEF/DEF with user 0 is

		activated. The activation for user 0 provides the default dialogs.
AIPTGRP, TGRP	Terminal group	Activates all dialogs for the terminal group. The system only activates the dialogs that are explicitly configured for the terminal group.

In general, dialogs that are configured for a terminal have a higher priority than dialogs for terminal groups. If dialogs are activated for a terminal, the dialogs of the respective terminal group are ignored.

If no dialogs are activated for the terminal group or for the terminal number, the terminal loads the dialogs that are activated for user number 0.

Porting notes: from AIP to AIP2 / workflow with one workflow step

The difference in the terminal script processing in AIP and AIP2 in case of a workflow with only one workflow step is as follows:

Example:

Workflow: [U_TST]
with only one dialog step: [WF_U_TST]

In the **AIP**, all "dynamic dialog user exits" have been performed as **workflow** script ("U_TST").

In the **AIP2**, the following activities are performed independent of the dynamic workflow/dialog configuration:

- The user exit "DynDlgInit_" is always executed as **workflow** script ("U_TST").

- All other user exits are called in the **dialog** tab script ("WF_U_TST").

As of the AIP2 version 8.2.1.10, you can use the following workflow configuration to specify the same processing in the workflow script ("U_TST") as in the AIP:

"Step 1" "WF_U_TST" (STEP:1= WF_U_TST)
"Script" "W" (WFSCR:1=W)

13 Dynamic Dialogs

Overview

Menu	System administration → Terminals → Dynamic dialogs
Transaction code	ddconf
Function authorization	ddconf

Purpose

You can use the configuration of the dynamic dialogs to change the AIP input dialogs in a quick and efficient manner according to the user's requirements.

The system delivery includes a basic dialog configuration. You can edit and change this configuration using the functions described in the following.

You can use the function "Dynamic dialogs" to configure and customize the dialogs on the AIP. With this function, the general dialog parameters and specific AIP options are specified.

The application *Dynamic dialogs* also includes the configuration of fields and function keys. You can therefore easily configure the input dialogs in this one application.

The complete functionality of the dialog configuration includes a lot of options, but it is also very complex. For this reason, we recommend to change the dialogs only after consultation with MPDV and only by experts.

Integration

To define the tab order, the fields and the buttons of dialogs, you need not only use the application *Dynamic dialogs*, but also the applications *Dynamic dialogs - Workflow*, *Dynamic dialogs - Fields* and *Dynamic dialogs - Function keys*.

Requirements

Some of the functions require the development license MDS-AIS to be fully available. The restricted functions are marked in the document using "(*)".

Basics of the functions without development license:

- Existing data can be changed. Only data for default dialogs with user 0 must not be changed without development license.

- You cannot create new data without development license, except fields in existing dialogs.
- You can copy the existing data to terminal groups or terminals and then change them. Without development license, you cannot copy to default dialogs with user 0.

Selection criteria

The fields **Dialog, Type** and **User** are provided as selection criteria.

Toolbars

The application *Dynamic dialogs* provides several toolbars.

Toolbar *Main page*

This toolbar includes functions to edit dialogs. The different functions are described below.

Toolbar *Dynamic dialogs - fields*

The toolbar *Dynamic dialogs - fields* includes functions to edit fields.

Toolbar *Dynamic dialogs - function keys*

The toolbar *Dynamic dialogs - function keys* includes functions to edit function keys.

Functions of toolbar *Main page*

Insert (*)

Use the function *Insert* to create new dynamic dialogs in the system.

Copy (*)

Use the function *Copy* to copy complete configurations, single dialogs or parts of a dialog.

(*) Without development license, you can only copy complete dialogs or complete dialog configurations. Without development license, you cannot copy to default dialogs with user 0.

You require a development license for all other functions.

Edit

Use the function *Edit* to edit existing dynamic dialogs in the system.

Delete (*)

The function *Delete* deletes the selected dialogs including fields and buttons.

An undo function does not exist.

If you delete default dialogs (AIPDEF and DEF with user 0) (*), we strongly recommend to backup the dialogs before deletion.

(*) Without development license, you cannot delete default dialogs for user 0.

Test dialog

Function authorization: ddconf.test

Use the function *Test dialog* to call the dialog. You can test its functions, fields and function buttons.

Activate dialogs

Function authorization: ddconf.activate

The function *Activate dialogs* activates the dynamic dialogs. The dialogs are then available on the terminals.

Save dialog configuration

Function authorization: none

The function *Save dialog configuration* saves the current dialog configuration on the server.

Enable simple dialogs

Function authorization: ddconf.actsdlg

Use the function *Enable simple dialogs* to activate the simplified dialogs that can be used when needed. These dialogs show all data that must be entered on one page.

Workflow

Function authorization: ddconfw.*

The function *Workflow* calls the application Workflow.

Field description

Dialog

Dialog ID

Type

Dialog type:

- AIPDEF/DEF – standard dialog
- AIPTNR/TNR – terminal dialog
- AIPTGRP/TGRP – dialog for terminal group

User

According to type: terminal number or terminal group.

Note for the fields **Type** and **User**: there are some effects that are described in section **Activating dynamic dialogs**.

Key text

The field *Key text* is not used.

Resolution

You can use the field *Resolution* to scale the dialog and the controls (default = empty).

Short text

Use the field *Short text* to configure the tab title of the dialog. This text is shown when the dialog is used as tab in a workflow.

Long text

Use the field *Long text* to configure the dialog title of the dialog. This text is only shown if the dialog is a one-page dialog without workflow configuration.

Function 1

Use the field *Function 1* to select the function that is called when the dialog is opened.

If an entry starts with "FKT=", the function specified is called in the dialog script using the function DynDlgFunctions_<DLG>().

Optionally, a preassignment of field *yield* is possible when A_AB or A_UN

in function 1 (when dialog is opened) or 2 (when ANR is exited):

- SET_RESTME
  Remaining quantity = target quantity - yield – scrap (up to now) – scrap (in current dialog)
- SET_RESTM2
  Remaining quantity = target quantity – yield

Other entries are still available for reasons of downward compatibility (CTWIN), but are not used in current software versions.

Function 2

Use the field *Function 2* to select the function that is called just before the dialog is closed.

Entries starting with "FKT=" are possible here. MPDV recommendation: with script functions called on closing a dialog, trigger these script functions via the script of the relevant function key because the context is known then.

Comment

The field *Comment* is not used.

Height

Use the field *Height* to configure the height of the dialog window.

Only relevant with CTWIN dialogs and POPUP windows (see options 2).

Width

Use the field *Width* to configure the width of the dialog window.

Only relevant with CTWIN dialogs and POPUP windows (see options 2).

Key

If you configure a "+" in field *Key*, the dialog ID is shown in the window title of a one-page dialog (e.g. "Log on OP and person" > "Log on OP and person <A_P_AN>"). This information is useful for non-German customers and for the support.

Key ID

The field *Key ID* is not used.

Activation

The field *Active* specifies if the dialog is active or not active. Cannot be modified.

AIP options

Licenses

Use the field *Licenses* to optionally define one or several licenses, separated by semicolon.

- If the field *Licenses* is empty, the dialog step is always active.

- If licenses are entered in field *Licenses*, then the dialog step is only displayed if at least one of the licenses is available (OR conjunction).

Otherwise, the dialog step is **not displayed.**

Example:

DNC-BP;WRM-BP

Static condition

You can define one or several conditions via AND conjunction in field *Static condition*. The condition refers to the values of acronyms. For each acronym, you can enter one or several valid values, separated by semicolon.

Static conditions do not change in the course of a dialog. Static conditions are only evaluated when the workflow is opened.

If no condition is specified, the dialog step is always active.

If the condition is not fulfilled, the dialog step is **not displayed.**

Syntax for conditions:

Example of a value request:

```
MNR.MGRP=100
```

The machine group must be 100.

Example of array access/comparison

```
TNR.PARAM3[5]=5
```

The fifth character must be 5 (counting starts with 1).

```
TNR.PARAM3[3..5]=345
```

The characters 3 to 5 must be 345.

Example of negated conditions, several values are allowed

```
XXX<>12;34 & YYY=34;56
```

This condition is true if the content of <XXX> does not equal "12“ or "34“ and the content of <YYY> is equal to "34“ or "56“.

Example of programmed functions

```
PRG:EMPTY->ABC
```

The condition is true if the acronym ABC is empty or the acronym does not exist.

```
PRG:[NOT]EMPTY->ABC
```

The condition is true if the ID ABC exists and is not empty.

Dynamic condition

You can define one or several conditions via AND conjunction in field *Dynamic condition*. The condition refers to the values of acronyms. For each acronym, you can enter one or several valid values, separated by semicolon.

Dynamic conditions refer to acronyms that you can enter or change in the dialog. They are evaluated when the system changes to the next workflow tab.

If no condition is specified, the dialog step is always active.

If the condition is not fulfilled, the dialog step is **deactivated.**

The syntax of dynamic conditions is the same as the syntax of static conditions.

Forced fields

You can use the field *Forced fields* to configure fields that are required for the processing but that are not configured as hidden fields. You configure several field acronyms using semicolon (e.g. `KNR;PNR;..`). The field acronyms are then available in the dialog buffer.

User defined 1

Additional configuration options:

BUTTONHEIGHT=50

In case of workflows or dialogs with one tab only, this configuration specifies the initial height of the button bar before scaling (default 30).

The dialog field positions do not automatically change.

User defined 2

Additional configuration options:

Configuration of a pop-up window with the configured height and width before scaling.

Example: POPUP:150:50:clYellow

Syntax	pop-up window	POPUP	key	word
	X position	150	X position top left corner (default 5)	
	Y position	50	Y position top left corner (default 5)	
	color	clYellow	color of dialog background (default $A0FFFF)	

Copying dynamic dialogs

You can use the function *Copy dynamic dialogs* (button *Copy* in the toolbar) to copy complete configurations, single dialogs or parts of a dialog.

Function selection

Copy entire configuration

You can use the function *Copy entire configuration* to copy a complete configuration, e.g. from the default configuration (AIPDEF 0) to a terminal (e.g. AIPTNR nnn, with nnn= terminal number) or to a terminal group (AIPTGRP nnn, with nnn = terminal group). If you copy the configuration, the default configuration is still used with all terminals that do not have their own configuration. And later on you can change the custom configuration without affecting other users. In this mode, the input fields *Dialog from* and *Dialog to* are hidden.

Copy complete dialog

Use the function *Copy complete dialog* to call the following three copy operations in one operation.

Copy dialog without buttons and fields (*)

Use the function *Copy dialog without buttons and fields* to copy the basic or dialog information of the dialog. Fields and function keys are not copied.

Copy buttons of a dialog (*)

Use the function *Copy buttons of a dialog* to copy only function keys. The target dialog must exist before the copy operation.

Copy fields of a dialog (*)

Use the function *Copy fields of a dialog* to copy only fields. The target dialog must exist before the copy operation.

When you copy dialogs, you can specify type and user for the source (*From*) and the target (*To*). If the complete configuration is copied, the input fields of *Dialog from* and *Dialog to* are hidden.

(*) Without development license, you can only copy in mode *Copy entire configuration* or *Copy complete dialog*. Without development license, you cannot copy to default dialogs with user 0.

Activate dynamic dialogs

If you use the function *Activate dialog*, the configured dynamic dialogs are available on the server and can then be downloaded to the terminals. Without activation, the dialogs and the possible changes are saved on the system, but the terminals still download the version activated last.

You can activate dialogs for single terminals or for terminal groups. When you activate a dialog, you specify the activation type and the user number that can be a terminal or a terminal group.

Type	Value of *User*	Description
AIPTNR, TNR	Terminal number	Activates all dialogs for the terminal. If a dialog is not explicitly configured for the terminal number, the dialog is activated with type AIPDEF/DEF and user 0. The activation for user 0 provides the default dialogs. The activated dialogs are stored as files on the server: Schema: \\<Server>\<InstDir>\<SystemNr>\spool\aip<User>.* Example: \\MyServer\mip3\3\spool\aip10.* or: Schema: \\<Server>\<InstDir>\<SystemNr>\spool\<User>.* Example: \\MyServer\mip3\3\spool\10.*
AIPTGRP, TGRP	Terminal group	Activates all dialogs for the terminal group. The system only activates the dialogs that are explicitly configured for the terminal group. The activated data does not include default dialogs. The activated dialogs are stored as files on the server: Schema: \\<Server>\<InstDir>\<SystemNr>\spool\aiptgrp<User>.* Example: \\MyServer\mip3\3\spool\aiptgrp900.* or: Schema: \\<Server>\<InstDir>\<SystemNr>\spool\tgrp<User>.* Example: \\MyServer\mip3\3\spool\tgrp10.*

When you **load the dialogs from the terminal**, only the activated configurations are used. The following priority applies:

1. If dialogs are activated for the terminal, only the dialogs activated for the terminal are loaded. No other dialogs are loaded.
2. If dialogs are activated for the terminal group, only the dialogs activated for the terminal group are loaded. No other dialogs are loaded.
3. If no dialogs are activated for the terminal or the terminal group, then the default dialogs are loaded. The default dialogs are the dialogs activated for terminal/AIP terminal 0.

Enable simple dialogs

In the system, simplified dialogs are available that may be used if required. On one page, these dialogs show all data that must be entered. The dialogs are activated for the default terminal user AIPDEF using the function *Enable simple dialogs*.

This setting affects the following input dialogs:

- Log on order → A_AN
- Log on order + person → A_P_AN
- Interrupt order → A_UN
- Finish order → A_AB
- Post part quantities (partial confirmation) → A_TR

The activation of the simplified dialogs can only be performed once.
You can only undo the activation if you manually change the workflow.

With older installations: It is possible that with the simple dialogs "Log on order (A_AN)" or "Log on order + person (A_P_AN)" the data transfer to the dialog fields does not work after selection of an operation. Here, the configuration in the ctaiplay.ini must be corrected.

In section [WF@ANR], you must enter the current standard file ctaiplay.ini in line DATAFIELDS.

14 Dynamic Dialogs - Fields

Overview

Menu	System administration → Terminals → Dynamic dialogs, Tab *Dynamic Dialogs - fields*
Transaction code	ddconf
Function authorization	ddconf

Other option:

Menu	System administration → Terminals → Dynamic dialogs - fields
Transaction code	ddconff
Function authorization	ddconff

Purpose

You can use the configuration of the dynamic dialogs to change the AIP input dialogs in a quick and efficient manner according to the user's requirements.

The system delivery includes a basic dialog configuration. You can edit and change this configuration using the functions described in the following.

You can edit the dialog fields in two places: in the tab integrated in the application *Dynamic dialogs* and in the application *Dynamic dialogs - fields*. You can change existing fields (e.g. positioning), create new fields or delete fields.

The complete functionality of the dialog configuration includes a lot of options, but it is also very complex. For this reason, we recommend to change the dialogs only after consultation with MPDV and only by experts.

Integration

To define the tab order, the dialogs and the buttons, you need not only use the application *Dynamic dialogs - fields*, but also the applications *Dynamic dialogs - Workflow, Dynamic dialogs* and *Dynamic dialogs - Function keys*.

Requirements

The dialog must exist in the *Dynamic dialogs* function.

Some of the functions require the development license MDS-AIS to be fully available. Basics of the functions without development license:

- Existing data can be changed. Only data for default dialogs with user 0 must not be changed without development license.
- You cannot create new data without development license, except fields in existing dialogs.
- You can copy the existing data to terminal groups or terminals and then change them. Without development license, you cannot copy to default dialogs with user 0.

With the fields of the dynamic dialogs, all functions are available without development license. But without development license, you cannot create, edit or delete default dialogs for user 0.

Selection criteria

The application provides the following selection criteria. The selection criteria are read-only. The values entered in the previous application are automatically preassigned.

Dialog

Selection by dialog

Type

Selection by dialog types:

- AIPDEF/DEF – standard dialog
- AIPTNR/TNR – terminal dialog
- AIPTGRP/TGRP – dialog for terminal group

User

Selection by terminal number or terminal group

Editing functions

The application "Dynamic dialogs - fields" only provides the usual editing functions: insert, edit and delete.

If you use the application *Dynamic dialogs*, you can change to the toolbar tab *Dynamic dialogs - fields.* This toolbar provides the usual editing functions and additionally the detail application *Edit fields*. Using this editing application, you can easily manage and edit the fields of a dialog.

Detail application *Edit fields*

In the detail application *Edit fields*, you can right-click the table view to open a **context menu**. The context menu provides the following functions:

New row

Inserts a new row in the grid for the definition of a field.

Several new rows

Inserts the specified number of rows in the grid.

Copy row

Copies the currently select row and inserts it in the grid.

Delete row(s)

Deletes the currently selected row(s) from the grid.

Swap fields

The selected rows are swapped. You can select one of the two methods:

- **Swap positions**
  Swaps the X and Y positions of texts, fields and units of the two selected entries.
- **Swap field numbers**
  The field numbers and the tab order of the fields are swapped.

Align fields

Automatically aligns function keys in the X or Y direction.

Move fields

Moves one or several fields in the X or Y direction. Buttons are moved using the specified offset ("Move by").

Apply fields from other dialog

Takes over several fields from the selected dialog.

Field description

Tab *General*

Activated

This option is only available for reasons of downward compatibility. Always enable the option.

Field no.

Consecutive number of input field in dialog. Specifies the tab order in the dialog.

Text

Text label of field (on the left of the field).

Unit

Text for unit (on the right of the field).

Information

Information text displayed in tooltip if focused with mouse (mouseover).

Identifier

Identification of the data field in the dialog data string, e.g. ANR → |ANR=123456780100|

ID index

Index of field identification for similar data fields, e.g. |EGR:GUT=12340|.

Tab *Position*

X pos. text

X position of field label

Y pos. text

Y position of field label

X pos. field

X position of data field

Y pos. field

Y position of data field

Unit X pos.

X position of text for unit

Unit Y pos.

Y position of text for unit

Tab *Format*

Alignment

Alignment of text in input field.

"L"	left	left-aligned
"R"	right	right-aligned

Category

INPUT	Field is an input field.	
TEXT	Alphanumeric text	
GRID	Table	
OPTION	Option field	
RADIO	Selection group	(as of CTAIP: see "field attribute 1-8" → "EXTENDED")

Input type

Input type of data field.

ALPHA	alphanumeric
NUMERISCH	numeric without decimal places
FLIESS	numeric with decimal places
DATUM	date
ZEIT	time (00:00:00 - 23:59:59)
DAUER	time period (00:00:00 - 9...9:59:59)

Alternatively, a you can edit a preconfigured field type for user fields in the input type using the selection list, for example ANR. Only MPDV can change the field type for user fields.

Length

Total length of input field in characters.

Formatting

If a field type for user fields is entered in the "Input type" field, the formatting is done using the formatting rules defined for the field type.

If you use the simple input types (FLIESS,...), you can specify in field *Formatting* how the contents are displayed. Exceptions: types NUMERISCH and ALPHA; you cannot store a format for these types.

Sample configurations of input fields of the different input types:

FLIESS	###,### or -###,### or ##########	with or without algebraic sign. The decimal separator must be "," or ".".
DAUER	hhhhh:mm:ss	maximum 99999:59:59, h: hours with leading zeros
-dddd:mm:ss	maximum 9999:59:59 + sign, d:	hours without leading zeros, -: sign allowed.
	dd,iiii:	industrial format
DATE	dd.mm.yyyy	Default for date fields (need not be specified)
TIME	hh:mm:ss	Default for time indications
	hh:mm	Time without seconds (with leading zeros)
	(must be specified (mandatory))	

Allowed characters

You can restrict the characters that can be used. The restriction only applies for the category "INPUT". If no characters are entered, there is no restriction.

e.g. A-Za-z0-9

Default

You can store a default value that is preassigned to the input field.

e.g. 1.5.

From

Lower limit of default (only relevant for EINGABE category)

To

Upper limit of default (only relevant for EINGABE category)

Optional field 1

Only relevant for OPTION category

Optional field 1: Value, if active

Optional field 2

Only relevant for OPTION category

Optional field 2: Value, if inactive

Radio button

Labels and values for radio buttons (only relevant for RADIO category),

For example: J:Yes:F7;N:No:F8;V:Maybe:F9

1st Value: Return value (identification = X), if selected

2nd Value: Text next to radio button

3rd Value: Function key (optional)

On the AIP, the function keys are automatically assigned to the dialog buttons using the sequence displayed. If the function key of the radio button is additionally configured as function key of a dialog button that triggers actions or is automatically assigned on the AIP, then the dialog button takes priority over the radio button.

Tab *Functions*

Field attribute 1 to 8

Field attributes for input fields

- You can use "*@AKRONYM/KENNUNG@"* to configure an alternative identification to initialize dialog variables.

  Application example: (dialog "TRANRLIST“)

  Dialog field/ID "MATPUF“. The column "MPUFF" of the MNR.LST file includes the value initializing the field.

  The field will be initialized properly when opening the dialog, if the field "MATPUF" is configured with field attribute "@MNR.MPUFF@". An additional terminal script is not required.

- If the field attribute *"DATA.LEN=xxx"* is configured, the input length of a field can be changed. The display length is not affected. Example: *DATA.LEN=40* (input length 40).
- *MANUELL* – Input field is visible and editable
- *STATUS* – Input field is visible but not editable
- *NULL* - Input field may remain empty and is assigned with default value
- *FOCUS* - Input field is focused when the dialog is shown
- *FOCUSNEXTFIELDONBARC* – After editing the field using a serial barcode, the subsequent field is focused.
- *BARCODE* - Data can only be entered via barcode (keyboard locked)
- *READONLY* - This field cannot be edited.
- *SETVALUE* - if configured, the value included in the "default" field is used as the default value, e.g. for statuses (ID "MST") in the "log on OP" dialog. Without this identification, the default value is not written into the field. Note the following on the processing:
  If a field is configured several times with SETVALUE (i.e. the same field is included in more than one tab of a workflow), the configuration with the lowest tab index "takes priority".
  If a field is configured several times with SETVALUE on one workflow/tab page, the configuration with the greatest field index (no.) takes priority.
- *UPPERCASE* – Data input is converted into capital letters.
- *PASSWORD* – Input without visible characters ('***') - transmission is not encrypted
- *PWD* - Input without visible characters ('***') - transmission is performed with Blowfish encryption
- PWDRSA - Input without visible characters ('***') - transmission is performed with RSA encryption (only AIP2 as of V# 8.2.1.10 and hypdm32.dll V# 8.2.1.24)
- *EMPTY* – For NUMERIC fields only: if the field value is deleted, an empty input field is displayed instead of value 0.
- *UNSELECT* – prevents the whole field content from being selected, if the field is focused.
- *DIALOGLISTE* - modal list dialog that can be called (button behind input field)
  Dialog list function must be filled (see below)
- *COMBOBOX* – Field contains combobox; Combobox function must be filled. (Is not used on the terminal).

General field attributes:

- *AUTO* - Input field is not visible on the terminal, but field ID is assigned
- *LABELFONT* – Font type and background color of label are used to display the input field. In combination with the NOBORDER parameter, an input field can be created that is displayed as a label.
- *NOBORDER* - The input field is displayed without depth effect.
- *FIELDLABELFONT* – The input field font and background color do not depend on the label. Font type and background color are displayed as configured in the file "dialog.ini" (section "layout", values of "FieldLabelFont") (as of CTAIP).

- *COLORLABELFONT* – The input field font and background color do not depend on the label. Font type and background color are displayed as configured in the file "dialog.ini" (section "layout", values of "ColorLabelFont") (as of CTAIP). For further information, refer to the section "Further descriptions", paragraph "Field attribute <COLORLABELFONT>".
- *FIELD* – shows variable "data" labels. Select the category "TEXT". The "identification" field must be filled (only CTWIN).
- EXTENDED – If configured in the field attribute, you can configure further properties with category <RADIO> in the following fields as of AIP:
  - Number of columns in field "length"
  - Width in field "X position of unit"
  - Height in field "Y position of unit"
- *AUTOTAB* – If the field is completely filled with characters, the cursor goes to the next input field. Example: If a barcode reader is connected via keyboard, the scan can take up several input fields (only CTWIN).

Field attributes of category GRID:

- <*XXX*>*_GRID* – Definition/function of table (<*XXX*> = variable; setting only by MPDV)

Entry	Description
AG_GRID	Table with running operations at the machine selected (A_AUT_HU, A_AUT_MPL, A_AUT_RF).
C_MG_BLZ_GRID	Table with quantity info on the input batches for the operation of the machine selected (C_MG_BLZ).
CA_INFO_GRID	Table with batches preceding the operation selected (C_VLOS_MPL, C_VLOS_RF, C_VLOS_S)
CE_ASW_GRID	Table with components of the operation selected (CE_ASW_RF).
CE_GRID	Table of components/input batches of the operation selected with processing (A_AN_[MPL,RF,S], A_P_AN_[MPL,RF], CE_WL[MPL,RF,S]).
CE_INFO_GRID	Table of components/input batches of the operation selected for display (CA_WL_MPL).
FHM_GRID	Table of the resources used at the selected machine (RES_WL).
KOMP_VERB_GRID	Table with components of the selected operation to enter discrete consumption (A_VERB).
PAL_GRID	Table to display the batches of a pallet (C_PALETTE).
SCRIPT_GRID	Table for the display of variable contents. The configuration/processing is defined in the relevant dialog script.
WF_GRID	Table for the display of variable contents. Here, the configuration/processing is performed via the configuration in the section (field *Dialog list function*) in the layout file (ctwinlay.ini/ctaiplay.ini).

- *AUTOFILTERFIELD* – Display of an auto filter field in a table. The column(s) affected by this filter are stored in the file "ctaiplay.ini" in the relevant list as value of AUTOFILTERCOL. (Only in connection with field attributes: SCRIPT_GRID, WF_GRID / as of CTAIP)

- *METER* – Progression display. The value is displayed as a percentage. (CTAIP and CTWIN)
- *TEXTVIEW* – Display of a text file
- *IMAGE* – shows pictures (as of CTAIP)
  - *STRETCH* – shows pictures / adjusts pictures (as of CTAIP)
  - *STRETCH_PROP* – shows pictures / adjusts and fits proportions (as of CTAIP)
  - *MOUSEDOWN* – shows pictures / calls user exit <DynDlgFieldExit_..>(from CTAIP)
- *SHAPE* – shows a line defined in group "position" in the "general" tab: field X/Y → start position
  Unit X/Y → width and height(As of CTAIP)
- *INDICATOR* – Display of a measured value indicator (As of CTAIP)
  ONCHANGE – attribute triggering animation of the data displayed in the measured value indicator when measured values are entered in the input field. To do so, the attribute ONCHANGE must be set for the input field.
- *CHART* – Display of control charts and histograms. (As of CTAIP)
- *FIELDPAGER* – You can use this component to perform the following entry if configured accordingly:
  - Multiple input field (e.g. to enter multiple scrap values / dialog: A_TR)
  - Multiple function keys (e.g. for the status change / dialog: M_MST_Q)

  For further information, refer to section "Further descriptions", paragraph "Field attribute <FIELDPAGER>".

As mentioned above, field attributes must be written in capital letters.

Dialog list function

Function for the dialog list, which can be called on the terminal for dynamic dialogs.

Available dialog lists (entry in the LISTE field; case sensitive):

MNR_LISTE:	List of operations running at the machine
VAG_LISTE:	Order sequencing list (only CTWIN)
VMST_LISTE:	Machine status list
VGGRD_LISTE:	List of deviation reasons
VAGRD_LISTE:	List of scrap quantity reasons
VNCH_LISTE:	List of rework reasons (as of ADE 7.2/ MW2.0)
VPRB_LISTE:	List of problem quantity reasons (as of ADE 7.2/ MW2.0)
VLPKZ_LISTE:	List of premium indicators
VBPOS_LISTE:	List of operator positions
ZLO_LISTE:	List of material buffers (MPL)
TPE_LISTE:	List of transport units (MPL)
HZTYP_LISTE:	List of material types (MPL)

REQRES_LIST : List of the allowed and assigned resources of a required resource (only AIP2)

Combobox function

Leave this field empty with current configurations. This option is only available for backward compatibility reasons and was used for selection lists on the clients from older system versions (before MW 3).

Select file

File used to read the list (only relevant in combination with the *Combobox function*).

Function 1

Function started upon entering the field, called by parameter list in/out.

Function 2

Function started upon leaving the field, called by parameter list in/out

Options

Blocked

If the field is blocked, it won't be made available by the AIP when the dialog is activated. Therefore,it is unknown in the AIP.

Visible

Invisible fields are processed in the AIP but not displayed. Invisible fields can be used to send fixed acronyms from the AIP to the server. A target value should be added to invisible fields and the field attribute SETVALUE.

Visible/invisible with dialog control

The meaning of this field/control changes whether the option *Visible* is set or not.

- 'Visible' set > Not visible when dialog control is enabled
- 'Visible' not set > Visible when dialog control is enabled

Depending on how the "*Visible*" option is set, you can use identifiers to specify when the field/control is displayed or not.

For further information, refer to the section "Further descriptions", paragraph "*Dialog control* ".

DB table 1

DB tables, reserved for customization.

DB field 1

DB fields, reserved for customization.

DB table 2

DB tables, reserved for customization.

DB field 2

DB fields, reserved for customization.

User defined 1

Additional configuration options:

If your customer documentation includes no other specification, enter AUTO in this field.

User defined 3

Additional configuration options:

If you configure "FLD.LEN=xxx", you can configure a display length of a field that deviates from the standard, e.g. FLD.LEN=17 (display/input length 17 digits).

Customers often need this configuration to enter longer, customer-specific batch numbers. The input length of the default batch number is configured in the "length of batch no." field in the basic parameter settings.

Further descriptions

Dialog control

Configurations

A field *Dialog control* is provided with the following configurations:

- Machine/workplace configuration
- Order type configuration
- Terminal configuration (not yet available via GUI)

In field *Dialog control*, you enter an "identifier", which controls the further processing.

These identifiers are used in the application *Dynamic dialogs - fields* in tab *Options* in the fields -Visible J/N

- Visible with dialog control/Not visible with dialog control

to integrate the required control of the input fields.

Functionality

An input field/control in a dynamic dialog is displayed, if it is

- visible = J

or

- visible = N, but the *Visible with dialog control* field includes a (sub) string transferred by the *Dialog control* field of one of the configurations. Several "dialog control identifiers" can be specified, separated by semicolon.

An input field/control is not displayed, if it is

- visible = N

or

- visible = J, but the *Not visible with dialog control* field includes a (sub) string transferred by the *Dialog control* field from one of the three configurations. Several "dialog control identifiers" can be specified, separated by semicolon.

Example:

- Machine 4711, Field *Dialog control* = M1
- Order type 0, Field *Dialog control* = A1

In the application *Dynamic dialogs - fields*, the configuration is as follows:

- [ ] Visible
- Visible if [A1;M]

The input field or control is displayed if either M1 or A1 or both is true. This means: The posting is made for a machine where the field *Dialog control* includes M1 and/or an order/OP is logged on where the order type is configured with A1 in field *Dialog control*.

The fields that are hidden via dialog control are not sent to the server via dialog strings.

Field attribute <FIELDPAGER>

With a FIELDPAGER, the AIP generates several input fields for a configured field in the entry dialog depending on a list on the AIP. This can be used to collect scrap with several scrap reasons for partial confirmations and to delete and interrupt operations. The FIELDPAGER then generates automatically an input field for a valid scrap reason on any machine.

The multiple entry is only suitable if you need not display more than 80 elements.

The following fields are used from the dialog configuration:

Identifier

The identification gets the prefix and no „$CT.“ and no index. Example:

Scrap: „$CT.AUS:“

Rework: „$CT.NCH:“

Open quantity: „$CT.PRB:“

If the machine status is entered hierarchically, the identification is MST.

Positions

Position Field	FPOSX and FPOSY	= Top left corner of the input object
Position Unit	EPOSX and EPOSY	= Width and height of the input object

Alignment

"Left"

Category

„Grid“

Length

Input length of multiple fields, e.g. 8

Create an input format

The input format by default is integer. If inputs with decimal places are required, the input format can be the same as for normal input fields with the input type "FLIESS" and the formatting "####.##" (7 digits with 2 decimal places) or via a configured input type. The input types are restricted to numeric field types (with or without decimal places).

Field attribute 1

„FIELDPAGER“.

Dialog list function

Preconfigured fieldpager from a PAGER - section in the file ctaiplay.ini, for example "PAGER-AGRD.LST" (see below)

User defined 1

„AUTO“

The fields are generated using the font/size configurations designed for workflow fields (see "Dialog.ini“)

➔ LabelFont… (Name, Size, Style, Color) (for multiple input field identifiers)

➔ FieldFont … (Name, Size, Style, Color) (for multiple input field)

You can configure the available FIELDPAGER in a PAGER-section of the file ctaiplay.ini. The following configuration are prepared in the file ctaiplay.ini in the standard.

Section	Purpose
PAGER-AGRD.LST	Collection of scrap with several scrap reasons in case of partial confirmations, interruption and termination of operations. The FIELDPAGER generates a valid scrap reason for the input field at the machine.
PAGER-AGRD-NCH.LST	Collection of rework quantities with several reasons for partial confirmations or interrupting and terminating operations. The FIELDPAGER generates a valid rework reason for the input field at the machine.
PAGER-AGRD-PRB.LST	Collection of open quantities with several scrap reasons for partial confirmations or interruption and termination of operations. The FIELDPAGER generates a valid reason for open quantities for the input field at the machine.
MST-BUTTON-PAGER	Hierarchically input of the machine status.

Example for "Multiple scrap input".

Entry	Comment
Section **[PAGER-AGRD.LST]**	Configuration in workflows (dynamic dialogs) ➔ Field attribute:1 = FIELDPAGER ➔ List = PAGER-AGRD.LST
FILE=agrd.lst	Name of file used to generate multiple input fields
INI=	INI/configuration file including grid layout definition (Default = ctaiplay.ini)
SECTION=WF-PAGERPANEL-AGRD.LST	Section that includes the definition of the grid layout. The section "WF-PAGERPANEL-AGRD.LST" is required because the sorting does not work properly with ORDER sorting if file contents are unsorted. (Numeric sorting 1,2,3,4,5,10,11,22,... / alphanumeric 1,10,11,2.22.3.4.5....)

Entry	Comment
FILTER=MNR=<MNR> & ART=A	Possible filter to generate multiple input fields from the file <*FILE*>
ORDER=GRTXT	Possible sorting to generate multiple input fields from the file <*FILE*>
LabelColumn=GRTXT	Configuration of the column from the file <*FILE*> that has been designed as identifier for multiple input fields. (Default GRTXT)
LABELBEFOREFIELD=true	Optional: Identifier input field (Default=False)
LABELHEIGHTFAKTOR=1.25	Optional: Factor for calculating the height of a multi input field for positioning (default = 1.25)
LABELCHARCOUNT=20	Optional: Number of characters displayed for the label text of a multiple input field (see *LabelColumn*) (default=20)
IDCOLUMN=GR	Optional: Configuration of the column from the file <*FILE*> that includes the "KeyValue“ of the row. (Default GR)
MODE=… **SHOWIDCOLUMN=1|** **SHOWKEYLABEL=1|** **BUTTON-PAGER=1|** **BUTTON-RESULT=1|** or **BUTTON-RESULT=0|** **BUTTON-SKIN=BUTTON_BIG|**	Extended options - shows (<IDCOLUMN>) in label (Default=0) - shows hotkeys 'a' .. 'z'(default=0) - item is shown as button (default=0) - button closes the dialog (default=1/active) ..as of V# 2.0.3.45 - the dialog remains open. The user exit „DynDlgFunctions_<dlg>“ with the function "ON(<KENN>)CLICK" is executed. - skin for button (default=BUTTON_BIG)
INCREMENT-BUTTON-COLOR=clSilver **MODE=..** **INCREMENT-BUTTON=TRUE**	As of CTAIP V# 2.0.3.10 If the mode *"INCREMENT-BUTTON=TRUE"* is set, you can increment the value by 1 if you click on the label of the input field. Use the configuration „*INCREMENT-BUTTON-COLOR=clSilver*“ to change the label background. Default is *"clSilver“*.

Entry	Comment
Section **[WF-PAGERPANEL-AGRD.LST]**	Configuration of a grid layout for the generation of multiple input fields
GR=N10,100,R	Definition that enables numeric sorting.
GRTXT=C10,200,L	Standard configuration for alphanumeric sorting.

If you use user exits at the AIP as part of the MES Development Suite, bear in mind that the DynDlgFieldchange event is not normally triggered for the fieldpager when a field is changed. This is required to calculate a sum field that might exist.

If the field attribute "**FIELDCHANGE**" is added to the Fieldpager, the event is also triggered for its input fields. You must insert the following line in DynDlgFieldchange_XYZ in your user exits to ensure that totals are still calculated.

Example for Fieldpager with ID $CT.AUS::

```
Sub DynDlgFieldChange_A_TR
  Select Case VDlg("DLG.FLD")
    Case "$CT.AUS:"
      DLGVAR=Item("$CT.AUS:SUM",VDlg("$CT.AUS:SUM"))
  End Select
End Sub
```

Field attribute <COLORLABELFONT>

You can use the field attribute <COLORLABELFONT> to display texts with a special font. (see "Dialog.ini" -> ColorLabelFont[Name,Size,..])

The default field color is < blue > and can be used for the following dynamic dialog fields

GRID - for Grid/Image/Shape/... Header
IMAGE
SHAPE
FIELDPAGER

TEXTVIEW

METER

TEXT - for text display
Input - for field label

Use {c~color} to switch the color.
For example: **"Log on {c~clblack} <ANR> {c~clblue} order"**
➔ **"Log on AU0010001AG01 order"**

The color configuration may also be used for the workflow caption. Here, the default font color is black.

Translation: If you want to translate the new texts/labels, you must add the texts to the relevant translation file and translate them (standard: ctaip.mld, custom: ctaipkd.mld).

15 Dynamic Dialogs - Function Keys

Overview

Menu	System administration → Terminals → Dynamic dialogs, tab "Dynamic dialogs - function keys"
Transaction code	ddconf
Function authorization	ddconf

Other option:

Menu	System administration → Terminals → Dynamic dialogs - function keys
Transaction code	ddconfb
Function authorization	ddconfb

Purpose

You can use the dialog configuration to change the AIP input dialogs in a quick and efficient manner according to the user's requirements.

The system delivery includes a basic dialog configuration. You can edit and change this configuration using the functions described in the following.

You can edit the function keys of dialogs in two places: in the tab integrated in the application "Dynamic dialogs" and in the application "Dynamic dialogs - function keys". You can change existing function keys (e.g. positioning), create new function keys or delete others.

The complete functionality of the dialog configuration includes a lot of options, but it is also very complex. For this reason, we recommend to change the dialogs only after consultation with MPDV and only by experts.

Integration

To define dialogs; fields and the order of tabs, you must not only use the application "Dynamic dialogs - function keys", but also the applications "Dynamic dialogs - workflow", "Dynamic dialogs" and "Dynamic dialogs - fields".

Requirements

The dialog that you want to configure must already exist in the "Dynamic dialogs".

Some of the functions require the development license MDS-AIS to be fully available. The restricted functions are marked in the document using "(*)".

Basics of the functions without development license:

- Existing data can be changed. Only data for default dialogs with user 0 must not be changed without development license.
- You cannot create new data without development license, except fields in existing dialogs.
- You can copy the existing data to terminal groups or terminals and then change them. Without development license, you cannot copy to default dialogs with user 0.

Selection criteria

The application provides the following selection criteria:

Dialog

Selection by dialog

Type

You can select from different dialog types:

- DEF – standard dialog
- TNR – terminal dialog
- TGRP – dialog for terminal group

User

Selection by terminal number or terminal group

Editing functions

The application "Dynamic dialogs - function keys" only provides the usual editing functions: insert, edit and delete.

If you use the application "Dynamic dialogs", you can change into the toolbar tab "Dynamic dialogs - function keys". This toolbar provides the usual editing functions and additionally the detail application "Edit function keys". Using this editing application, you can easily manage and edit the function keys of a dialog.

Detailed application "Process function keys"

In the detail application "Edit function keys", you can right-click the table view to open a **context menu**. The context menu provides the following functions:

New row (*)

The function "New row (*)" adds a new empty row to the grid to define a function key.

Several new rows (*)

The function "Several new rows (*)" adds the specified number of new empty rows to the grid.

Copy row (*)

The function "Copy row (*)" copies the currently selected row and adds it to the grid.

Delete row(s) (*)

The function "Delete row(s) (*)" deletes the currently selected row(s) from the grid.

Swap function keys (*)

Using the function "Swap function keys (*)", the selected rows are swapped. The following types of swapping exist:

- **Swap position**

  With "Swap positions", the X and Y positions for function keys of the two selected entries are swapped (relevant CTWIN + ACTIONBUTTON).

- **Swap button no.**

  With "Swap button no.", the button number and therefore also the tab order of the function keys is swapped.

Align buttons (*)

Using the function "Align buttons (*)", an automatic alignment of the buttons in the x or y direction is possible.

Move buttons (*)

Using the function "Move buttons (*)", you can move one or several function keys in the x or y direction. Buttons are moved using the specified offset ("Move by").

Apply function keys from other dialog (*)

Using the function "Apply function keys from other dialog (*)", you can take over several function keys from the dialog selected.

Field description

Button no.

The field "Button no. (*)" specifies the sequence number of the button in the dialog. This number specifies the tab order.

Return code

The field "Return code" defines the further processing after confirmation of the function key.

0: Dialog is closed and the dialog string is sent to the server.

1: Dialog is canceled.

7: Dialog is closed, the dialog string is returned, but not sent.

8: Dialog is not closed; the virtual keyboard must still be displayed. (relevant with CTWIN)

9: Dialog is not closed.

Identifier

The field "Acronym" includes the value of the acronym for the return value; is returned as BTN=<acronym>

ID index

The field "Acronym index" includes the index for the return value in case of several similar data fields; is supplied with KENN, e.g. "..|BTN=BTN:UNDO|.."

Activated

This option is only available for reasons of downward compatibility. Select the option "Always" as of MW 3.x.

Key

Only CTWIN: You can use the field "Key" to configure the function key for the activation (hotkey) F1 to F12. By default, the key assignment is displayed on the button. The display is blocked if you place a "*" before the definition (e.g. KEY=*F1)

AIP: The function keys of the dialog are automatically assigned in the order of display to the function keys F1 to F12 of the keyboard.

Text

The field "Text" includes the button text (label) displayed.

X pos.

The field "X pos." specifies the X position of the button (top left corner of the button). (Relevant with CTWIN + ACTIONBUTTON).

Y pos.

The field "Y pos." specifies the Y position of the button (top left corner of the button). (Relevant CTWIN + ACTIONBUTTON).

Width

The field "Width" specifies the button width. (Relevant CTWIN + ACTIONBUTTON).

Height

The field "Height" specifies the button height. (Relevant CTWIN + ACTIONBUTTON).

Information

The field "Information" specifies the info text displayed as tooltip if the button is moused over.

Symbol

The field "Symbol" specifies the name of the assigned button icon.

(*): AIP: The available icons are included in the file pict.zip of the installation directory of the terminal.

Function

Function called via in/out.

For example:

Entry	Description
DLG=…	Calling a dialog
DLG=…;BREAK-ON-CANCEL	The dialog is not closed upon cancellation of the script dialog called, regardless of the "Return code" configured.
FKT=… (*)	Calling a script function
A_INFO	Calling the operation information (if an operation number is available in the dialog context).
A_AB_MPL A_UN_MPL A_TR	Calling the dialog "Log OP off", "Interrupt OP", "Partial confirmation" from the dialog "Interr/logoff/part.conf. OP" at an MPL machine.
A_AB_RF A_UN_RF A_TR_RF	Obsolete, not processed anymore.
C_VLOS	Calling the dialog "Show preceding batches" (C_VLOS_MPL,C_VLOS_RF)
CE_MLD	If you perform the function "Post batch", the dialogs - "Log off batch" (CE_AB,CE_AB_RF)" and - "Log on batch" (CE_AN,CE_AN_RF)" are used to change batches.
C_PAL_GEN	Generating a new batch number for a new pallet
CNR_ABF	Calling the dialog "Batch waste"
CNR_ADD	Execute function "PALTR.INSERT"
CNR_CHG	Calling the dialog "Modify batch length (C_CNR_LEN)" for update with "PALTR.UPDATE"
CNR_DEL	Execute function "PALTR.DELETE"
CNR-UNDO	UNDO function of a new batch number (only in dialog "Enter GR batch (C_GEN)")
DLG_CHECK	General function to check dialog input (only in dialog "Quantity balancing (C_MG_BLZ)")
ELW	Calling the dialog "Change input batch (CE_WL_MPL,CE_WL_RF)"
ELW_AB ELW_WL	Only in dialog "Quantity balancing (C_MG_BLZ)" Calling the dialog "Log off input batch (CE_AB,CE_AB_RF)" Performing the functions dialog "Log off input batch (CE_AB, CE_AB_RF)" and

	dialog "Log on input batch (CE_AN, CE_AN_RF)"
	General functions for navigating in tables (only CTWIN)
GRID_DOWN	next row
GRID_UP	previous row
GRID_LEFT	column left
GRID_RIGHT	column right
GRID_PAGEDOWN	next page
GRID_PAGEUP	previous page
PRINT	Calling the print (only CTWIN and in the dialogs "Form pallet (C_PAL_ASW)" and "Pallet (C_PALETTE)")
SEND	General function to perform server posting (only in the dialogs "Enter GR batch (C_GEN)" and "Repost batch (C_UMB)")
VERBRAUCH:RELOAD (consumption:reload)	Function "Reset (refresh consumption)" only in dialog "Component consumption posting (A_VERB)"
VERBRAUCH:START (consumption:start)	Calling the dialog "Component consumption posting (A_VERB)"
VTST	Showing/hiding the virtual keyboard (only required with CTWIN)

License

Optional: The field "License" includes the license required to activate the function key. If the field is empty, the key is always active.

User defined 1

The field "User def. 1" can include additional configuration options.

If you specify the value ACTIONBUTTON in field "User def. 2", you can configure the button layout in field "User def. 1".

Example: „clYellow,2,5,clBlack“

Syntax	color	clYellow	(default $0080FF)
	layout	2=rectangle	(0=capsule (default), 1=ellipse, 2=rectangle)
	corner radius	5	(default 10, only with layout 2=rectangle)
	font color	clBlack	(default clBlack)

User defined 2

In field "User def. 2", you can configure the following configuration options:

FORM-VALIDATION

Before executing the button function, the system checks if the contents of the input fields are valid, as it does when the dialog is closed.

ACTIONBUTTON:

As with "FORM-VALIDATION", the system checks if the contents of the input fields are valid.

You are free to position the ACTIONBUTTONS in the dialog. You specify position and size using the fields "X pos.", "Y pos.", "Width" and "Height". You specify the layout in field "User def. 1".

Blocked

If the field "Blocked" is selected, the button is not displayed and not processed. If dialogs are activated, the button is not passed to the terminal.